

Quantum Algorithms from First Principles

*Derivations and Implementation Guides to
Shor, Grover, Hamiltonian Simulation, Quantum Chemistry,
Linear Algebra, and Block Encoding Algorithms*

Muhammad Faryad

Free PDF edition

Version 1.0, October 2026

Copyright © 2026 Muhammad Faryad. All rights reserved.

You may download and print this PDF for your own study and teaching. Please do not repost or redistribute the file. Share this link instead: maven.com/faryad

Print and Kindle editions: get it from AMAZON.

ISBN (paperback): 9798175584913

ISBN (hardcover): 9798175591652

How to cite this book

M. Faryad, *Quantum Algorithms from First Principles: Derivations and Implementation Guides to Shor, Grover, Hamiltonian Simulation, Quantum Chemistry, Linear Algebra, and Block Encoding Algorithms*. Indy Pub, 2026. ISBN 9798175584913.

```
@book{faryad2026quantum,  
  author    = {Faryad, Muhammad},  
  title     = {Quantum Algorithms from First Principles: Derivations  
              and Implementation Guides to Shor, Grover, Hamiltonian  
              Simulation, Quantum Chemistry, Linear Algebra, and  
              Block Encoding Algorithms},  
  publisher = {Indy Pub},  
  year      = {2026},  
  isbn      = {9798175584913}  
}
```

Errata

Corrections are welcome at muhammad.faryad@lums.edu.pk. An errata list is maintained. Quote the version number above when you report an error.

Disclaimer

The code and worked examples in this book are for educational use. They are provided as is, without warranty of any kind. The code was tested with Qiskit 2.5 and may need changes for other versions. Qiskit and IBM Quantum are trademarks of IBM. This book is not affiliated with or endorsed by IBM.

An earlier and shorter version of this material was posted as arXiv:2507.11565 under a Creative Commons BY-NC-SA 4.0 license. The present edition is substantially revised and expanded, and is published under the copyright terms stated above.

*For my daughters,
Shireen and Eshaal*

Preface

What this book is

This book is a derivation-first guide to quantum algorithms. It grew out of a semester-long course taught at the Lahore University of Management Sciences (LUMS) in Fall 2021 and Summer 2024, and it is written for readers who want to understand *why* these algorithms work well enough to implement them.

Three commitments shape it.

Every step is derived. Where other treatments state a result and move on, this book works through the algebra. The intent is that a reader with a firm grasp of linear algebra, and no prior quantum mechanics, can follow every argument without needing a second source open alongside.

Every algorithm ends with an implementation guide. Understanding a derivation is not the same as being able to code it. Each algorithm chapter closes with register counts, the exact classical pre- and post-processing, the traps that produce plausible-looking wrong answers, and an ordered testing strategy. Appendix D collects the methodology those guides share.

Claims are checked, not asserted. Every mathematical statement, worked example and circuit in this book was verified numerically during preparation. Operators were built as explicit matrices, circuits were simulated, and error scalings were measured. Where a construction is exact only under conditions, those conditions are stated. Where a widely quoted speed-up carries caveats, the caveats appear next to it.

Who it is for

The prerequisites are linear algebra and comfort with complex numbers. From linear algebra you need vector spaces, eigenvalues, and unitary and Hermitian matrices. No prior quantum mechanics is assumed; Chapter 1 supplies what is needed. Some familiarity with programming will make the implementation guides far more useful, but the mathematics stands alone.

The book suits three audiences: students taking a first course in quantum algorithms, physicists and computer scientists moving into the field, and practitioners

who have used quantum software libraries and now want to know what the routines actually do.

How to use this book

Chapters 1–3 are foundations and should be read in order. After that the book has three largely independent strands, and readers with limited time can follow one:

- **The Fourier strand.** Chapters 4 and 5. The quantum Fourier transform, phase estimation, period finding, Shor’s factoring algorithm and the discrete logarithm. This is where the exponential speed-ups are.
- **The search strand.** Chapter 6. Grover search, amplitude amplification and estimation. Quadratic speed-ups, but far more widely applicable.
- **The simulation strand.** Chapters 7–9. Hamiltonian simulation, quantum chemistry, optimization and linear systems. This is the application area with the clearest physical case.

Chapter 10 unifies all three and is best read last.

If you want to ...	read ...
understand Shor’s algorithm	1–3, 4, 5
understand Grover and its uses	1–3, 6
simulate a molecule	1–3, 7, 8
solve optimization or linear systems	1–3, 4, 7, 9
work on fault-tolerant algorithms	all, ending with 10
teach a one-semester course	1–7, then 8 or 9 as time allows

Chapters 1–6 may be read as a companion to *Quantum Computation and Quantum Information* by Nielsen and Chuang, supplying the intermediate steps that text leaves to the reader.

Work the problems. Each chapter ends with twelve to twenty problems, ordered roughly by difficulty, 162 in all. Appendix E gives hints for the harder ones, and Appendix F gives the final answer to every problem whose answer is a number or a formula. I have kept circuits and proofs out of the answer key on purpose. Those you should check by building them.

Each chapter then closes with three **research problems and projects**. These are a different kind of exercise. They are open-ended, they take weeks rather than hours, and each one names a concrete deliverable: a measured scaling curve, a resource-count table, a reproduction of a published claim together with an extension of it. They are where the implementation guides pay off, and they are suitable as term projects or as the starting point of a thesis.

Read Appendix A before writing code. Most implementation failures in this subject are convention mismatches rather than mathematical errors. The usual culprits are a factor of two in a rotation angle, a reversed bit order, and an inverted occupation convention. That appendix collects every convention used here, and flags the ones that most often go wrong.

What is in each chapter

Chapter 1 reviews linear algebra and the postulates of quantum mechanics, then builds single- and multi-qubit gates. **Chapter 2** addresses state preparation and the construction of oracles. **Chapter 3** presents the foundational algorithms: entanglement, teleportation, Deutsch–Jozsa, Bernstein–Vazirani and Simon.

Chapter 4 develops the quantum Fourier transform and phase estimation, including the iterative single-ancilla variant. **Chapter 5** applies them to period finding, factoring, RSA and the discrete logarithm, with the classical number theory developed alongside.

Chapter 6 covers Grover search, quantum counting, amplitude amplification, and search as continuous-time evolution.

Chapter 7 treats Hamiltonian simulation: Trotter–Suzuki decomposition, sparse Hamiltonians, and linear combinations of unitaries. Its central construction, the circuit for exponentiating a Pauli string, is used by the three chapters that follow. **Chapter 8** maps fermionic systems to qubits via the Jordan–Wigner, parity and Bravyi–Kitaev transformations, and assembles the full quantum chemistry pipeline. **Chapter 9** covers combinatorial optimization by adiabatic evolution and QAOA, and linear systems by the HHL algorithm.

Chapter 10 introduces quantum signal processing and the singular value transformation, and shows that most of the preceding algorithms are the same algorithm with different polynomials. It closes with three algorithms of practical importance for fault-tolerant hardware: qubit-efficient amplitude estimation, Trotter-free simulation via qubitization, and linear solvers that improve HHL’s dependence on the condition number.

Variational algorithms are deliberately not covered. Their analysis is empirical rather than mathematical. Convergence, barren plateaus and optimizer choice remain open questions. Even QAOA, which is usually taught as a variational method, appears in Chapter 9 with a fixed schedule derived from adiabatic evolution, so that its cost can be written down. Everything presented here can be costed exactly. That is what makes it teachable.

Acknowledgements

I am deeply grateful to Muhammad Faizan and Amber Riaz, both MS students in the Department of Physics at LUMS, for their invaluable assistance in typesetting these notes in $\text{\LaTeX}$. Without their dedication and effort, these notes would likely not have taken their present form.

My graduate students Zain Shahid, Maaz Khan and Muhammad Abbas read the first drafts of this book with the care that only a student who has to learn from the text can bring. They caught errors that I had stopped seeing, and they told me plainly which passages did not teach. The book is clearer for it.

Where standard results are presented, sources are cited; the derivations, worked examples, exercises and exposition are my own.

Errors that remain are mine. Corrections are welcome at `muhammad.faryad@lums.edu.pk`, and an errata list will be maintained.

Muhammad Faryad
Lahore

Contents

Preface	v
1 Fundamentals	1
1.1 Review of Linear Algebra	1
1.2 Dimension and Basis of a Vector Space	3
1.3 Dirac Notation	4
1.3.1 Kets	4
1.3.2 Bras	4
1.3.3 Inner Product	4
1.3.4 Linear Operators	5
1.3.5 Outer Product	6
1.3.6 Tensor Product	7
1.4 Postulates of Quantum Mechanics	8
1.4.1 Postulate 1: State Space	8
1.4.2 Postulate 2: Evolution	11
1.4.3 Postulate 3: Measurement	16
1.4.4 Postulate 4: Composite Systems	18
1.5 Single-Qubit Gates in Outer-Product Form	21
1.5.1 Hadamard Gate	22
1.5.2 Phase Gate	23
1.5.3 Exponentiation of Operators	24
1.5.4 Rotation Gates	25
1.5.5 The Z–Y Decomposition of a Single-Qubit Gate	26
1.6 Two-Qubit Gates in Outer-Product Form	28
1.7 Entangled States	29
1.7.1 When is a State Entangled?	29
1.7.2 Bell States	31
1.8 Two-Qubit Gates	32
1.8.1 Controlled-NOT Gate	32
1.8.2 Controlled-Z Gate	34
1.8.3 SWAP Gate	35
1.9 Summary	36

1.10	End-of-Chapter Problems	37
	Research Problems and Projects	38
2	Quantum Circuits and Building Blocks	41
2.1	Reading a Quantum Circuit	42
2.2	The Standard Gate Set	43
2.2.1	Single-qubit gates	43
2.2.2	Two-qubit gates	44
2.2.3	Three-qubit gates	44
2.3	Controlled Operations	45
2.3.1	Building controlled- U from hardware gates	45
2.3.2	Controlling on $ 0\rangle$	46
2.3.3	Two controls from a square root	47
2.3.4	Many controls	48
2.4	Universality	50
2.4.1	Any unitary is a product of two-level unitaries	50
2.4.2	From two-level unitaries to circuits	52
2.4.3	Approximating with a finite gate set	54
2.4.4	Why approximation is hard in general	57
2.5	Quantum Oracles	58
2.5.1	Why oracles exist	58
2.5.2	Making f reversible	59
2.5.3	Phase kickback	60
2.5.4	Bit oracles and phase oracles	61
2.5.5	Building oracles for a given f	63
2.5.6	Garbage and uncomputation	64
2.6	Summary	66
2.7	End-of-Chapter Problems	66
	Research Problems and Projects	68
3	Foundational Quantum Algorithms	71
3.1	Preparation of Bell States	72
3.1.1	Measurement in the Bell basis	73
3.1.2	Measurement in the Hadamard basis	74
3.2	Quantum Teleportation	74
3.2.1	The protocol	75
3.2.2	The teleportation circuit	77
3.3	The Deutsch–Jozsa Algorithm	77
3.3.1	The problem	77
3.3.2	Classical cost	78
3.3.3	The quantum algorithm	78
3.4	The Bernstein–Vazirani Algorithm	82

3.4.1	The problem	82
3.4.2	Classical cost	83
3.4.3	The quantum algorithm	83
3.5	Simon's Algorithm	85
3.5.1	The problem	86
3.5.2	Classical cost	87
3.5.3	The quantum algorithm	88
3.5.4	Solving for p	91
3.6	Measurement Error Mitigation	92
3.6.1	Sources of error	92
3.6.2	Characterizing readout error	93
3.6.3	Recovering the ideal distribution	94
3.7	Summary	95
3.8	End-of-Chapter Problems	96
	Research Problems and Projects	97
4	The Quantum Fourier Transform and Phase Estimation	99
4.1	The Quantum Fourier Transform	100
4.1.1	From the DFT to the QFT	100
4.1.2	One qubit	101
4.1.3	Two qubits	102
4.1.4	The general product form	104
4.1.5	The circuit	104
4.1.6	Graphical interpretation	107
4.1.7	The inverse QFT	108
4.2	Quantum Phase Estimation	109
4.2.1	The problem	109
4.2.2	Why the eigenvalue is a pure phase	109
4.2.3	The key idea: powers of U shift the binary point	110
4.2.4	The circuit	111
4.2.5	Step by step	112
4.2.6	When θ is not a dyadic rational	113
4.2.7	Superpositions of eigenvectors	115
4.3	Iterative Phase Estimation	116
4.3.1	Motivation	116
4.3.2	Where the idea comes from	117
4.3.3	The algorithm	118
4.4	Summary	121
4.5	End-of-Chapter Problems	122
	Research Problems and Projects	123
5	Period Finding, Factoring and Discrete Logarithms	127

5.1	Period Finding	128
5.1.1	The problem	128
5.1.2	The algorithm	130
5.1.3	Continued fractions	133
5.1.4	Implementation notes	135
5.2	Number Theory for Factoring	136
5.2.1	Primes and factorizations	136
5.2.2	Greatest common divisors	137
5.2.3	Bézout's identity and modular inverses	138
5.2.4	From order finding to factoring	138
5.2.5	Implementation notes	140
5.3	Shor's Algorithm	140
5.3.1	The complete procedure	140
5.3.2	The quantum subroutine as phase estimation	141
5.3.3	The circuit	143
5.3.4	Worked example: factoring $N = 15$	144
5.3.5	Implementation notes	146
5.4	Breaking RSA	149
5.4.1	The RSA scheme	149
5.4.2	Implementation notes	151
5.5	The Discrete Logarithm Algorithm	151
5.5.1	The problem	151
5.5.2	The trick: a two-dimensional period	151
5.5.3	The circuit	153
5.5.4	The derivation	154
5.5.5	Recovering s	154
5.5.6	Implementation notes	155
5.6	Summary	155
5.7	End-of-Chapter Problems	156
	Research Problems and Projects	157
6	Grover Search and Amplitude Amplification	159
6.1	The Search Problem	160
6.2	Grover's Algorithm	161
6.2.1	The circuit	161
6.2.2	The oracle	162
6.2.3	The diffuser	163
6.2.4	The two-dimensional picture	163
6.2.5	One Grover iteration	164
6.2.6	How many iterations?	166
6.2.7	Multiple solutions	168
6.2.8	Implementation guide	170

6.3	Quantum Counting	171
6.3.1	The idea	171
6.3.2	The circuit	173
6.3.3	Implementation guide	174
6.4	Amplitude Amplification	175
6.4.1	Generalizing Grover	175
6.4.2	Amplitude estimation	177
6.4.3	Derandomizing Grover	178
6.4.4	Implementation guide	179
6.5	Quantum Search as Hamiltonian Simulation	180
6.5.1	Guessing the Hamiltonian	180
6.5.2	Evolving	181
6.5.3	Simulating the Hamiltonian	181
6.5.4	Circuits	182
6.5.5	Implementation guide	184
6.6	Summary	184
6.7	End-of-Chapter Problems	185
	Research Problems and Projects	186
7	Hamiltonian Simulation	189
7.1	The Trotter–Suzuki Method	190
7.1.1	Locality	190
7.1.2	First-order Trotter	191
7.1.3	Second-order (symmetric) Trotter	192
7.1.4	Implementation guide	194
7.2	Simulating Sparse Hamiltonians	194
7.2.1	Sparsity and oracles	194
7.2.2	The 1-sparse case	195
7.2.3	The algorithm	196
7.2.4	Implementation guide	199
7.3	Linear Combination of Unitaries	199
7.3.1	The problem	199
7.3.2	The construction	200
7.3.3	Oblivious amplitude amplification	202
7.3.4	Hamiltonian simulation via LCU	204
7.3.5	Implementation guide	206
7.4	Simulating Pauli Strings	206
7.4.1	Every Hamiltonian is a sum of Pauli strings	206
7.4.2	The building block: $e^{-i\theta Z}$	207
7.4.3	$e^{-i\theta Z \otimes Z}$: the parity trick	207
7.4.4	The general recipe	208
7.4.5	Putting it together: a sum of Pauli strings	211

7.4.6	Implementation guide	212
7.5	Summary	213
7.6	End-of-Chapter Problems	213
	Research Problems and Projects	215
8	Quantum Chemistry on a Quantum Computer	217
8.1	The Molecular Hamiltonian	218
8.1.1	Many-particle Schrödinger equation	218
8.1.2	Born–Oppenheimer approximation	219
8.2	Identical Particles and Antisymmetry	219
8.2.1	Exchange symmetry	219
8.2.2	Slater determinants and the Pauli principle	219
8.3	Second Quantization	220
8.3.1	Fock space	220
8.3.2	Creation and annihilation operators	220
8.3.3	Anticommutation and the ordering sign	221
8.3.4	The second-quantized Hamiltonian	221
8.4	Jordan–Wigner Transformation	223
8.4.1	The mapping	223
8.4.2	Useful special cases	223
8.4.3	The cost	224
8.5	Parity Transformation	225
8.6	Bravyi–Kitaev Transformation	226
8.6.1	The idea	226
8.6.2	The matrix	227
8.6.3	The three sets	228
8.6.4	The operator mapping	229
8.6.5	Comparing the three encodings	231
8.6.6	Qubit tapering	232
8.7	From Qubit Hamiltonian to Circuit	232
8.7.1	Preparing the reference state	232
8.7.2	Evolving under the Hamiltonian	233
8.7.3	Measuring the energy	234
8.7.4	Implementation guide	236
8.8	Summary	238
8.9	End-of-Chapter Problems	239
	Research Problems and Projects	240
9	Optimization and Linear Algebra	243
9.1	QUBO and the Ising Hamiltonian	244
9.1.1	Quadratic unconstrained binary optimization	244
9.1.2	MaxCut	244

9.1.3	Converting to QUBO form	245
9.1.4	Converting to an Ising Hamiltonian	246
9.2	Adiabatic Quantum Computation	247
9.2.1	The adiabatic theorem	247
9.2.2	The interpolation	248
9.2.3	Discretizing	248
9.3	QAOA	249
9.3.1	The algorithm	249
9.3.2	The circuit	251
9.3.3	Implementation guide	253
9.4	The Hadamard Test	254
9.5	The HHL Algorithm	256
9.5.1	The problem	256
9.5.2	The three steps	257
9.5.3	Implementing the $1/\lambda$ rotation	259
9.5.4	Cost, and four caveats	261
9.5.5	Implementation guide	263
9.6	Summary	264
9.7	End-of-Chapter Problems	265
	Research Problems and Projects	266
10	Quantum Singular Value Transformation Algorithms	269
10.1	Block Encoding	270
10.2	Quantum Signal Processing	272
10.2.1	The signal operator	272
10.2.2	Why Chebyshev polynomials appear	273
10.2.3	Finding the phases	274
10.3	Qubitization	275
10.3.1	The walk operator	275
10.4	The Quantum Singular Value Transformation	277
10.4.1	Recovering the algorithms of earlier chapters	278
10.5	Application 1: Qubit-Efficient Amplitude Estimation	280
10.5.1	The problem, and why it matters	280
10.5.2	The textbook version, and its cost	281
10.5.3	Removing the phase estimation	281
10.5.4	Implementation guide	284
10.6	Application 2: Trotter-Free Simulation via Qubitization	284
10.6.1	The idea	284
10.6.2	Implementation guide	287
10.7	Application 3: Linear Systems Beyond HHL	288
10.7.1	Inversion as polynomial approximation	288
10.7.2	Implementation guide	290

10.8 Summary	290
10.9 End-of-Chapter Problems	291
Research Problems and Projects	293
A Notation and Conventions	295
A.1 Dirac notation	295
A.2 Scalars and counts	296
A.3 Bit ordering: the endianness trap	298
A.4 Rotation angles: the factor of two	299
A.5 Fourier sign convention	299
A.6 Occupation and ladder operators	299
A.7 Bravyi–Kitaev index sets	300
A.8 Units	300
B Gate and Identity Reference	301
B.1 Single-qubit gates	301
B.2 Identities used repeatedly	302
B.3 The Pauli exponential circuit	302
B.4 These gates in Qiskit	303
C Algorithm Resource Summary	305
D Verifying an Implementation	309
D.1 The five tests	309
D.2 Symptom-to-cause table	310
D.3 A note on published demonstrations	310
E Hints for Selected Problems	311
F Answers to Selected Problems	315
Bibliography	327
Index	329

Chapter 1

Fundamentals

Everything in this book rests on the material in this chapter. The good news is that it is almost entirely linear algebra over the complex numbers. If you can multiply matrices and take a dot product, you already have the tools. What is new is the notation, the physical interpretation, and a handful of rules (the postulates) that tell you which mathematical objects correspond to which physical things.

I have kept the review of vector spaces compact, because I want to get you to Dirac's bra-ket notation, the postulates of quantum mechanics, and the single- and two-qubit gates that every quantum algorithm is built from. Those are the things you will use on every page that follows.

What you should be able to do after this chapter

- Write a qubit state as a column vector and as a ket, convert between them, and locate it on the Bloch sphere from its amplitudes.
- Compute inner products, outer products, and tensor products by hand.
- State the four postulates and explain what each one is for.
- Apply X , Y , Z , H , S and the rotation gates, in both matrix and Dirac form.
- Write down CX , CZ and SWAP, predict their action on any basis state, and decide whether a two-qubit state is entangled.

1.1 Review of Linear Algebra

A *vector space* over the complex numbers consists of two sets of objects and two algebraic rules:

1. a set of *vectors* $V = \{|v_1\rangle, |v_2\rangle, |v_3\rangle, \dots\}$ and a set of *scalars*, here the complex numbers $\mathbb{C}$, whose elements we write $a, b, c, \dots$;

2. a rule for vector *addition* and a rule for *multiplication by a scalar*.

These two rules are not arbitrary. They must satisfy the axioms below. You have almost certainly used every one of these for ordinary arrows in the plane without ever writing them down. I list them because they are *all* we assume. Any collection of objects obeying them behaves like a space of vectors, whether the objects are arrows, polynomials, or quantum states.

a) **Addition rule.** Vector addition has the following properties.

- **Closure:** if $|v_1\rangle$ and $|v_2\rangle$ are elements of V , so is their sum:

$$\text{if } |v_1\rangle, |v_2\rangle \in V \Rightarrow |v_1\rangle + |v_2\rangle \in V.$$

- **Commutativity:** $|v_1\rangle + |v_2\rangle = |v_2\rangle + |v_1\rangle$.
- **Associativity:**

$$(|v_1\rangle + |v_2\rangle) + |v_3\rangle = |v_1\rangle + (|v_2\rangle + |v_3\rangle).$$

- **Existence of a null vector:** there is a vector $|N\rangle \in V$ such that for every $|v\rangle$,

$$|N\rangle + |v\rangle = |v\rangle + |N\rangle = |v\rangle.$$

- **Existence of an inverse:** each $|v\rangle$ has an inverse $-|v\rangle$ with

$$|v\rangle + (-|v\rangle) = (-|v\rangle) + |v\rangle = |N\rangle.$$

b) **Scalar multiplication rule.** Multiplication of vectors by scalars (drawn from $\mathbb{R}$ or, for us, $\mathbb{C}$) has the following properties.

- **Closure:** the product of a scalar with a vector is a vector in the same space:

$$\text{if } a \in \mathbb{C} \text{ and } |v\rangle \in V \Rightarrow a|v\rangle \in V.$$

- **Distributivity:**

$$a(|v_1\rangle + |v_2\rangle) = a|v_1\rangle + a|v_2\rangle, \quad (a+b)|v\rangle = a|v\rangle + b|v\rangle.$$

- **Associativity with respect to scalar multiplication:**

$$a(b|v\rangle) = (ab)|v\rangle.$$

- **Unit and zero scalars:** the scalar 1 and the scalar 0 satisfy

$$1|v\rangle = |v\rangle \quad \text{and} \quad 0|v\rangle = |N\rangle.$$

Watch the types

$0|v\rangle = |N\rangle$: multiplying a vector by the *number* zero gives the *null vector*, not the number zero. Scalars and vectors are different kinds of object, and an equation can never equate one to the other. Keep this straight now. It will save you a surprising amount of confusion later, when operators, scalars and vectors all appear in the same expression.

1.2 Dimension and Basis of a Vector Space

A set of nonzero vectors $\{|v_1\rangle, |v_2\rangle, \dots, |v_n\rangle\}$ is *linearly independent* if the only solution of

$$\sum_{i=1}^n c_i |v_i\rangle = |N\rangle$$

is $c_1 = c_2 = \dots = c_n = 0$. If instead there is a solution in which at least one c_i is nonzero, the set is *linearly dependent*. In that case one of the vectors can be written as a linear combination of the others, so it carries no information the others do not already carry.

Span. The *span* of a set of vectors is the collection of all their linear combinations. A set *spans* V if every vector in V can be written as such a combination.

Basis. A *basis* of V is a set of vectors that is both linearly independent and spanning. Equivalently: every $|w\rangle \in V$ can be written in exactly one way as

$$|w\rangle = \sum_{i=1}^n c_i |v_i\rangle.$$

The words “exactly one way” are what linear independence buys you. The coefficients c_i are unambiguous, so they can be used as coordinates.

Dimension. Every basis of a given vector space has the same number of elements, and that number is the *dimension* of the space. A single qubit lives in a two-dimensional space, so every basis for it has exactly two vectors.

It is almost always convenient to choose a basis that is *orthonormal*, meaning the basis vectors are mutually orthogonal and each has unit length:

$$\langle v_i | v_j \rangle = \delta_{ij},$$

$$\text{where } \delta_{ij} = \begin{cases} 0 & i \neq j, \\ 1 & i = j. \end{cases}$$

(The inner product $\langle \cdot | \cdot \rangle$ is defined in the next section; if you have not met it before, read on and return here.) Orthonormality makes extracting coefficients trivial: if $|w\rangle = \sum_i c_i |v_i\rangle$, then $c_j = \langle v_j | w \rangle$. Almost every calculation in this book uses that one line, so I want you to be able to write it down without thinking.

1.3 Dirac Notation

Physicists write quantum states using a notation introduced by Paul Dirac. It looks strange at first, but it is designed so that the shape of the bracket tells you what kind of object you are holding.

1.3.1 Kets

A *ket* $|v\rangle$ denotes a vector in the state space. Once a basis is chosen, it is represented by a column of numbers. For example, in $\mathbb{C}^2$ an arbitrary ket is

$$|v\rangle = \begin{pmatrix} a \\ b \end{pmatrix}, \quad a, b \in \mathbb{C}.$$

Let me be pedantic about this once. The ket *is* the abstract vector; the column of numbers is its *representation* in a particular basis. The same state has different columns in different bases, just as the same arrow has different components in different coordinate systems.

1.3.2 Bras

For every ket $|v\rangle$ there is a corresponding *bra* $\langle v|$, a row vector obtained by transposing and complex-conjugating:

$$\text{if } |v\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \text{ then } \langle v| = (|v\rangle)^\dagger = [|v\rangle^*]^T = \begin{pmatrix} a^* & b^* \end{pmatrix},$$

where a^* and b^* are the complex conjugates of a and b . The dagger $\dagger$ means “conjugate transpose” and will appear constantly.

1.3.3 Inner Product

The inner product of two vectors is written $\langle v | w \rangle$, a *bra-ket*, which is where the names come from. With $|v\rangle = \begin{pmatrix} a \\ b \end{pmatrix}$ and $|w\rangle = \begin{pmatrix} c \\ d \end{pmatrix}$,

$$\langle v | w \rangle = \begin{pmatrix} a^* & b^* \end{pmatrix} \begin{pmatrix} c \\ d \end{pmatrix}$$

$$= a^*c + b^*d.$$

Watch the order-reversal rule,

$$\langle v|w\rangle = (\langle w|v\rangle)^*,$$

which the complex conjugation forces on us. Unlike the real dot product, the complex inner product is *not* symmetric.

The inner product of a vector with itself is always a nonnegative real number:

$$\langle v|v\rangle = a^*a + b^*b = |a|^2 + |b|^2 \geq 0,$$

with equality if and only if $|v\rangle$ is the null vector. Its square root $\| |v\rangle \| = \sqrt{\langle v|v\rangle}$ is the *norm*, or length, of the vector.

Orthogonal and orthonormal vectors. Two vectors are *orthogonal* if their inner product vanishes: $\langle v|w\rangle = \langle w|v\rangle = 0$. A *set* of vectors is *orthonormal* if its members are pairwise orthogonal and each has norm 1. An orthonormal set is automatically linearly independent, so if it contains n vectors in an n -dimensional space it is an orthonormal *basis*. Two orthonormal vectors form a basis of $\mathbb{C}^2$, but they would not form a basis of $\mathbb{C}^3$, so the count matters.

What is a Hilbert space? A *Hilbert space* is a vector space equipped with an inner product that is also *complete*: every sequence whose terms get arbitrarily close together converges to a limit inside the space. Completeness is a technical condition that matters in infinite dimensions. In the finite-dimensional spaces $\mathbb{C}^2, \mathbb{C}^4, \dots$ used throughout this book it is automatic. For our purposes, “Hilbert space” means “complex vector space with an inner product”.

Common misconception

A Hilbert space is *not* a space in which all vectors are normalized. It contains vectors of every length, including the null vector. It is the *physical states* that are required to be unit vectors. That is a postulate about physics, imposed on top of the mathematics, and we meet it in Section 1.4.

1.3.4 Linear Operators

An operator $\hat{A}$ is a rule that turns a vector into another vector in the same space. It is *linear* if for any vectors $|v_1\rangle, |v_2\rangle$ and any complex numbers c_1, c_2 ,

$$\hat{A}(c_1|v_1\rangle + c_2|v_2\rangle) = c_1\hat{A}|v_1\rangle + c_2\hat{A}|v_2\rangle.$$

Linearity is the reason quantum mechanics is tractable. To know what an operator does to *every* state, it is enough to know what it does to the basis states. In a d -dimensional space, a linear operator is represented by a $d \times d$ matrix.

For example, the operator

$$\hat{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

acts on the two basis vectors as

$$\hat{X} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \hat{X} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

It exchanges the two basis vectors. We will meet it again as the quantum NOT gate.

1.3.5 Outer Product

Where the inner product $\langle v|w\rangle$ multiplies a row by a column and returns a *number*, the outer product $|v\rangle\langle w|$ multiplies a column by a row and returns a *matrix*, i.e. an operator. With $|v\rangle = \begin{pmatrix} a \\ b \end{pmatrix}$ and $|w\rangle = \begin{pmatrix} c \\ d \end{pmatrix}$,

$$|v\rangle\langle w| = \begin{pmatrix} a \\ b \end{pmatrix} \begin{pmatrix} c^* & d^* \end{pmatrix} = \begin{pmatrix} ac^* & ad^* \\ bc^* & bd^* \end{pmatrix}.$$

The way I read $|v\rangle\langle w|$ is as an instruction: “*absorb a $|w\rangle$, emit a $|v\rangle$* ”. Acting on a ket $|u\rangle$ it gives $|v\rangle\langle w| |u\rangle = \langle w|u\rangle |v\rangle$. The number $\langle w|u\rangle$ falls out and multiplies $|v\rangle$. Once you internalize that, outer-product algebra becomes bookkeeping.

Taking $|0\rangle$ and $|1\rangle$ as the two basis vectors, their four outer products are

$$\begin{aligned} |0\rangle\langle 0| &= \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, & |0\rangle\langle 1| &= \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \\ |1\rangle\langle 0| &= \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, & |1\rangle\langle 1| &= \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}. \end{aligned}$$

Each picks out one entry of a 2×2 matrix, so any $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ can be written as

$$A = a |0\rangle\langle 0| + b |0\rangle\langle 1| + c |1\rangle\langle 0| + d |1\rangle\langle 1|.$$

This is the *outer-product form* of an operator, and we will use it throughout Section 1.5.

Reading the notation at a glance

$\langle v w\rangle$	row $\times$ column	a <i>number</i> (scalar)
$ v\rangle\langle w $	column $\times$ row	a <i>matrix</i> (operator)
$\hat{A} v\rangle$	matrix $\times$ column	a <i>ket</i>
$\langle v \hat{A}$	row $\times$ matrix	a <i>bra</i>
$\langle v \hat{A} w\rangle$	row $\times$ matrix $\times$ column	a <i>number</i>

1.3.6 Tensor Product

Suppose H_1 and H_2 are Hilbert spaces of dimension N_1 and N_2 . To describe two systems at once we combine them into a larger space using the *tensor product*, written $\otimes$:

$$H = H_1 \otimes H_2, \quad \dim(H) = N_1 N_2.$$

The dimension *multiplies*. I regard this as the single most important fact in quantum computing. Each qubit has a two-dimensional state space, so n qubits together have a space of dimension $2 \times 2 \times \cdots \times 2 = 2^n$, and a general state of n qubits needs 2^n complex amplitudes to write down. A register of n classical bits, by contrast, is described by n numbers. In plain words, every qubit you add doubles the number of amplitudes. A classical computer that stores the full state of n qubits needs 2^n complex numbers of memory, and once n passes fifty or so that is more memory than the largest supercomputer has. That exponential growth is both the source of quantum computers' promise and the reason they are hard to simulate classically.

If $|\phi\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \in H_1$ and $|\chi\rangle = \begin{pmatrix} c \\ d \end{pmatrix} \in H_2$, the composite state is

$$|\psi\rangle = |\phi\rangle \otimes |\chi\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} a \begin{pmatrix} c \\ d \end{pmatrix} \\ b \begin{pmatrix} c \\ d \end{pmatrix} \end{pmatrix} = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix}.$$

The recipe is mechanical: take each entry of the first vector in turn and multiply the whole of the second vector by it, stacking the results.

The same block rule applies to operators. If $\hat{A}_1$ acts on H_1 and $\hat{A}_2$ acts on H_2 , then

$$\begin{aligned} \hat{A} = \hat{A}_1 \otimes \hat{A}_2 &= \begin{pmatrix} a_1 & b_1 \\ c_1 & d_1 \end{pmatrix} \otimes \begin{pmatrix} a_2 & b_2 \\ c_2 & d_2 \end{pmatrix} \\ &= \begin{pmatrix} a_1 \begin{pmatrix} a_2 & b_2 \\ c_2 & d_2 \end{pmatrix} & b_1 \begin{pmatrix} a_2 & b_2 \\ c_2 & d_2 \end{pmatrix} \\ c_1 \begin{pmatrix} a_2 & b_2 \\ c_2 & d_2 \end{pmatrix} & d_1 \begin{pmatrix} a_2 & b_2 \\ c_2 & d_2 \end{pmatrix} \end{pmatrix} \\ &= \begin{pmatrix} a_1 a_2 & a_1 b_2 & b_1 a_2 & b_1 b_2 \\ a_1 c_2 & a_1 d_2 & b_1 c_2 & b_1 d_2 \\ c_1 a_2 & c_1 b_2 & d_1 a_2 & d_1 b_2 \\ c_1 c_2 & c_1 d_2 & d_1 c_2 & d_1 d_2 \end{pmatrix}. \end{aligned}$$

Here is a sanity check I want you to make a habit of: the bottom-right entry of $\hat{A}_1 \otimes \hat{A}_2$ must be the product of the bottom-right entries, $d_1 d_2$. In my experience,

sign and index slips in tensor products are the most common source of error in this subject.

1.4 Postulates of Quantum Mechanics

The linear algebra above is pure mathematics. The postulates are the bridge to physics. They tell us which mathematical objects represent states, how states change, and what happens when we look. There are four, and every later chapter is an application of them. I use the formulation that is standard in quantum information theory [24].

1.4.1 Postulate 1: State Space

Postulate 1. Associated with any isolated physical system is a Hilbert space known as the *state space* of the system. The system is completely described by its *state vector*, which is a *unit* vector in that space.

The simplest quantum mechanical system is the *qubit*, whose state space is two-dimensional. Writing $|0\rangle$ and $|1\rangle$ for an orthonormal basis of that space (the *computational basis*), an arbitrary state vector is

$$|\psi\rangle = a|0\rangle + b|1\rangle, \quad (1.1)$$

with $a, b \in \mathbb{C}$. Because $|\psi\rangle$ must be a unit vector, $\langle\psi|\psi\rangle = 1$, which is equivalent to

$$|a|^2 + |b|^2 = 1. \quad (1.2)$$

This is the *normalization condition*. The numbers a and b are called *amplitudes*. Postulate 3 will tell us that $|a|^2$ and $|b|^2$ are the probabilities of the two measurement outcomes. Equation (1.2) is therefore just the statement that the probabilities sum to one.

Contrast this with a classical bit, which is either 0 or 1. A qubit is described by a *pair of complex numbers*, and the set of allowed pairs is continuous. It is tempting, and wrong, to conclude that a qubit therefore stores an unlimited amount of information. Postulate 3 will show that a single measurement extracts only one bit. The extra structure is not directly readable. It does determine how states interfere, though, and interference is where quantum algorithms get their power.

Geometrical Representation: the Bloch Sphere

A qubit state can be drawn as a point on the surface of a unit sphere, called the *Bloch sphere* after the physicist Felix Bloch. Let us count parameters to see why a sphere is the right picture.

The state (1.1) contains two complex numbers, hence *four* real numbers. Two of these are redundant:

- (i) Normalization, $|a|^2 + |b|^2 = 1$, removes one.
- (ii) The overall (*global*) phase removes another. Only the *relative* phase between the two amplitudes has physical meaning.¹

That leaves *two* real parameters. A set described by two parameters is a surface, in this case the surface of a sphere in three dimensions. Let us carry out the reduction explicitly. We write the amplitudes in polar form:

$$\begin{aligned} |\psi\rangle &= |a|e^{i\phi_1} |0\rangle + |b|e^{i\phi_2} |1\rangle \\ &= e^{i\phi_1} \left(|a| |0\rangle + |b|e^{i(\phi_2-\phi_1)} |1\rangle \right). \end{aligned}$$

The normalization condition $|a|^2 + |b|^2 = 1$ lets us set $|a| = \cos(\theta/2)$ and $|b| = \sin(\theta/2)$ for some angle θ , since $\cos^2 + \sin^2 = 1$ automatically. Writing $\varphi = \phi_2 - \phi_1$ for the relative phase,

$$|\psi\rangle = e^{i\phi_1} \left[\cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\varphi} \sin\left(\frac{\theta}{2}\right) |1\rangle \right]. \quad (1.3)$$

The prefactor $e^{i\phi_1}$ is the global phase. Discarding it,

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\varphi} \sin\left(\frac{\theta}{2}\right) |1\rangle, \quad 0 \leq \theta \leq \pi, \quad 0 \leq \varphi < 2\pi. \quad (1.4)$$

The angles (θ, φ) are exactly the polar and azimuthal angles of a point on a sphere, shown in Fig. 1.1.

Why the half-angle?

The factor of $\frac{1}{2}$ in $\cos(\theta/2)$ is not a typo, and it is not a convention we are free to change. Orthogonal states must sit at *antipodal* points of the sphere. Setting $\theta = 0$ gives $|0\rangle$ at the north pole. We need $|1\rangle$, which is orthogonal to it, at the south pole, $\theta = \pi$. Substituting $\theta = \pi$ into (1.4) gives $\cos(\pi/2) = 0$ and $\sin(\pi/2) = 1$, i.e. $|\psi\rangle = |1\rangle$, as required. Without the half-angle the two poles would be separated by only $\pi/2$ in θ and the picture would not close up.

¹The phase of a quantum state is not directly measurable; see the box on global versus relative phase below.

Global versus relative phase

The states $|\psi\rangle$ and $e^{i\alpha}|\psi\rangle$ are physically *identical*: no measurement can tell them apart, because probabilities involve $|\cdot|^2$ and $|e^{i\alpha}| = 1$. For example, $i|0\rangle$ and $|0\rangle$ are the same physical state.

The states $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ are *not* identical. Here the minus sign is a *relative* phase between two terms of a superposition, and it is perfectly measurable. These two states are orthogonal, so a suitable measurement distinguishes them with certainty. Confusing these two situations is one of the most common errors I see newcomers make.

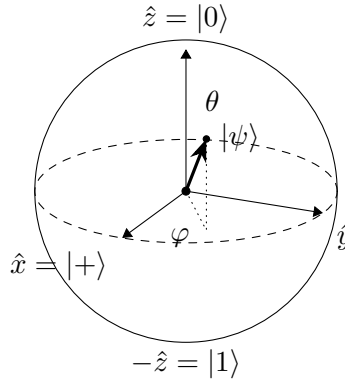


Figure 1.1: Bloch sphere representation of a qubit state. The polar angle θ is measured from the $+z$ axis and the azimuthal angle φ in the equatorial plane from the $+x$ axis.

I want you to memorize four points.

- i) $\theta = 0$ gives $|\psi\rangle = |0\rangle$ (north pole). Here φ is undefined, as it is at any pole.
- ii) $\theta = \pi$ gives $|\psi\rangle = |1\rangle$ (south pole).
- iii) $\theta = \pi/2$, $\varphi = 0$ gives $|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$.
- iv) $\theta = \pi/2$, $\varphi = \pi$ gives $|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$.

The pair $\{|+\rangle, |-\rangle\}$ is orthonormal ($\langle + | - \rangle = \frac{1}{2}(1 - 1) = 0$) and so is itself a perfectly good basis for a qubit, called the *Hadamard* or *X-basis*. Poles of the y -axis give a third common basis, $|\pm i\rangle = \frac{1}{\sqrt{2}}(|0\rangle \pm i|1\rangle)$.

Exercise 1.1

Find θ and φ , and hence the Bloch vector, for $|\psi\rangle = \frac{1}{2}|0\rangle + \frac{\sqrt{3}}{2}e^{i\pi/3}|1\rangle$.

Answer: $\theta = 2\pi/3$, $\varphi = \pi/3$, $\vec{r} = (\frac{\sqrt{3}}{4}, \frac{3}{4}, -\frac{1}{2})$.

Bloch vector

It is often convenient to name the point directly. The *Bloch vector* of a state $|\psi\rangle$ is the real unit vector

$$\vec{r} = (r_x, r_y, r_z), \quad r_k = \langle \psi | \sigma_k | \psi \rangle,$$

where $\sigma_x, \sigma_y, \sigma_z$ are the Pauli matrices X, Y, Z defined in the next subsection. In terms of the angles, $\vec{r} = (\sin \theta \cos \varphi, \sin \theta \sin \varphi, \cos \theta)$. Every single-qubit gate acts on $\vec{r}$ as a *rotation*, which is what makes the sphere so useful for building intuition.

1.4.2 Postulate 2: Evolution

How does a state change with time? For a system that is perfectly isolated, the answer is remarkably restrictive.

Postulate 2. The evolution of a *closed* quantum system is described by a *unitary transformation*. The state $|\psi\rangle$ at time t_1 is related to the state $|\psi'\rangle$ at time t_2 by a unitary operator U depending only on t_1 and t_2 :

$$|\psi'\rangle = U |\psi\rangle. \quad (1.5)$$

An operator U is *unitary* if its inverse equals its conjugate transpose, $U^{-1} = U^\dagger$, equivalently $U^\dagger U = U U^\dagger = I$.

Why unitary? Because Postulate 1 demands that states stay normalized, and unitary operators are exactly the ones that preserve the norm of every vector. The next example proves this.

Example 1.1

Show that the norm is preserved under a unitary transformation.

Solution.

$$\begin{aligned}
 |\psi'\rangle &= U |\psi\rangle, \\
 \langle\psi'| &= (U |\psi\rangle)^\dagger = \langle\psi| U^\dagger, \\
 \Rightarrow \langle\psi'|\psi'\rangle &= \langle\psi| U^\dagger U |\psi\rangle \\
 &= \langle\psi| I |\psi\rangle \quad (\text{since } U \text{ is unitary}) \\
 &= \langle\psi|\psi\rangle = 1.
 \end{aligned}$$

So $|\psi'\rangle$ is again a unit vector.

One consequence follows at once: **every quantum gate is reversible**. Since $U^{-1} = U^\dagger$ always exists, no quantum gate can destroy information the way a classical AND gate does. This is a genuine constraint on circuit design, and we will return to it when we build arithmetic circuits.

The Pauli Gates and the Hadamard Gate

In quantum computing, unitary operators acting on qubits are called *gates*. The most important single-qubit gates are the three Pauli matrices together with the Hadamard gate:

$$\begin{aligned}
 X &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, & Y &= \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \\
 Z &= \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, & H &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.
 \end{aligned}$$

- i) X takes $|0\rangle \mapsto |1\rangle$ and $|1\rangle \mapsto |0\rangle$. It is the quantum analog of the classical NOT gate and is often called the *bit-flip*.
- ii) Y takes $|0\rangle \mapsto i|1\rangle$ and $|1\rangle \mapsto -i|0\rangle$: it flips the qubit *and* attaches a phase.
- iii) Z leaves $|0\rangle$ unchanged and takes $|1\rangle \mapsto -|1\rangle$. It is called the *phase-flip*.
- iv) H maps the computational basis onto the superposition basis:

$$\begin{aligned}
 H|0\rangle &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) = |+\rangle, \\
 H|1\rangle &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) = |-\rangle.
 \end{aligned}$$

Creating superposition is the first step of nearly every quantum algorithm, so H is the gate you will use most.

Check two identities now and remember them: each of X, Y, Z, H is its own inverse ($X^2 = Y^2 = Z^2 = H^2 = I$), and each is both Hermitian and unitary.

Example 1.2

Verify that the Pauli matrices are unitary.

Solution. An operator A is unitary if $A^\dagger A = AA^\dagger = I$. Each Pauli is its own conjugate transpose, $X^\dagger = (X^T)^* = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = X$, $Y^\dagger = (Y^T)^* = \begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}^* = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = Y$ and $Z^\dagger = (Z^T)^* = Z$, so it is enough to square each one:

$$\begin{aligned} XX^\dagger &= X^\dagger X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I, \\ YY^\dagger &= Y^\dagger Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} -i^2 & 0 \\ 0 & -i^2 \end{pmatrix} = I, \\ ZZ^\dagger &= Z^\dagger Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I. \end{aligned}$$

Example 1.3

Verify that the Hadamard gate is unitary.

Solution. $H^\dagger = H$, so

$$HH^\dagger = H^\dagger H = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} = I.$$

In all four cases $A^\dagger = A$ as well, so X, Y, Z, H are Hermitian too. That is a special feature, not shared by, say, the S gate of Section 1.5.2.

Action of Gates on the Bloch Sphere

Because every single-qubit unitary is a rotation of the Bloch vector, we can picture what each gate does. The rules for the Paulis are:

$$\begin{aligned} X : (r_x, r_y, r_z) &\mapsto (r_x, -r_y, -r_z), \\ Y : (r_x, r_y, r_z) &\mapsto (-r_x, r_y, -r_z), \\ Z : (r_x, r_y, r_z) &\mapsto (-r_x, -r_y, r_z). \end{aligned} \tag{1.6}$$

Each is a rotation by π about the corresponding axis: the component *along* the rotation axis is unchanged and the other two flip sign.

Figures 1.2–1.5 show the action on a generic state with $\theta = 50^\circ$, $\varphi = 35^\circ$,

A rotation is not an inversion

It is tempting to draw $X|\psi\rangle$ as the point diametrically opposite $|\psi\rangle$. That is wrong. The antipodal map sends $(r_x, r_y, r_z) \mapsto (-r_x, -r_y, -r_z)$, which is *not* a rotation and does not correspond to any unitary. Compare with (1.6): X leaves r_x alone. The two agree only for states that happen to lie in the y - z plane. Sketch the general case carefully, as in Fig. 1.2, and this trap disappears.

chosen so that it lies in none of the coordinate planes. Its Bloch vector is $\vec{r} \approx (0.63, 0.44, 0.64)$, and I suggest you check each figure against (1.6).

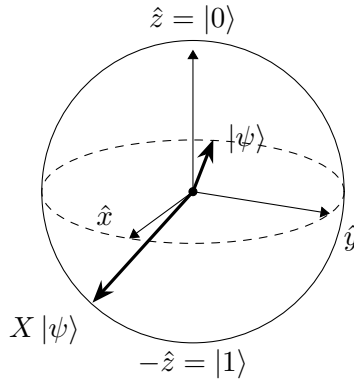


Figure 1.2: Action of X : a rotation by π about the x -axis. The x -component of the Bloch vector is unchanged; r_y and r_z change sign. Notice that $X|\psi\rangle$ is not diametrically opposite $|\psi\rangle$.

For Z the algebra is easy enough to do explicitly:

$$\begin{aligned} Z|\psi\rangle &= \cos\left(\frac{\theta}{2}\right) Z|0\rangle + e^{i\varphi} \sin\left(\frac{\theta}{2}\right) Z|1\rangle \\ &= \cos\left(\frac{\theta}{2}\right) |0\rangle - e^{i\varphi} \sin\left(\frac{\theta}{2}\right) |1\rangle \\ &= \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i(\varphi+\pi)} \sin\left(\frac{\theta}{2}\right) |1\rangle, \end{aligned}$$

so θ is unchanged and $\varphi \mapsto \varphi + \pi$: the point swings half way round its line of latitude.

The Hadamard gate acts as

$$\begin{aligned} H|\psi\rangle &= \cos\left(\frac{\theta}{2}\right) H|0\rangle + e^{i\varphi} \sin\left(\frac{\theta}{2}\right) H|1\rangle \\ &= \frac{1}{\sqrt{2}} \left[\cos\left(\frac{\theta}{2}\right) + e^{i\varphi} \sin\left(\frac{\theta}{2}\right) \right] |0\rangle \end{aligned}$$

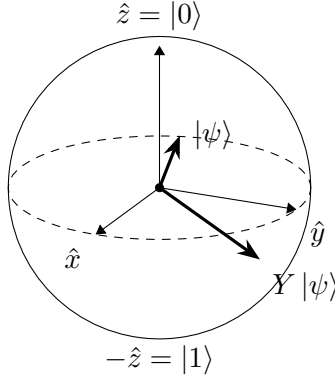


Figure 1.3: Action of Y : a rotation by π about the y -axis.

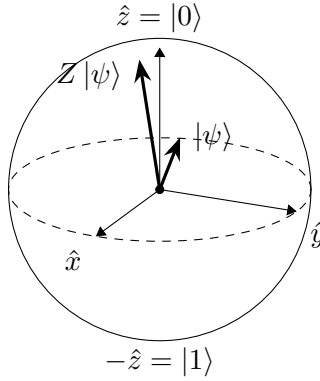


Figure 1.4: Action of Z : a rotation by π about the z -axis. The height r_z is unchanged.

$$+ \frac{1}{\sqrt{2}} \left[\cos\left(\frac{\theta}{2}\right) - e^{i\varphi} \sin\left(\frac{\theta}{2}\right) \right] |1\rangle,$$

which on the Bloch sphere is the map $(r_x, r_y, r_z) \mapsto (r_z, -r_y, r_x)$, a rotation by π about the diagonal axis $(\hat{x} + \hat{z})/\sqrt{2}$. It therefore exchanges the x - and z -axes, which is exactly the statement $H|0\rangle = |+\rangle$ and $H|+\rangle = |0\rangle$. Equivalently, it can be built from two rotations: first $\pi/2$ about y , then π about x (see Section 1.5.4 for the notation),

$$H = i R_x(\pi) R_y(\pi/2),$$

where the leading i is an unobservable global phase. Watch the order: in a product of operators the *rightmost* acts first.

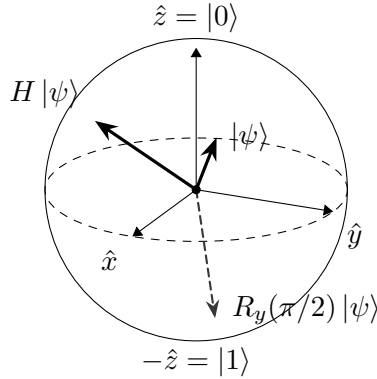


Figure 1.5: Action of H , built as a rotation of $\pi/2$ about y (dashed) followed by a rotation of π about x .

1.4.3 Postulate 3: Measurement

Postulate 2 describes a *closed* system. But to learn anything we must interact with the system, and that interaction is not unitary. Measurement gets its own postulate.

Postulate 3. Quantum measurements are described by a collection $\{M_m\}$ of *measurement operators* acting on the state space of the system being measured. The index m labels the possible outcomes. If the state immediately before measurement is $|\psi\rangle$, then the probability of outcome m is

$$p(m) = \langle \psi | M_m^\dagger M_m | \psi \rangle, \quad (1.7)$$

and the state immediately after the measurement is

$$\frac{M_m |\psi\rangle}{\sqrt{\langle \psi | M_m^\dagger M_m | \psi \rangle}}.$$

The operators must satisfy the *completeness relation*

$$\sum_m M_m^\dagger M_m = I.$$

The completeness relation is exactly the statement that the probabilities sum to one:

$$\sum_m p(m) = \sum_m \langle \psi | M_m^\dagger M_m | \psi \rangle = \langle \psi | \left(\sum_m M_m^\dagger M_m \right) | \psi \rangle = \langle \psi | \psi \rangle = 1.$$

Measurement in the computational basis. The case we use constantly is

m is a label, not an eigenvalue

In Postulate 3 the index m is simply a name for an outcome, such as “the detector on the left clicked”. It carries no numerical meaning and is *not* an eigenvalue of M_m . Eigenvalues enter with an *observable*, a Hermitian operator whose eigenvalues are the numerical values the quantity can take. That is the special case of Postulate 3 in which M_m projects onto the eigenspace of the eigenvalue m , and only there do the label and the eigenvalue coincide.

measurement in the basis $\{|0\rangle, |1\rangle\}$, for which $M_0 = |0\rangle\langle 0|$ and $M_1 = |1\rangle\langle 1|$. These are projectors: $M_0^\dagger M_0 = (|0\rangle\langle 0|)(|0\rangle\langle 0|) = |0\rangle\langle 0| = M_0$, and similarly for M_1 . Take $|\psi\rangle = a|0\rangle + b|1\rangle$. Then

$$p(0) = \langle \psi | M_0^\dagger M_0 | \psi \rangle = \langle \psi | |0\rangle\langle 0| | \psi \rangle = \langle \psi | 0 \rangle \langle 0 | \psi \rangle = |a|^2,$$

and likewise $p(1) = |b|^2$. This is the *Born rule*: the probability of an outcome is the squared modulus of the corresponding amplitude. The post-measurement states are $|0\rangle$ and $|1\rangle$ respectively, and the superposition is gone.

I want to state three consequences plainly, because they constrain everything we can do with a quantum computer.

- (i) **Measurement is irreversible.** Unlike unitary evolution, it cannot be undone. Once a qubit has been measured, the amplitudes a and b are lost.
- (ii) **One measurement yields one bit.** From a single qubit you learn only “0” or “1”. To *estimate* $|a|^2$ you need many repetitions of the whole circuit. That is why quantum programs are run for thousands of “shots”. The count is easy to estimate. Let $p = |a|^2$ be the probability you want and let N be the number of shots, that is, the number of times the circuit is prepared, run and measured. The fraction of shots that return 0 has a statistical spread of about $\sqrt{p(1-p)/N}$, which is at most $1/(2\sqrt{N})$. So to pin p down to within $\pm\epsilon$ you need roughly $N \approx 1/(4\epsilon^2)$ shots. In plain words: one more decimal digit of p costs a hundred times more runs of the circuit.
- (iii) **Global phase is invisible.** Replacing $|\psi\rangle$ by $e^{i\alpha}|\psi\rangle$ leaves (1.7) unchanged, confirming what we assumed when drawing the Bloch sphere.

Example 1.4

A qubit is prepared in $|+\rangle$ and measured in the computational basis. What are the outcome probabilities, and what if we instead measure in the $\{|+\rangle, |-\rangle\}$ basis?

Solution. In the computational basis, $a = b = 1/\sqrt{2}$, so $p(0) = p(1) = \frac{1}{2}$: the result is a fair coin toss.

In the $\{|+\rangle, |-\rangle\}$ basis take $M_+ = |+\rangle\langle +|$ and $M_- = |-\rangle\langle -|$. Then $p(+) = |\langle +|+\rangle|^2 = 1$ and $p(-) = |\langle -|+\rangle|^2 = 0$: the outcome is certain.

The lesson I want you to take away is that “random” is not a property of the state alone. The same state is maximally uncertain in one basis and perfectly definite in another. Choosing the right measurement basis is a design decision in every algorithm.

1.4.4 Postulate 4: Composite Systems

Postulate 4. The state space of a composite physical system is the *tensor product* of the state spaces of the components. If the components are numbered 1 through n and component i is prepared in state $|\psi_i\rangle$, then the joint state is $|\psi_1\rangle \otimes |\psi_2\rangle \otimes \cdots \otimes |\psi_n\rangle$.

So for Hilbert spaces $H_1, H_2, \dots, H_n$ of a composite system combining into $H = H_1 \otimes H_2 \otimes \cdots \otimes H_n$, with $|\psi_i\rangle \in H_i$, we build

$$|\psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle \otimes \cdots \otimes |\psi_n\rangle \in H.$$

The tensor product is linear in each argument:

$$\begin{aligned} |v\rangle \otimes [|w_1\rangle + |w_2\rangle] &= |v\rangle \otimes |w_1\rangle + |v\rangle \otimes |w_2\rangle, \\ [|w_1\rangle + |w_2\rangle] \otimes |v\rangle &= |w_1\rangle \otimes |v\rangle + |w_2\rangle \otimes |v\rangle, \end{aligned}$$

and for any scalar $\alpha \in \mathbb{C}$,

$$|v\rangle \otimes (\alpha |w\rangle) = (\alpha |v\rangle) \otimes |w\rangle = \alpha (|v\rangle \otimes |w\rangle).$$

Scalars may be moved freely across the $\otimes$ sign; vectors may not.

Basis of the composite space. If $\{|u_i\rangle\}_{i=1}^{N_1}$ is a basis of H_1 and $\{|v_j\rangle\}_{j=1}^{N_2}$ a basis of H_2 , then a basis of $H_1 \otimes H_2$ is the set of *all* pairwise products

$$|w_{ij}\rangle = |u_i\rangle \otimes |v_j\rangle, \quad i = 1, \dots, N_1, \quad j = 1, \dots, N_2. \quad (1.8)$$

There are $N_1 N_2$ of them, matching $\dim(H_1 \otimes H_2) = N_1 N_2$. Taking only the “diagonal” combinations $|u_i\rangle \otimes |v_i\rangle$ would give just N_1 vectors, far too few to span the space.

For brevity, $|\phi\rangle \otimes |\chi\rangle$ is usually written $|\phi\rangle |\chi\rangle$ or simply $|\phi\chi\rangle$.

Example 1.5

The basis of a single qubit is $\{|0\rangle, |1\rangle\}$. What is the basis of a two-qubit system?

Solution. Take all $2 \times 2 = 4$ pairwise tensor products:

$$\begin{aligned} |w_{11}\rangle &= |0\rangle \otimes |0\rangle = |00\rangle, & |w_{12}\rangle &= |0\rangle \otimes |1\rangle = |01\rangle, \\ |w_{21}\rangle &= |1\rangle \otimes |0\rangle = |10\rangle, & |w_{22}\rangle &= |1\rangle \otimes |1\rangle = |11\rangle. \end{aligned}$$

In general n qubits have 2^n computational basis states, labeled by the 2^n bit strings of length n .

Inner Products in the Composite Space

Inner products factorize. If $|w_1\rangle = |u_1\rangle \otimes |v_1\rangle$ and $|w_2\rangle = |u_2\rangle \otimes |v_2\rangle$, then

$$\langle w_1 | w_2 \rangle = (\langle u_1 | \otimes \langle v_1 |) (|u_2\rangle \otimes |v_2\rangle) = \langle u_1 | u_2 \rangle \langle v_1 | v_2 \rangle.$$

This makes orthogonality checks in large spaces almost free: if either factor is orthogonal, the whole inner product vanishes.

Operators and Tensor Products

If $\hat{A}$ acts on H_1 and $\hat{B}$ on H_2 , the operator $\hat{A} \otimes \hat{B}$ acts on $H = H_1 \otimes H_2$ factor by factor:

$$(\hat{A} \otimes \hat{B}) (|u\rangle \otimes |v\rangle) = \hat{A} |u\rangle \otimes \hat{B} |v\rangle.$$

In matrix form we use the block rule of Section 1.3.6. For example, with X acting on the first qubit and Y on the second,

$$X \otimes Y = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \otimes \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & -i \\ 0 & 0 & i & 0 \\ 0 & -i & 0 & 0 \\ i & 0 & 0 & 0 \end{pmatrix}.$$

A gate acting on only one qubit of a pair is written with an identity in the other slot: applying H to the first qubit alone is $H \otimes I$. Which slot of the tensor product belongs to which wire is a matter of convention. The one I have chosen for this book is stated next and drawn in Fig. 1.6.

Qubit ordering: a warning worth heeding early

A two-qubit state is written $|q_0q_1\rangle$ throughout this book. The *leftmost* symbol is qubit 0, so $A \otimes B$ means “ A on qubit 0, B on qubit 1”, and the basis is ordered $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. Qubit 0 is also the top wire of every circuit, and it carries the most significant bit (big-endian).

Some software, Qiskit in particular, uses the opposite (little-endian) convention, in which the *rightmost* symbol is qubit 0. The same circuit then produces a matrix whose rows and columns are permuted relative to ours. Neither convention is wrong, but in my experience mixing them silently is the single most common bug in beginners’ quantum programs. Whenever you compare a hand calculation with code output, check the ordering first.

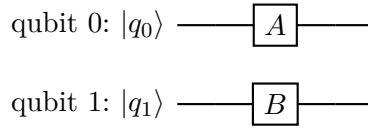


Figure 1.6: The qubit-ordering convention used throughout this book. The top wire is qubit 0 and supplies the *left* factor of a tensor product, so the circuit shown is $A \otimes B$ acting on $|q_0q_1\rangle$. Reading the wires the other way round would give $B \otimes A$, a different matrix whenever $A \neq B$.

This ordering is used without further comment for the rest of the book. In particular, the control wire of every controlled gate drawn in this chapter is the top wire, qubit 0, and it supplies the left factor of the tensor product.

Tensor Products of Two-Qubit Basis States

Applying the column-stacking rule to the four computational basis states:

$$\begin{aligned}
 |00\rangle &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, & |01\rangle &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \\
 |10\rangle &= \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, & |11\rangle &= \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.
 \end{aligned}$$

Each has a single 1, in the position given by reading the label as a binary number with qubit 0 as the most significant bit: $|00\rangle \rightarrow$ row 0, $|01\rangle \rightarrow$ row 1, $|10\rangle \rightarrow$ row 2, $|11\rangle \rightarrow$ row 3. That indexing rule extends to any number of qubits. Commit it to memory.

Example 1.6

Write the state $|\phi\rangle = \frac{1}{\sqrt{3}} \begin{pmatrix} i \\ 0 \\ i \\ 1 \end{pmatrix}$ in the computational basis of a two-qubit system.

Solution. Read off the coefficients row by row:

$$\begin{aligned} \frac{1}{\sqrt{3}} \begin{pmatrix} i \\ 0 \\ i \\ 1 \end{pmatrix} &= \frac{1}{\sqrt{3}} [i|00\rangle + 0|01\rangle + i|10\rangle + 1|11\rangle], \\ \Rightarrow |\phi\rangle &= \frac{1}{\sqrt{3}} [i|00\rangle + i|10\rangle + |11\rangle]. \end{aligned} \quad (1.9)$$

As a check, the norm is $\frac{1}{3} (|i|^2 + 0 + |i|^2 + 1) = \frac{1}{3}(1 + 1 + 1) = 1$. Verifying normalization costs one line and catches most arithmetic slips.

1.5 Single-Qubit Gates in Outer-Product Form

We have written gates as matrices. There is a second representation, the outer-product or Dirac form. It is often faster to manipulate and it generalizes cleanly to many qubits. Learning to move between the two is a core skill, and I will use both forms freely from here on.

An operator $\hat{A}$ acts on kets and bras according to

$$\hat{A}|\psi\rangle = |\psi'\rangle, \quad \langle\psi|\hat{A}^\dagger = \langle\psi'|.$$

On a d -dimensional space an operator is a $d \times d$ matrix, so single-qubit gates are 2×2 .

Unitary Operator

An operator $\hat{U}$ is unitary if its inverse equals its Hermitian conjugate, $\hat{U}^{-1} = \hat{U}^\dagger$, so that

$$\hat{U}\hat{U}^\dagger = \hat{U}^\dagger\hat{U} = \hat{I}.$$

Take an arbitrary gate G :

$$G = \begin{pmatrix} g_{11} & g_{12} \\ g_{21} & g_{22} \end{pmatrix}. \quad (1.10)$$

Decomposing it entry by entry,

$$G = g_{11} \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + g_{12} \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} + g_{21} \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} + g_{22} \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

So the four matrices $\{ \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \}$ form a basis for the space of 2×2 operators. And as we saw in Section 1.3.5, these are exactly the outer products of the basis vectors: $|0\rangle\langle 0|$, $|0\rangle\langle 1|$, $|1\rangle\langle 0|$, $|1\rangle\langle 1|$. Hence any single-qubit gate can be written

$$G = g_{11} |0\rangle\langle 0| + g_{12} |0\rangle\langle 1| + g_{21} |1\rangle\langle 0| + g_{22} |1\rangle\langle 1|. \quad (1.11)$$

The index pattern is easy to remember. The coefficient g_{rc} sits in front of $|r-1\rangle\langle c-1|$, so the ket labels the *row* and the bra labels the *column*.

Exercise 1.2

Express X , Y and Z in Dirac notation and hence find $X|0\rangle$, $X|1\rangle$, $Y|0\rangle$, $Y|1\rangle$, $Z|0\rangle$, $Z|1\rangle$.

Hint. Use (1.10) and (1.11) together with $(|a\rangle\langle b|)|b\rangle = |a\rangle$ and $(|a\rangle\langle b|)|a\rangle = |N\rangle$ for orthonormal $|a\rangle, |b\rangle$.

Answer: $X = |0\rangle\langle 1| + |1\rangle\langle 0|$, $Y = -i|0\rangle\langle 1| + i|1\rangle\langle 0|$, $Z = |0\rangle\langle 0| - |1\rangle\langle 1|$.

1.5.1 Hadamard Gate

From the matrix $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ we read off the Dirac form directly using (1.11):

$$H = \frac{1}{\sqrt{2}} (|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| - |1\rangle\langle 1|).$$

Its action on the basis states follows by letting each bra meet the incoming ket:

$$\begin{aligned} H|0\rangle &= \frac{1}{\sqrt{2}} (|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| - |1\rangle\langle 1|)|0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) = |+\rangle, \\ H|1\rangle &= \frac{1}{\sqrt{2}} (|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| - |1\rangle\langle 1|)|1\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) = |-\rangle. \end{aligned}$$

Only the terms whose bra is $\langle 0|$ survive in the first line, and only those whose bra is $\langle 1|$ in the second. The rest are killed by $\langle 0|1\rangle = 0$. This selection is what makes Dirac-form algebra quick.

Exercise 1.3

Find the action of H on $|+\rangle$ and $|-\rangle$, and explain the result geometrically using Fig. 1.5.

Answer: $H|+\rangle = |0\rangle$, $H|-\rangle = |1\rangle$. Since $H^2 = I$, the Hadamard gate is its own inverse and simply exchanges the x - and z -axes of the Bloch sphere.

Example 1.7

Using the outer-product form, show that $Z = HXH$.

Solution. Write $H = \frac{1}{\sqrt{2}}(|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| - |1\rangle\langle 1|)$ and $X = |0\rangle\langle 1| + |1\rangle\langle 0|$, and use $\langle i|j\rangle = \delta_{ij}$ at every step. First HX :

$$\begin{aligned} HX &= \frac{1}{\sqrt{2}}(|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| - |1\rangle\langle 1|)(|0\rangle\langle 1| + |1\rangle\langle 0|) \\ &= \frac{1}{\sqrt{2}}(|0\rangle\langle 1| + |0\rangle\langle 0| + |1\rangle\langle 1| - |1\rangle\langle 0|). \end{aligned}$$

Now multiply that on the right by H . Each product $|i\rangle\langle j||k\rangle\langle l| = \delta_{jk}|i\rangle\langle l|$ survives only when $j = k$, so collecting the eight surviving terms:

$$\begin{aligned} HXH &= \frac{1}{2} \left((|0\rangle\langle 0| - |0\rangle\langle 1|) + (|0\rangle\langle 0| + |0\rangle\langle 1|) \right. \\ &\quad \left. + (|1\rangle\langle 0| - |1\rangle\langle 1|) - (|1\rangle\langle 0| + |1\rangle\langle 1|) \right) \\ &= \frac{1}{2} (2|0\rangle\langle 0| - 2|1\rangle\langle 1|) = |0\rangle\langle 0| - |1\rangle\langle 1| = Z, \end{aligned}$$

the four bracketed pairs coming, in order, from the $|0\rangle\langle 1|$, $|0\rangle\langle 0|$, $|1\rangle\langle 1|$ and $-|1\rangle\langle 0|$ terms of HX , and the $|0\rangle\langle 1|$ and $|1\rangle\langle 0|$ terms canceling in pairs. $X = HZH$ follows in the same way, or immediately from $H^2 = I$: multiply $Z = HXH$ on both sides by H .

1.5.2 Phase Gate

The phase gate S is

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/2} \end{pmatrix}, \quad \text{i.e.} \quad S = |0\rangle\langle 0| + i|1\rangle\langle 1|.$$

Its action is

$$\begin{aligned} S|0\rangle &= (|0\rangle\langle 0| + i|1\rangle\langle 1|)|0\rangle = |0\rangle, \\ S|1\rangle &= (|0\rangle\langle 0| + i|1\rangle\langle 1|)|1\rangle = i|1\rangle. \end{aligned}$$

It leaves $|0\rangle$ alone and multiplies $|1\rangle$ by i . On the Bloch sphere this is a rotation of $\pi/2$ about the z -axis, so $S^2 = Z$.

Observe that $S^\dagger \neq S$. Unlike the Paulis and H , the phase gate is *not* Hermitian, and $S^\dagger$ (sometimes written S^{-1}) is a distinct gate. Both belong to the general phase-gate family P , which leaves $|0\rangle$ alone and multiplies $|1\rangle$ by a phase of your choosing, so that $S = P(\pi/2)$ and $S^\dagger = P(-\pi/2)$. That family is tabulated in Chapter 2. The related T gate, with $e^{i\pi/4}$ in place of i , is $P(\pi/4)$. It satisfies $T^2 = S$ and will matter a great deal when we discuss universal gate sets.

1.5.3 Exponentiation of Operators

Most gates used in practice are not fixed matrices but families parameterized by a continuous angle. These are built by exponentiating a Pauli matrix. The key identity is the following.

Let U be both Hermitian and unitary (all three Paulis and H qualify). Then for any real η ,

$$e^{-i\eta U} = \cos \eta I - i \sin \eta U. \quad (1.12)$$

This is the operator version of Euler's formula $e^{-i\eta} = \cos \eta - i \sin \eta$, and the proof follows the same route. Expand the series and sort into even and odd powers:

$$\begin{aligned} e^{-i\eta U} &= I + (-i\eta U) + \frac{(-i\eta U)^2}{2!} + \frac{(-i\eta U)^3}{3!} + \frac{(-i\eta U)^4}{4!} + \dots \\ &= I - i\eta U + (-i)^2 \frac{\eta^2}{2!} U^2 + (-i)^3 \frac{\eta^3}{3!} U^3 + (-i)^4 \frac{\eta^4}{4!} U^4 + \dots \end{aligned}$$

Because U is Hermitian ($U^\dagger = U$) and unitary ($U^\dagger U = I$), we have $U^2 = U^\dagger U = I$. Hence every even power is I and every odd power is U , giving

$$\begin{aligned} e^{-i\eta U} &= \left(1 - \frac{\eta^2}{2!} + \frac{\eta^4}{4!} - \dots\right) I - i \left(\eta - \frac{\eta^3}{3!} + \frac{\eta^5}{5!} - \dots\right) U \\ &= \cos \eta I - i \sin \eta U. \end{aligned}$$

Mind the sign

Equation (1.12) has $e^{-i\eta U}$ on the left, matching the minus sign in $-i \sin \eta U$ on the right. The opposite sign gives

$$e^{+i\eta U} = \cos \eta I + i \sin \eta U.$$

Sign slips here propagate into every rotation angle downstream, so check the case $\eta = \pi/2$, $U = Z$ by hand once: it should give $-iZ$.

1.5.4 Rotation Gates

Exponentiating the Pauli matrices produces the three families of *rotation gates*, which rotate the Bloch vector about the x -, y - and z -axes by an angle η :

$$R_x(\eta) = e^{-i\eta X/2} = \cos\left(\frac{\eta}{2}\right) I - i \sin\left(\frac{\eta}{2}\right) X = \begin{pmatrix} \cos\left(\frac{\eta}{2}\right) & -i \sin\left(\frac{\eta}{2}\right) \\ -i \sin\left(\frac{\eta}{2}\right) & \cos\left(\frac{\eta}{2}\right) \end{pmatrix},$$

$$R_y(\eta) = e^{-i\eta Y/2} = \cos\left(\frac{\eta}{2}\right) I - i \sin\left(\frac{\eta}{2}\right) Y = \begin{pmatrix} \cos\left(\frac{\eta}{2}\right) & -\sin\left(\frac{\eta}{2}\right) \\ \sin\left(\frac{\eta}{2}\right) & \cos\left(\frac{\eta}{2}\right) \end{pmatrix},$$

$$R_z(\eta) = e^{-i\eta Z/2} = \cos\left(\frac{\eta}{2}\right) I - i \sin\left(\frac{\eta}{2}\right) Z = \begin{pmatrix} e^{-i\eta/2} & 0 \\ 0 & e^{i\eta/2} \end{pmatrix}.$$

The $\eta/2$ in the exponent is deliberate: it makes $R_k(\eta)$, with $k \in \{x, y, z\}$, rotate the *Bloch vector* by η , which is the angle you actually want to specify. These continuous families are the gates whose angles a compiler tunes when it builds an arbitrary single-qubit operation, and they will reappear constantly.

A note on the angle symbol. I call the rotation angle η here because θ is already in use in this chapter as the Bloch polar angle of (1.4). Later chapters and Appendix A often write the same argument as θ . Only the letter changes. The half-angle convention $R_z(\eta) = e^{-i\eta Z/2}$ is used everywhere in this book, so $e^{-i\eta Z}$ is $R_z(2\eta)$ and not $R_z(\eta)$.

Useful identities, all verifiable in one line from the matrices above:

$$R_k(\eta_1)R_k(\eta_2) = R_k(\eta_1 + \eta_2), \quad R_k(\eta)^{-1} = R_k(-\eta), \quad R_k(2\pi) = -I.$$

Pause on the last one: rotating a qubit through a full turn returns $-I$, not I . The minus sign is a global phase and therefore unobservable. It becomes observable if the rotation is applied to only one qubit of an entangled pair, where it is a *relative* phase.

Pauli gates versus rotation gates

It is often said that X “is” a rotation by π about the x -axis. That is true of the Bloch vector, but the matrices differ by a phase:

$$R_x(\pi) = \begin{pmatrix} 0 & -i \\ -i & 0 \end{pmatrix} = -iX, \quad \text{so} \quad X = i R_x(\pi).$$

Likewise $Y = iR_y(\pi)$ and $Z = iR_z(\pi)$. The distinction is invisible for a single isolated qubit. As with $R_k(2\pi) = -I$, it matters when the gate is one factor of a controlled operation. Write $X = iR_x(\pi)$, not $X = R_x(\pi)$.

Exercise 1.4

Express $R_x(\eta)$, $R_y(\eta)$ and $R_z(\eta)$ in Dirac notation.

Answer (for R_z): $R_z(\eta) = e^{-i\eta/2} |0\rangle\langle 0| + e^{i\eta/2} |1\rangle\langle 1|$.

1.5.5 The Z – Y Decomposition of a Single-Qubit Gate

We now know a handful of specific gates. How many are there in total, and can they all be built from rotations? The answer is yes, and it is the reason hardware only needs to implement a couple of gate families. The result below is standard [24]. Read as a resource statement, it says that *any* single-qubit gate costs at most three rotation gates, one R_y and two R_z , each with an arbitrary real angle. The global phase costs nothing, because no circuit element is needed to produce it.

Theorem (Z – Y decomposition)

For any single-qubit unitary U there exist real numbers $\alpha, \beta, \gamma, \delta$ such that

$$U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta). \quad (1.13)$$

As a circuit the decomposition is a single wire carrying three rotations, drawn in Fig. 1.7. Watch the reversal: the factor written *rightmost* in (1.13) is the gate placed *leftmost* on the wire, because a circuit is read in time order while a matrix product is read right to left.

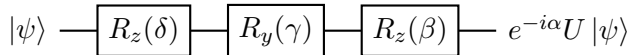


Figure 1.7: The Z – Y decomposition read as a circuit. Time runs left to right, so $R_z(\delta)$ acts first even though it is the rightmost factor in (1.13). The global phase $e^{i\alpha}$ is not drawn, since no single-qubit circuit element produces an observable overall phase.

Why four parameters? Let us count. A 2×2 complex matrix has four complex entries, hence eight real parameters. Unitarity, $UU^\dagger = I$, imposes four independent real conditions:

$$\begin{aligned} UU^\dagger &= \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} a^* & c^* \\ b^* & d^* \end{pmatrix} = I \\ \Rightarrow \begin{pmatrix} |a|^2 + |b|^2 & ac^* + bd^* \\ ca^* + db^* & |c|^2 + |d|^2 \end{pmatrix} &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \end{aligned}$$

that is,

$$|a|^2 + |b|^2 = 1, \quad |c|^2 + |d|^2 = 1, \quad ac^* + bd^* = 0$$

(the fourth equation is the complex conjugate of the third, so the off-diagonal condition counts as two real constraints). Eight parameters minus four constraints leaves *four* free real parameters, exactly the count in (1.13): three rotation angles β, γ, δ and one global phase α . In plain words, a single-qubit gate is specified by three numbers you can observe and one you cannot.

Verifying the form. Multiplying out the right-hand side of (1.13) with the matrices from Section 1.5.4 gives

$$e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta) = e^{i\alpha} \begin{pmatrix} e^{-i\beta/2} e^{-i\delta/2} \cos \frac{\gamma}{2} & -e^{-i\beta/2} e^{i\delta/2} \sin \frac{\gamma}{2} \\ e^{i\beta/2} e^{-i\delta/2} \sin \frac{\gamma}{2} & e^{i\beta/2} e^{i\delta/2} \cos \frac{\gamma}{2} \end{pmatrix}. \quad (1.14)$$

Comparing entries with a general unitary U shows the parameters can always be matched: γ is fixed by the magnitudes ($|a| = |\cos(\gamma/2)|$), and α, β, δ by the three independent phases.

Exercise 1.5

Carry out the matrix multiplication that produces (1.14), and check the special case $\alpha = \beta = \delta = 0$, $\gamma = \pi/2$.

Corollary (the ABC form). A general unitary can also be written

$$U = e^{i\alpha} A X B X C,$$

where A, B, C are unitary and satisfy $ABC = I$. A valid choice is

$$\begin{aligned} A &\equiv R_z(\beta) R_y(\gamma/2), \\ B &\equiv R_y(-\gamma/2) R_z\left(-\frac{\delta + \beta}{2}\right), \\ C &\equiv R_z\left(\frac{\delta - \beta}{2}\right). \end{aligned}$$

Checking $ABC = I$ is immediate: the two R_y factors cancel, and the R_z angles sum to $\beta - \frac{\delta + \beta}{2} + \frac{\delta - \beta}{2} = 0$.

This form looks like a curiosity, but it is the standard route to building *controlled* versions of arbitrary gates, as we will see in Chapter 2.

Example 1.8

Prove that $AXBXC = R_z(\beta)R_y(\gamma)R_z(\delta)$.

Solution. We use the conjugation identities

$$XR_y(\eta)X = R_y(-\eta), \quad XR_z(\eta)X = R_z(-\eta),$$

which follow from $XYX = -Y$, $XZX = -Z$ and (1.12). Insert $XX = I$ between the two factors of B and conjugate each factor in turn:

$$XBX = XR_y(-\gamma/2)R_z\left(-\frac{\delta+\beta}{2}\right)X = R_y(\gamma/2)R_z\left(\frac{\delta+\beta}{2}\right).$$

Therefore

$$\begin{aligned} AXBXC &= R_z(\beta)R_y(\gamma/2) \cdot R_y(\gamma/2)R_z\left(\frac{\delta+\beta}{2}\right) \cdot R_z\left(\frac{\delta-\beta}{2}\right) \\ &= R_z(\beta)R_y(\gamma)R_z\left(\frac{\delta+\beta}{2} + \frac{\delta-\beta}{2}\right) = R_z(\beta)R_y(\gamma)R_z(\delta). \end{aligned}$$

1.6 Two-Qubit Gates in Outer-Product Form

A two-qubit state lives in a four-dimensional space, so a two-qubit gate is a 4×4 matrix:

$$G = \begin{pmatrix} g_{11} & g_{12} & g_{13} & g_{14} \\ g_{21} & g_{22} & g_{23} & g_{24} \\ g_{31} & g_{32} & g_{33} & g_{34} \\ g_{41} & g_{42} & g_{43} & g_{44} \end{pmatrix}.$$

Exactly as in the single-qubit case, we decompose it entry by entry,

$$G = g_{11} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} + \cdots + g_{44} \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad (1.15)$$

and each of the sixteen matrices in (1.15) is an outer product of two-qubit basis states, for instance

$$|00\rangle\langle 00| = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

So in Dirac notation

$$G = g_{11} |00\rangle\langle 00| + g_{12} |00\rangle\langle 01| + \cdots + g_{44} |11\rangle\langle 11|,$$

with the same rule as before: ket labels the row, bra labels the column, reading 00, 01, 10, 11 as rows/columns 1, 2, 3, 4.

Exercise 1.6

Write $X \otimes X$ in Dirac notation.

Hint. Compute the 4×4 tensor product first, then read off the nonzero entries.

Answer: $X \otimes X = |00\rangle\langle 11| + |01\rangle\langle 10| + |10\rangle\langle 01| + |11\rangle\langle 00|.$

1.7 Entangled States

Entanglement is the feature of quantum mechanics with no classical counterpart, and it is the resource that most quantum algorithms consume. Suppose two systems A and B are entangled. Then measurement outcomes on A are correlated with outcomes on B in a way that no assignment of independent properties to each system separately can reproduce. Equivalently, an entangled pair has no description in which each half has a state of its own.

1.7.1 When is a State Entangled?

Not every two-qubit state is entangled. If a joint state *can* be written as a tensor product of individual states, it is called *separable* (or a *product state*) and each subsystem has a well-defined state of its own. If it cannot, it is *entangled*.

For example, $|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |01\rangle)$ is separable, because

$$|\psi\rangle = \frac{|0\rangle \otimes (|0\rangle + |1\rangle)}{\sqrt{2}} = |0\rangle \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |0\rangle \otimes |+\rangle,$$

so the first qubit is definitely in $|0\rangle$ and the second definitely in $|+\rangle$. By contrast,

$$|\phi\rangle = \frac{|0\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle}{\sqrt{2}}$$

cannot be factored (try it), so $|\phi\rangle$ is entangled.

Quick test for a two-qubit pure state. Write the state in the computational basis as

$$|\psi\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle = \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} \in \mathbb{C}^4.$$

Then $|\psi\rangle$ is **entangled if and only if** $ad \neq bc$, and separable if $ad = bc$.

Where this comes from: if $|\psi\rangle = (a_1|0\rangle + b_1|1\rangle) \otimes (a_2|0\rangle + b_2|1\rangle)$ then $(a, b, c, d) = (a_1a_2, a_1b_2, b_1a_2, b_1b_2)$, so $ad - bc = a_1b_1a_2b_2 - a_1b_1a_2b_2 = 0$. The quantity $ad - bc$ is the determinant of $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$; a product state gives a rank-one matrix, whose determinant vanishes.

This test is specific to *two* qubits in a *pure* state. For more qubits, or for mixed states, entanglement detection is much harder. The research problems at the end of this chapter survey what replaces it.

Example 1.9

Express $|++\rangle$ in the computational basis and determine whether it is entangled.

Solution. Expanding the product,

$$\begin{aligned} |++\rangle &= \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2} \\ &= \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}. \end{aligned}$$

It is written as a product by construction, so it is separable. The quick test agrees: $a = b = c = d = \frac{1}{2}$, so $ad = bc = \frac{1}{4}$.

I want you to notice that a state can be a superposition of all four basis states and still be completely unentangled. Superposition and entanglement are different things.

Example 1.10

Show that $\{|++\rangle, |+-\rangle, |-+\rangle, |--\rangle\}$ is an orthonormal basis of the two-qubit space.

Solution. Four normalized, mutually orthogonal vectors in a four-dimensional space form a basis, so those two properties are all we need. Their column forms are

$$\begin{aligned} |++\rangle &= \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, & |+-\rangle &= \frac{1}{2} \begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix}, \\ |-+\rangle &= \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ -1 \\ -1 \end{pmatrix}, & |--\rangle &= \frac{1}{2} \begin{pmatrix} 1 \\ -1 \\ -1 \\ 1 \end{pmatrix}. \end{aligned}$$

Each has four entries of modulus $\frac{1}{2}$, so each is normalized: $\| |++\rangle \| = \sqrt{\langle ++ | ++ \rangle} = \sqrt{4 \times \left(\frac{1}{2}\right)^2} = 1$, and identically for the other three.

For orthogonality, the factorization rule of Postulate 4 does the work in one line rather than six. Since $\langle ++ | ++ \rangle = \langle -- | -- \rangle = 1$ and $\langle +- | -+ \rangle = \langle -+ | +- \rangle = 0$,

$$\langle s_1 s_2 | t_1 t_2 \rangle = \langle s_1 | t_1 \rangle \langle s_2 | t_2 \rangle,$$

which vanishes whenever the labels differ in *either* slot. All six distinct pairs therefore have zero inner product, and the set is an orthonormal basis.

1.7.2 Bell States

The four *Bell states* are the most important entangled states in the subject. Each is maximally entangled, and together they form an orthonormal basis of the two-qubit space known as the *Bell basis*:

$$\begin{aligned} |\psi^{00}\rangle &= \frac{|00\rangle + |11\rangle}{\sqrt{2}}, \\ |\psi^{01}\rangle &= \frac{|01\rangle + |10\rangle}{\sqrt{2}}, \\ |\psi^{10}\rangle &= \frac{|00\rangle - |11\rangle}{\sqrt{2}}, \\ |\psi^{11}\rangle &= \frac{|01\rangle - |10\rangle}{\sqrt{2}}. \end{aligned} \tag{1.16}$$

All four are captured by the compact formula

$$|\psi^{xy}\rangle = \frac{|0y\rangle + (-1)^x |1\bar{y}\rangle}{\sqrt{2}}, \tag{1.17}$$

where $\bar{y}$ denotes the complement of the bit y . Substituting the four values of (x, y) reproduces (1.16): the bit x controls the sign and the bit y controls whether the two halves agree or disagree.

What entanglement means here. Suppose Alice holds the first qubit of $|\psi^{00}\rangle$ and Bob the second, and they are far apart. If Alice measures in the computational basis, her outcome is completely random: 0 and 1 each occur with probability $\frac{1}{2}$, since $|1/\sqrt{2}|^2 = \frac{1}{2}$. Bob's outcome, taken alone, is equally random. Yet when they later compare notes they find that their results agreed *every single time*. Neither qubit had a definite value beforehand, but the pair had a definite *relationship*.

Correlated does not always mean equal

Perfect agreement is a property of $|\psi^{00}\rangle$ and $|\psi^{10}\rangle$, whose terms are $|00\rangle$ and $|11\rangle$. For $|\psi^{01}\rangle$ and $|\psi^{11}\rangle$, built from $|01\rangle$ and $|10\rangle$, the outcomes are perfectly *anti*-correlated: Bob always gets the opposite of Alice. All four are maximally entangled; they differ in the *form* the correlation takes.

Let me be clear about what this does *not* allow. Alice's measurement does not transmit anything to Bob. His outcome is a fair coin toss whatever she does, and the correlation is only visible once they compare results over an ordinary classical channel. Entanglement is a correlation resource, not a signaling one.

Exercise 1.7

Use the $ad \neq bc$ test to confirm that all four Bell states are entangled, and verify that they are mutually orthogonal.

Answer: for $|\psi^{00}\rangle$, $(a, b, c, d) = (\frac{1}{\sqrt{2}}, 0, 0, \frac{1}{\sqrt{2}})$, so $ad - bc = \frac{1}{2} \neq 0$.

1.8 Two-Qubit Gates

Two-qubit gates make the qubits interact. They are what allow entanglement to be created, and no set of single-qubit gates alone can do it. A circuit built only from single-qubit gates always maps product states to product states. This is why every quantum computer must implement at least one entangling two-qubit gate [18].

1.8.1 Controlled-NOT Gate

The controlled-NOT (CX , or CNOT) gate has a *control* qubit and a *target* qubit. The control is left unchanged; the target is flipped if and only if the control is 1.

Table 1.1: Truth table for the controlled-NOT gate. Here $\oplus$ is addition modulo 2 (XOR). Both output qubits are listed: the control is unchanged and the target becomes $c \oplus t$.

Input $ ct\rangle$	Output $ c, c \oplus t\rangle$
$ 00\rangle$	$ 00\rangle$
$ 01\rangle$	$ 01\rangle$
$ 10\rangle$	$ 11\rangle$
$ 11\rangle$	$ 10\rangle$

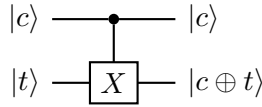


Figure 1.8: Controlled-NOT gate. The filled dot marks the control wire.

In Fig. 1.8 the control $|c\rangle$ is the top wire, qubit 0, and the target $|t\rangle$ is qubit 1, so the input is $|ct\rangle$ with c the left symbol. Reading Table 1.1 row by row, with each input supplying one column, gives the matrix

$$CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

and the outer-product form

$$CX = |00\rangle\langle 00| + |01\rangle\langle 01| + |11\rangle\langle 10| + |10\rangle\langle 11|. \quad (1.18)$$

Each term is the instruction “if you see this input, emit that output”. A more compact and more revealing way to write the same thing is

$$CX = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X, \quad (1.19)$$

which says explicitly: project the control (the left factor, qubit 0) onto $|0\rangle$ and do nothing to the target, or project it onto $|1\rangle$ and apply X to the target. Every controlled gate has this shape, with X replaced by the gate being controlled.

Checking against the truth table:

$$CX|00\rangle = |00\rangle, \quad CX|01\rangle = |01\rangle, \quad CX|10\rangle = |11\rangle, \quad CX|11\rangle = |10\rangle.$$

The two-gate circuit of Fig. 1.9 is the standard way to turn a product state into an entangled one. The example below traces it amplitude by amplitude.

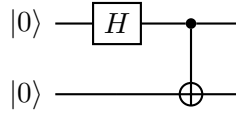


Figure 1.9: The canonical entangling circuit. A Hadamard on qubit 0 creates the superposition $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$, and the CNOT then flips qubit 1 only in the $|1\rangle$ branch of the control, producing the Bell state $|\psi^{00}\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

Example 1.11

Show that the circuit “ H on the first qubit, then CX ” turns $|00\rangle$ into a Bell state.

Solution.

$$|00\rangle \xrightarrow{H \otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{CX} \frac{|00\rangle + |11\rangle}{\sqrt{2}} = |\psi^{00}\rangle.$$

The CX acts term by term: it leaves $|00\rangle$ alone (control is 0) and maps $|10\rangle \mapsto |11\rangle$ (control is 1). You will meet this two-gate circuit in almost every protocol in this book.

Exercise 1.8

Apply the controlled-NOT gate to the state $|++\rangle$.

Hint. Write $|++\rangle$ in the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ first.

Answer: $CX|++\rangle = |++\rangle$. The state is unchanged, and in particular no entanglement is created. Now try $CX|+-\rangle$; you should find $|- -\rangle$, an effect known as *phase kickback* that we exploit repeatedly in later chapters.

1.8.2 Controlled- Z Gate

The controlled- Z gate applies a Z to the target when the control is 1, i.e. it attaches a minus sign to $|11\rangle$ and leaves the other three basis states alone.

Table 1.2: Truth table for the controlled- Z gate

Input	Output
$ 00\rangle$	$ 00\rangle$
$ 01\rangle$	$ 01\rangle$
$ 10\rangle$	$ 10\rangle$
$ 11\rangle$	$- 11\rangle$

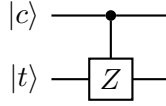


Figure 1.10: Controlled- Z gate. The joint output state is $(-1)^{ct} |ct\rangle$.

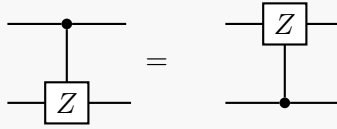
In Dirac notation,

$$CZ = |00\rangle\langle 00| + |01\rangle\langle 01| + |10\rangle\langle 10| - |11\rangle\langle 11| = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes Z,$$

so as a matrix $CZ = \text{diag}(1, 1, 1, -1)$.

Example 1.12

Show that the controlled- Z gate is symmetric under exchange of control and target:



Solution. From Table 1.2, the phase -1 appears exactly when *both* qubits are 1. The output can be written as $(-1)^{ct} |ct\rangle$, and the product ct is unchanged if c and t are swapped. Equivalently, $CZ = \text{diag}(1, 1, 1, -1)$ is unchanged by the relabeling of basis states $|01\rangle \leftrightarrow |10\rangle$, since those two entries are equal. Contrast this with CX , which is *not* symmetric: swapping its control and target gives a different gate. The symmetry of CZ is why it is a natural native gate on several hardware platforms.

1.8.3 SWAP Gate

The SWAP gate exchanges the states of two qubits.

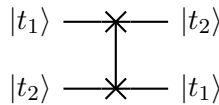


Figure 1.11: SWAP gate

Its action on the computational basis is $|00\rangle \mapsto |00\rangle$, $|01\rangle \mapsto |10\rangle$, $|10\rangle \mapsto |01\rangle$,

$|11\rangle \mapsto |11\rangle$, so

$$\text{SWAP} = |00\rangle\langle 00| + |10\rangle\langle 01| + |01\rangle\langle 10| + |11\rangle\langle 11| = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Observe that $\text{SWAP}^2 = I$, and that SWAP creates no entanglement. It merely relabels which physical qubit holds which state.

A useful identity is that SWAP factors into three controlled-NOTs with alternating control and target. In Fig. 1.12 the first and third CNOTs are controlled by the top wire, qubit 0, and the middle one by the bottom wire, qubit 1: This

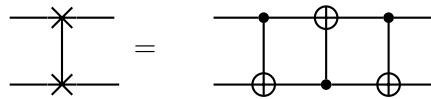


Figure 1.12: SWAP decomposed into three CNOT gates

matters in practice. Most hardware can only apply two-qubit gates to physically adjacent qubits, so moving a state across a chip costs a chain of SWAPs. Let us count. Suppose the qubit you need to interact with sits k positions away along a line of qubits. Bringing the two next to each other takes $k - 1$ SWAPs, and each SWAP is three CNOTs, so the move costs $3(k - 1)$ two-qubit gates before the gate you actually wanted is applied. On current hardware the two-qubit gates are the noisiest operations, so each SWAP is three CNOTs' worth of noise. In plain words, a gate between distant qubits can cost several times the noise of a gate between neighbors. That is why minimizing SWAP count is a central concern of quantum circuit compilation.

Exercise 1.9

Verify the decomposition of Fig. 1.12 by multiplying the three CNOT matrices, remembering that the middle one has control and target interchanged.

1.9 Summary

- A qubit state is a unit vector in $\mathbb{C}^2$, $|\psi\rangle = a|0\rangle + b|1\rangle$ with $|a|^2 + |b|^2 = 1$, drawn as a point on the Bloch sphere at angles (θ, φ) .
- Global phase is unphysical; relative phase is not.
- The four postulates: states are unit vectors (P1); closed-system evolution is unitary, hence reversible (P2); measurement outcomes follow the Born rule

and collapse the state (P3); composite systems combine by tensor product, so n qubits need 2^n amplitudes (P4).

- Every operator has a matrix form and an equivalent outer-product form; $|i\rangle\langle j| |k\rangle\langle l| = \delta_{jk} |i\rangle\langle l|$ makes Dirac-form algebra fast.
- Key single-qubit gates: X, Y, Z (rotations by π about the coordinate axes, up to phase), H (creates superposition), S (quarter-turn phase), and the continuous families R_x, R_y, R_z .
- Any single-qubit unitary factors as $e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$.
- A two-qubit pure state is entangled iff $ad \neq bc$. The four Bell states are maximally entangled and form a basis.
- CX, CZ and SWAP are the standard two-qubit gates; H followed by CX is the canonical entangling circuit.

1.10 End-of-Chapter Problems

- 1.1 Normalize the state $|\psi\rangle \propto 3|0\rangle - 4i|1\rangle$ and find its Bloch angles.
- 1.2 Show that $\{|+\rangle, |-\rangle\}$ is an orthonormal basis, and express $|0\rangle$ and $|1\rangle$ in that basis.
- 1.3 Prove that $XY = iZ$, $YZ = iX$ and $ZX = iY$. Deduce that no two distinct Pauli matrices commute.
- 1.4 Show that any 2×2 matrix can be written as $c_0I + c_xX + c_yY + c_zZ$ and find the coefficients for H .
- 1.5 A qubit in state $|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\varphi}\sin(\theta/2)|1\rangle$ is measured in the computational basis. Find $p(0)$ and $p(1)$, and explain why the answer does not depend on φ .
- 1.6 Compute $H \otimes H$ as a 4×4 matrix and verify that $(H \otimes H)|00\rangle = |++\rangle$.
- 1.7 Determine whether each of the following is entangled: (a) $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$; (b) $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)$; (c) $\frac{1}{\sqrt{3}}(|00\rangle + |01\rangle + |11\rangle)$.
- 1.8 Find the state produced by applying CX to $(\alpha|0\rangle + \beta|1\rangle) \otimes |0\rangle$, and state the condition on α, β for the result to be entangled.
- 1.9 Show that $(H \otimes H)CX(H \otimes H)$ equals CX with the roles of control and target interchanged.
- 1.10 Show that $R_z(\eta_1)R_z(\eta_2) = R_z(\eta_1 + \eta_2)$, and use this to evaluate $R_z(2\pi)$. Comment on the sign.

- 1.11** Verify explicitly that $S^2 = Z$ and $T^2 = S$, where $T = \text{diag}(1, e^{i\pi/4})$.
- 1.12** Starting from $|00\rangle$, design circuits producing each of the four Bell states, using only H , X , Z and CX .

Research Problems and Projects

The problems above have answers. The three projects below do not, at least not in the form given. Each is meant to occupy a strong MS or beginning PhD student for several weeks and to end in a written report with reproducible code and figures.

R1.1 Numerical conditioning of the Z – Y decomposition. Equation (1.13) guarantees that every single-qubit unitary factors as $e^{i\alpha}R_z(\beta)R_y(\gamma)R_z(\delta)$, but it says nothing about how stably β, γ, δ can be recovered from the four matrix entries. The parametrization degenerates when $\gamma \rightarrow 0$ or $\gamma \rightarrow \pi$, where only the combination $\beta + \delta$ (respectively $\beta - \delta$) is determined. Write an extraction routine, feed it Haar-random unitaries in double and in reduced precision, and measure the operator-norm error $\|U - e^{i\alpha}R_z(\beta)R_y(\gamma)R_z(\delta)\|$ as a function of $\sin \gamma$. Then repeat with a deliberately stabilized variant, for example one that solves for $\beta + \delta$ and $\beta - \delta$ from the two diagonal and two off-diagonal entries using `atan2` rather than inverting a cosine. *Deliverable:* a measured error-versus-sin γ curve for at least two extraction algorithms over 10^6 random unitaries, a statement of the observed worst-case error, and a short argument for which algorithm a compiler should use. A useful extension is to ask what the accumulated error looks like after the decomposition is applied to every gate of a depth- d circuit. Note that the individual pieces here are elementary; the work is in the careful measurement, not in the mathematics.

R1.2 What survives of the $ad \neq bc$ test? The separability box in the section on entangled states gives a complete and trivially checkable entanglement criterion for a two-qubit *pure* state. Trace what happens to that criterion as the setting is generalized, working from the literature rather than from first principles: the Schmidt rank for pure states of arbitrary bipartite dimension, the positive-partial-transpose criterion and the precise dimensions in which it is both necessary and sufficient, and the known computational hardness of deciding separability for general mixed states [24]. Reproduce, numerically and in full detail, one published example of a bound entangled state, that is, an entangled state with positive partial transpose. *Deliverable:* a survey report with a table whose rows are criteria and whose columns are (dimensions covered, necessary?, sufficient?, cost), together with a worked, verified counterexample for every cell where a criterion fails. Be careful to state which questions are settled and which are genuinely open. The bipartite mixed-state separability problem is known to be computationally hard in general, so an honest report

will identify where efficient criteria exist for *structured* states rather than claim the general problem is open.

R1.3 Bloch-vector tomography as a device benchmark. The Bloch vector of Section 1.4 is defined by $r_k = \langle \psi | \sigma_k | \psi \rangle$, and each component is estimated on hardware by measuring in the corresponding Pauli basis over many shots. Implement single-qubit state tomography on a simulator with a configurable noise model and, if you have access, on a real device. Prepare a grid of states $|\psi(\theta, \varphi)\rangle$ from (1.4), reconstruct $\vec{r}$, and study two things: how the estimator variance falls with the number of shots N_{shots} , and how the reconstructed length $|\vec{r}|$ shrinks below 1 as identity padding is inserted before the measurement. *Deliverable:* a resource table giving shots-per-state against reconstruction error for a fixed target accuracy, a fitted decay curve for $|\vec{r}|$ against idle time, and a comparison of the extracted decay constant with the coherence times the device or noise model reports. The reconstruction can return $|\vec{r}| > 1$ from sampling noise alone; say how you handle that, since the naive fix of rescaling to the sphere biases the estimate.

Chapter 2

Quantum Circuits and Building Blocks

Chapter 1 gave us states, gates and measurement. This chapter turns them into an engineering toolbox. Almost every algorithm in the rest of the book is assembled from a small number of constructions introduced here: controlled operations, multi-controlled gates, and oracles.

I have organized the chapter around two questions.

First, what can we actually build? Real hardware offers a fixed, finite menu: typically arbitrary single-qubit rotations plus one two-qubit entangling gate. Yet algorithms are written in terms of arbitrary unitaries on many qubits. Section 2.4 closes that gap: any unitary can be decomposed into gates from a small set, and a *finite* set suffices to approximate any unitary to any accuracy. This is why quantum computers are programmable at all.

Second, how does a quantum algorithm access a problem? Classical algorithms call a function. Quantum algorithms cannot, because a general function is not reversible and Postulate 2 requires unitarity. The resolution is the *oracle*, a reversible embedding of f that can be queried in superposition. Section 2.5 builds oracles carefully, because almost every speed-up in this book is really a statement about how few oracle queries are needed.

What you should be able to do after this chapter

- Read and write circuit diagrams, and convert a circuit to a matrix.
- Recall the standard gate set from memory and know what each is for.
- Build a controlled- U from CNOTs and single-qubit gates.
- Build $C^n(U)$ with and without ancillas, and state the cost of each.
- Explain why a finite gate set suffices, and quote the Solovay–Kitaev bound.
- Construct a bit oracle or a phase oracle for a given Boolean function.
- Explain phase kickback and use it to convert between oracle types.
- Uncompute garbage, and say why failing to do so destroys interference.

How the rest of the book uses this chapter

Phase kickback (§2.5.3)	Deutsch–Jozsa, Bernstein–Vazirani, phase estimation
Phase oracles (§2.5.4)	Grover search, amplitude amplification
Controlled- U powers (§2.3)	Phase estimation, order finding
Multi-controlled gates (§2.3.4)	Grover diffusion, arithmetic circuits
Uncomputation (§2.5.6)	Every circuit using scratch space
Universality (§2.4)	Compilation, resource estimates

2.1 Reading a Quantum Circuit

A circuit diagram is read *left to right* in time, with one horizontal wire per qubit. This is the opposite of matrix notation, where the rightmost operator acts first. A circuit showing A then B then C corresponds to the matrix product CBA . In my experience that reversal is a persistent source of sign and ordering errors, so I want you to fix it in your mind now.

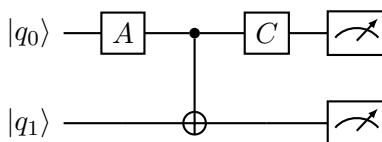


Figure 2.1: Circuit conventions. Time flows left to right. The filled dot is a control, $\oplus$ is the NOT target, and the meter symbol is a measurement in the computational basis. This circuit implements $(C \otimes I) \text{CNOT} (A \otimes I)$.

Qubit ordering, again

As in Chapter 1, we write $|q_0q_1 \dots q_{n-1}\rangle$ with qubit 0 *leftmost*, and the top wire of a circuit is qubit 0. A gate on qubit 0 of a two-qubit register is $A \otimes I$. Qiskit uses the opposite (little-endian) convention: its top wire is the *least* significant bit, so the same circuit produces a matrix with permuted rows and columns. When you check a hand calculation against code, check the ordering before you check the physics. In my experience this is the single most common source of “my circuit is right but the simulator disagrees”.

2.2 The Standard Gate Set

Before building anything, here is the working vocabulary. You met X, Y, Z, H, S and the rotations in Chapter 1. This section collects them with the additions we need, and says what each is *for*.

2.2.1 Single-qubit gates

Table 2.1: Single-qubit gates used throughout this book.

Gate	Matrix	Bloch action	Typical use
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	π about x	bit flip; NOT; building controls on $ 0\rangle$
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	π about y	bit + phase flip
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	π about z	phase flip; marking $ 1\rangle$
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	π about $\frac{x+z}{\sqrt{2}}$	create superposition; change to X -basis
S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$	$\pi/2$ about z	$S^2 = Z$; Clifford group
$S^\dagger$	$\begin{pmatrix} 1 & 0 \\ 0 & -i \end{pmatrix}$	$-\pi/2$ about z	inverse of S
T	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$	$\pi/4$ about z	$T^2 = S$; the non-Clifford gate
$P(\theta)$	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$	θ about z	general phase; controlled-phase
$R_x(\eta)$	$\begin{pmatrix} \cos \frac{\eta}{2} & -i \sin \frac{\eta}{2} \\ -i \sin \frac{\eta}{2} & \cos \frac{\eta}{2} \end{pmatrix}$	η about x	variational ansätze
$R_y(\eta)$	$\begin{pmatrix} \cos \frac{\eta}{2} & -\sin \frac{\eta}{2} \\ \sin \frac{\eta}{2} & \cos \frac{\eta}{2} \end{pmatrix}$	η about y	real-amplitude state preparation
$R_z(\eta)$	$\begin{pmatrix} e^{-i\eta/2} & 0 \\ 0 & e^{i\eta/2} \end{pmatrix}$	η about z	phase evolution

$P(\theta)$ versus $R_z(\theta)$

These differ by a global phase:

$$P(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}, \quad R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} = e^{-i\theta/2} P(\theta).$$

On a single isolated qubit they are interchangeable. **Their controlled versions are not.** $C\text{-}P(\theta)$ multiplies only $|11\rangle$ by $e^{i\theta}$, whereas $C\text{-}R_z(\theta)$ multiplies $|10\rangle$ by $e^{-i\theta/2}$ and $|11\rangle$ by $e^{i\theta/2}$. These are two different two-qubit gates. We will meet this distinction again in Section 2.3, where getting it wrong silently breaks the construction.

2.2.2 Two-qubit gates

Table 2.2: Two-qubit gates. Recall $CZ = (I \otimes H) \text{CNOT} (I \otimes H)$.

Gate	Matrix	Notes
$\text{CNOT} = C(X)$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$	Not symmetric in its two qubits. The standard entangling gate.
CZ	$\text{diag}(1, 1, 1, -1)$	Symmetric: control and target may be exchanged. Native on several platforms.
$C\text{-}P(\theta)$	$\text{diag}(1, 1, 1, e^{i\theta})$	Symmetric. The workhorse of the quantum Fourier transform.
SWAP	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$	Creates no entanglement. Costs three CNOTs.

2.2.3 Three-qubit gates

Figure 2.2: The Toffoli gate CCX (left) flips the target when both controls are 1. The Fredkin gate $CSWAP$ (right) swaps the two targets when the control is 1. Both are *universal for classical reversible computation*: any Boolean circuit can be rebuilt from Toffoli gates alone, which is why a quantum computer can run any classical algorithm.

The Toffoli gate acts as $|c_1 c_2 t\rangle \mapsto |c_1 c_2, t \oplus c_1 c_2\rangle$. Its truth table is the identity on all basis states except the last two, which it exchanges:

Table 2.3: Truth table of the Toffoli gate. Only the last two rows are non-trivial; every other basis state is unchanged. A correct table must be a *permutation*, with each output appearing exactly once, since the gate is unitary and hence invertible.

In	Out	In	Out
$ 000\rangle$	$ 000\rangle$	$ 100\rangle$	$ 100\rangle$
$ 001\rangle$	$ 001\rangle$	$ 101\rangle$	$ 101\rangle$
$ 010\rangle$	$ 010\rangle$	$ 110\rangle$	$ 111\rangle$
$ 011\rangle$	$ 011\rangle$	$ 111\rangle$	$ 110\rangle$

2.3 Controlled Operations

“If A is true, then do B ” is the most useful primitive in classical computing, and it is no less useful in quantum computing, with one difference that matters. When the control qubit is in superposition, *both* branches happen at once, and their relative phase is what algorithms exploit.

For an arbitrary single-qubit unitary U , the controlled- U gate acts as

$$C(U) = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U, \quad |c\rangle |t\rangle \mapsto |c\rangle U^c |t\rangle. \quad (2.1)$$

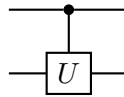


Figure 2.3: Controlled- U operation.

2.3.1 Building controlled- U from hardware gates

Hardware does not offer a controlled version of every gate. It typically offers single-qubit rotations and CNOT. So we must build $C(U)$ from those. The tool is the ABC decomposition proved in Chapter 1: for any single-qubit unitary there exist α and unitaries A, B, C with

$$U = e^{i\alpha} A X B X C, \quad A B C = I. \quad (2.2)$$

The construction works because of a simple observation. If the control is $|0\rangle$, the two CNOTs do nothing, so the target sees $A B C = I$ and is left unchanged. If the control is $|1\rangle$, both CNOTs fire, so the target sees $A X B X C$, and the leftover phase $e^{i\alpha}$ is supplied separately.

That phase is the subtle part, and I want you to look at it closely. Applying $e^{i\alpha}I$ to the *target* conditional on the control means multiplying both $|10\rangle$ and $|11\rangle$ by $e^{i\alpha}$. That is exactly what the single-qubit gate $P(\alpha)$ does when placed on the *control* wire:

$$P(\alpha) \otimes I : \quad |00\rangle \mapsto |00\rangle, \quad |01\rangle \mapsto |01\rangle, \quad |10\rangle \mapsto e^{i\alpha} |10\rangle, \quad |11\rangle \mapsto e^{i\alpha} |11\rangle.$$

Do not use $R_z(2\alpha)$ here

It is tempting to say that $e^{i\alpha}I$ “is just a z -rotation by 2α ”. It is not. The two matrices are

$$e^{i\alpha}I = \begin{pmatrix} e^{i\alpha} & 0 \\ 0 & e^{i\alpha} \end{pmatrix}, \quad R_z(2\alpha) = \begin{pmatrix} e^{-i\alpha} & 0 \\ 0 & e^{i\alpha} \end{pmatrix},$$

which agree only in the lower entry; they are not even equal up to a global phase. Substituting $C\text{-}R_z(2\alpha)$ for the phase step would give $|10\rangle \mapsto e^{-i\alpha} |10\rangle$ and $|11\rangle \mapsto e^{i\alpha} |11\rangle$. That is a different gate, and the construction silently breaks. Use $P(\alpha)$ on the control wire. Its action on the pair $(|10\rangle, |11\rangle)$, with no control at all, is exactly the required $e^{i\alpha}$ on both.

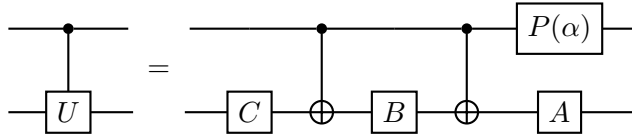


Figure 2.4: Controlled- U built from two CNOTs and four single-qubit gates. The target sees $ABC = I$ when the control is $|0\rangle$ and $AXBXC$ when it is $|1\rangle$; $P(\alpha)$ on the control supplies the phase $e^{i\alpha}$.

Why this matters later

Phase estimation (Chapter 4) needs controlled- U^{2^k} for a user-supplied U . Everything in that algorithm rests on being able to build controlled versions of gates the hardware does not natively provide. This construction is what makes that possible.

2.3.2 Controlling on $|0\rangle$

An open circle $\circ$ denotes a control that fires when the qubit is $|0\rangle$. It is implemented by conjugating an ordinary control with X gates, since X exchanges $|0\rangle$ and $|1\rangle$.

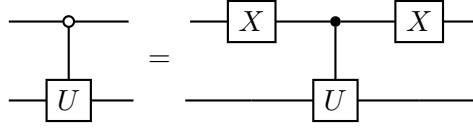


Figure 2.5: A control on $|0\rangle$, written as an open circle.

This trivial-looking device is what lets us mark *any* basis state rather than only $|11\cdots 1\rangle$, and it is used constantly in Section 2.5.

Exercise 2.1

Construct a $C^2(X)$ gate that fires on the control pattern $|01\rangle$.

Answer: put an open circle on the first control and a filled dot on the second; equivalently, conjugate the first control wire with X gates.

2.3.3 Two controls from a square root

Suppose we have access to V with $V^2 = U$. Then $C^2(U)$ can be built from three controlled- V gates and two CNOTs, five two-qubit gates in all, with no ancilla.

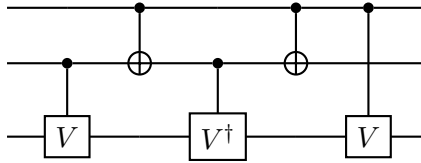


Figure 2.6: $C^2(U)$ from V with $V^2 = U$. Verified for $U = X, H$ and T .

The logic is a counting argument on the exponent of V applied to the target. Reading left to right, with controls c_1, c_2 :

Table 2.4: Why Fig. 2.6 works. The first CNOT replaces c_2 by $c_1 \oplus c_2$, so the $V^\dagger$ fires exactly when $c_1 \oplus c_2 = 1$; the second CNOT restores c_2 .

$c_1 c_2$	first V	$V^\dagger$	last V	net
00	none	none	none	I
01	V	$V^\dagger$	none	I
10	none	$V^\dagger$	V	I
11	V	none	V	$V^2 = U$

For the Toffoli gate $U = X$, we need $V = \sqrt{X}$.

Example 2.1

Show that $V = \frac{1+i}{2}(I - iX)$ satisfies $V^2 = X$.

Solution. Diagonalize X . Its eigenvalues are $\lambda_{1,2} = \pm 1$ with eigenvectors $|+\rangle$ and $|-\rangle$, so with $P = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ and $P^{-1} = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ we have $X = PDP^{-1}$, $D = \text{diag}(1, -1)$. Taking the principal square root $\sqrt{D} = \text{diag}(1, i)$,

$$\sqrt{X} = P\sqrt{D}P^{-1} = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix} = \frac{1+i}{2} \begin{pmatrix} 1 & -i \\ -i & 1 \end{pmatrix} = \frac{1+i}{2}(I - iX).$$

Squaring gives $\frac{(1+i)^2}{4}(I - iX)^2 = \frac{2i}{4}(I - 2iX - I) = X$, using $X^2 = I$. One checks directly that $VV^\dagger = I$.

Remark. A square root is not unique: $-\sqrt{X}$ works equally well, as does the other branch $\text{diag}(1, -i)$ in the eigenbasis. Any choice with $V^2 = X$ makes Fig. 2.6 correct.

2.3.4 Many controls

To build $C^n(U)$, compute the AND of all n controls into an ancilla, apply a single controlled- U , then *undo* the AND computation to return the ancillas to $|0\rangle$.

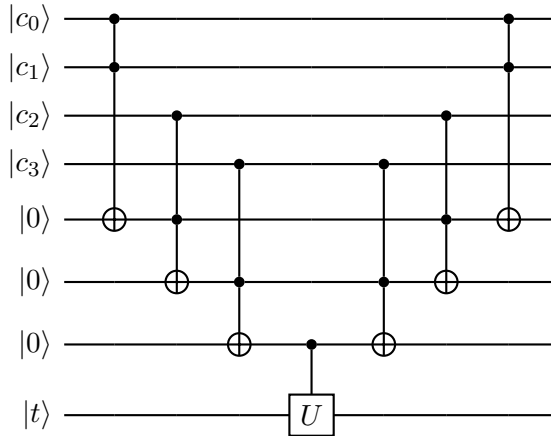


Figure 2.7: $C^4(U)$ using three ancillas. The left half computes $c_0c_1c_2c_3$ into the last ancilla; the right half is its mirror image and restores every ancilla to $|0\rangle$.

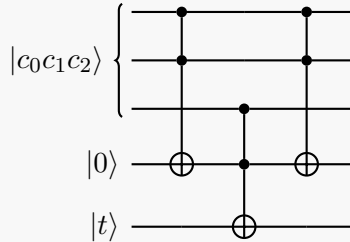
The mirror half is not optional

Beginners often delete the right-hand Toffoli, reasoning that the ancillas are “no longer needed”. That is wrong. An ancilla left holding $|c_0c_1c_2c_3\rangle$ is *entangled* with the control register. That entanglement acts as a measurement record: it destroys the interference between computational paths that the algorithm depends on. Section 2.5.6 makes this precise.

Example 2.2

Build $C^3(X)$ from Toffoli gates and one ancilla.

Solution.



The first Toffoli writes c_0c_1 into the ancilla; the second flips the target if c_2 and the ancilla are both 1, i.e. if $c_0c_1c_2 = 1$; the third restores the ancilla to $|0\rangle$. Verified numerically on all 16 inputs with the ancilla starting in $|0\rangle$.

Cost. Let me count what Fig. 2.7 uses, with n the number of controls. The first Toffoli ANDs two controls into a fresh ancilla, and each of the remaining $n - 2$ controls is ANDed with the previous ancilla into a new one. That is $n - 1$ Toffolis and $n - 1$ ancillas to compute the AND of all n controls. The mirror half repeats the $n - 1$ Toffolis in reverse. So $C^n(U)$ uses $n - 1$ ancillas and costs $2n - 2$ Toffoli gates plus one controlled- U . For $U = X$ the final controlled- X is itself a Toffoli, so the last ancilla is not needed and the layout folds down to $n - 2$ ancillas and $2n - 3$ Toffoli gates, as in Example 2.2. The Toffolis in each half form a chain, each waiting for the ancilla written by the one before, so either layout has depth $O(n)$: the number of time steps grows in proportion to the number of controls. Ancilla-free constructions exist but cost $O(n^2)$ elementary gates (CNOTs and single-qubit gates), so doubling the number of controls costs about four times as many gates. With a single dirty ancilla, a borrowed qubit in an unknown state that is returned unchanged, the cost drops back to $O(n)$ elementary gates. In practice the trade-off between ancilla count and gate count is a compilation decision.

A note on software

Multi-controlled X is available directly in Qiskit as `MCXGate`, and via the `QuantumCircuit.mcx` method, for any number of controls. Both were checked against Qiskit 2.5, the current release as this book went to press.

Know what the compiler is doing, though: the ancilla count and depth it chooses will dominate your resource estimates. The call that reported an `MCXGate`'s ancilla count was deprecated in Qiskit 2.1, because the transpiler now picks a synthesis method during high-level synthesis from how many ancillas are free. Count them from the transpiled circuit instead. Appendix B gives the Qiskit name of every gate in this book and repeats this warning. Library APIs change, so check the current documentation rather than any printed listing, including that one.

2.4 Universality

We now answer the first motivating question: given only a small menu of gates, what can we build? The answer comes in three stages, each narrowing the gate set:

- (i) any $d \times d$ unitary is a product of *two-level* unitaries (§2.4.1);
- (ii) any two-level unitary is a product of multi-controlled single-qubit gates, hence of CNOTs and single-qubit gates (§2.4.2);
- (iii) single-qubit gates, which form a continuum, can be *approximated* to any accuracy by a *finite* set such as $\{H, T\}$ (§2.4.3).

Stage (iii) is the one that makes physical hardware possible: a machine with a finite instruction set can run any quantum algorithm.

2.4.1 Any unitary is a product of two-level unitaries

A *two-level* unitary acts non-trivially on at most two components of a vector, leaving all others alone. The claim is that any U on a d -dimensional space factors as $U = V_1 V_2 \cdots V_k$ with each V_i two-level.

The proof is constructive: it is nothing more than Gaussian elimination with unitary row operations [24]. We illustrate it for $d = 3$ with

$$U = \begin{pmatrix} a & d & g \\ b & e & h \\ c & f & j \end{pmatrix}, \quad (2.3)$$

and find two-level U_1, U_2, U_3 with $U_3 U_2 U_1 U = I$; then $U = U_1^\dagger U_2^\dagger U_3^\dagger$ is the desired product.

Step 1: zero the (2, 1) entry. If $b = 0$, take $U_1 = I$. Otherwise set $m = \sqrt{|a|^2 + |b|^2}$ and

$$U_1 = \begin{pmatrix} a^*/m & b^*/m & 0 \\ b/m & -a/m & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \text{giving} \quad U_1 U = \begin{pmatrix} a' & d' & g' \\ 0 & e' & h' \\ c' & f' & j' \end{pmatrix}.$$

Check: the new (2, 1) entry is $(b/m)a + (-a/m)b = 0$, and the new (1, 1) entry is $(|a|^2 + |b|^2)/m = m$, so $a' = m$ is real and positive.

Step 2: zero the (3, 1) entry. If $c' = 0$, take $U_2 = I$. Otherwise set $n = \sqrt{|a'|^2 + |c'|^2}$ and

$$U_2 = \begin{pmatrix} a'^*/n & 0 & c'^*/n \\ 0 & 1 & 0 \\ c'/n & 0 & -a'/n \end{pmatrix}. \quad (2.4)$$

Note the conjugations in (2.4)

The third row is $(c'/n, 0, -a'/n)$, *without* complex conjugation, exactly mirroring the second row of U_1 . If the row is written with conjugates instead, the matrix is not unitary and the (3, 1) entry does not vanish. The new (3, 1) entry would be $(c'^*a' - a'^*c')/n = 2i \operatorname{Im}(a'c'^*)/n$, which is nonzero whenever $a'c'^*$ is not real. The pattern to remember is that the two rows of a two-level block are $(\bar{u}, \bar{v})$ and $(v, -u)$.

This gives

$$V \equiv U_2 U_1 U = \begin{pmatrix} a'' & d'' & g'' \\ 0 & e'' & h'' \\ 0 & f'' & j'' \end{pmatrix}.$$

Since V is a product of unitaries it is unitary, so $V^\dagger V = I$. Reading off the (1, 1) entry gives $|a''|^2 = 1$; the (1, 2) and (1, 3) entries then give $a''^* d'' = a''^* g'' = 0$, and since $a'' \neq 0$ we get $d'' = g'' = 0$. Tracing the construction shows $a'' = n > 0$, hence $a'' = 1$ and

$$U_2 U_1 U = \begin{pmatrix} 1 & 0 & 0 \\ 0 & e'' & h'' \\ 0 & f'' & j'' \end{pmatrix}.$$

Step 3. Take $U_3 = V^\dagger$, which is two-level (it is the identity on the first component). Then $U_3 U_2 U_1 U = I$, so

$$U = U_1^\dagger U_2^\dagger U_3^\dagger = V_1 V_2 V_3, \quad V_i = U_i^\dagger.$$

General d . The same elimination clears the first column with $d - 1$ two-level unitaries, then the second column with $d - 2$, and so on, giving at most

$$k = (d - 1) + (d - 2) + \cdots + 1 = \frac{d(d - 1)}{2} = O(d^2) \quad (2.5)$$

two-level factors. Here d is the dimension of the space, and k counts two-level unitaries, not yet gates; each factor still has to be turned into a circuit, which is the next subsection. For n qubits, $d = 2^n$ and $k = O(4^n)$: every extra qubit multiplies the number of factors by about four.

Exercise 2.2

Carry out the elimination for a 2×2 unitary. How many two-level factors does (2.5) predict, and does the answer make sense?

Answer: $k = 1$. A 2×2 unitary is already two-level.

2.4.2 From two-level unitaries to circuits

A two-level unitary on n qubits is *not* a single-qubit gate: it acts on the full 2^n -dimensional space, just non-trivially on only two basis states. To implement it we use Gray codes.

Consider three qubits, so an 8-dimensional space with basis $\{|000\rangle, \dots, |111\rangle\}$, and the two-level unitary

$$U = \begin{pmatrix} a & 0 & 0 & 0 & 0 & 0 & 0 & c \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ b & 0 & 0 & 0 & 0 & 0 & 0 & d \end{pmatrix}, \quad \tilde{U} = \begin{pmatrix} a & c \\ b & d \end{pmatrix}, \quad (2.6)$$

which mixes only $|000\rangle$ and $|111\rangle$.

The obstruction is that $|000\rangle$ and $|111\rangle$ differ in *all three* bits, whereas a controlled single-qubit gate can only act on states differing in *one* bit. The fix is to walk from one state to the other one bit at a time, following a *Gray code*. Each step of the walk is a multi-controlled NOT. Apply $\tilde{U}$ at the end of the walk, then retrace the walk to restore everything.

q_0	q_1	q_2
1	1	1
0	1	1
0	0	1
0	0	0

- (i) $|111\rangle \rightarrow |011\rangle$: qubits 1 and 2 agree in both states, so apply $C^2(X)$ on qubit 0 controlled by qubits 1 and 2.
- (ii) $|011\rangle \rightarrow |001\rangle$: qubits 0 and 2 agree, so apply $C^2(X)$ on qubit 1 controlled by qubits 0 and 2 (with qubit 0's control on $|0\rangle$).
- (iii) $|001\rangle \rightarrow |000\rangle$: qubits 0 and 1 agree, so apply $C^2(\tilde{U})$ on qubit 2 controlled by qubits 0 and 1, both on $|0\rangle$.
- (iv) Reverse steps (i)–(ii) to restore the labeling.

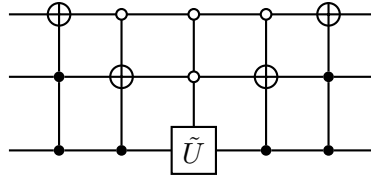


Figure 2.8: Circuit for the two-level unitary (2.6). The outer pairs of gates are mirror images; only the middle gate does anything non-trivial. Verified numerically: this circuit reproduces (2.6) exactly.

Exercise 2.3

Verify that Fig. 2.8 is two-level by applying it to each of the eight basis states.

Hint: every state other than $|000\rangle$ and $|111\rangle$ should emerge unchanged. Track the Gray-code walk to see why.

Total cost. Let me assemble the count, with n the number of qubits. The two basis states that a two-level unitary mixes differ in at most n bits, so the Gray-code walk has at most $n - 1$ steps, and with the retrace the circuit contains at most $2n - 1$ multi-controlled gates, each controlled on $n - 1$ qubits. By Section 2.3.4, one such gate costs $O(n)$ elementary gates, meaning CNOTs and single-qubit gates. So one two-level unitary costs $O(n) \times O(n) = O(n^2)$ single-qubit and CNOT gates.

Combining with the $O(4^n)$ two-level factors from (2.5): an arbitrary unitary on n qubits needs

$$O(n^2 4^n) \quad (2.7)$$

single-qubit and CNOT gates. In plain words, adding one qubit multiplies the gate count by a little more than four. This counts gates, not queries or qubits; the construction uses ancillas only inside the multi-controlled gates.

Read that bound carefully

Equation (2.7) is exponential in n . It says that *most* unitaries are hopelessly expensive. That is not a defect of the construction but a fact about the space. A general n -qubit unitary is specified by 4^n real parameters, so no circuit family with polynomially many gates could reach them all.

Quantum algorithms are therefore never “apply an arbitrary unitary”. They are always built from *structured* unitaries with efficient circuits: the Fourier transform, a Grover diffusion, a Trotterized Hamiltonian. Recognizing that structure is the real work of algorithm design.

2.4.3 Approximating with a finite gate set

There is still a problem. A single-qubit unitary $U = e^{i\alpha} R_{\hat{n}}(\beta) R_{\hat{m}}(\gamma) R_{\hat{n}}(\delta)$ has continuous parameters, so an exact implementation would need hardware capable of every angle. That would be an infinite instruction set. No finite set of gates can produce every unitary exactly.

What we can do is *approximate*. Define the error of implementing V in place of U as the worst-case output deviation

$$E(U, V) = \max_{|\psi\rangle} \|(U - V)|\psi\rangle\|, \quad (2.8)$$

the maximum over normalized $|\psi\rangle$. This is the operator norm of $U - V$. It has the property we need: errors add at worst linearly. If a circuit contains m gates and each gate is implemented to accuracy ϵ_1 in the sense of (2.8), then the whole circuit is accurate to $m\epsilon_1$. So a target accuracy ϵ for the whole circuit can be met by asking $\epsilon_1 = \epsilon/m$ of every gate.

Why this definition and not another

One might instead compare expectation values, $\max_{|\psi\rangle} |\langle\psi|U|\psi\rangle - \langle\psi|V|\psi\rangle|$. That quantity is not a norm on the difference and does not obey the linear error-accumulation bound, so it is the wrong tool for chaining gates together. Definition (2.8) is the standard one and is what makes the rest of this section work.

The gate set $\{H, T, \text{CNOT}\}$. We show H and T generate a dense set of single-qubit rotations. First,

$$\begin{aligned} HTH &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \\ &= \frac{1}{2} \begin{pmatrix} 1 + e^{i\pi/4} & 1 - e^{i\pi/4} \\ 1 - e^{i\pi/4} & 1 + e^{i\pi/4} \end{pmatrix} = \frac{e^{i\pi/8}}{2} \begin{pmatrix} e^{-i\pi/8} + e^{i\pi/8} & e^{-i\pi/8} - e^{i\pi/8} \\ e^{-i\pi/8} - e^{i\pi/8} & e^{-i\pi/8} + e^{i\pi/8} \end{pmatrix} \\ &= e^{i\pi/8} \begin{pmatrix} \cos \frac{\pi}{8} & -i \sin \frac{\pi}{8} \\ -i \sin \frac{\pi}{8} & \cos \frac{\pi}{8} \end{pmatrix} = e^{i\pi/8} R_x\left(\frac{\pi}{4}\right). \end{aligned}$$

(Both diagonal entries carry a *plus* sign; a minus in either would make the matrix non-unitary.)

Recall the general rotation about an axis $\hat{n}$,

$$R_{\hat{n}}(\theta) = e^{-i\frac{\theta}{2}\hat{n}\cdot\vec{\sigma}} = \cos \frac{\theta}{2} I - i \sin \frac{\theta}{2} \hat{n} \cdot \vec{\sigma}, \quad \vec{\sigma} = (X, Y, Z). \quad (2.9)$$

Now compose T with HTH . Since $T = e^{i\pi/8} R_z(\pi/4)$,

$$THTH \propto R_z\left(\frac{\pi}{4}\right) R_x\left(\frac{\pi}{4}\right) = \cos^2 \frac{\pi}{8} I - i \sin \frac{\pi}{8} \left(\cos \frac{\pi}{8}, \sin \frac{\pi}{8}, \cos \frac{\pi}{8} \right) \cdot \vec{\sigma}.$$

Write $\vec{m} = (\cos \frac{\pi}{8}, \sin \frac{\pi}{8}, \cos \frac{\pi}{8})$, so $|\vec{m}| = \sqrt{1 + \cos^2 \frac{\pi}{8}}$ and $\hat{m} = \vec{m}/|\vec{m}|$. Comparing with (2.9),

$$\cos \frac{\theta}{2} = \cos^2 \frac{\pi}{8}, \quad \sin \frac{\theta}{2} = \sin \frac{\pi}{8} \sqrt{1 + \cos^2 \frac{\pi}{8}}. \quad (2.10)$$

Multiply, do not divide

The coefficient of $\hat{m} \cdot \vec{\sigma}$ is $\sin \frac{\pi}{8} |\vec{m}|$, because pulling the unit vector out of $\vec{m} \cdot \vec{\sigma}$ *multiplies* by $|\vec{m}|$. Writing $\sin \frac{\pi}{8} / \sqrt{1 + \cos^2 \frac{\pi}{8}}$ instead gives 0.2811, whereas $\sin(\theta/2) = 0.5210$; the resulting “rotation” would not be unitary. As a check, verify $\cos^2 \frac{\theta}{2} + \sin^2 \frac{\theta}{2} = 1$ with the values in (2.10).

Numerically $\theta = 2 \arccos(\cos^2 \frac{\pi}{8}) = 1.0961 \text{ rad} = 62.80^\circ$. The property that matters is that θ is an *irrational* multiple of 2π . A second axis is obtained by conjugation: using $HXH = Z$, $HYH = -Y$, $HZH = X$,

$$H R_{\hat{m}}(\theta) H = R_{\hat{n}}(\theta), \quad \hat{n} \propto \left(\cos \frac{\pi}{8}, -\sin \frac{\pi}{8}, \cos \frac{\pi}{8} \right),$$

and $\hat{m}, \hat{n}$ are non-parallel. Rotations about two non-parallel axes generate all of $SU(2)$.

Why irrationality gives density

Because $\theta/2\pi$ is irrational, the angles $\theta, 2\theta, 3\theta, \dots \pmod{2\pi}$ never repeat and fill the circle densely. So for any target angle α and any tolerance ϵ_1 , some power $R_{\hat{n}}(\theta)^k$ lands within ϵ_1 of $R_{\hat{n}}(\alpha)$.

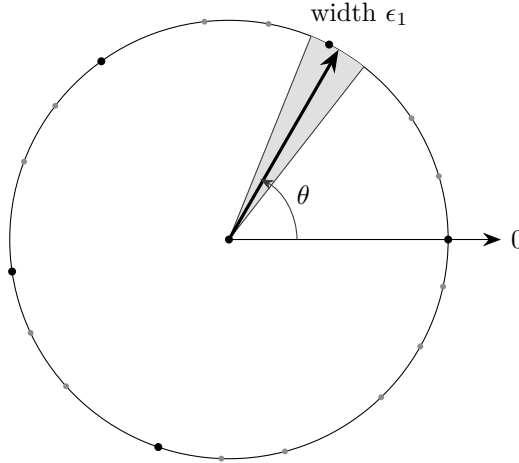


Figure 2.9: Because θ is an irrational multiple of 2π , the iterates $k\theta \pmod{2\pi}$ never repeat and eventually enter any target window of width ϵ_1 . Dividing the circle into $2\pi/\epsilon_1$ windows shows that $k = O(1/\epsilon_1)$ repetitions typically suffice.

A crude count gives the cost. Here ϵ_1 is the accuracy we want for one rotation, in the norm (2.8). Divide the circle into $2\pi/\epsilon_1$ windows of width ϵ_1 . Among any $2\pi/\epsilon_1 + 1$ iterates, two must share a window. Their difference is some power $R_{\hat{n}}(\theta)^j$ with $j \leq 2\pi/\epsilon_1$, and it is a non-zero rotation by less than ϵ_1 . Stepping around the circle in multiples of that small rotation then reaches every window. When the small rotation is not much smaller than ϵ_1 , which is the typical case, the whole process needs $k = O(1/\epsilon_1)$ applications of $R_{\hat{n}}(\theta)$ to approximate any single-qubit rotation to accuracy ϵ_1 . In plain words, halving the tolerance doubles the count. Each $R_{\hat{n}}(\theta)$ is, up to a phase, a word of four gates in H and T , so this is $O(1/\epsilon_1)$ gates from the finite set $\{H, T\}$ for a single rotation.

Solovay–Kitaev theorem

The $O(1/\epsilon_1)$ count above is very pessimistic. The Solovay–Kitaev theorem [18, 24] applies to any gate set that is universal for $SU(2)$ and closed under inverses. For such a set, any single-qubit unitary can be approximated to accuracy ϵ_1 using

$$O(\log^c(1/\epsilon_1))$$

gates from the finite set, with $c \approx 2$ for the standard proof. The dependence is *polylogarithmic*, not linear, in $1/\epsilon_1$.

That changes the resource picture. For a circuit of m gates and total error ϵ , each gate needs accuracy $\epsilon_1 = \epsilon/m$, so the compiled circuit has $O(m \log^c(m/\epsilon))$ gates from the finite set, the original gate count times a logarithmic factor. Whenever you see a compiled gate count in the literature, this is the theorem being invoked. The counting argument above is used only because it is elementary; this is the bound that matters in practice.

Read the two bounds side by side. The crude count says that improving the accuracy by a factor of ten costs ten times as many gates. The Solovay–Kitaev count says that the gate count grows only like a power of the *number of correct digits*, since $\log(1/\epsilon_1)$ is that number up to a constant. Going from three correct digits to six then costs about $2^c \approx 4$ times as many gates, not a thousand times. Both bounds count gates drawn from the finite set $\{H, T\}$ for one single-qubit rotation. The classical computation that finds the gate sequence is a separate cost, also polylogarithmic in $1/\epsilon_1$, and it does not run on the quantum computer.

2.4.4 Why approximation is hard in general

Combining the pieces: an arbitrary unitary on n qubits needs $O(n^2 4^n)$ elementary gates (CNOTs and single-qubit gates), and each single-qubit gate needs $O(\log^c(1/\epsilon_1))$ gates from a finite set to reach accuracy ϵ_1 . The product is still exponential in n . Is *exponential* cost unavoidable? A counting argument says yes.

Suppose our gate set has g gates, each acting on at most f qubits. Consider a circuit of m gates on n qubits. Each gate can be chosen in g ways and placed on at most $\binom{n}{f}$ subsets of qubits, so the circuit can be arranged in at most $\left(g \binom{n}{f}\right)^m$ ways, and at most that many distinct output states are reachable.

Now ask how many states we need to reach. The set of n -qubit pure states is the complex projective space $\mathbb{CP}^{2^n-1}$, of *real* dimension

$$D = 2(2^n - 1) = 2^{n+1} - 2. \quad (2.11)$$

Let ϵ be the accuracy to which we want to reach every state, measured in the norm (2.8). Covering a D -dimensional space with patches of radius ϵ takes $\Omega(1/\epsilon^D)$

patches. For the reachable set to cover the state space we need

$$\left(g \binom{n}{f}\right)^m \gtrsim \frac{1}{\epsilon^D}.$$

Using $\binom{n}{f} \leq n^f$ and taking logarithms,

$$mf \log n \gtrsim D \log \frac{1}{\epsilon} \quad \implies \quad m \gtrsim \frac{2^{n+1} \log(1/\epsilon)}{f \log n}.$$

Here m is the number of gates from the finite set, n the number of qubits, f the largest number of qubits a gate touches, and the dropped $\log g$ only changes the constant. So there exist n -qubit states requiring exponentially many gates to prepare even approximately: adding one qubit roughly doubles the lower bound. The bound carries over to unitaries at once. Any such state is $U|0\rangle^{\otimes n}$ for some unitary U , so a circuit implementing U also prepares the state and must be at least this long.

So exponential cost is unavoidable. Look carefully at what the argument counts, though: *states*, whose real dimension is (2.11). It therefore delivers a lower bound of order 2^n , against the upper bound of order 4^n in (2.7). Closing that gap needs the dimension of the unitary group rather than of the state space, which is the subject of the second research problem at the end of this chapter.

Sanity-check the dimension

For $n = 1$, (2.11) gives $D = 2$: a single qubit's pure states form the *surface* of the Bloch sphere, parameterized by the polar and azimuthal angles (θ, φ) , a two-dimensional surface sitting in three-dimensional space. It is easy to write 3 here by thinking of the ball rather than the sphere; the count that enters the argument is the number of free parameters, which is 2.

The conclusion is not pessimism but guidance: efficient quantum algorithms must exploit structure. Nothing is gained by hoping a generic unitary will be cheap.

2.5 Quantum Oracles

2.5.1 Why oracles exist

Most quantum algorithms are stated as questions about a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$: is it constant or balanced? which string is hidden inside it? which input does it map to 1? The algorithm is given f as a “black box” it may query, and the figure of merit is the *number of queries* needed. Speed-ups in this book are almost always query-complexity statements.

Two reasons make this abstraction the right one.

It isolates the quantum content. We want to study the interference technique, not the details of some particular f . Suppose a quantum algorithm needs $O(1)$ queries to f , a number that does not grow with the input length n , where any classical algorithm needs $\Omega(2^n)$ queries, a number that doubles with every extra input bit. That separation is a fact about quantum mechanics, independent of what f is.

It is honest about cost. A real implementation must build the oracle from gates, and that circuit may be large. Query complexity deliberately does not count that cost: it counts calls to O_f , not the gates inside O_f . A query speed-up is therefore a real result, but not by itself a claim of practical advantage. Keeping the two separate will save you from over-reading many headline results.

2.5.2 Making f reversible

We cannot simply build a circuit $|x\rangle \mapsto |f(x)\rangle$. Postulate 2 forces every gate to be unitary, hence invertible, but f generally is not: if $f(0) = f(1) = 0$ we could never recover x from the output. The standard fix embeds f reversibly using an extra register:

$$O_f : |x\rangle |b\rangle \mapsto |x\rangle |b \oplus f(x)\rangle, \quad (2.12)$$

where $\oplus$ is addition modulo 2. The input $|x\rangle$ is preserved, which makes the map a permutation of basis states and therefore unitary. Applying O_f twice returns the original state, so $O_f^2 = I$ and $O_f = O_f^\dagger$.

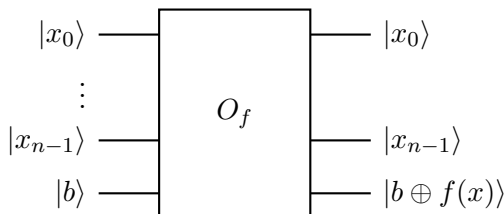


Figure 2.10: The standard oracle. The top n wires carry the input $|x\rangle = |x_0 x_1 \cdots x_{n-1}\rangle$, with x_0 the most significant bit on the top wire; $|b\rangle$ is a single *ancilla* qubit.

The whole point is that O_f is linear, so it acts on superpositions:

$$O_f \left(\frac{1}{\sqrt{2^n}} \sum_x |x\rangle \right) |b\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle |b \oplus f(x)\rangle. \quad (2.13)$$

A single query has touched all 2^n inputs. This is often called “quantum parallelism”, but the name oversells it: measuring (2.13) returns one random $(x, f(x))$ pair, no better than one classical query. The value is not in the parallelism but in what we do next: arranging the *phases* so that the answers we want interfere constructively.

2.5.3 Phase kickback

Phase kickback is the mechanism that converts the oracle’s bit answer into a phase. It appears in nearly every algorithm in this book.

The observation is that $|-\rangle$ is an eigenvector of X :

$$X|-\rangle = X \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|1\rangle - |0\rangle}{\sqrt{2}} = -|-\rangle,$$

with eigenvalue -1 . So applying X to a $|-\rangle$ ancilla does not change the ancilla; it produces a factor of -1 . In a *controlled* operation that factor attaches to the control’s $|1\rangle$ branch, becoming a relative phase on the control register. It is “kicked back”.

Example 2.3

Compute the action of CNOT on $|+\rangle|-\rangle$, with the first qubit as control.

Solution. Expand the control:

$$\begin{aligned} \text{CNOT}|+\rangle|-\rangle &= \frac{1}{\sqrt{2}} \left(\text{CNOT}|0\rangle|-\rangle + \text{CNOT}|1\rangle|-\rangle \right) \\ &= \frac{1}{\sqrt{2}} \left(|0\rangle|-\rangle + |1\rangle X|-\rangle \right) \\ &= \frac{1}{\sqrt{2}} \left(|0\rangle|-\rangle - |1\rangle|-\rangle \right) = |-\rangle|-\rangle. \end{aligned}$$

The target is unchanged; the *control* has moved from $|+\rangle$ to $|-\rangle$. Nothing flipped, but the phase relationship inside the control register did.

Which qubit is which

In Example 2.3 the control is the *first* ket and the $|-\rangle$ ancilla the second, matching the circuit convention that the ancilla is the bottom wire. If you write $\text{CNOT}|-\rangle|+\rangle$ and expect $|-\rangle|-\rangle$, you are implicitly making the *second* qubit the control. Both conventions appear in the literature; fix one and state it.

The general statement: if $U|u\rangle = e^{i\phi}|u\rangle$, then

$$C(U)(\alpha|0\rangle + \beta|1\rangle)|u\rangle = (\alpha|0\rangle + \beta e^{i\phi}|1\rangle)|u\rangle. \quad (2.14)$$

The eigenvector is untouched; its eigenvalue lands on the control as a relative phase. Figure 2.11 draws this as a circuit. Phase estimation (Chapter 4) is nothing more than (2.14) run in reverse: measure the control register to learn ϕ .

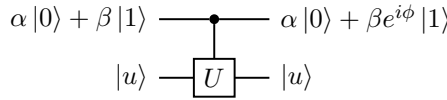
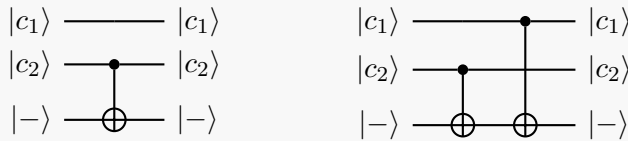


Figure 2.11: Phase kickback in its general form, Eq. (2.14). The target is an eigenvector of U , so it leaves the circuit unchanged. The eigenvalue $e^{i\phi}$ appears instead as a relative phase on the control wire. Compare Fig. 2.3: the gate is the same, only the target state is special.

Example 2.4

Two small circuits, with the $|-\rangle$ ancilla on the bottom wire.



In each case the ancilla returns to $|-\rangle$ and the control register picks up a phase:

$$\text{left: } |c_1 c_2\rangle \mapsto (-1)^{c_2} |c_1 c_2\rangle, \quad \text{right: } |c_1 c_2\rangle \mapsto (-1)^{c_1 \oplus c_2} |c_1 c_2\rangle.$$

Both verified on all four inputs. I want you to notice that the right-hand circuit computes the parity $c_1 \oplus c_2$ as a phase using no extra qubits. This is exactly the trick behind the Bernstein–Vazirani algorithm [3].

2.5.4 Bit oracles and phase oracles

The same unitary O_f behaves in two quite different ways depending on how we prepare the ancilla. Both forms are used constantly, so I will state the dictionary once and clearly.

Bit oracle (ancilla in $|0\rangle$). Setting $b = 0$ in (2.12) gives

$$|x\rangle |0\rangle \mapsto |x\rangle |f(x)\rangle, \quad (2.15)$$

so the answer appears as a *bit* in the ancilla. This is the form to use when a later part of the circuit must condition on the value of f .

Phase oracle (ancilla in $|-\rangle$). Setting $b = |-\rangle$,

$$|-\oplus f(x)\rangle = \begin{cases} |-\rangle & f(x) = 0, \\ -|-\rangle & f(x) = 1, \end{cases} = (-1)^{f(x)} |-\rangle,$$

so

$$|x\rangle |-\rangle \mapsto (-1)^{f(x)} |x\rangle |-\rangle. \quad (2.16)$$

The ancilla factors out entirely and can be dropped from the bookkeeping. We write the effective action as

$$O_f^\pm : |x\rangle \mapsto (-1)^{f(x)} |x\rangle.$$

The dictionary

	Bit oracle	Phase oracle
ancilla prepared in	$ 0\rangle$	$ -\rangle$
action	$ x\rangle 0\rangle \mapsto x\rangle f(x)\rangle$	$ x\rangle \mapsto (-1)^{f(x)} x\rangle$
answer stored as	a bit	a phase
ancilla afterwards	entangled with $ x\rangle$	unchanged, unentangled
used by	arithmetic, reversible logic	Deutsch–Jozsa, Grover

They are the *same unitary*. Only the ancilla preparation differs, so starting from an ancilla in $|1\rangle$ the conversion costs two Hadamards and nothing else. From $|0\rangle$ an extra X is needed on each side, because $H|0\rangle = |+\rangle$ is unchanged by the oracle and no phase would appear.

Figure 2.12 draws that conversion explicitly. It is the block that appears at the front of almost every oracle-based algorithm in this book.

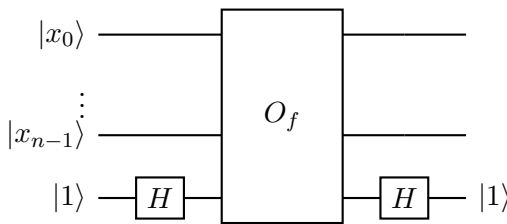


Figure 2.12: Turning the bit oracle into the phase oracle. The first Hadamard sends the ancilla $|1\rangle$ to $|-\rangle$, so O_f contributes the factor $(-1)^{f(x)}$ and leaves the ancilla in $|-\rangle$; the second Hadamard returns it to $|1\rangle$. The ancilla is therefore unchanged and unentangled, and the block acts on the top register alone as $O_f^\pm : |x\rangle \mapsto (-1)^{f(x)} |x\rangle$.

Why the phase form is the useful one

A phase is invisible to measurement, since $\left|(-1)^{f(x)}\right|^2 = 1$, so at first the phase oracle looks useless. The point is that phases become visible after *interference*. Applying $H^{\otimes n}$ to $\sum_x (-1)^{f(x)} |x\rangle$ makes the terms add or cancel according to the pattern of signs, converting an invisible phase pattern into a measurable amplitude. Every algorithm in Chapters 3 and 6, Grover search [15] included, has this shape: mark with phases, then interfere.

2.5.5 Building oracles for a given f

We now construct $O_f^\pm$ explicitly. The recipe is uniform: to mark a basis state $|s\rangle$, use a multi-controlled gate whose controls are filled dots where s has a 1 and open circles where s has a 0.

One qubit

There are four functions $f : \{0, 1\} \rightarrow \{0, 1\}$.



Figure 2.13: Phase oracles for the four one-bit functions. Left to right: $f \equiv 0$ (do nothing); $f \equiv 1$ (a global -1 , unobservable); $f(x) = x$; and $f(x) = 1 \oplus x$. The last two are the two *balanced* functions.

The two constant functions differ only by a global phase and are therefore physically indistinguishable. That is consistent with Deutsch's algorithm [9], which distinguishes constant from balanced but never tells you *which* constant.

Two qubits

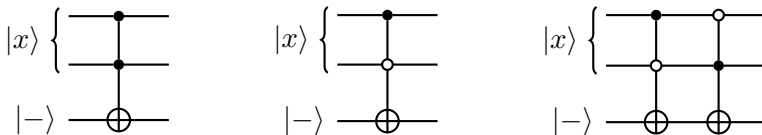


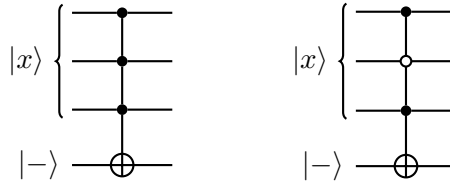
Figure 2.14: Two-qubit phase oracles. Left: marks $|11\rangle$ only. Middle: marks $|10\rangle$ only. Right: the middle circuit followed by a second gate that marks $|01\rangle$, so it marks $|10\rangle$ and $|01\rangle$.

Table 2.5: Action of the three circuits of Fig. 2.14 on the computational basis. All three verified numerically.

$ x\rangle$	marks $ 11\rangle$	marks $ 10\rangle$	marks $ 10\rangle, 01\rangle$
$ 00\rangle$	$+ 00\rangle$	$+ 00\rangle$	$+ 00\rangle$
$ 01\rangle$	$+ 01\rangle$	$+ 01\rangle$	$- 01\rangle$
$ 10\rangle$	$+ 10\rangle$	$- 10\rangle$	$- 10\rangle$
$ 11\rangle$	$- 11\rangle$	$+ 11\rangle$	$+ 11\rangle$

Because each multi-controlled gate marks exactly one basis state, and the gates commute (they are all diagonal in the computational basis), *marking any subset S costs one multi-controlled gate per element of S* . That is the whole construction. In gate terms: on n input qubits each of those gates is a $C^n(X)$, which by Section 2.3.4 costs $O(n)$ Toffolis with $O(n)$ ancillas that every gate can reuse, so the oracle for S costs $O(n|S|)$ Toffolis. The whole oracle counts as *one* query, however many gates it contains.

Three qubits and beyond

**Figure 2.15:** Three-qubit phase oracles marking $|111\rangle$ (left) and $|101\rangle$ (right). The control pattern is read directly off the marked string.

Exercise 2.4

Draw a three-qubit phase oracle that marks both $|011\rangle$ and $|001\rangle$.

Answer: two multi-controlled gates in series, with control patterns $(\circ, \bullet, \bullet)$ and $(\circ, \circ, \bullet)$ respectively. Equivalently, since the two strings differ only in the middle bit, a single gate with controls $(\circ, \bullet)$ on qubits 0 and 2 does the same job with one fewer gate. That is a small instance of the logic minimization that oracle compilers perform.

2.5.6 Garbage and uncomputation

A realistic oracle circuit needs scratch space. Computing $f(x)$ may require intermediate results held in ancillas, and at the end of the computation those

ancillas hold *garbage* $g(x)$:

$$|x\rangle |0\rangle |0\rangle \mapsto |x\rangle |f(x)\rangle |g(x)\rangle.$$

This is a serious problem, not an untidiness. Feed in a superposition:

$$\frac{1}{\sqrt{2^n}} \sum_x |x\rangle |f(x)\rangle |g(x)\rangle.$$

The garbage register is now entangled with $|x\rangle$. If g is different for different x , those branches can no longer interfere. The garbage acts as a permanent record of which branch was taken, exactly as a measurement would. Interference is the only reason quantum algorithms work, so garbage destroys the speed-up.

The fix is the *uncomputation trick* (Bennett):

- (i) Compute: $|x\rangle |0\rangle |0\rangle \mapsto |x\rangle |f(x)\rangle |g(x)\rangle$.
- (ii) Copy the answer to a fresh output qubit with a CNOT.
- (iii) Run the computation *backwards* ($U^\dagger$), erasing both the working copy of $f(x)$ and all garbage.

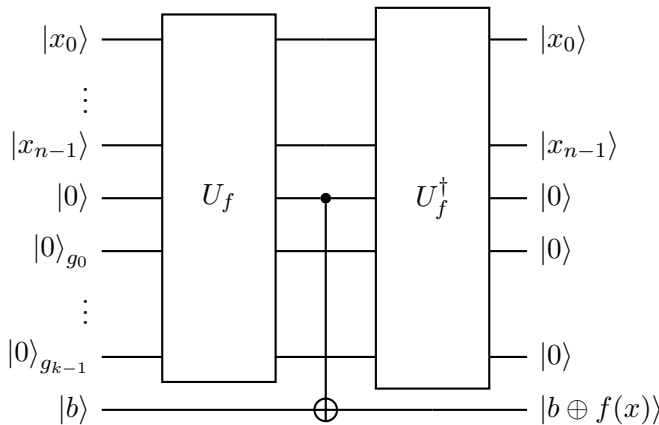


Figure 2.16: Uncomputation. The top n wires hold the input x . The single wire below them holds $f(x)$ and the k wires $g_0, \dots, g_{k-1}$ hold garbage; running $U_f^\dagger$ after copying the answer returns all of them to $|0\rangle$, leaving a clean oracle of the form (2.12). The cost is a factor of two in gate count, plus one CNOT and one output qubit.

This is why the mirror-image Toffolis in Fig. 2.7 were not optional, and why almost every circuit in this book is symmetric about its middle. Whenever you see that shape, it is uncomputation.

Rule of thumb

Any qubit that is not part of the final answer must be returned to $|0\rangle$ before the circuit ends. More precisely, it must end in a state independent of x . If you cannot argue that it does, the circuit is probably wrong.

2.6 Summary

- Circuits read left to right; matrix products read right to left.
- $C(U) = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U$ is built from two CNOTs and the ABC decomposition, with the phase supplied by $P(\alpha)$ on the *control* wire.
- $P(\theta)$ and $R_z(\theta)$ differ by a global phase; their *controlled* versions are different gates.
- $C^2(U)$ needs only a square root V with $V^2 = U$; for the Toffoli gate $V = \sqrt{X} = \frac{1+i}{2}(I - iX)$.
- $C^n(U)$ costs $O(n)$ Toffolis with $n - 1$ ancillas, which must be uncomputed.
- Any $d \times d$ unitary is a product of $O(d^2)$ two-level unitaries; each two-level unitary becomes a circuit via a Gray code; total $O(n^2 4^n)$ CNOT and single-qubit gates on n qubits, which is exponential, so useful algorithms must exploit structure.
- $\{H, T, \text{CNOT}\}$ is universal. $HTH \propto R_x(\pi/4)$ and $THTH \propto R_{\hat{m}}(\theta)$ with $\theta/2\pi$ irrational, giving a dense set of rotations. Solovay–Kitaev: one gate to accuracy ϵ_1 costs $O(\log^c(1/\epsilon_1))$ gates from the finite set.
- An oracle embeds f reversibly as $|x\rangle|b\rangle \mapsto |x\rangle|b \oplus f(x)\rangle$.
- Ancilla $|0\rangle$ gives a bit oracle; ancilla $|-\rangle$ gives a phase oracle $|x\rangle \mapsto (-1)^{f(x)}|x\rangle$. Same unitary, different preparation.
- Phase kickback: $C(U)$ with $U|u\rangle = e^{i\phi}|u\rangle$ moves $e^{i\phi}$ onto the control.
- Mark a basis state with a multi-controlled gate; filled dots for 1s, open circles for 0s. Mark a set by composing one gate per element.
- Garbage entangled with the input destroys interference. Uncompute it.

2.7 End-of-Chapter Problems

2.1 Write out the 4×4 matrix of $C(H)$ and verify it is unitary.

2.2 Show that $CZ = (I \otimes H) \text{CNOT} (I \otimes H)$, and explain why this makes CZ symmetric under exchange of its two qubits while CNOT is not.

- 2.3** Find A , B , C and α explicitly for $U = H$, and check $ABC = I$ and $e^{i\alpha}AXBXC = H$.
- 2.4** Verify Table 2.4 by applying the circuit of Fig. 2.6 to each of the eight computational basis states, taking $V = \sqrt{X}$.
- 2.5** Show that $C\text{-}P(\theta)$ is symmetric in its two qubits, but $C\text{-}R_z(\theta)$ is not. (This is the distinction that the box in Section 2.2 warns about.)
- 2.6** Build a $C^2(Z)$ gate from Toffoli gates and Hadamards. How many two-qubit gates does your construction use?
- 2.7** Prove that the Toffoli gate is universal for classical reversible computation by showing how to build AND, NOT and FANOUT from it. What must the ancilla inputs be in each case?
- 2.8** For the two-level unitary of (2.6), write down a different Gray code from 111 to 000 and draw the corresponding circuit. Does the gate count change?
- 2.9** Compute θ in (2.10) to four decimal places, and verify numerically that $\cos^2(\theta/2) + \sin^2(\theta/2) = 1$.
- 2.10** Using (2.11), how many gates does the counting argument say are needed to prepare a generic 10-qubit state to accuracy $\epsilon = 10^{-3}$? Then count the number of ϵ -patches the argument had to cover, and compare that with the number of atoms in the observable universe.
- 2.11** Let $f : \{0, 1\}^2 \rightarrow \{0, 1\}$ be $f(x) = x_0 \wedge x_1$. Draw both the bit oracle and the phase oracle, and confirm they are the same circuit with different ancilla preparation.
- 2.12** Let $f(x) = x \cdot s \bmod 2$ for a fixed string s (the Bernstein–Vazirani function). Show that the phase oracle for f is a product of CZ gates, and identify which ones. *This is the entire content of the Bernstein–Vazirani algorithm; you will meet it again in Chapter 3.*
- 2.13** A circuit computes $f(x)$ into an ancilla but leaves one garbage qubit in the state $|x_0\rangle$. Show explicitly, for $n = 1$, that the interference in Deutsch’s algorithm fails, by computing the final state with and without the garbage qubit.
- 2.14** Show that marking the set $S = \{|01\rangle, |10\rangle, |11\rangle\}$ can be done with fewer than three multi-controlled gates. *Hint:* mark the complement and use a global phase.

Research Problems and Projects

The problems above are exercises with known answers. The three projects below are open-ended. Each is sized for several weeks of work by a strong MS or beginning PhD student, and each ends in a concrete artifact: a measured curve, a resource table, or a piece of working software with a benchmark behind it. Where a project touches something genuinely unsettled, that is said plainly; where the work is engineering rather than discovery, that is said too.

R2.1 Measure the Solovay–Kitaev overhead you are actually paying.

Implement the recursive Solovay–Kitaev compiler for the gate set $\{H, T\}$. Build the base case by exhaustive enumeration of all gate words up to some length ℓ , then recurse using the group-commutator balancing step. Compile a sample of at least 10^3 Haar-random single-qubit unitaries at target accuracies $\epsilon_1 = 10^{-2}, 10^{-3}, \dots, 10^{-6}$, recording for each run the word length, the T -count, and the achieved error measured with the operator norm of (2.8). *Deliverable:* a log-log plot of gate count against $\log(1/\epsilon_1)$ with a fitted exponent c , to be compared against the $c \approx 2$ quoted in the Solovay–Kitaev box of Section 2.4.3 [18, 24], plus a table of the fitted c as a function of base-case net size ℓ . Then compare against exact number-theoretic synthesis of z -rotations, which achieves T -count linear in $\log(1/\epsilon_1)$, and report the accuracy at which the recursive algorithm stops being competitive. The theorem itself is settled; what is not settled, and what your measurement is for, is the constant in front and how it degrades for structured rather than random targets.

R2.2 How loose is the $O(n^2 4^n)$ bound? Section 2.4.2 derives the upper bound (2.7) of $O(n^2 4^n)$ elementary gates for a generic n -qubit unitary. The counting argument of Section 2.4.4 is a lower bound of a weaker kind: it covers the *state* space, of real dimension (2.11), and so yields only $\Omega(2^n)$. Start by redoing that argument over the unitary group itself, whose real dimension is 4^n , to obtain a matching $\Omega(4^n)$ gate count. Then locate in the literature the best published upper bound on the CNOT count for a generic n -qubit unitary and the best published lower bound, and restate each in the notation of this chapter, keeping every constant. *Deliverable:* a short review paper with one table giving, for $n = 1, \dots, 10$, the chapter’s bound, the best published upper bound, your dimension-counting lower bound, and the ratio between the last two; plus an explicit statement of what part of the gap is an artifact of the proof technique and what part is genuinely unknown. The exponent is settled at $\Theta(4^n)$; the constant factor is not, and a careful account of exactly where the remaining factor lives is a useful contribution on its own. Nielsen and Chuang [24] and Kitaev, Shen and Vyalıy [18] are the right starting points for the classical statements.

R2.3 Write an oracle compiler and benchmark it honestly. Build a tool

that takes a Boolean function, given either as a truth table or as a formula, and emits a reversible circuit implementing the bit oracle (2.12), with every scratch qubit returned to $|0\rangle$ by the uncomputation of Section 2.5.6. Support both the ancilla-heavy and the ancilla-free multi-controlled constructions of Section 2.3.4, so that the trade-off between ancilla count and gate count becomes a knob rather than a fixed choice. Benchmark on three families: random k -CNF formulas, the inner-product function $f(x) = x \cdot s \bmod 2$ of Bernstein and Vazirani [3], and the single-marked-string oracle used by Grover search [15]. *Deliverable:* a resource table reporting ancilla count, Toffoli count, T -count and depth for each family at $n = 4, \dots, 16$, against a stock library compiler as the baseline; a plot of ancilla count against Toffoli count showing the trade-off; and a verification harness that simulates all 2^n inputs for $n \leq 14$ and checks that every scratch qubit really ends in $|0\rangle$, which is the rule of thumb at the end of Section 2.5.6 turned into a test. This project is engineering rather than an open problem. Its value is a reproducible baseline, and the interesting result is usually the gap between what the textbook cost model predicts and what the compiler actually emits.

Chapter 3

Foundational Quantum Algorithms

This chapter puts the machinery of Chapters 1 and 2 to work. We build the entangled resource states that quantum protocols consume, and use them to teleport an unknown qubit. We then study the three algorithms that established, for the first time, that quantum computers can outperform classical ones on well-posed problems.

The three algorithms share a single architecture. I want to name it before we start, because everything else is variation:

- (i) **Superpose.** Hadamards on $|0\rangle^{\otimes n}$ create a uniform superposition over all 2^n inputs.
- (ii) **Mark.** One oracle query attaches a phase $(-1)^{f(x)}$ to every branch simultaneously.
- (iii) **Interfere.** A second layer of Hadamards makes the branches add or cancel, concentrating amplitude on the answer.
- (iv) **Measure.**

Step (iii) is where the power lies. Step (ii) alone gives nothing. If you measure right after the oracle, you get one random $(x, f(x))$ pair, exactly what one classical query would give. It is the interference that converts a global property of f into a measurable outcome.

Simon's algorithm varies step (ii). Its oracle writes $f(x)$ into a second register rather than into a phase, and that register is then measured or discarded (§3.5). Steps (i), (iii) and (iv) are unchanged.

What you should be able to do after this chapter

- Prepare and measure in the Bell basis, and explain why the measurement circuit is the preparation circuit reversed.
- Trace the teleportation protocol algebraically and say precisely what is and is not transmitted.
- State the Deutsch–Jozsa, Bernstein–Vazirani and Simon problems, and derive the query counts for each.
- Explain why all three circuits need an extra register beyond the input, and where it goes.
- Set up and solve the linear system in Simon’s algorithm.
- Build a measurement-error calibration matrix and apply it.

Table 3.1: Query complexity of the three algorithms. Every entry counts calls to the oracle, and n is the number of input bits. The second row is important and often omitted. If a small probability of error is allowed, Deutsch–Jozsa gives *no* asymptotic advantage, because a few random classical queries detect a balanced function with high probability. Simon’s separation survives bounded error. That is why Simon’s algorithm, and not Deutsch–Jozsa, is the direct ancestor of Shor’s algorithm.

Problem	Classical queries	Quantum queries	Separation
Deutsch–Jozsa	$2^{n-1} + 1$ (exact)	1	exponential (exact)
Deutsch–Jozsa	$O(1)$ (bounded error)	1	none
Bernstein–Vazirani	n	1	linear
Simon	$\Theta(2^{n/2})$	$O(n)$	exponential

3.1 Preparation of Bell States

The four Bell states of Chapter 1 are the standard entangled resource. All four are produced by the same two-gate circuit, differing only in the input.

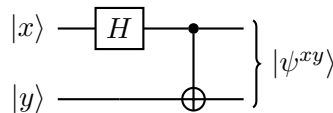


Figure 3.1: Bell-state preparation. A Hadamard on the first qubit followed by a CNOT.

Trace the input $|00\rangle$. The Hadamard gives $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$, and

the CNOT flips the second qubit only in the $|1\rangle$ branch:

$$|00\rangle \mapsto \frac{|00\rangle + |11\rangle}{\sqrt{2}} = |\psi^{00}\rangle.$$

The other three inputs give the other three Bell states. All four verified numerically against the labeling of Chapter 1:

Table 3.2: Action of Fig. 3.1. The first input bit x controls the sign; the second bit y controls whether the halves agree or disagree.

Input	Output
$ 00\rangle$	$ \psi^{00}\rangle = \frac{1}{\sqrt{2}}(00\rangle + 11\rangle)$
$ 01\rangle$	$ \psi^{01}\rangle = \frac{1}{\sqrt{2}}(01\rangle + 10\rangle)$
$ 10\rangle$	$ \psi^{10}\rangle = \frac{1}{\sqrt{2}}(00\rangle - 11\rangle)$
$ 11\rangle$	$ \psi^{11}\rangle = \frac{1}{\sqrt{2}}(01\rangle - 10\rangle)$

3.1.1 Measurement in the Bell basis

Measurement hardware only ever measures in the computational basis. To measure in some other orthonormal basis $\{|b_i\rangle\}$, we apply the unitary that maps $|b_i\rangle \mapsto |i\rangle$ and then measure normally. Since Fig. 3.1 maps $|ij\rangle \mapsto |\psi^{ij}\rangle$, the circuit we want is its *inverse*, the same gates in reverse order.

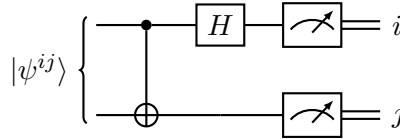


Figure 3.2: Bell-basis measurement: Fig. 3.1 run backwards. Verified to send $|\psi^{ij}\rangle \mapsto |ij\rangle$ for all four Bell states.

Concretely, write a two-qubit state in the computational basis as

$$|\psi\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle.$$

The same state expanded in the Bell basis reads

$$|\psi\rangle = a'|\psi^{00}\rangle + b'|\psi^{01}\rangle + c'|\psi^{10}\rangle + d'|\psi^{11}\rangle. \quad (3.1)$$

Measuring directly returns outcome probabilities $|a|^2, \dots$ in the computational basis, which is not what we want. Applying Fig. 3.2 first converts (3.1) into

$$a'|00\rangle + b'|01\rangle + c'|10\rangle + d'|11\rangle,$$

so an ordinary measurement now returns $|a'|^2, |b'|^2, |c'|^2, |d'|^2$.

The general rule

To measure in the basis $\{|b_i\rangle\}$, find the unitary B with $B|i\rangle = |b_i\rangle$, apply $B^\dagger$, then measure in the computational basis. Every “measurement in the X basis”, “measurement in the Bell basis” and so on in this book is an instance of this one rule. It is also why the last thing before a measurement, in almost every algorithm, is a layer of Hadamards.

3.1.2 Measurement in the Hadamard basis

The same rule with $B = H^{\otimes n}$. Since $H^\dagger = H$, the circuit is just Hadamards.

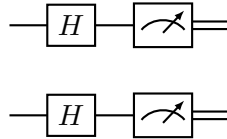


Figure 3.3: Measurement in the $\{|+\rangle, |-\rangle\}$ basis on two qubits. The circuit maps $|++\rangle \mapsto |00\rangle$, $|+-\rangle \mapsto |01\rangle$, $|-+\rangle \mapsto |10\rangle$, $|--\rangle \mapsto |11\rangle$.

Exercise 3.1

Write the state $|\psi\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)$ in the Hadamard basis, and predict the outcome probabilities of Fig. 3.3.

Answer: $|\psi\rangle = \frac{1}{2}(|++\rangle + |+-\rangle + |-+\rangle - |--\rangle)$, so all four outcomes have probability $\frac{1}{4}$.

3.2 Quantum Teleportation

Alice holds an unknown qubit

$$|\phi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1, \quad (3.2)$$

and wants Bob to end up holding it. She may not measure it, since that would collapse it. She cannot copy it either, by the no-cloning theorem. What she *can* do is consume a shared entangled pair and send two classical bits. You will find a standard textbook treatment of the protocol below in [24].

What teleportation is and is not

Teleportation transfers an unknown *quantum state*. It does not transfer matter, and it does not transmit information faster than light: Bob's qubit is useless until Alice's two classical bits arrive, and those travel no faster than light. It also *destroys* the original. Alice's copy is gone by the end, as no-cloning requires. The resource accounting is exact: one shared entangled pair plus two classical bits buys the transmission of one qubit.

3.2.1 The protocol

Step 0. Alice and Bob prepare an EPR pair using Fig. 3.1 and separate, Alice keeping qubit A and Bob qubit B :

$$|\psi^{00}\rangle = \frac{|0\rangle_A |0\rangle_B + |1\rangle_A |1\rangle_B}{\sqrt{2}}. \quad (3.3)$$

Step 1. The full three-qubit state is the product of (3.2) with (3.3):

$$\begin{aligned} |\phi\rangle \otimes |\psi^{00}\rangle &= [\alpha |0\rangle + \beta |1\rangle] \otimes \frac{|00\rangle + |11\rangle}{\sqrt{2}} \\ &= \frac{\alpha |000\rangle + \alpha |011\rangle + \beta |100\rangle + \beta |111\rangle}{\sqrt{2}}. \end{aligned}$$

Step 2. Re-group the first two qubits into Bell states. Write each term twice with a compensating factor of $\frac{1}{2}$, then add and subtract matching terms:

$$\begin{aligned} |\phi\rangle \otimes |\psi^{00}\rangle &= \frac{1}{2\sqrt{2}} \left[\alpha |000\rangle + \alpha |110\rangle + \alpha |011\rangle + \alpha |101\rangle \right. \\ &\quad + \alpha |000\rangle - \alpha |110\rangle + \alpha |011\rangle - \alpha |101\rangle \\ &\quad + \beta |100\rangle + \beta |010\rangle + \beta |111\rangle + \beta |001\rangle \\ &\quad \left. + \beta |100\rangle - \beta |010\rangle + \beta |111\rangle - \beta |001\rangle \right]. \end{aligned}$$

Factoring the last qubit out of each group,

$$\begin{aligned} |\phi\rangle \otimes |\psi^{00}\rangle &= \frac{1}{2\sqrt{2}} \left[(|00\rangle + |11\rangle) (\alpha |0\rangle + \beta |1\rangle) \right. \\ &\quad + (|00\rangle - |11\rangle) (\alpha |0\rangle - \beta |1\rangle) \\ &\quad + (|01\rangle + |10\rangle) (\alpha |1\rangle + \beta |0\rangle) \\ &\quad \left. + (|01\rangle - |10\rangle) (\alpha |1\rangle - \beta |0\rangle) \right]. \end{aligned}$$

Each bracket on the left is $\sqrt{2}$ times a Bell state, and each bracket on the right is a Pauli operator acting on $|\phi\rangle$:

$$\begin{aligned}\alpha|0\rangle + \beta|1\rangle &= I|\phi\rangle, & \alpha|0\rangle - \beta|1\rangle &= Z|\phi\rangle, \\ \alpha|1\rangle + \beta|0\rangle &= X|\phi\rangle, & \alpha|1\rangle - \beta|0\rangle &= XZ|\phi\rangle.\end{aligned}$$

Therefore

$$|\phi\rangle \otimes |\psi^{00}\rangle = \frac{1}{2} \left[|\psi^{00}\rangle \otimes I|\phi\rangle + |\psi^{10}\rangle \otimes Z|\phi\rangle + |\psi^{01}\rangle \otimes X|\phi\rangle + |\psi^{11}\rangle \otimes XZ|\phi\rangle \right]. \quad (3.4)$$

This identity is exact, and I have verified it numerically. It is the whole protocol. Nothing has happened physically yet. We have only rewritten the same state in a different basis.

Step 3. Alice applies Fig. 3.2 to her two qubits, converting the Bell labels to computational ones:

$$\mapsto \frac{1}{2} \left[|00\rangle \otimes |\phi\rangle + |10\rangle \otimes Z|\phi\rangle + |01\rangle \otimes X|\phi\rangle + |11\rangle \otimes XZ|\phi\rangle \right],$$

and measures, obtaining one of the four outcomes with probability $\frac{1}{4}$ each. The outcome is independent of α, β . Alice learns nothing about the state she is sending, as she must not.

Step 4. Alice sends her two bits to Bob over a classical channel. Bob applies the *inverse* of the corresponding operator:

Table 3.3: Teleportation corrections. All four verified numerically to return exactly $|\phi\rangle$.

Alice's outcome	Bob's qubit is	Bob applies
00	$ \phi\rangle$	nothing
01	$X \phi\rangle$	X
10	$Z \phi\rangle$	Z
11	$XZ \phi\rangle$	ZX (i.e. X first, then Z)

Order matters in the last row

Bob holds $XZ|\phi\rangle$, so he must apply $(XZ)^{-1} = Z^{-1}X^{-1} = ZX$. Since X and Z *anticommute*, $ZX = -XZ$, so the order is not free.

In circuit terms this means X first, then Z , reading left to right. Applying “ XZ ” as a circuit instead gives $XZXZ|\phi\rangle = -|\phi\rangle$, which happens to be correct up to a global phase, so the error is invisible here. It is *not* invisible if the teleported qubit is entangled with something else, where a sign becomes a relative phase. Write ZX .

3.2.2 The teleportation circuit

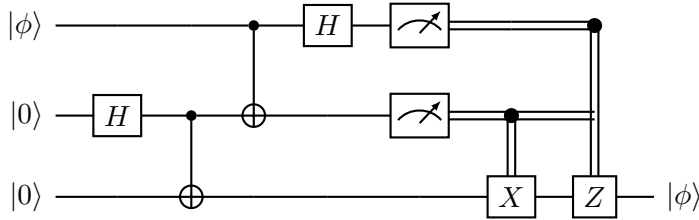


Figure 3.4: Quantum teleportation. The first two gates, the H on the middle wire and the CNOT below it, create the EPR pair of (3.3): Alice keeps the middle wire and Bob the bottom one. Alice also holds the unknown qubit $|\phi\rangle$ on the top wire. She then applies the Bell measurement of Fig. 3.2 to her two wires and sends the two classical bits (double lines) that condition Bob’s corrections. The top wire’s bit controls Z and the middle wire’s bit controls X , matching Table 3.3. The correction order is X then Z .

LOCC

Teleportation is the standard example of *local operations and classical communication* (LOCC): each party acts only on qubits they hold, and they exchange only classical bits. Entanglement is consumed as a resource, not created at a distance. Without the classical message, Bob’s qubit is in the maximally mixed state $I/2$, carrying no information at all. That is why no faster-than-light signaling occurs.

Exercise 3.2

Work out the teleportation protocol when Alice and Bob share $|\psi^{01}\rangle$, $|\psi^{10}\rangle$ or $|\psi^{11}\rangle$ instead of $|\psi^{00}\rangle$. What changes?

Answer: only the lookup table. Each shared Bell state permutes which outcome calls for which correction; the protocol structure is identical. This is why teleportation still works if the entangled pair has been corrupted by a known Pauli error.

3.3 The Deutsch–Jozsa Algorithm

3.3.1 The problem

We are given a black box computing $f : \{0, 1\}^n \rightarrow \{0, 1\}$, with a promise. The function f is either *constant*, meaning the same value on every input, or *balanced*, meaning value 0 on exactly half the inputs and 1 on the other half. Determine which. The problem and its one-query quantum solution are due to Deutsch and

Jozsa [9].

$$f \text{ is } \begin{cases} \text{constant: } f(x) = 0 \ \forall x, \quad \text{or} \quad f(x) = 1 \ \forall x, \\ \text{balanced: } f(x) = 0 \text{ for } 2^{n-1} \text{ inputs, } f(x) = 1 \text{ for the rest.} \end{cases}$$

The promise matters: most functions are neither, and the algorithm gives no guarantee on those.

3.3.2 Classical cost

We count calls to the oracle, and n is the number of input bits, so there are 2^n inputs in all. In the best case two queries suffice: any two different outputs prove the function is balanced. In the worst case an adversary returns the same value for the first 2^{n-1} queries, which is exactly half of the inputs. At that point both answers are still possible. The function could be constant, or it could be balanced with all its other values sitting on the half we have not asked about. Only query number $2^{n-1} + 1$ settles it. So a *deterministic* classical algorithm needs

$$2^{n-1} + 1 \quad \text{queries in the worst case.}$$

In plain words, you must look at half the inputs and then one more, and that count doubles every time n grows by one bit.

The honest comparison

That exponential bound is for *exact* classical computation. If we accept a small error probability, classical does far better: query k random inputs, and if all agree, guess “constant”. For a balanced function the first query fixes a value and each later query matches it with probability $\frac{1}{2}$, so all k agree with probability only $2^{-(k-1)}$. Thus $k = 21$ queries give error below one in a million, *independent of n* .

So Deutsch–Jozsa gives an exponential advantage over exact classical algorithms, and **no asymptotic advantage at all** over randomized ones. Its importance is historical and pedagogical. It was the first clean proof that quantum circuits can do something classical circuits cannot, and its structure is the template for everything that follows. Simon’s algorithm (§3.5) is the one whose separation survives randomization.

3.3.3 The quantum algorithm

We use the *phase oracle* of Chapter 2: the ancilla is prepared in $|-\rangle$, so that

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle. \quad (3.5)$$

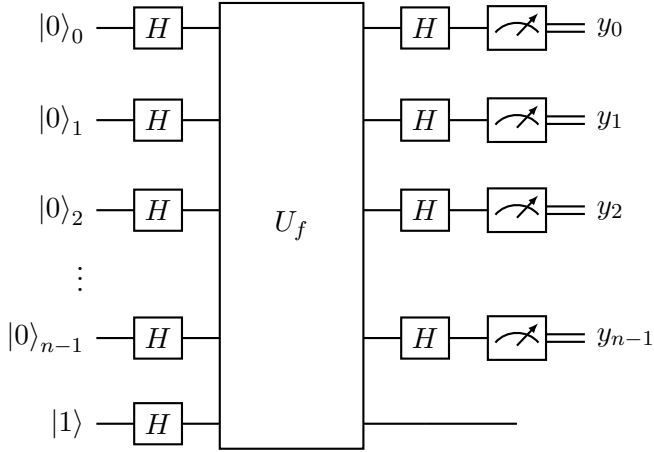


Figure 3.5: The Deutsch–Jozsa algorithm. The bottom wire is the ancilla: it starts in $|1\rangle$ and the Hadamard turns it into $|-\rangle$, which is what makes U_f act as a phase oracle. It is never measured. The function is constant if and only if every y_i is 0. I have drawn the register as individual wires rather than one bundle, so that each qubit visibly gets its own Hadamard and its own measurement, and the ancilla is visibly a separate physical qubit.

Here U_f is the standard bit oracle of Chapter 2,

$$U_f |x\rangle |b\rangle = |x\rangle |b \oplus f(x)\rangle. \quad (3.6)$$

Step-by-step

(1) Initialize.

$$|\psi_0\rangle = |0\rangle^{\otimes n} \otimes |1\rangle. \quad (3.7)$$

(2) Hadamards. On the register,

$$\begin{aligned} |\psi_1\rangle &= H^{\otimes n} |0\rangle^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)^{\otimes n} \\ &= \frac{1}{\sqrt{2^n}} \left(|00 \cdots 0\rangle + |00 \cdots 1\rangle + \cdots + |11 \cdots 1\rangle \right), \end{aligned} \quad (3.8)$$

and on the ancilla, $H|1\rangle = |-\rangle$. Writing $N = 2^n$ and reading each bit string as a binary integer,¹

$$|\psi_1\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle \otimes |-\rangle. \quad (3.9)$$

¹Following the book's convention, the leftmost symbol is qubit 0 and carries the most significant bit. If $|j\rangle = |j_0 j_1 \cdots j_{n-1}\rangle$ then $j = j_0 2^{n-1} + j_1 2^{n-2} + \cdots + j_{n-1}$. For $n = 3$, $|111\rangle = |7\rangle$.

(3) **Query.** By (3.5),

$$|\psi_2\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} (-1)^{f(x)} |x\rangle \otimes |-\rangle. \quad (3.10)$$

The ancilla has factored out and stays out; we drop it from now on.

The ancilla is not optional

A common drawing of Figure 3.5 shows only the n -qubit register with U_f acting on it and no ancilla wire at all. That is inconsistent with the mathematics: U_f in (3.6) is a *two-register* unitary, and the phase $(-1)^{f(x)}$ in (3.10) appears only because the ancilla sits in the eigenstate $|-\rangle$ of X . Without the ancilla wire, the phase has nowhere to come from.

(4) **Interfere.** Using $H^{\otimes n} |x\rangle = \frac{1}{\sqrt{N}} \sum_k (-1)^{x \cdot k} |k\rangle$, where $x \cdot k = \bigoplus_i x_i k_i$ is the bitwise inner product mod 2,

$$|\psi_3\rangle = \frac{1}{N} \sum_{x=0}^{N-1} \sum_{k=0}^{N-1} (-1)^{f(x)+x \cdot k} |k\rangle = \sum_{k=0}^{N-1} C_k |k\rangle, \quad C_k = \frac{1}{N} \sum_{x=0}^{N-1} (-1)^{f(x)+x \cdot k}. \quad (3.11)$$

Reading the answer

Everything hinges on the amplitude of the all-zeros outcome. Setting $k = 0$ kills the inner product, leaving

$$C_0 = \frac{1}{N} \sum_{x=0}^{N-1} (-1)^{f(x)}. \quad (3.12)$$

Keep the sum

In my experience the usual slip is to write $C_0 = \frac{1}{N} (-1)^{f(x)}$ after setting $k = 0$. That expression is meaningless, because x is a bound summation index and cannot appear outside the sum. The whole argument depends on the sum in (3.12) surviving, because it is the *cancellation* between terms that distinguishes the two cases.

Constant f . Every term is the same sign, so

$$C_0 = \frac{1}{N} \sum_{x=0}^{N-1} (\pm 1) = \pm 1 \quad \implies \quad |C_0|^2 = 1.$$

Since $|\psi_3\rangle$ is normalized, every other amplitude is zero: the measurement returns $00\cdots 0$ with certainty.

Balanced f . Exactly half the terms are $+1$ and half are -1 , so they cancel:

$$C_0 = \frac{1}{N} \left(\underbrace{+1 \cdots +1}_{N/2} \underbrace{-1 \cdots -1}_{N/2} \right) = 0 \quad \implies \quad |C_0|^2 = 0.$$

The all-zeros outcome *never* occurs.

The decision rule

Measure all n register qubits.

- All outcomes 0 $\implies f$ is **constant**.
- Any outcome 1 $\implies f$ is **balanced**.

This is exact, not probabilistic, and uses exactly one oracle query.

Example 3.1

Work through $n = 1$ (Deutsch’s original problem) explicitly.

Solution. Here $N = 2$ and $|\psi_1\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. After the oracle,

$$|\psi_2\rangle = \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} = (-1)^{f(0)} \frac{|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle}{\sqrt{2}}.$$

Ignoring the global phase, this is $|+\rangle$ if $f(0) = f(1)$ and $|-\rangle$ otherwise. A final Hadamard sends $|+\rangle \mapsto |0\rangle$ and $|-\rangle \mapsto |1\rangle$, so the measured bit is exactly $f(0) \oplus f(1)$: 0 for constant, 1 for balanced.

Look at what the algorithm computed: a *global* property $f(0) \oplus f(1)$, using one query, without ever learning $f(0)$ or $f(1)$ individually. Global information comes cheaply, local information not at all. That trade is the recurring theme of the whole subject.

Example 3.2

Verify the decision rule numerically for $n = 3$.

Solution. Computing $|C_0|^2$ gives 1.0000 for $f(x) = 0$ and for $f(x) = 1$, both constant, and 0.0000 for the parity function $f(x) = x_0 \oplus x_1 \oplus x_2$ and for the first-bit function $f(x) = x_0$, both balanced. The two constant functions are indistinguishable from one another, since they differ by a global phase -1 : the algorithm answers “constant or balanced?”, never “which constant?”.

Exercise 3.3

Repeat the derivation with the register initialized to $|1\rangle^{\otimes n}$ instead of $|0\rangle^{\otimes n}$. What is the decision rule now?

Answer: $H^{\otimes n} |1\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_x (-1)^{x \cdot \mathbf{1}} |x\rangle$, so the marked outcome moves from $|0 \cdots 0\rangle$ to $|1 \cdots 1\rangle$: the function is constant iff every measured bit is 1.

3.4 The Bernstein–Vazirani Algorithm

3.4.1 The problem

The oracle now computes the bitwise inner product of the input with a fixed but unknown *secret string* s :

$$f(x) = s \cdot x \bmod 2 = s_0 x_0 \oplus s_1 x_1 \oplus \cdots \oplus s_{n-1} x_{n-1}. \quad (3.13)$$

Find s . The problem and the one-query algorithm below are due to Bernstein and Vazirani [3].

Example 3.3

For $n = 3$ and $s = 101$:

Table 3.4: The Bernstein–Vazirani oracle for $s = 101$: each entry is the parity of $x \wedge s$.

x	000	001	010	011	100	101	110	111
$f(x) = s \cdot x$	0	1	0	1	1	0	1	0

The function f here is always balanced (unless $s = 0$), so Deutsch–Jozsa would answer “balanced” and stop. Bernstein–Vazirani extracts strictly more information from the same single query.

3.4.2 Classical cost

We count oracle queries again, and n is the length of s . Query $x = 10 \cdots 0$, the string whose only 1 sits in position 0: only the term $s_0 x_0$ survives in (3.13), so the answer is s_0 . Query $x = 010 \cdots 0$: the answer is s_1 . Continuing, n queries with the n strings of weight one reveal all n bits, and the last of them, $00 \cdots 01$, reveals s_{n-1} . Fewer cannot suffice, because each classical query returns one bit and s contains n bits of information. So the classical cost is exactly n queries, one per bit of the secret.

3.4.3 The quantum algorithm

The circuit is identical to Deutsch–Jozsa. Only the analysis differs.

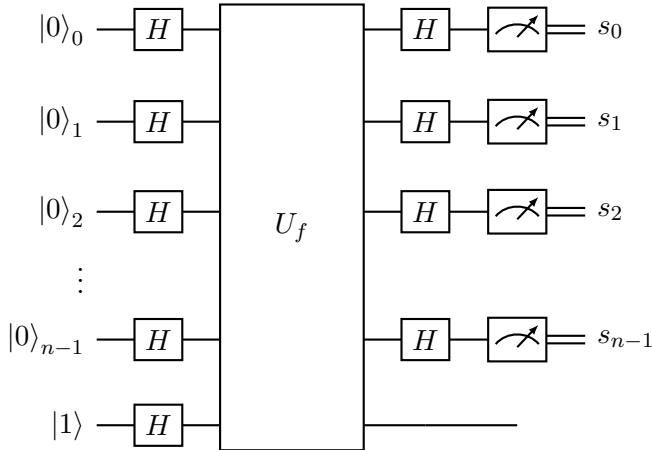


Figure 3.6: The Bernstein–Vazirani algorithm. The measured bit string *is* the secret s , obtained with certainty from one query.

Steps (1)–(3) are as before, giving

$$|\psi_2\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} (-1)^{s \cdot x} |x\rangle. \quad (3.14)$$

Applying the final Hadamards,

$$|\psi_3\rangle = \frac{1}{N} \sum_{k=0}^{N-1} \left(\sum_{x=0}^{N-1} (-1)^{s \cdot x} (-1)^{x \cdot k} \right) |k\rangle = \sum_{k=0}^{N-1} C_k |k\rangle, \quad (3.15)$$

where, exactly as in (3.11), C_k is the amplitude of $|k\rangle$:

$$C_k = \frac{1}{N} \sum_{x=0}^{N-1} (-1)^{x \cdot (s \oplus k)}. \quad (3.16)$$

The exponents here are bits, so the two signs combine, and the resulting exponent collapses termwise:

$$(-1)^{s \cdot x} (-1)^{x \cdot k} = (-1)^{s \cdot x \oplus x \cdot k}, \quad s \cdot x \oplus x \cdot k = \bigoplus_i (s_i \oplus k_i) x_i = x \cdot (s \oplus k).$$

Case $k = s$. Then $s \oplus k = 0$, every exponent vanishes, and

$$C_s = \frac{1}{N} \sum_{x=0}^{N-1} (-1)^0 = \frac{N}{N} = 1.$$

Case $k \neq s$. Then $s \oplus k \neq 0$, so it has at least one bit set. Call its position j , and let e_j be the string with a 1 in position j and 0 everywhere else. Pair each x with $x \oplus e_j$: the two differ in bit j alone, so their exponents differ by 1 and the terms cancel. Summing over all $N/2$ such pairs,

$$C_k = 0.$$

Example 3.4

Check the cancellation for $n = 3$, $s = 101$, $k = 001$.

Solution. $s \oplus k = 100$, so

$$\begin{aligned} C_{001} &= \frac{1}{8} \sum_{x=0}^7 (-1)^{x \cdot 100} \\ &= \frac{1}{8} \left(\underbrace{(-1)^0 + (-1)^0 + (-1)^0 + (-1)^0}_{x=000,001,010,011} \right. \\ &\quad \left. + \underbrace{(-1)^1 + (-1)^1 + (-1)^1 + (-1)^1}_{x=100,101,110,111} \right) \\ &= \frac{1}{8} (4 - 4) = 0. \end{aligned}$$

The four strings with leading bit 0 contribute +1 and the four with leading bit 1 contribute -1.

Hence only one term survives (3.15):

$$|\psi_3\rangle = C_s |s\rangle = |s\rangle, \quad (3.17)$$

and the measurement returns s with probability 1. I have verified this numerically for $n = 3, 4, 5$: the measured string equals s with probability 1.0000 in every case.

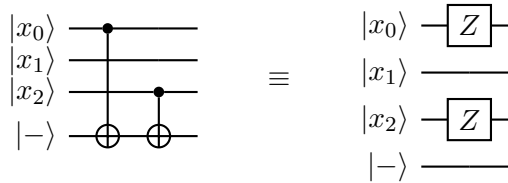


Figure 3.7: Inside the Bernstein–Vazirani oracle, for $n = 3$ and $s = 101$. On the left, one CNOT runs into the ancilla from each qubit i with $s_i = 1$, which writes $s \cdot x$ onto the ancilla. Because the ancilla sits in $|-\rangle$, each of those CNOTs kicks a factor $(-1)^{x_i}$ back onto its control and leaves the ancilla alone, so the same oracle is the product of Z gates shown on the right. Sandwiched between the Hadamard layers of Fig. 3.6, each Z becomes an X , which flips exactly the qubits where $s_i = 1$.

Where the speed-up comes from

Classically, s has n bits and each query returns one bit, so n oracle queries are information-theoretically necessary. Quantumly, one oracle query returns one n -bit string. That works only because the phase oracle wrote all n bits of s into the *phase pattern* across 2^n branches at once, and the Hadamards decoded that pattern in one shot. No individual value $f(x)$ is ever learned. The connection to Chapter 2 is exact: the phase oracle for $f(x) = s \cdot x$ is just a product of Z gates, one on each qubit where $s_i = 1$. The Bernstein–Vazirani entry in that chapter’s end-of-chapter problems asks you to prove it. Sandwiched between Hadamards, $HZH = X$, so the circuit flips exactly those qubits, which is why the answer appears directly in the measurement. Figure 3.7 draws both forms of the oracle for the $s = 101$ instance of Table 3.4.

Exercise 3.4

Draw the Bernstein–Vazirani oracle explicitly for $n = 4$, $s = 1011$, using CNOT gates into the $|-\rangle$ ancilla. How many CNOTs are needed in general?
Answer: one CNOT from each qubit i with $s_i = 1$ into the ancilla; here three.
 In general, the Hamming weight of s .

3.5 Simon's Algorithm

Simon’s algorithm [27] is the most important in this chapter. It was the first to show an *exponential* separation that survives randomization. Its structure, find a hidden period and then solve linear equations, is the direct ancestor of Shor’s factoring algorithm [26].

3.5.1 The problem

We are given $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ with a promise: there is a hidden string p such that

$$f(x_1) = f(x_2) \iff x_1 = x_2 \text{ or } x_1 \oplus x_2 = p. \quad (3.18)$$

If $p = 00 \dots 0$ the function is one-to-one; otherwise it is two-to-one, with inputs paired by XOR with p . Find p .

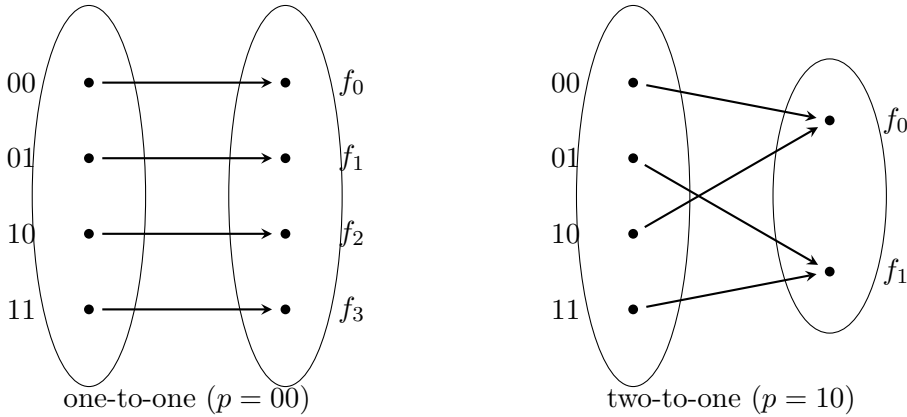


Figure 3.8: The two cases of Simon’s promise, for $n = 2$. On the right, $p = 10$ pairs $00 \leftrightarrow 10$ and $01 \leftrightarrow 11$, and every pair XORs to the *same* string p . A map that merely collapses inputs in pairs is not enough. The pairing must be by a common p .

Label your inputs from zero

I have seen “two-to-one” illustrated with something like $f(1) = 1$, $f(2) = 2$, $f(3) = 1$, $f(4) = 2$. That is a valid two-to-one map but *not* a valid Simon instance: the pairs are $\{1, 3\}$ and $\{2, 4\}$, and $1 \oplus 3 = 2$ while $2 \oplus 4 = 6$. There is no common p .

The inputs are bit strings, so they must be labeled $0, 1, \dots, 2^n - 1$. Relabeled from zero the pairs become $\{0, 2\}$ and $\{1, 3\}$, both XORing to $2 = 10_2$, and the instance is valid with $p = 10$.

Example 3.5

Write the truth table for $n = 3$ with $p = 010$.

Solution. Inputs pair up as $x \leftrightarrow x \oplus 010$:

x	$f(x)$	x	$f(x)$
000	f_0	100	f_2
010	f_0	110	f_2
001	f_1	101	f_3
011	f_1	111	f_3

Each of the four output values is hit exactly twice, and every pair XORs to 010. Verified: all four pairings are consistent with $p = 010$.

3.5.2 Classical cost

Classically we must find a *collision*, that is, two inputs with the same output. Knowing one collision x_1, x_2 immediately gives $p = x_1 \oplus x_2$.

We count oracle queries, and n is the number of input bits. By the birthday argument, querying k distinct random inputs produces $\binom{k}{2} \approx k^2/2$ candidate pairs. Each pair collides with probability $1/(2^n - 1)$, because every input has exactly one partner $x \oplus p$ among the $2^n - 1$ others. The expected number of collisions is therefore about $k^2/2^{n+1}$, and it reaches order one once $k \approx 2^{n/2}$. With fewer queries a collision is unlikely; with more, it is likely. No classical algorithm, randomized or not, can do much better than this, so the bound is tight, and the bounded-error classical cost is

$$\Theta(2^{n/2}) \quad \text{queries.}$$

In plain words, the classical cost is the square root of the number of inputs. Each extra input bit multiplies it by $\sqrt{2}$ rather than by 2, but it is still exponential in n .

Not $2^{n-1} + 1$

I have seen lecture notes carry over the Deutsch–Jozsa bound $2^{n-1} + 1$ and state it here, but that is the wrong figure. The birthday bound $2^{n/2}$ is much smaller:

n	$2^{n/2}$	2^{n-1}
10	32	512
20	1,024	524,288
30	32,768	536,870,912

Quoting the larger number overstates the classical difficulty. The true separation is $O(n)$ quantum oracle queries versus $\Theta(2^{n/2})$ classical ones, which is still exponential. It is also a *bounded-error* separation, and that is what makes Simon’s result far stronger than Deutsch–Jozsa’s.

3.5.3 The quantum algorithm

Simon’s algorithm needs two n -qubit registers and uses the *bit* oracle (ancilla-style, not phase-style), because we genuinely need $f(x)$ as a value to measure. Here O_f is the standard oracle $O_f |x\rangle |a\rangle = |x\rangle |a \oplus f(x)\rangle$.

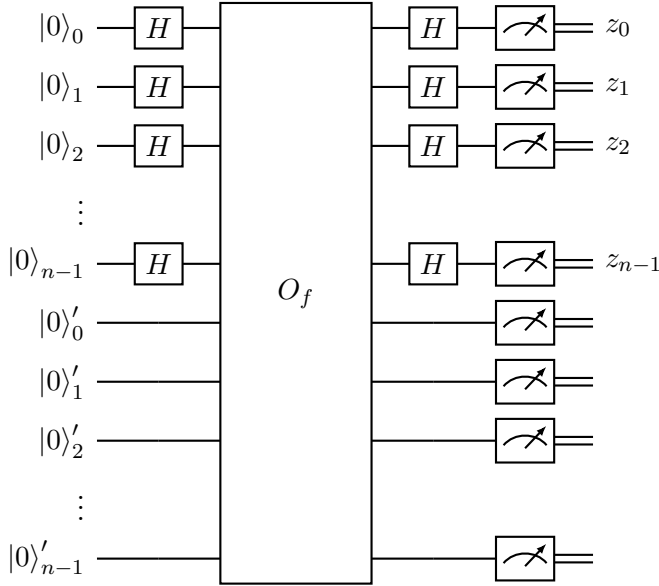


Figure 3.9: Simon's algorithm. The top block is the n -qubit input register and the bottom block, with primed labels, the n -qubit output register. The output register is measured first, or equivalently discarded. That is what collapses the input register onto a two-term superposition.

Step-by-step

(1) $|\psi_0\rangle = |0\rangle^{\otimes n} \otimes |0\rangle^{\otimes n}$.

(2) Hadamards on the first register:

$$|\psi_1\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle \otimes |0\rangle^{\otimes n}, \quad N = 2^n. \quad (3.19)$$

(3) Query:

$$|\psi_2\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle \otimes |f(x)\rangle. \quad (3.20)$$

Because f is two-to-one, the terms come in pairs sharing an output value: grouping x with $x \oplus p$,

$$|\psi_2\rangle = \frac{1}{\sqrt{N}} \sum_{\text{pairs}} (|x\rangle + |x \oplus p\rangle) \otimes |f(x)\rangle. \quad (3.21)$$

(4) **Measure the second register.** Suppose the outcome is $f(m)$ for some m . The first register collapses onto exactly the two inputs consistent with that outcome:

$$|\psi_3\rangle = \frac{|m\rangle + |m \oplus p\rangle}{\sqrt{2}}. \quad (3.22)$$

The factor $1/\sqrt{2}$ is renormalization after collapse. The value m is uniformly random and unknown to us, which is why we cannot just read off p . Figure 3.10 shows the collapse for the $n = 3$ instance of Example 3.5.

Why measure the output register at all?

We never use the value $f(m)$. Measuring the second register is a device for *collapsing* the first onto a two-term superposition. Equivalently we could just discard those qubits, since tracing them out has the same effect on the statistics of the first register. Drawing the measurement makes the logic explicit, and it is how the algorithm is usually implemented.

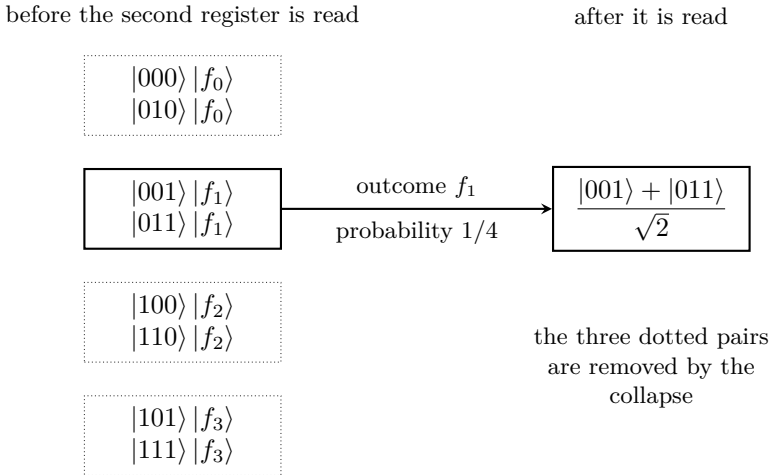


Figure 3.10: Step (4) of Simon's algorithm, for the $n = 3$, $p = 010$ function of Example 3.5. Before the second register is read, all four input pairs are present in $|\psi_2\rangle$. Reading it returns one output value, here f_1 , and the input register is left in an equal superposition of exactly the two inputs that map to it. Those two inputs differ by p , which is the only thing about p that a single run reveals.

(5) Hadamards on the first register. Using $H^{\otimes n} |y\rangle = \frac{1}{\sqrt{N}} \sum_z (-1)^{y \cdot z} |z\rangle$,

$$\begin{aligned}
 |\psi_4\rangle &= \frac{1}{\sqrt{2N}} \sum_{z=0}^{N-1} \left[(-1)^{m \cdot z} + (-1)^{(m \oplus p) \cdot z} \right] |z\rangle \\
 &= \frac{1}{\sqrt{2N}} \sum_{z=0}^{N-1} (-1)^{m \cdot z} [1 + (-1)^{p \cdot z}] |z\rangle.
 \end{aligned} \tag{3.23}$$

The second line factors out $(-1)^{m \cdot z}$ using the same bitwise identity as in (3.16), namely $(m \oplus p) \cdot z = m \cdot z \oplus p \cdot z$.

(6) Measure. The bracket is 2 when $p \cdot z = 0$ and 0 when $p \cdot z = 1$. So the outcomes with nonzero probability are exactly the strings orthogonal to p :

$$p \cdot z = 0 \pmod{2}, \quad (3.24)$$

each occurring with probability $2/N$. I verified this numerically for $n = 3$ with $p = 110$: every measured string satisfied (3.24), and the outcomes observed were exactly 000, 001, 110 and 111, the four solutions, each with probability $\frac{1}{4}$.

3.5.4 Solving for p

Each run gives one linear equation over $\mathbb{F}_2$. Collect them:

$$p \cdot z_1 = 0, \quad p \cdot z_2 = 0, \quad \dots, \quad p \cdot z_k = 0. \quad (3.25)$$

Solve by Gaussian elimination mod 2. That step is classical post-processing. With about n equations in n unknowns it costs $O(n^3)$ bit operations, polynomial in n and never the bottleneck.

How many runs, exactly?

We need $n - 1$ independent equations, not n : the solution space must be cut down to $\{0, p\}$, of dimension 1, so $n - 1$ linearly independent constraints are needed, and with n the only solution would be $p = 0$. And not every run helps: $z = 00 \cdots 0$ satisfies (3.24) trivially, as does any z already in the span of previous outcomes. Collecting $n - 1$ independent vectors takes $O(n)$ runs in expectation, and a coupon-collector argument shows the failure probability after $n - 1 + k$ runs falls off like 2^{-k} . If the only solution is $p = 0$, the function was one-to-one.

Let me show where the $O(n)$ comes from. Runs and oracle queries coincide here, because each run of Fig. 3.9 calls O_f exactly once, so the query count is the run count. Suppose i independent vectors have been collected so far. They span 2^i of the 2^{n-1} strings that satisfy (3.24), and each new run returns one of those 2^{n-1} strings uniformly at random. So the new run is wasted with probability $2^i/2^{n-1}$, which is at most $\frac{1}{2}$ as long as $i \leq n - 2$, and the expected number of runs until the next useful vector is $1/(1 - 2^{i-n+1})$, never more than 2. Summing over $i = 0, \dots, n - 2$ gives $n - 1$ useful runs plus fewer than two wasted ones on average, a little over n runs in all. In my simulations the average for $n = 8$ was about nine runs. The 2^{-k} tail comes from the same bookkeeping, and the end-of-chapter problems ask you to prove it.

Example 3.6

For $n = 3$, three runs return $z_1 = 011$, $z_2 = 101$, $z_3 = 110$. Find p .

Solution. Writing $p = (p_0 p_1 p_2)$, the equations over $\mathbb{F}_2$ are $p_1 \oplus p_2 = 0$, $p_0 \oplus p_2 = 0$ and $p_0 \oplus p_1 = 0$. The first two give $p_1 = p_2$ and $p_0 = p_2$, and the third is then automatically satisfied, so it was *not* independent: only two of the three runs were useful, exactly the $n - 1 = 2$ we need. So $p_0 = p_1 = p_2$, i.e. $p \in \{000, 111\}$, and discarding the trivial solution, $p = 111$. Check: $111 \cdot 011 = 1 \oplus 1 = 0 \checkmark$, $111 \cdot 101 = 1 \oplus 1 = 0 \checkmark$, $111 \cdot 110 = 1 \oplus 1 = 0 \checkmark$.

Exercise 3.5

For $n = 4$, runs return $z_1 = 0011$, $z_2 = 0101$, $z_3 = 1000$. Determine p , or explain what further runs are needed.

Answer: the three vectors are independent, giving $n - 1 = 3$ constraints $p_2 = p_3$, $p_1 = p_3$, $p_0 = 0$, so $p \in \{0000, 0111\}$ and $p = 0111$. No further runs needed.

Why Simon matters

This algorithm has a definite shape: prepare a superposition, query once, discard a register to collapse onto a periodic subset, Fourier-transform, and read off a constraint on the period. That is exactly the shape of Shor's algorithm, where the group is $\mathbb{Z}_N$ rather than $\mathbb{F}_2^n$, the Hadamard becomes the quantum Fourier transform (Chapter 4), and the linear algebra becomes continued fractions.

3.6 Measurement Error Mitigation

The algorithms above are exact in theory. On hardware they are not. The shallowest circuits, Deutsch–Jozsa and Bernstein–Vazirani, are exactly the ones where the dominant error is in the *readout* rather than the gates. That makes them the best candidates for the simple error mitigation technique below.

3.6.1 Sources of error

- (i) **State preparation.** Qubits are not initialized perfectly; the starting state has small admixtures of the wrong state.
- (ii) **Gate errors.** Every gate is slightly off-target, and qubits decay and dephase even when idle.

- (iii) **Readout errors.** The detector reports 1 when the qubit was 0, or vice versa. On current hardware this is often the largest single error rate, typically a few percent.

Decoherence

Decoherence is the loss of the definite phase relationships between components of a superposition. Coherence is preserved only for a perfectly isolated system; real qubits interact with their environment, which effectively records information about the system's state. As Chapter 2 argued for garbage qubits, any such record destroys interference, and interference is what the algorithms in this chapter run on. Decoherence is the same mechanism, imposed by the environment rather than by our own sloppy circuit design.

What this does not fix

The technique below corrects source (iii), *readout* error, only. Gate infidelity and decoherence during idle time have already corrupted the state before measurement, and no post-processing of the outcome distribution can undo them. Mitigating those requires different methods (zero-noise extrapolation, probabilistic error cancellation) or true error correction.

3.6.2 Characterizing readout error

Rather than model each error separately, we measure the *net* effect on readout. Prepare each computational basis state in turn and record the observed outcome distribution.

Ideally, preparing $|00\rangle$ gives outcome 00 with probability 1. In practice one might observe

$$\Pr(00) = 0.90, \quad \Pr(01) = 0.05, \quad \Pr(10) = 0.04, \quad \Pr(11) = 0.01.$$

Collect these as a column vector, and repeat for all four basis states:

$$P_{\text{noisy}}^{00} = \begin{pmatrix} 0.90 \\ 0.05 \\ 0.04 \\ 0.01 \end{pmatrix}, \quad P_{\text{noisy}}^{01} = \begin{pmatrix} 0.01 \\ 0.98 \\ 0.002 \\ 0.008 \end{pmatrix}, \quad P_{\text{noisy}}^{10} = \begin{pmatrix} 0.02 \\ 0.04 \\ 0.91 \\ 0.03 \end{pmatrix}, \quad P_{\text{noisy}}^{11} = \begin{pmatrix} 0.01 \\ 0.03 \\ 0.04 \\ 0.92 \end{pmatrix}.$$

Stacking them as columns gives the *calibration matrix*

$$M = \begin{pmatrix} 0.90 & 0.01 & 0.02 & 0.01 \\ 0.05 & 0.98 & 0.04 & 0.03 \\ 0.04 & 0.002 & 0.91 & 0.04 \\ 0.01 & 0.008 & 0.03 & 0.92 \end{pmatrix}, \quad P_{\text{noisy}} = M P_{\text{ideal}}. \quad (3.26)$$

Sanity check your calibration matrix

Each *column* of M is a probability distribution over outcomes, so every column must sum to 1. I checked (3.26): all four columns sum to exactly 1.0000. A matrix that fails this test has a bookkeeping error. Most often, the noisy distributions were entered as rows instead of columns.

Example 3.7

Using M from (3.26), predict the measured distribution for the Bell state $|\psi^{00}\rangle$.

Solution. Ideally $\Pr(00) = \Pr(11) = \frac{1}{2}$, so $P_{\text{ideal}} = (0.5, 0, 0, 0.5)^T$ and

$$P_{\text{noisy}} = M \begin{pmatrix} 0.5 \\ 0 \\ 0 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 0.455 \\ 0.040 \\ 0.040 \\ 0.465 \end{pmatrix}.$$

So about 8% of shots land on 01 or 10, outcomes the ideal state never produces. The entries sum to 1, as they must.

3.6.3 Recovering the ideal distribution

Inverting (3.26),

$$P_{\text{ideal}} = M^{-1}P_{\text{noisy}}. \quad (3.27)$$

Applying M^{-1} to the result of Example 3.7 returns $(0.5, 0, 0, 0.5)^T$ exactly. Figure 3.11 collects the two passes this requires: a calibration pass that builds M , and a correction pass that uses it.

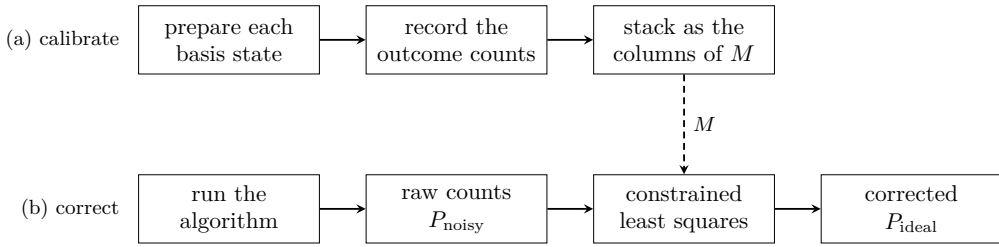


Figure 3.11: The two passes of readout-error mitigation. Pass (a) is run once per device and costs 2^n calibration circuits, one for every computational basis state of the n measured qubits. Pass (b) is run for every circuit whose distribution you want corrected, and it consumes the matrix M produced by pass (a). The dashed arrow is the only coupling between them, which is why a stale calibration silently corrupts every corrected result.

Why the raw inverse is not used in practice

M^{-1} is not a stochastic matrix, so (3.27) can return *negative* entries on real, statistically noisy data, and a vector with negative components is not a probability distribution. Real implementations therefore solve a constrained problem: find the P_{ideal} minimizing $\|MP_{\text{ideal}} - P_{\text{noisy}}\|^2$ subject to $P_i \geq 0$ and $\sum_i P_i = 1$. This small quadratic program, solved on a classical computer, is what standard software actually runs. The scaling matters too. With n measured qubits, M is $2^n \times 2^n$, so full calibration requires 2^n calibration circuits, one per basis state. Beyond about ten qubits one uses tensored or correlated-subset approximations.

3.7 Summary

- H then CNOT maps $|xy\rangle \mapsto |\psi^{xy}\rangle$; the reverse circuit measures in the Bell basis. To measure in any basis $\{|b_i\rangle\}$, apply $B^\dagger$ and measure normally.
- Teleportation sends one qubit using one shared EPR pair and two classical bits. It destroys the original and cannot signal faster than light. For outcome 11, Bob applies ZX , and the order matters.
- All three algorithms share the pattern: superpose, query once, interfere, measure. Deutsch–Jozsa and Bernstein–Vazirani mark with a phase; Simon writes $f(x)$ into a second register and discards it.
- Deutsch–Jozsa decides constant-versus-balanced in one query, exactly. The separation is over *deterministic* classical algorithms only.
- Bernstein–Vazirani recovers an n -bit secret in one query instead of n .
- Simon finds a hidden XOR-period with $O(n)$ oracle queries, one per run, versus $\Theta(2^{n/2})$ classical queries. That is an exponential, bounded-error separation,

and the template for Shor.

- Simon needs $n - 1$ *independent* equations; $z = 0$ is always a useless outcome.
- Deutsch–Jozsa and Bernstein–Vazirani both need an ancilla in $|-\rangle$; Simon needs a full second register, and it is measured or discarded.
- Readout error is characterized by a column-stochastic calibration matrix M ; mitigation solves a constrained least-squares problem, not a raw matrix inverse.

3.8 End-of-Chapter Problems

- 3.1** Verify by direct matrix multiplication that the circuit of Fig. 3.1 maps all four computational basis states to the four Bell states as listed in Table 3.2.
- 3.2** Show that Fig. 3.2 is the inverse of Fig. 3.1 by computing the product of the two circuit matrices.
- 3.3** Prove the teleportation identity (3.4) directly, by expanding each Bell state and collecting terms.
- 3.4** In teleportation, show that Bob’s qubit before the classical message arrives is in the maximally mixed state $I/2$, independent of α and β . Explain why this rules out faster-than-light signaling.
- 3.5** Show that $(XZ)^2 = -I$, and hence that applying XZ instead of ZX in the final teleportation step gives the right state up to a global phase. Construct a scenario in which the difference becomes observable.
- 3.6** For $n = 2$, list all six balanced functions and all two constant functions. Confirm that Deutsch–Jozsa distinguishes the two classes in one query for each.
- 3.7** Show that if f is neither constant nor balanced, the Deutsch–Jozsa circuit still runs but its output is not meaningful. What is $\Pr(00 \cdots 0)$ in general?
- 3.8** Show that a randomized classical algorithm decides constant-versus-balanced with error probability below δ using $O(\log(1/\delta))$ queries, independent of n .
- 3.9** Draw the Bernstein–Vazirani oracle for $n = 5$, $s = 11010$, and count the CNOT gates.
- 3.10** Show that the Bernstein–Vazirani phase oracle equals $\bigotimes_i Z^{s_i}$, and hence that the full circuit reduces to X^{s_i} on each qubit. *Hint:* $HZH = X$.
- 3.11** For Simon’s problem with $n = 3$ and $p = 101$, write out the full truth table of a valid f , and verify that every pair XORs to p .

- 3.12** In Simon's algorithm, show that each valid outcome z (i.e. each z with $p \cdot z = 0$) occurs with probability exactly $2/N$, and that there are $N/2$ such outcomes.
- 3.13** Show that the probability of failing to collect $n - 1$ independent vectors after $n - 1 + k$ runs of Simon's algorithm is at most 2^{-k} .
- 3.14** Prove that the columns of a calibration matrix M must each sum to 1, and give a physical interpretation of the entry M_{ij} .
- 3.15** Construct a noisy distribution P_{noisy} for which $M^{-1}P_{\text{noisy}}$ has a negative entry, using M from (3.26). What should be reported instead?

Research Problems and Projects

The problems above have answers you can check in an afternoon. The three below do not. Each is a small project: expect several weeks of work, and a written report with code, figures and an honest error analysis.

- R3.1 How Simon's algorithm degrades under readout noise.** Build a state-vector simulator for Simon's algorithm at $n = 3, \dots, 12$. Add readout noise only, using the calibration-matrix model of §3.6 with an independent flip probability ε on each measured wire. The chapter's clean claims are that the expected number of runs is $O(n)$, and that the failure probability after $n - 1 + k$ runs is at most 2^{-k} . Both assume exact sampling from the uniform distribution on $\{z : p \cdot z = 0\}$. Readout error breaks that assumption, because a flipped bit produces a z that violates (3.24) and injects a false equation into the linear system. One false equation is enough to make Gaussian elimination return the wrong p with confidence. Measure, as a function of n and ε , the distribution of the number of runs needed and the probability that the recovered p is correct. Then compare two repair strategies: majority filtering over repeated runs, and solving the system in a maximum-likelihood rather than exact sense. *Deliverable:* a measured scaling curve of expected runs against n , at three or four values of ε . Add an estimate of the largest ε at which the algorithm still succeeds with probability above $1/2$ at $n = 10$, and a comparison of the two repair strategies against the unrepaired baseline. The outcome is a quantitative curve, not a theorem. The interesting part is where the curve bends.
- R3.2 What the three original papers actually prove.** Table 3.1 makes a claim that many lecture notes get wrong. Deutsch–Jozsa is an exact-versus-exact separation and evaporates under bounded error, while Simon's separation survives it. Read the three sources [9, 3, 27] against that table. For each, extract the precise computational model, the precise separation

proved, and the lower-bound argument that supports it. Then record where the modern textbook statement differs from the original. Bernstein and Vazirani in particular prove a great deal more than the one-query secret-string result taught in §3.4. Their paper also contains a recursive Fourier sampling separation and the definition of the class BQP. Work out which of the results commonly attributed to them is which, and which one the name “Bernstein–Vazirani algorithm” actually refers to. *Deliverable*: a survey of ten to fifteen pages, built around one table giving, for each paper, the theorem as stated in the original and the version usually quoted. Include a bibliography of at least ten secondary sources you checked, and a list of the misstatements you found in them. Be clear with yourself about what this is. The complexity-theoretic picture here is settled, so the contribution is accuracy and attribution, not a new result.

R3.3 Readout mitigation on Bernstein–Vazirani, end to end. Implement Bernstein–Vazirani for $n = 2, \dots, 10$ on a simulator with a realistic readout-noise model, and on real hardware if you have access to it. For each n , build the full $2^n \times 2^n$ calibration matrix of (3.26) and record what it costs in circuits, shots and wall-clock time. Then compare three ways of recovering the ideal distribution: the raw inverse M^{-1} , the constrained least-squares solve of §3.6, and a tensored approximation that treats the n qubits as having independent readout error. *Deliverable*: a resource-count table against n , together with a plot of the probability of reading s correctly under each of the three methods. State the value of n at which full calibration stops being practical on your machine. Then extend the study. Bernstein–Vazirani has a known correct answer on every shot, which makes it a self-checking benchmark. Propose a figure of merit built on that property, test it, and say honestly whether it measures anything that standard randomized benchmarking does not already measure.

Chapter 4

The Quantum Fourier Transform and Phase Estimation

Chapter 3 ended with Simon’s algorithm: prepare a superposition, query once, collapse onto a periodic subset, apply Hadamards, and read off a constraint on the hidden period. That worked because the Hadamard transform is the Fourier transform on the group $\mathbb{F}_2^n$, the group of bit strings under XOR.

Most interesting periods are not XOR-periods. The order of an element modulo N , the period of $a^x \bmod N$ that Shor’s algorithm needs [26], lives in $\mathbb{Z}_N$ instead. To find those we need the Fourier transform on $\mathbb{Z}_N$: the *quantum Fourier transform*.

This chapter builds the QFT, shows that on n qubits it needs only $O(n^2)$ gates, and then uses it to solve the single most reusable problem in quantum computing.

What you should be able to do after this chapter

- State the QFT, compute it by hand for one and two qubits, and derive the product form.
- Draw the QFT and inverse QFT circuits and count their gates.
- Explain why the QFT is *not* a speed-up for computing Fourier coefficients.
- Derive the phase-estimation circuit and explain where the inverse QFT enters.
- Compute the success probability when θ is not a dyadic rational, and choose the number of counting qubits for a target accuracy.
- Run iterative phase estimation with a single ancilla and state its trade-offs.

Why phase estimation is worth a chapter

Given a unitary U and one of its eigenvectors $|u\rangle$ with $U|u\rangle = e^{2\pi i\theta}|u\rangle$, find θ . That sounds narrow. It is not:

- **Shor's factoring algorithm** is phase estimation applied to the modular-multiplication operator; the phase encodes the period.
- **Quantum chemistry** computes molecular ground-state energies by estimating the phase of e^{-iHt} (Chapters 7 and 9).
- **HHL** [16] and other linear-algebra routines use it to invert eigenvalues.
- **Amplitude estimation** [5], which covers counting solutions and Monte Carlo speed-ups, is phase estimation on the Grover operator (Chapter 6). Chapter 10 shows how to do it without a counting register at all [28].

If you learn one subroutine from this book, learn this one.

4.1 The Quantum Fourier Transform

4.1.1 From the DFT to the QFT

The discrete Fourier transform takes a vector of N complex numbers, written $(x_0, \dots, x_{N-1})$, to another vector of the same length, written $(y_0, \dots, y_{N-1})$:

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k / N}. \quad (4.1)$$

The QFT is the same transformation, applied to the *amplitudes* of a quantum state. On basis states it acts as

$$\text{QFT } |j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i j k / N} |k\rangle, \quad N = 2^n, \quad (4.2)$$

where a basis state is labeled by the integer its bit string represents:

$$|j\rangle = |j_0 j_1 \dots j_{n-2} j_{n-1}\rangle, \quad j = j_0 2^{n-1} + j_1 2^{n-2} + \dots + j_{n-1} 2^0.$$

As everywhere else in this book, the leftmost symbol is qubit 0; here it is also the *most* significant bit. For instance $|101\rangle$ has $j = 4 + 0 + 1 = 5$, so $|101\rangle = |5\rangle$.

Two conventions to fix now

Sign. The classical DFT is usually written with $e^{-2\pi ijk/N}$. Equations (4.1) and (4.2) both use $+$, which is self-consistent, but it means our QFT is what some references call the *inverse* DFT.

Bit ordering. In $|j_0j_1 \cdots j_{n-1}\rangle$ qubit 0 is the leftmost symbol, carries the *most* significant bit and is drawn on the **top** wire, as in Chapters 1–3. In my experience bit ordering is the single easiest thing to get wrong in this chapter. Qiskit uses the opposite (little-endian) convention, in which the *rightmost* symbol is qubit 0 and the top wire is the least significant bit, so its built-in QFT circuit looks like a mirror image of ours. Check the sign and the ordering before concluding that anything is broken.

The QFT circuit still ends with a layer of SWAP gates. That is not to reconcile two conventions; the circuit simply produces the output qubits in reverse order. I verified this numerically: the two-qubit circuit of Fig. 4.3 reproduces (4.2) exactly under this convention, and would be wrong under the other.

The QFT is not a fast Fourier transform

It is tempting to read “ $O(n^2)$ quantum gates versus $O(N \log N)$ classical arithmetic operations”, with n the number of qubits and $N = 2^n$ the number of amplitudes, as an exponential speed-up for Fourier analysis. It is not. The classical FFT outputs all N coefficients y_k , which you can read; the QFT leaves them as *amplitudes* of a state you cannot read. Measuring returns a single k with probability $|y_k|^2$, and learning all the y_k to reasonable precision would take $\Omega(N)$ repetitions of the whole circuit. So there is no known way to use the QFT to speed up general signal processing. What it is good for is extracting a *single global feature* of the amplitude pattern, such as a period or a phase, in one shot. That is exactly what phase estimation does, and it is why the QFT appears inside algorithms rather than as an algorithm.

4.1.2 One qubit

With $n = 1$ and $N = 2$, (4.2) reads

$$\text{QFT} |j\rangle = \frac{1}{\sqrt{2}} \sum_{k=0}^1 e^{\pi ijk} |k\rangle = \frac{1}{\sqrt{2}} (|0\rangle + e^{\pi ij} |1\rangle).$$

So

$$\text{QFT} |0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle, \quad \text{QFT} |1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle.$$

The single-qubit QFT is exactly the Hadamard gate. This was verified as matrices.

$$|j\rangle \longrightarrow \boxed{H} \longrightarrow |\tilde{j}\rangle$$

Figure 4.1: The one-qubit QFT.

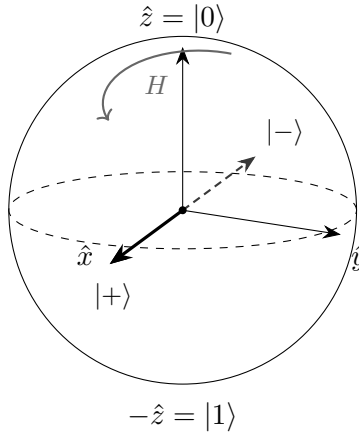


Figure 4.2: The one-qubit QFT on the Bloch sphere. The Hadamard rotates by π about the axis $(\hat{x} + \hat{z})/\sqrt{2}$, carrying the poles $|0\rangle, |1\rangle$ to the equatorial points $|+\rangle, |-\rangle$. Every QFT output qubit ends up on the equator: the QFT converts *which basis state* into a *pattern of relative phases*.

4.1.3 Two qubits

For $n = 2$, split the sum over $k = 2k_0 + k_1$, where k_0 is the leading bit of $|k\rangle = |k_0 k_1\rangle$:

$$\begin{aligned} |\tilde{j}\rangle &= \frac{1}{2} \sum_{k=0}^3 e^{2\pi i j k / 4} |k\rangle = \frac{1}{2} \sum_{k_0=0}^1 \sum_{k_1=0}^1 e^{2\pi i j (2k_0 + k_1) / 4} |k_0 k_1\rangle \\ &= \frac{1}{\sqrt{2}} \sum_{k_0=0}^1 e^{\pi i j k_0} |k_0\rangle \otimes \frac{1}{\sqrt{2}} \sum_{k_1=0}^1 e^{\pi i j k_1 / 2} |k_1\rangle \\ &= \frac{|0\rangle + e^{\pi i j} |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{\pi i j / 2} |1\rangle}{\sqrt{2}}. \end{aligned}$$

The sum has factorized into a *product* of single-qubit states. That is the key structural fact about the QFT, and the reason it has an efficient circuit. Substituting $j = 2j_0 + j_1$ and using $e^{2\pi i \nu} = 1$ for integer ν ,

$$|\tilde{j}\rangle = \frac{|0\rangle + e^{i\pi j_1} |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{i\pi j_0} e^{i\pi j_1 / 2} |1\rangle}{\sqrt{2}}.$$

The first factor depends only on j_1 , the least significant input bit; the second needs both bits. That is what the controlled-phase gate in the circuit supplies.

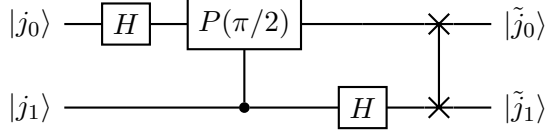


Figure 4.3: The two-qubit QFT. The top wire carries j_0 , the *most* significant bit, as everywhere in this book. Before the SWAP the top wire holds the factor $(|0\rangle + e^{2\pi i 0.j_0 j_1} |1\rangle)/\sqrt{2}$ and the bottom wire holds $(|0\rangle + e^{2\pi i 0.j_1} |1\rangle)/\sqrt{2}$; the SWAP puts them the right way round. Verified numerically to equal (4.2) exactly.

Example 4.1

Compute QFT $|00\rangle$ and QFT $|01\rangle$.

Solution. Here $N = 4$. For $|00\rangle$, $j = 0$:

$$|\widetilde{00}\rangle = \frac{1}{2} \sum_{k=0}^3 |k\rangle = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2} = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |++\rangle.$$

For $|01\rangle$, $j = 1$:

$$\begin{aligned} |\widetilde{01}\rangle &= \frac{1}{2} \sum_{k=0}^3 e^{\pi i k/2} |k\rangle = \frac{|00\rangle + i|01\rangle - |10\rangle - i|11\rangle}{2} \\ &= \frac{|0\rangle - |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + i|1\rangle}{\sqrt{2}} = |-\rangle \otimes |+i\rangle. \end{aligned}$$

Both verified numerically. I want you to notice that the QFT of a basis state is always an unentangled product state of equatorial Bloch vectors. It is never entangled.

Exercise 4.1

Compute QFT $|10\rangle$ and QFT $|11\rangle$, and describe the outputs on the Bloch sphere.

Answer: QFT $|10\rangle = |+\rangle \otimes |-\rangle$ and QFT $|11\rangle = |-\rangle \otimes |-i\rangle$. Reading the four results together: the first output qubit alternates between $|+\rangle$ and $|-\rangle$ as j increases (azimuth advancing by π), while the second advances by $\pi/2$ each time. The second qubit turns at half the rate of the first. That is the binary counter structure the general formula makes explicit.

4.1.4 The general product form

Repeating the factorization for general n : writing $k = \sum_{l=1}^n k_{l-1} 2^{n-l}$ and splitting the exponential,

$$\text{QFT } |j\rangle = \frac{|0\rangle + e^{2\pi i j/2} |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{2\pi i j/2^2} |1\rangle}{\sqrt{2}} \otimes \dots \otimes \frac{|0\rangle + e^{2\pi i j/2^n} |1\rangle}{\sqrt{2}}. \quad (4.3)$$

Each factor is a single qubit on the equator of the Bloch sphere, at azimuth $2\pi j/2^l$. Now expand j in binary. Because $e^{2\pi i \nu} = 1$ for any integer ν , only the *fractional* part of $j/2^l$ survives:

$$\frac{j}{2} \equiv \frac{j_{n-1}}{2}, \quad \frac{j}{4} \equiv \frac{j_{n-2}}{2} + \frac{j_{n-1}}{4}, \quad \frac{j}{2^l} \equiv \frac{j_{n-l}}{2} + \frac{j_{n-l+1}}{4} + \dots + \frac{j_{n-1}}{2^l} \pmod{1}.$$

Using the binary-fraction notation $0.j_{n-l}j_{n-l+1} \dots j_{n-1}$ for the right-hand side, (4.3) becomes the standard form

$$\begin{aligned} \text{QFT } |j_0 j_1 \dots j_{n-1}\rangle &= \frac{|0\rangle + e^{2\pi i 0.j_{n-1}} |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{2\pi i 0.j_{n-2}j_{n-1}} |1\rangle}{\sqrt{2}} \otimes \dots \\ &\quad \otimes \frac{|0\rangle + e^{2\pi i 0.j_0 j_1 \dots j_{n-1}} |1\rangle}{\sqrt{2}}. \end{aligned} \quad (4.4)$$

Read (4.4) as an instruction. The l -th output qubit, which is the l -th wire from the top, needs a Hadamard to create the superposition. It then needs phase rotations conditioned on the input bits $j_{n-l}, \dots, j_{n-1}$. The top output qubit depends on the single bit j_{n-1} ; the bottom one depends on all n input bits.

4.1.5 The circuit

The phase rotations are the gates

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix} = P\left(\frac{2\pi}{2^k}\right). \quad (4.5)$$

R_1 is not the Hadamard gate

Substituting $k = 1$ into (4.5) gives $R_1 = \text{diag}(1, e^{2\pi i/2}) = \text{diag}(1, -1) = Z$, not H ; Z is diagonal and H is not. What is true is that R_1 never appears in the circuit. The phase gates start at $R_2 = P(\pi/2) = S$, and the role of “create the superposition” is played by H , a separate gate. Verified: $R_2 = P(\pi/2)$ and $R_3 = P(\pi/4)$ as expected.

The circuit works down the register from the top wire. To each qubit in turn it applies a Hadamard followed by controlled- R_k gates from every *less* significant bit,

that is, from every wire still below it. It finishes with a layer of SWAPs that reverses the wire order. Figure 4.4 shows the standard construction [24]. Before you read it, I want to clear up a naming trap that costs students real debugging time.

R_k is a phase gate, not a rotation gate

The name R_k is standard for this gate and I keep it, but it is misleading in one specific and expensive way. Read the subscript as an integer index, not as an axis. The R_x, R_y, R_z of Section 1.5.4 are the *rotation* gates, $R_z(\eta) = e^{-i\eta Z/2}$, and that section writes the family as $R_k(\eta)$ with $k \in \{x, y, z\}$. The R_k of (4.5) is a member of the *phase* family P instead. It carries no factor of $\frac{1}{2}$ in the exponent, so

$$R_k = P\left(\frac{2\pi}{2^k}\right) = e^{i\pi/2^k} R_z\left(\frac{2\pi}{2^k}\right) \neq R_z\left(\frac{2\pi}{2^k}\right).$$

The two differ by the global phase $e^{i\pi/2^k}$, which is invisible on an isolated qubit. **Every R_k in the QFT circuit is controlled.** Chapter 2, Section 2.2 shows that a global phase on the target of a controlled gate becomes a *relative* phase, so $C\text{-}P(\eta)$ and $C\text{-}R_z(\eta)$ are different two-qubit gates. Building the QFT from controlled- $R_z(2\pi/2^k)$ therefore gives a wrong circuit, and wrong in the worst way: it still produces plausible output. In Qiskit the gate you want is `PhaseGate`, or its controlled form `CPhaseGate`, never `RZGate`.

A quick self-check that fixes the convention: $R_k^{2^k} = P(2\pi) = I$, whereas $R_z(2\pi) = -I$. If your gate raised to the 2^k power is not the identity, it is not the R_k of this chapter.

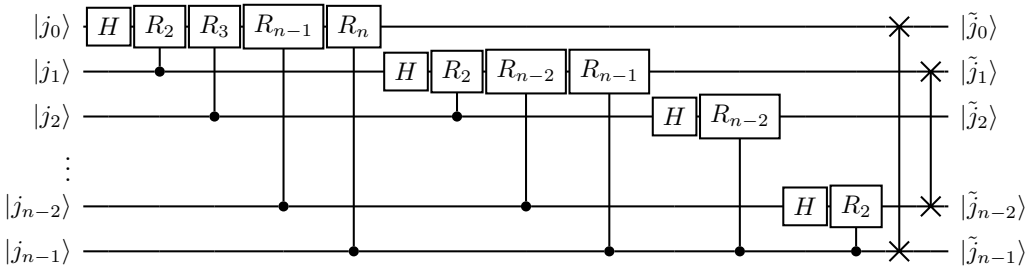


Figure 4.4: The n -qubit QFT. The most significant input bit j_0 (top wire) is processed first: a Hadamard, then controlled- R_k gates from every less significant input bit. On the top wire R_k is controlled by j_{k-1} . Working down the register, each wire needs one rotation fewer, and the bottom wire needs only a Hadamard. At that point wire q carries the factor with phase $2\pi 0.j_q j_{q+1} \cdots j_{n-1}$, which is the factor (4.4) assigns to wire $n-1-q$. The final layer of $\lfloor n/2 \rfloor$ SWAPs reverses the wire order and puts every factor where it belongs, so the output labels run in the same direction as the inputs. Verified numerically for $n = 2, 3, 4, 5$: the circuit equals (4.2) exactly, and fails without the SWAP layer.

Gate count

Count wire by wire, with n the number of qubits. The top wire receives one H and one controlled- R_k from each of the $n - 1$ wires below it, so n gates. The next wire has one fewer wire below it and takes $n - 1$ gates, and so on down to the bottom wire, which takes only its H . Summing, $n + (n - 1) + \cdots + 1 = n(n + 1)/2$ gates: n Hadamards and $n(n - 1)/2$ two-qubit controlled phases. Add the $\lfloor n/2 \rfloor$ SWAPs of the final layer, and, dropping the floor,

$$\frac{n(n + 1)}{2} + \frac{n}{2} = \frac{n^2}{2} + n = O(n^2).$$

The classical FFT on $N = 2^n$ points costs $O(N \log N) = O(n2^n)$ arithmetic operations, so the QFT is exponentially cheaper *in gate count*, subject to the caveat above that you cannot read the answer out.

In plain words: the count is dominated by the two-qubit controlled phases, and doubling the number of qubits roughly quadruples the number of gates. Each SWAP is itself three CNOTs, so the two-qubit gate count is also $O(n^2)$. This is a count of gates, not of depth. The controlled phases are all diagonal and commute with one another, so a careful schedule runs many of them in parallel and the depth is considerably smaller than the gate count.

A practical note: tiny phases

The gate R_k applies the phase $2\pi/2^k$. For $n = 50$ qubits, R_{50} applies about 5.6×10^{-15} radians, far below any achievable gate fidelity and utterly swamped by noise. In practice one *truncates* the QFT, dropping all R_k with $k > k_{\max} \approx \log_2(n/\epsilon)$, where ϵ is the error you are willing to accept in the output state. The resulting approximate QFT has error at most ϵ and only $O(n \log n)$ gates; Figure 4.5 shows which rotations survive the cut. This is standard in any real implementation: the textbook circuit is never the one that runs.

Here is where those two numbers come from. Take n to be the number of qubits and ϵ the largest change you will tolerate in the output state, measured as the operator-norm distance between the exact and the truncated QFT. Dropping a phase gate with angle η changes the state by at most η in that norm. On any one wire the dropped angles are $2\pi/2^{k_{\max}+1}, 2\pi/2^{k_{\max}+2}, \dots$, a geometric tail that sums to less than $2\pi/2^{k_{\max}}$. There are n wires, so the total error is below $2\pi n/2^{k_{\max}}$, and asking that this be at most ϵ gives $k_{\max} \approx \log_2(n/\epsilon)$, up to an additive constant of order one. As for the count, each wire now keeps its Hadamard and at most $k_{\max} - 1$ controlled phases, so the whole circuit has fewer than $nk_{\max}$ gates plus

the SWAPs. For a fixed target error that is $O(n \log n)$ gates instead of $O(n^2)$, and it is $O(n \log(n/\epsilon))$ if you keep ϵ in view. I checked the error bound numerically for $n = 5, 6, 7$ and $k_{\max} = 2, 3, 4$; the true operator-norm error is always below the bound, by a factor of three or more.

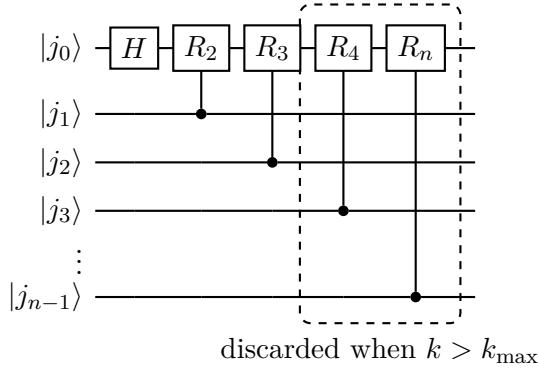


Figure 4.5: The truncated, or approximate, QFT, with $k_{\max} = 3$. Only the gates acting on the top wire are drawn; every wire below it is cut in the same way. The wire keeps its Hadamard and the two short-range phase gates R_2 and R_3 , and every longer-range one is dropped. A dropped R_k has $k \geq k_{\max} + 1$, so its phase is at most $2\pi/2^{k_{\max}+1}$, which is why the accumulated error is controlled by $k_{\max}$ alone. Each wire then carries at most $k_{\max} - 1$ phase gates, R_2 through $R_{k_{\max}}$, instead of $n - 1$, so the gate count falls from $O(n^2)$ to $O(nk_{\max})$. Compare Fig. 4.4, where nothing is dropped.

4.1.6 Graphical interpretation

Equation (4.4) has a clean picture. Every output qubit sits on the equator of its own Bloch sphere. The input integer j is encoded entirely in the *azimuthal angles*, which form a binary odometer. The last qubit turns by $2\pi j/2^n$, the one before by twice that, and so on up to πj for the first.

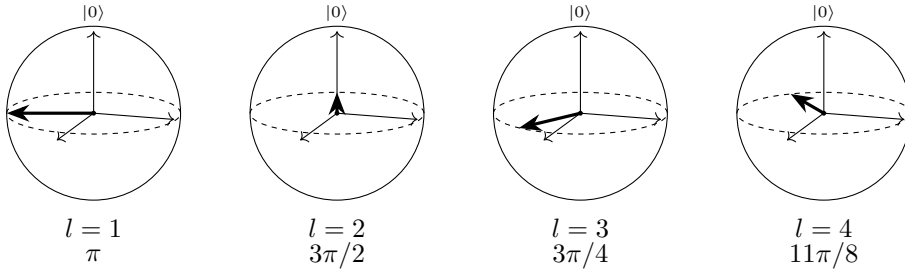


Figure 4.6: The four output qubits of QFT $|1011\rangle$, viewed from above the equator. Here $j = 1011_2 = 11$ and $n = 4$, so output qubit l sits at azimuth $2\pi j/2^l \bmod 2\pi$, giving π , $3\pi/2$, $3\pi/4$ and $11\pi/8$ respectively. Verified: the product of these four single-qubit states equals QFT $|1011\rangle$ exactly. All four vectors lie on the equator, so the information is entirely in the angles.

4.1.7 The inverse QFT

Phase estimation needs the QFT run *backwards*, so the inverse quantum Fourier transform is worth writing out explicitly. For any product of unitaries,

$$(ABCD)^{-1} = D^\dagger C^\dagger B^\dagger A^\dagger,$$

so the inverse circuit is the original circuit with the gate order reversed and every gate replaced by its adjoint. For the phase gates,

$$R_k^\dagger = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}^\dagger = \begin{pmatrix} 1 & 0 \\ 0 & e^{-2\pi i/2^k} \end{pmatrix},$$

while $H^\dagger = H$ and $\text{SWAP}^\dagger = \text{SWAP}$. In kernel form,

$$\text{QFT}^\dagger |k\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{-2\pi i j k / N} |j\rangle, \quad (4.6)$$

verified numerically for $n = 2, 3, 4$.

Exercise 4.2

Verify for $n = 2$ that the circuit of Fig. 4.7 inverts Fig. 4.3, by computing the product of the two circuit matrices.

Answer: the product is the identity, because each gate meets its own adjoint in the middle and cancels outward in pairs.

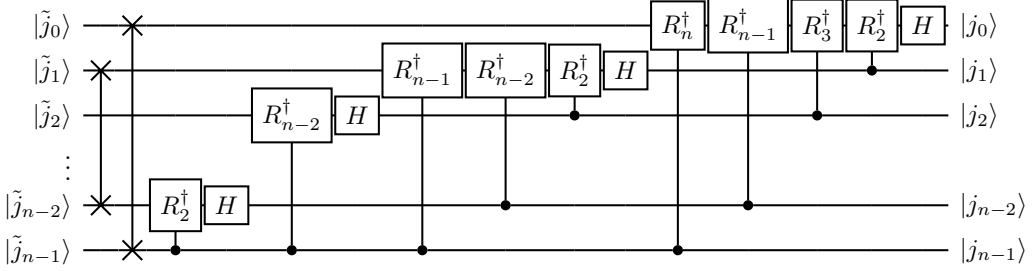


Figure 4.7: The inverse QFT: Fig. 4.4 reversed, with every R_k replaced by $R_k^\dagger$. The SWAP layer now comes first. The bottom wire, which needs only a Hadamard, is the first to be finished. Since $H^\dagger = H$, the Hadamards are unchanged. Verified numerically for $n = 2, 3, 4$: this circuit equals $\text{QFT}^\dagger$ exactly.

4.2 Quantum Phase Estimation

4.2.1 The problem

Given a unitary U and an eigenvector $|u\rangle$ with

$$U|u\rangle = e^{2\pi i\theta}|u\rangle, \quad 0 \leq \theta < 1, \quad (4.7)$$

estimate θ . That is the problem of quantum phase estimation.

I write the eigenvalue as $e^{2\pi i\theta}$ rather than $e^{i\phi}$ on purpose. It makes θ a number in $[0, 1)$, so its binary expansion $\theta = 0.\theta_1\theta_2\theta_3\cdots$ is exactly what the algorithm will read off, bit by bit.

4.2.2 Why the eigenvalue is a pure phase

Example 4.2

Show that every eigenvalue of a unitary matrix has modulus 1.

Solution. Let $U|v\rangle = \lambda|v\rangle$ with $|v\rangle \neq 0$. Taking the adjoint, $\langle v|U^\dagger = \lambda^*\langle v|$. Multiplying the two and using $U^\dagger U = I$ gives $\langle v|v\rangle = \langle v|U^\dagger U|v\rangle = |\lambda|^2 \langle v|v\rangle$. Since $\langle v|v\rangle \neq 0$ we may divide, giving $|\lambda|^2 = 1$, hence $|\lambda| = 1$.

So $\lambda = e^{i\phi}$ for some real ϕ , and setting $\phi = 2\pi\theta$ puts it in the form (4.7).

Exercise 4.3

Find the eigenvalues and eigenvectors of $A = \begin{pmatrix} 6 & -1 \\ 2 & 3 \end{pmatrix}$. Is A unitary?

Answer: $\det(A - \lambda I) = \lambda^2 - 9\lambda + 20 = 0$, so $\lambda = 4, 5$ with eigenvectors $(1, 2)^T$ and $(1, 1)^T$. The eigenvalues do not have modulus 1, so A is not unitary, consistent with Example 4.2.

4.2.3 The key idea: powers of U shift the binary point

Applying U repeatedly multiplies the phase:

$$U^x |u\rangle = e^{2\pi i x \theta} |u\rangle. \quad (4.8)$$

Take $x = 2^t$. Then the phase becomes $2\pi 2^t \theta$, and since $e^{2\pi i \nu} = 1$ for integer ν , only the fractional part matters:

$$2^t \theta \equiv 0.\theta_{t+1}\theta_{t+2}\cdots \pmod{1}.$$

Multiplying by 2^t shifts the binary point t places to the right, as Figure 4.8 illustrates. So controlled- U^{2^t} applied to a $|+\rangle$ control writes the phase $0.\theta_{t+1}\theta_{t+2}\cdots$ onto that control by phase kickback (Chapter 2, Section 2.5.3):

$$|+\rangle |u\rangle \mapsto \frac{|0\rangle + e^{2\pi i 2^t \theta} |1\rangle}{\sqrt{2}} \otimes |u\rangle.$$

Collect c such controls, one for each $t = 0, 1, \dots, c-1$, and assign them to wires so that the phases appear in the order (4.4) demands. The leading factor there carries $2\pi j/2$, a single bit. So counting qubit 0, the top wire, must take $t = c-1$, the *largest* power of U , which leaves it only the single bit $0.\theta_c$. Counting qubit $c-1$ at the bottom takes $t = 0$ and carries every bit. In general counting qubit q controls $U^{2^{c-1-q}}$. The result is exactly the product form (4.4) of QFT $|2^c \theta\rangle$, so applying QFT † recovers $|2^c \theta\rangle$ and a measurement reads off the bits, most significant on top.

That is the whole algorithm. The rest is bookkeeping.

$$\begin{array}{rcl}
 \theta & = & 0 \cdot \theta_1 \quad \theta_2 \quad \theta_3 \quad \theta_4 \quad \theta_5 \quad \cdots \\
 & \nearrow \times 2 & \\
 2\theta & = & \boxed{\theta_1} \cdot \theta_2 \quad \theta_3 \quad \theta_4 \quad \theta_5 \quad \cdots \equiv 0.\theta_2\theta_3\theta_4\theta_5\cdots \\
 & \nearrow \times 2^2 & \\
 2^3\theta & = & \boxed{\theta_1 \quad \theta_2 \quad \theta_3} \cdot \theta_4 \quad \theta_5 \quad \cdots \equiv 0.\theta_4\theta_5\cdots
 \end{array}$$

integer part: contributes $e^{2\pi i(\text{integer})} = 1$

Figure 4.8: Why powers of U are the right thing to apply. Multiplying θ by 2^t slides the binary point t places to the right. The bits that cross to the left of the point form an integer, and an integer contributes nothing to $e^{2\pi i 2^t \theta}$, so it is invisible to the measurement. What survives is the tail $0.\theta_{t+1}\theta_{t+2}\cdots$, and that is exactly the phase that controlled- U^{2^t} writes onto its control qubit. Choosing $t = c-1, c-2, \dots, 0$ therefore hands each counting qubit a different tail, which is the pattern (4.4) needs.

4.2.4 The circuit

The eigenvector register is a register, not a wire

In the circuit below, $|u\rangle$ is drawn as a single wire. That is a space-saving abbreviation, not a claim that one qubit suffices: $|u\rangle$ is an eigenvector of U , and U generally acts on many qubits. For Shor's algorithm the eigenvector register is the whole modular-arithmetic register. Where it matters, draw it out.

Notice also what the circuit does *not* require: we never need to know $|u\rangle$, only to be able to prepare it and to apply controlled powers of U . Section 4.2.7 shows what happens when we cannot prepare it exactly.

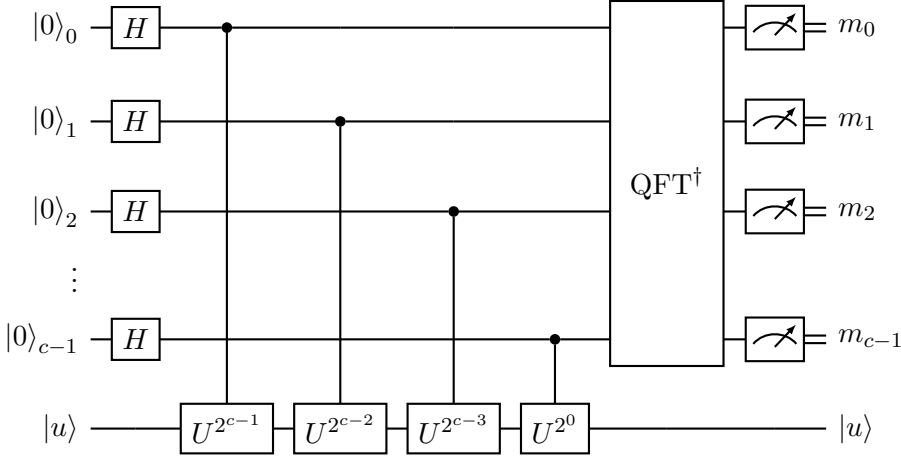


Figure 4.9: Quantum phase estimation with c counting qubits. Counting qubit q controls $U^{2^{c-1-q}}$, so the *top* wire controls the largest power $U^{2^{c-1}}$ and the bottom wire controls $U^{2^0} = U$. The inverse QFT converts the resulting phase pattern into a binary number. The eigenvector register is returned unchanged: $|u\rangle$ is never consumed, which is why the same $|u\rangle$ can be reused. The measured bits form $m = m_0 m_1 \cdots m_{c-1}$ with m_0 most significant, so $\theta \approx m/2^c = 0.m_0 m_1 \cdots m_{c-1}$. Verified numerically: for dyadic θ this circuit returns the correct m with probability 1, and it fails if the powers of U are assigned the other way up.

4.2.5 Step by step

(1) Initialize.

$$|\psi_1\rangle = |0\rangle^{\otimes c} \otimes |u\rangle.$$

(2) Hadamards on the counting register.

$$|\psi_2\rangle = \frac{1}{\sqrt{2^c}} \sum_{k=0}^{2^c-1} |k\rangle \otimes |u\rangle.$$

(3) Controlled powers. By (4.8) and phase kickback, each $|k\rangle$ picks up $e^{2\pi i k \theta}$:

$$|\psi_3\rangle = \frac{1}{\sqrt{2^c}} \sum_{k=0}^{2^c-1} e^{2\pi i \theta k} |k\rangle \otimes |u\rangle. \quad (4.9)$$

Equivalently, in product form,

$$|\psi_3\rangle = \frac{|0\rangle + e^{2\pi i 2^{c-1} \theta} |1\rangle}{\sqrt{2}} \otimes \cdots \otimes \frac{|0\rangle + e^{2\pi i 2^0 \theta} |1\rangle}{\sqrt{2}} \otimes |u\rangle,$$

where the leftmost factor is the top counting qubit, the one that controlled $U^{2^{c-1}}$. Comparing with (4.3), this is exactly $\text{QFT}|2^c \theta\rangle$ when $2^c \theta$ is an integer.

(4) Inverse QFT.

$$|\psi_4\rangle = |2^c\theta\rangle \otimes |u\rangle.$$

(5) Measure the counting register to get an integer m , and report

$$\theta \approx \frac{m}{2^c}. \quad (4.10)$$

Example 4.3

Estimate the phase of the S gate on eigenvector $|1\rangle$, using $c = 3$.

Solution. $S|1\rangle = i|1\rangle = e^{2\pi i(1/4)}|1\rangle$, so $\theta = 1/4 = 0.010_2$. With $c = 3$, $2^c\theta = 8 \times \frac{1}{4} = 2$, an integer, so step (4) gives exactly $|2\rangle = |010\rangle$ and the measurement returns $m = 2$ with probability 1. Then $\theta = 2/8 = 0.25 \checkmark$. Verified numerically along with $\theta = 1/8, 3/8, 1/2$ and $13/16$: whenever $2^c\theta$ is an integer, the correct m is obtained with probability 1.0000.

How many counting qubits?

Each counting qubit contributes one bit of θ , so c qubits give resolution 2^{-c} . If θ happens to be a dyadic rational $m/2^c$, the answer is exact. Otherwise, which is the usual case, we get the best c -bit approximation with high probability, quantified next.

The cost is not just qubits. Counting qubit q controls $U^{2^{c-1-q}}$, which is 2^{c-1-q} applications of U in a row, so the circuit applies U a total of $1 + 2 + \dots + 2^{c-1} = 2^c - 1$ times. Every extra bit of precision doubles that number, and doubling the number of bits squares it. In every application of phase estimation this count, the number of controlled applications of U , is the dominant cost.

4.2.6 When θ is not a dyadic rational

In general $2^c\theta$ is not an integer and the inverse QFT does not produce a single basis state. Starting from (4.9) and applying (4.6) with $N = 2^c$,

$$\text{QFT}^\dagger |\psi_3\rangle = \frac{1}{N} \sum_{m=0}^{N-1} \left(\sum_{k=0}^{N-1} e^{2\pi i(\theta - m/N)k} \right) |m\rangle = \sum_{m=0}^{N-1} \alpha_m |m\rangle.$$

The inner sum is a geometric series with ratio $a = e^{2\pi i(\theta - m/N)}$. Using

$$\sum_{k=0}^{N-1} a^k = \frac{1 - a^N}{1 - a} \quad (a \neq 1), \quad (4.11)$$

and writing $\delta = \theta - m/N$,

$$\alpha_m = \frac{1}{N} \left(\frac{1 - e^{2\pi i \delta N}}{1 - e^{2\pi i \delta}} \right) = \frac{1}{N} e^{i\pi \delta (N-1)} \frac{\sin(\pi \delta N)}{\sin(\pi \delta)},$$

where the second step factors $e^{i\pi \delta N}$ from the numerator and $e^{i\pi \delta}$ from the denominator. Hence

$$|\alpha_m|^2 = \frac{1}{N^2} \frac{\sin^2(\pi \delta N)}{\sin^2(\pi \delta)}, \quad \delta = \theta - \frac{m}{N}. \quad (4.12)$$

This formula was checked against direct simulation for $c = 4$, $\theta = 0.3$: it reproduces all sixteen outcome probabilities exactly.

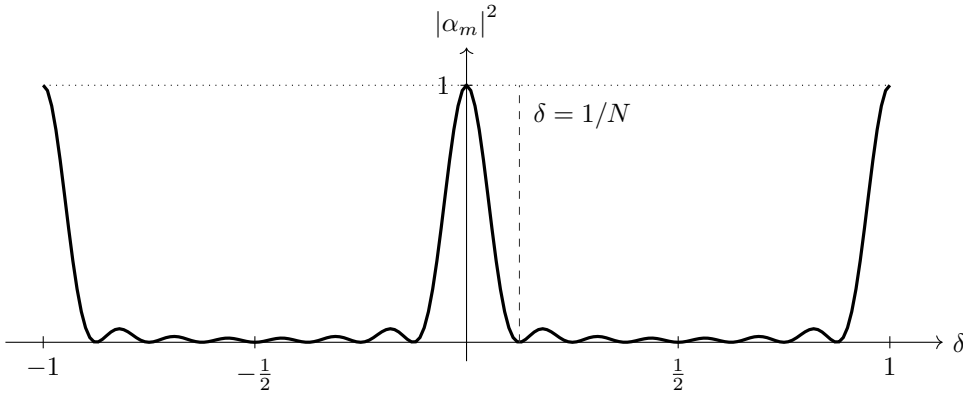


Figure 4.10: The phase-estimation outcome probability (4.12), drawn for $N = 2^c = 8$. The function is periodic with period 1 and peaks sharply at $\delta = 0$, i.e. when m/N equals θ . The first zero is at $\delta = 1/N$, so essentially all the probability sits on the one or two values of m closest to $2^c \theta$.

The small- δ limit is sinc^2 , not $\sin^2/(\cdot)$ squared

For $|\delta| \ll 1$ we may replace $\sin(\pi \delta)$ by $\pi \delta$ in the denominator of (4.12), giving

$$|\alpha_m|^2 \approx \left(\frac{\sin(\pi \delta N)}{\pi \delta N} \right)^2 = \text{sinc}^2(\pi \delta N).$$

The numerator of this sinc function is $\sin$, not $\sin^2$. Writing $(\sin^2(\pi \delta N)/(\pi \delta N))^2$ squares the sine an extra time and is badly wrong: at $\delta = 0.01$, $N = 16$ the true value is 0.919 while that expression gives 0.213. A quick check is $\delta \rightarrow 0$, where the correct expression tends to 1 as it must.

Success probability, and how to buy more of it

Let m_{best} be the integer nearest $2^c\theta$, so $|\delta| \leq 2^{-c-1}$. Evaluating (4.12) at the worst case gives the standard bound on the success probability

$$\Pr[m = m_{\text{best}}] \geq \frac{4}{\pi^2} \approx 0.405.$$

So a single run succeeds at least about 40% of the time, and in practice much more often. For $\theta = 0.3$ with $c = 4$ the simulated probability of the best outcome is 0.876, and the two nearest outcomes together carry 0.931.

To get n bits of θ correct with probability at least $1 - \epsilon$, use

$$c = n + \left\lceil \log_2 \left(2 + \frac{1}{2\epsilon} \right) \right\rceil$$

counting qubits. Here n is the number of bits of the *answer* you want, not the size of the register U acts on, and ϵ is the failure probability you are willing to accept. The extra qubits are cheap; the extra applications of U are not.

The formula is worth taking apart, because you will have to choose the size of a counting register again in Chapters 5 and 6. Write $c = n + p$: the first n counting qubits are the bits you want, and the p extra ones are insurance. With c qubits the outcome m is a c -bit number, and reading θ to n bits means m landing within 2^p or so of the best value m_{best} . From (4.12), the probability of an outcome d steps away from m_{best} falls off like $1/d^2$, since the numerator is bounded and $\sin^2(\pi\delta)$ grows like δ^2 . Summing that tail over every outcome more than $2^p - 1$ steps away gives a failure probability of at most $1/(2(2^p - 2))$ [24]. Setting this equal to ϵ and solving for p gives $p = \log_2(2 + 1/(2\epsilon))$, rounded up. In plain words, the insurance is cheap in qubits: $\epsilon = 0.01$ costs $p = 6$ extra qubits, and every further halving of ϵ costs about one more. It is not cheap in runtime, since each extra counting qubit doubles the number of applications of U . I checked the formula numerically for $n = 2, 3, 4$ and ϵ from 0.25 down to 0.01, scanning θ finely over $[0, 1)$: the probability of an n -bit-accurate answer never falls below $1 - \epsilon$.

4.2.7 Superpositions of eigenvectors

In practice we often cannot prepare an exact eigenvector. Suppose instead we prepare

$$|\psi\rangle = \sum_j c_j |u_j\rangle, \quad U |u_j\rangle = e^{2\pi i \theta_j} |u_j\rangle.$$

Because the circuit is linear, it acts on each term independently:

$$|\psi_4\rangle = \sum_j c_j |\tilde{\theta}_j\rangle \otimes |u_j\rangle, \quad (4.13)$$

where $|\tilde{\theta}_j\rangle$ is the estimate register for θ_j . Measuring the counting register returns θ_j with probability $|c_j|^2$, and *collapses* the second register onto the corresponding $|u_j\rangle$. The subscripted c_j are expansion amplitudes; the bare c is still the counting-register width.

This is a feature, not a workaround

Equation (4.13) says phase estimation samples the eigenvalue spectrum of U according to the overlap of your input state with each eigenvector. You need no knowledge of the eigenvectors at all.

This is exactly how quantum chemistry algorithms work. Prepare a cheap approximation to the ground state, say a Hartree–Fock state. Then run phase estimation on e^{-iHt} , and you obtain the ground-state energy with probability equal to the squared overlap [23]. Getting that overlap large is the central practical difficulty of the whole field.

It also explains a limitation of the iterative method in Section 4.3. That method measures and reuses a single ancilla across rounds, so a superposition input does not give a clean answer. Different rounds can collapse onto different eigenvectors.

Exercise 4.4

Suppose the counting register is initialized to $|1\rangle^{\otimes c}$ instead of $|0\rangle^{\otimes c}$. Does phase estimation still work?

Answer: $H|1\rangle = |-\rangle$, so step (2) produces $\frac{1}{\sqrt{N}} \sum_k (-1)^{k \cdot 1} |k\rangle$ instead of a uniform superposition. The controlled powers then build $\frac{1}{\sqrt{N}} \sum_k (-1)^{|k|} e^{2\pi i \theta k} |k\rangle$, which is not the QFT of any single basis state, and the inverse QFT does not concentrate on one outcome. The algorithm fails. The uniform superposition with *all positive* amplitudes is essential.

4.3 Iterative Phase Estimation

4.3.1 Motivation

The circuit of Fig. 4.9 needs c counting qubits to get c bits of precision, on top of the register that holds $|u\rangle$. On near-term hardware that is expensive twice over. Qubits are scarce, and every extra qubit adds decoherence and crosstalk to a circuit that must already stay coherent through $2^c - 1$ controlled applications of U .

Iterative phase estimation (IPE) trades qubits for repetitions. It uses **one** ancilla qubit and extracts the bits of θ one at a time, least significant first. Each measured bit is fed forward as a classical correction. Precision is set by the number of *rounds*, not the number of qubits. The idea goes back to Kitaev's original formulation of phase estimation [18].

Throughout this section c is again the number of bits of θ we are after, exactly as in Section 4.2. Standard phase estimation spends c qubits on it; IPE spends c rounds on one qubit. The bits are written $\theta_1\theta_2\cdots\theta_c$, most significant first, so they are the bits of the measured integer m of Section 4.2 in the same order: $\theta_1 = m_0$ and $\theta_c = m_{c-1}$.

4.3.2 Where the idea comes from

Look again at the inverse QFT in Fig. 4.7. Once the opening SWAP layer is done, the bottom wire needs only a Hadamard. No controlled rotations act on it before its measurement. The next wire up needs one rotation, conditioned on that first result. And so on.

But a rotation conditioned on a qubit that has already been measured is just a rotation conditioned on a *classical bit*. So we never need those qubits to coexist: measure one, record the bit, apply the corresponding classical correction to the next round, and reuse the same physical qubit.

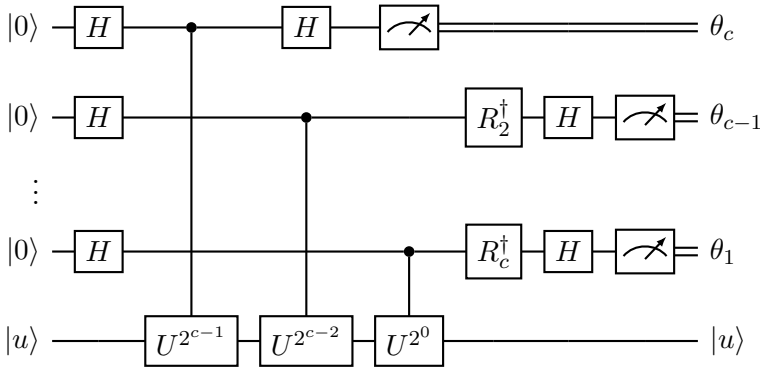


Figure 4.11: Phase estimation with the inverse QFT written out as Hadamards and controlled rotations. To keep the drawing readable, only one rotation per wire is shown and the control lines are left out: each $R_l^\dagger$ is controlled by a wire above it, after that wire's Hadamard. The top counting qubit controls the largest power $U^{2^{c-1}}$, so the only phase it carries is $2\pi 0.\theta_c$. It is measured with no incoming rotations at all, before anything else happens, and it returns the *least* significant bit θ_c . Each qubit below it needs only rotations conditioned on results already obtained. That is what makes the qubits separable in time. The SWAP layer of the inverse QFT has been absorbed into the output labels here, which is why they run in the opposite order to Fig. 4.9.

4.3.3 The algorithm

Write $\theta = 0.\theta_1\theta_2 \cdots \theta_c$ in binary, i.e.

$$\theta = \frac{\theta_1}{2} + \frac{\theta_2}{4} + \cdots + \frac{\theta_c}{2^c}.$$

Round 1 (least significant bit θ_c). Apply controlled- $U^{2^{c-1}}$ to a $|+\rangle$ ancilla. The kickback phase is

$$2\pi 2^{c-1}\theta \equiv 2\pi (0.\theta_c) = \pi\theta_c \pmod{2\pi},$$

because multiplying by 2^{c-1} shifts all the higher bits past the binary point, where they become integers and vanish. So the ancilla is

$$\frac{|0\rangle + e^{i\pi\theta_c}|1\rangle}{\sqrt{2}} = \begin{cases} |+\rangle & \theta_c = 0, \\ |-\rangle & \theta_c = 1, \end{cases}$$

and a Hadamard followed by measurement returns θ_c *deterministically*.

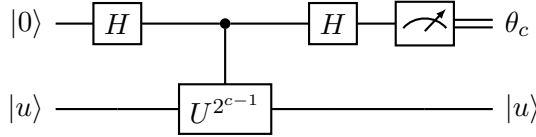


Figure 4.12: Round 1 of IPE: recovering the least significant bit θ_c . Watch the exponent: it is 2^{c-1} , not $2^c - 1$.

Round 2 (bit θ_{c-1}). Now apply controlled- $U^{2^{c-2}}$. The kickback phase is $2\pi (0.\theta_{c-1}\theta_c)$, which contains the bit we want *and* the bit we already know. Subtract the known part with a classical correction: apply $R_2^\dagger = P(-\pi/2)$ if and only if $\theta_c = 1$. What remains is $\pi\theta_{c-1}$, and a Hadamard plus measurement returns θ_{c-1} deterministically.

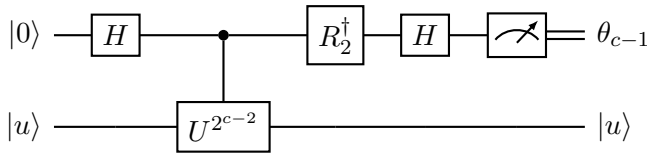


Figure 4.13: Round 2 of IPE. The gate $R_2^\dagger$ is applied classically, conditioned on the bit θ_c measured in round 1.

Round k (bit θ_{c-k+1}). Apply controlled- $U^{2^{c-k}}$, then undo every previously measured bit:

$$\omega_k = -2\pi \sum_{j=c-k+2}^c \frac{\theta_j}{2^{j-(c-k)}}, \quad (4.14)$$

implemented as a single $P(\omega_k)$, or as the product of $R_l^\dagger$ for each set bit, with $l = j - (c - k)$ running over $2, 3, \dots, k$. Then Hadamard and measure. Check the formula at $k = 2$: the sum has the single term $j = c$, giving $\omega_2 = -2\pi\theta_c/2^2 = -\pi\theta_c/2$, which is the $R_2^\dagger = P(-\pi/2)$ of round 2 above. Figure 4.15 shows how the rounds fit together, with the measured bits accumulating in a classical register that sets the next correction.

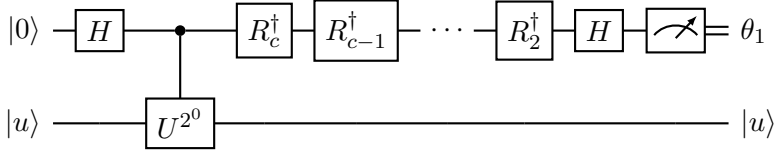


Figure 4.14: The final round of IPE, recovering the most significant bit θ_1 . Each $R_l^\dagger$ is applied only if the corresponding previously measured bit was 1.

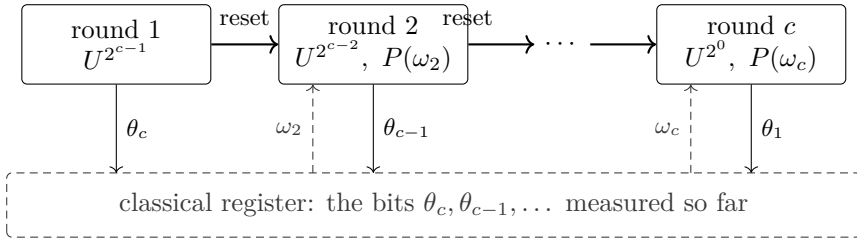


Figure 4.15: How the rounds of iterative phase estimation fit together. Every round uses the same single ancilla: prepare it in $|+\rangle$, apply controlled- $U^{2^{c-k}}$, apply the classical correction $P(\omega_k)$ of (4.14), then Hadamard, measure and reset. Solid arrows down are measured bits being recorded; dashed arrows up are those recorded bits setting the correction angle for a later round. Nothing quantum passes between rounds, which is why one physical qubit is enough. Contrast Fig. 4.11, where the same rotations are quantum-controlled and every counting qubit must therefore coexist.

Two things to get right

The exponents are powers of two. Round k applies $U^{2^{c-k}}$, so the sequence across rounds is $U^{2^{c-1}}, U^{2^{c-2}}, \dots, U^2, U$. It is easy to write U^{N-1} and U^{N-2} with $N = 2^c$, but those are different numbers: for $c = 4$ the first round needs U^8 , not U^{15} , and the second needs U^4 , not U^{14} .

The corrections are inverse phase gates. The whole point is to *remove* phase that earlier bits contributed, so the gates are $R_k^\dagger = P(-2\pi/2^k)$, not R_k . Applying R_k doubles the unwanted phase instead of canceling it, and the measurement stops being deterministic.

Example 4.4

Estimate the phase of the T gate using IPE.

Solution. $T|1\rangle = e^{i\pi/4}|1\rangle = e^{2\pi i(1/8)}|1\rangle$, so $\theta = 1/8 = 0.001_2$ and $c = 3$ suffices: $\theta_1 = 0, \theta_2 = 0, \theta_3 = 1$.

Round 1. Apply controlled- T^4 . Since $T^4 = Z$, the kickback phase is $2\pi \cdot 4 \cdot \frac{1}{8} = \pi$, giving $|-\rangle$; the Hadamard sends this to $|1\rangle$, so we measure $\theta_3 = 1 \checkmark$.

Round 2. Apply controlled- $T^2 = \text{controlled-}S$. Kickback phase $2\pi \cdot 2 \cdot \frac{1}{8} = \pi/2$. Correct with $R_2^\dagger = P(-\pi/2)$ since $\theta_3 = 1$: the phase becomes 0, giving $|+\rangle$ and measurement $\theta_2 = 0 \checkmark$.

Round 3. Apply controlled- T . Kickback phase $2\pi \cdot \frac{1}{8} = \pi/4$. Correct with $R_3^\dagger = P(-\pi/4)$ (since $\theta_3 = 1$; $\theta_2 = 0$ contributes nothing): phase becomes 0, giving $\theta_1 = 0 \checkmark$.

Reassembling, $\theta = 0.001_2 = 1/8 \checkmark$.

The full procedure was verified numerically for $\theta = \frac{1}{8}, \frac{3}{8}, \frac{5}{8}, \frac{13}{16}, \frac{3}{16}, \frac{15}{16}$; in every case the recovered bits reproduce θ exactly.

Let me assemble the cost of IPE, with c the number of bits of θ wanted, as before. Round k applies controlled- $U^{2^{c-k}}$ once, which is 2^{c-k} controlled applications of U in a row, so over the c rounds the applications of U total $2^{c-1} + 2^{c-2} + \dots + 1 = 2^c - 1$. That is the same number as the c -qubit circuit of Fig. 4.9, and it is still the dominant cost. What changes is everything around it. In qubits, IPE needs one ancilla plus the eigenvector register, instead of c counting qubits plus that register. In single-qubit gates, round k needs two Hadamards and one phase correction $P(\omega_k)$, so the whole run needs $3c$ of them, against the $O(c^2)$ gates of the inverse QFT. In classical work, each round needs one mid-circuit measurement and an update of the angle ω_k from the bits already recorded, a handful of arithmetic operations. The table collects the comparison.

Exercise 4.5

Run IPE by hand for the S gate on $|1\rangle$, with $c = 2$. What are the two bits, and which correction is needed?

Answer: $\theta = 1/4 = 0.01_2$, so $\theta_1 = 0, \theta_2 = 1$. Round 1 applies controlled- $S^2 = \text{controlled-}Z$, giving phase π and $\theta_2 = 1$. Round 2 applies controlled- S with correction $R_2^\dagger$, giving phase 0 and $\theta_1 = 0$.

QPE versus IPE

	Standard QPE	Iterative QPE
Ancilla qubits	c	1
Circuit depth	one long circuit	c short circuits
Total applications of U	$2^c - 1$	$2^c - 1$
Needs mid-circuit measurement	no	yes
Needs classical feed-forward	no	yes
Superposition of eigenvectors	works (samples spectrum)	unreliable

The total work is the same; IPE moves it from space into time. That is a good trade on hardware where qubits are the binding constraint and mid-circuit measurement with fast classical control is available. That is exactly the situation on current devices.

Exercise 4.6

What happens in IPE if θ is not a dyadic rational, say $\theta = 1/3$?

Answer: the measurements are no longer deterministic. Each round returns the “correct” bit only with some probability, and an error in an early (low-order) round corrupts the corrections in all later rounds. Robust variants repeat each round several times and take a majority vote, or use Bayesian updating. This error-propagation sensitivity is IPE’s main practical weakness compared with standard QPE, where a single bad outcome merely shifts m by one.

4.4 Summary

- QFT $|j\rangle = \frac{1}{\sqrt{N}} \sum_k e^{2\pi i jk/N} |k\rangle$ with $N = 2^n$. For one qubit it is exactly H .
- The QFT of a basis state *factorizes*: $\bigotimes_{l=1}^n (|0\rangle + e^{2\pi i 0 \cdot j_{n-l} \cdots j_{n-1}} |1\rangle) / \sqrt{2}$, with j_0 the most significant bit on the top wire. This is why an efficient circuit exists, and why the output is never entangled.
- The circuit costs about $\frac{n^2}{2} + n = O(n^2)$ gates on n qubits: n Hadamards, $n(n-1)/2$ two-qubit controlled- R_k phase gates, plus a layer of $\lfloor n/2 \rfloor$ SWAPs to reverse the wire order, which the circuit produces backwards.
- $R_k = P(2\pi/2^k)$ is a *phase* gate, not a rotation gate: $R_k \neq R_z(2\pi/2^k)$, and since every R_k here is controlled, the global phase between them is a real difference. In particular $R_1 = Z$, and R_1 does not appear in the circuit; the

superposition comes from H .

- The QFT is not a fast Fourier transform, because you cannot read the amplitudes out. It is useful for extracting one global feature, such as a phase.
- In practice the QFT is truncated: dropping R_k for $k > \log_2(n/\epsilon)$ gives operator-norm error at most ϵ with $O(n \log(n/\epsilon))$ gates, which is $O(n \log n)$ for fixed ϵ .
- Phase estimation: counting qubit q controls $U^{2^{c-1-q}}$, so the top qubit takes the largest power; kickback writes $2^{c-1-q}\theta$ onto it, producing QFT $|2^c\theta\rangle$, and the inverse QFT then reads off $\theta = 0.m_0m_1 \cdots m_{c-1}$ with m_0 on top.
- Exact when $2^c\theta$ is an integer. Otherwise $|\alpha_m|^2 = \sin^2(\pi\delta N) / (N^2 \sin^2(\pi\delta))$, peaked at $\delta = 0$, with success probability at least $4/\pi^2$.
- Cost: c counting qubits and $2^c - 1$ controlled applications of U . Each extra bit of precision doubles the runtime.
- On a superposition of eigenvectors, QPE samples eigenvalue θ_j with probability $|c_j|^2$ and collapses onto $|u_j\rangle$.
- IPE recovers the same c bits with one ancilla, using mid-circuit measurement and classical feed-forward. Round k applies $U^{2^{c-k}}$ and corrects with $R_l^\dagger$ gates.

4.5 End-of-Chapter Problems

- 4.1 Write out the 4×4 matrix of QFT_2 explicitly and verify it is unitary.
- 4.2 Compute $\text{QFT}|101\rangle$ for $n = 3$ using the product form (4.4), and check it against the direct sum (4.2).
- 4.3 Show that $\text{QFT}|0\rangle^{\otimes n}$ is the uniform superposition, and hence that QFT and $H^{\otimes n}$ agree on that one input. Give an input on which they disagree.
- 4.4 Prove that the QFT of any computational basis state is a product state (never entangled). Is the same true of the QFT of an arbitrary state?
- 4.5 Verify the gate count $\frac{n(n+1)}{2} + \lfloor n/2 \rfloor$ by drawing the circuit for $n = 4$ and counting.
- 4.6 Show that $R_k^\dagger = R_k^{2^k-1}$, and explain why this means an inverse phase gate costs no more than a forward one on hardware with only positive-angle gates. Note that this identity holds because $R_k^{2^k} = I$, which is true of the phase gate $P(2\pi/2^k)$ but not of $R_z(2\pi/2^k)$.

- 4.7** For the approximate QFT with all R_k , $k > k_{\max}$, dropped, estimate the total error as a function of n and $k_{\max}$. *Hint:* bound the error of each dropped gate by its phase angle.
- 4.8** Show that the c -qubit phase-estimation circuit applies U a total of $2^c - 1$ times, and that this is what makes precision exponentially expensive.
- 4.9** For $\theta = 1/3$ and $c = 4$, compute $|\alpha_m|^2$ for all sixteen values of m using (4.12). Which m is most likely, and what is the resulting estimate of θ ?
- 4.10** Derive the bound $\Pr[m = m_{\text{best}}] \geq 4/\pi^2$ from (4.12). *Hint:* use $|\delta| \leq 2^{-c-1}$ and $\sin x \leq x$, $\sin x \geq 2x/\pi$ on $[0, \pi/2]$.
- 4.11** Phase estimation is run on $U = Z$ with the input register in $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. What are the possible measurement outcomes and their probabilities, and what is the state of the second register afterwards?
- 4.12** Show that if $|u\rangle$ is not an eigenvector of U , the counting register and the second register end up entangled, and explain why this is what makes Eq. (4.13) work.
- 4.13** Work out all rounds of IPE for $\theta = 5/8$ with $c = 3$, listing the gate applied and the correction at each round.
- 4.14** Explain why IPE gives unreliable results when the second register is a superposition of eigenvectors, whereas standard QPE handles that case cleanly.
- 4.15** Estimate the number of physical applications of U needed to obtain 20 bits of θ , for both standard QPE and IPE. Comment on feasibility.

Research Problems and Projects

The problems above have a right answer and a fixed amount of work. The three below do not. Each is a project of several weeks, ending in a written report with code, figures and an honest account of what did not work.

R4.1 How far the QFT can be truncated before phase estimation breaks.

The chapter asserts that dropping every R_k with $k > k_{\max}$, as in Fig. 4.5, costs error ϵ once $k_{\max} \approx \log_2(n/\epsilon)$. That is a bound on the operator-norm error of the QFT alone. It is not a statement about the quantity you actually care about, which is the probability that phase estimation returns the right m . Build a state-vector simulator for the circuit of Fig. 4.9 with the inverse QFT replaced by its truncated version, and measure $\Pr[m = m_{\text{best}}]$ directly as a function of c and $k_{\max}$, for both dyadic and generic θ . Compare what you measure against (4.12), which assumes an exact inverse QFT. Then add

a depolarizing-noise model with per-gate error rate p and repeat. There is a crossover worth hunting for: below some p the truncation only hurts, but above it the gates you removed were doing more damage than good, so truncating *improves* the answer. *Deliverable*: a measured surface of success probability over $(c, k_{\max})$ at three or four values of p , a fitted threshold $k_{\max}^*(c, p)$ below which the algorithm fails, and a plot of the crossover in p . State clearly which of your curves are estimates from finite sampling and give error bars. The asymptotic bound is known; the shape of the curve at the small c a real device can run is not, and that is the useful part.

R4.2 What the cost of phase estimation actually is, across the literature.

This chapter gives two resource claims: $2^c - 1$ applications of U , and $c = n + \lceil \log_2(2 + 1/2\epsilon) \rceil$ counting qubits for n correct bits with confidence $1 - \epsilon$. Both are quoted everywhere and proved in relatively few places. Track them back to [24] and to Kitaev's original treatment in [18], and record exactly what each proves, under which assumptions about $|u\rangle$ and about the available gate set. Then set that beside the modern replacements for textbook phase estimation: qubitization [21], quantum signal processing and its generalization [20, 13], the unified account in [22], and the amplitude-estimation line running from [5] to the maximum-likelihood method of [28]. These papers do not use a common cost metric. Some count queries to a block encoding, some count applications of U , some count T gates, and the confidence parameter is not always defined the same way. *Deliverable*: a survey of ten to fifteen pages built around one table that puts every source on a single cost metric of your choosing, with the conversion shown explicitly, plus a numerical verification of at least two of the bounds at small instance sizes. Be honest about what this is. The asymptotics are settled, so the contribution is a like-for-like comparison and an accurate attribution, not a new bound.

R4.3 Standard phase estimation against the iterative variant, on real hardware.

Section 4.3 argues that IPE trades space for time and that the total work is unchanged. That argument ignores everything the trade costs in practice: mid-circuit measurement is slow, ancilla reset is imperfect, and the classical feed-forward of Fig. 4.15 adds a latency between rounds during which the eigenvector register keeps decohering. Implement both algorithms for a phase gate $U = P(2\pi\theta)$ on $|1\rangle$, and then for a two-qubit $U = e^{-iHt}$ with H a small Ising Hamiltonian whose spectrum you can compute exactly. Run each on a noisy simulator and, if you have access, on hardware that supports mid-circuit measurement with feed-forward. Measure the per-round bit-error rate, the total number of applications of U , the total wall-clock time, and the error in the recovered θ . Then implement the two robustness repairs mentioned in Section 4.3, majority voting over repeated rounds and Bayesian updating, and measure what each buys. *Deliverable*: a resource-count table against the

precision c for both algorithms on both platforms, a plot of recovered-phase error against the number of applications of U , and an identified value of c at which feed-forward latency rather than gate count becomes the limiting cost. Report the non-dyadic case as well as the dyadic one. The interesting result is the crossover, and it may well go the other way from what the chapter's table suggests.

Chapter 5

Period Finding, Factoring and Discrete Logarithms

This is the chapter the previous four have been building toward. Chapter 4 gave us phase estimation; here we use it to break the two problems on which almost all deployed public-key cryptography rests: integer factoring and the discrete logarithm. Both fall to a single quantum subroutine, due to Shor[26].

The logical chain is short and worth stating up front:

- (i) **Phase estimation** (Chapter 4) finds the eigenvalue phase of a unitary.
- (ii) Applied to the *modular multiplication* operator $V|y\rangle = |ay \bmod N\rangle$, the phase is s/r , where r is the *order* of a modulo N . So phase estimation is order finding.
- (iii) Order finding is period finding for $f(x) = a^x \bmod N$.
- (iv) A classical reduction, known since the 1970s, turns order finding into factoring.

Only step (ii) is quantum. Steps (i), (iii) and (iv) are bookkeeping and classical number theory. That is why I have put as much number theory as quantum mechanics into this chapter.

What you should be able to do after this chapter

- State the period-finding problem precisely, including the condition on f that makes it solvable.
- Run the continued-fractions algorithm to recover r from a measurement.
- Carry out every step of Shor's algorithm on a small N by hand.
- Explain which parts are quantum and which are classical, and count the qubits and gates each needs.
- Implement all of it: the chapter gives enough detail to write the code.
- Explain what Shor's algorithm does and does not break in RSA.
- Set up the discrete-logarithm circuit and recover s from the two measurement outcomes.

5.1 Period Finding

5.1.1 The problem

We are given an oracle for a function

$$f : \{0, 1\}^t \longrightarrow \{0, 1\}^n \quad (5.1)$$

with the promise that f is periodic: $f(x + r) = f(x)$ for some unknown $0 < r < 2^L$ with $L \leq t$. The task is to find r .

The output register must be wide enough

You may be tempted to state the problem with a single-bit output, $f : \{0, 1\}^t \rightarrow \{0, 1\}$. That version is **not solvable** by the algorithm below. Period finding requires f to be *injective within one period*:

$$f(x_1) = f(x_2) \iff x_1 \equiv x_2 \pmod{r}. \quad (5.2)$$

Measuring the output register must collapse the input register onto a *single* coset $\{x_0, x_0 + r, x_0 + 2r, \dots\}$. If f takes only two values, then for $r > 2$ several residues share an output. The collapse then leaves a *union* of cosets, and the Fourier transform of a union does not encode r . Shor's function $f(x) = a^x \bmod N$ satisfies (5.2) automatically, so this never bites in practice, but I want the condition stated.

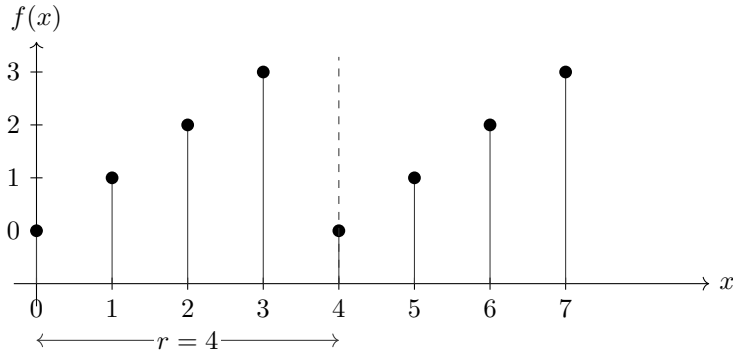


Figure 5.1: A periodic function with $r = 4$, here $f(x) = x \bmod 4$ on three input bits. The pattern over $x = 0, 1, 2, 3$ repeats exactly at $x = 4, 5, 6, 7$. The four values within one period are *distinct*. This is condition (5.2), and without it the algorithm below does not work.

What a one-bit output does to the outcomes

Take $t = 3$ and $f = (0, 1, 1, 0, 0, 1, 1, 0)$, which does satisfy $f(x + 4) = f(x)$. Measuring the output gives the set $\{0, 3, 4, 7\}$, and the inverse QFT then yields $m \in \{0, 2, 6\}$, whereas $r = 4$ requires $m \in \{0, 2, 4, 6\}$: the wrong answer. With $g(x) = x \bmod 4$, the two-bit function plotted above, the collapse gives a clean coset such as $\{1, 5\}$ and the outcomes are exactly $m \in \{0, 2, 4, 6\}$, i.e. $m/8 \in \{0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}\}$, giving $r = 4$. Both computations verified numerically.

The oracle is the standard one of Chapter 2:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle. \quad (5.3)$$

5.1.2 The algorithm

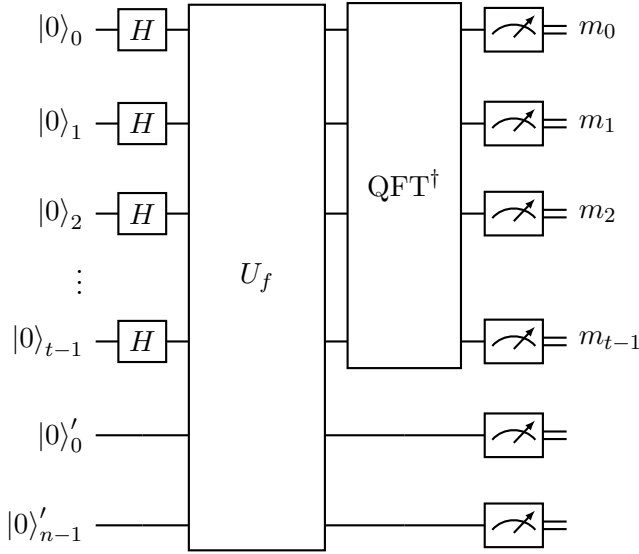


Figure 5.2: The period-finding circuit. The top register (t qubits) holds the argument, the bottom register (n qubits) the function value. The output register is measured (or simply discarded); this collapses the input register onto a single coset. The inverse QFT then converts the coset's periodicity into a measurable multiple of $2^t/r$. As everywhere in this book, the top wire of each register is qubit 0 and carries the *most* significant bit, so the measured bits assemble as $m = m_0 2^{t-1} + m_1 2^{t-2} + \dots + m_{t-1} 2^0$.

- (1) Start in $|0\rangle^{\otimes t} |0\rangle^{\otimes n}$.
- (2) Hadamards on the first register:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle |0\rangle.$$

- (3) Query the oracle:

$$|\psi_3\rangle = \frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle |f(x)\rangle. \quad (5.4)$$

The two registers are now entangled.

- (4) **Measure the second register.** Suppose the outcome is $f(x_0)$. By (5.2) the first register collapses onto exactly the inputs congruent to x_0 modulo r :

$$|\psi_4\rangle = \sqrt{\frac{r}{2^t}} \sum_{j=0}^{2^t/r-1} |x_0 + jr\rangle, \quad (5.5)$$

assuming for simplicity that r divides 2^t .

(5) Inverse QFT. The inverse quantum Fourier transform turns a comb of spacing r into a comb of spacing $2^t/r$:

$$|\psi_5\rangle = \frac{1}{\sqrt{r}} \sum_{l=0}^{r-1} e^{-2\pi i l x_0/r} \left| \frac{2^t l}{r} \right\rangle. \quad (5.6)$$

Here is the calculation, since the sign of that phase is easy to get wrong. The inverse transform sends $|x\rangle \mapsto 2^{-t/2} \sum_m e^{-2\pi i x m/2^t} |m\rangle$, the conjugate of the QFT convention fixed in Chapter 4. Applying it to (5.5) gives

$$|\psi_5\rangle = \frac{\sqrt{r}}{2^t} \sum_{m=0}^{2^t-1} e^{-2\pi i x_0 m/2^t} \left(\sum_{j=0}^{2^t/r-1} e^{-2\pi i j r m/2^t} \right) |m\rangle.$$

The inner sum is a geometric series in $e^{-2\pi i r m/2^t}$. It equals $2^t/r$ when $r m/2^t$ is an integer, that is when $m = 2^t l/r$ for some $l \in \{0, \dots, r-1\}$, and it vanishes for every other m . Putting $m = 2^t l/r$ into the leading phase turns $e^{-2\pi i x_0 m/2^t}$ into $e^{-2\pi i l x_0/r}$, and the prefactor becomes $(\sqrt{r}/2^t)(2^t/r) = 1/\sqrt{r}$. That is (5.6). The minus sign is inherited from the inverse transform. The end-of-chapter problems ask you to confirm that this phase does not affect the measurement.

The unknown offset x_0 has moved entirely into a phase, where it is harmless: the measurement probabilities do not depend on it. I want you to notice what has just happened. The offset we could not control has been made irrelevant. Figure 5.3 draws the two combs for $t = 5$ and $r = 4$.

(6) Measure the first register to get

$$m = \frac{2^t l}{r} \quad \implies \quad \frac{m}{2^t} = \frac{l}{r}, \quad (5.7)$$

for a uniformly random $l \in \{0, \dots, r-1\}$. Here m is the integer assembled from the measured bits with qubit 0 (the top wire) as the most significant, $m = \sum_{k=0}^{t-1} m_k 2^{t-1-k}$. Reading the register the other way round replaces m by the integer whose binary digits are reversed. That reversed integer is in general not a multiple of $2^t/r$ at all. The continued-fraction step then returns a denominator that has nothing to do with r , and it does so silently, without any error message.

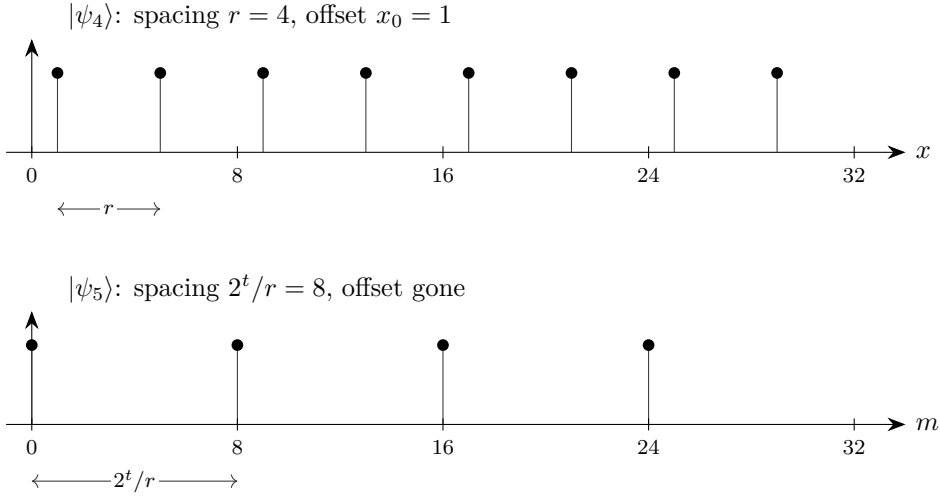


Figure 5.3: The inverse QFT turns one comb into another, and inverts the spacing. Upper panel: the state $|\psi_4\rangle$ of (5.5) for $t = 5$ and $r = 4$, an equal superposition of the eight arguments $x = x_0 + jr$ with $x_0 = 1$. Lower panel: the state $|\psi_5\rangle$ of (5.6), supported on the four multiples of $2^t/r = 8$. Every dot in a panel carries the same amplitude magnitude, so the offset x_0 is invisible to the measurement. It has moved out of the support of the state and into a phase.

Example 5.1

With $r = 4$ and $t = 4$, what outcomes are possible?

Solution. $m = 2^4 l / 4 = 4l$, so

l	m	binary	$m/2^t$	in lowest terms
0	0	0000	0/16	0/1
1	4	0100	4/16	1/4
2	8	1000	8/16	1/2
3	12	1100	12/16	3/4

Each occurs with probability $\frac{1}{4}$. The binary column is written most significant bit first, which is also the order the four counting wires are read from top to bottom. For $m = 4$ the wires read $m_0 m_1 m_2 m_3 = 0100$: the second wire from the top is the one that comes out in $|1\rangle$.

Do not take the LCM of the “ r values”

Example 5.1 shows why a single run is not enough. But the repair is not to “compute r for each outcome and take the least common multiple”.

Look at the table. The outcome $l = 0$ gives $m/2^t = 0/1$, whose denominator is 1: it carries *no information at all* and must be discarded. The outcome $l = 2$ gives $1/2$, whose denominator is 2, not 4, because l and r shared the factor 2. Only $l = 1$ and $l = 3$, which are coprime to r , reveal r directly.

The correct procedure is:

- (i) Discard the outcome $m = 0$.
- (ii) Apply the continued-fractions algorithm to $m/2^t$ to get a fraction l'/r' in lowest terms with $r' < N$.
- (iii) The candidate is r' . Test it: check whether $f(x) = f(x + r')$, or in Shor’s case whether $a^{r'} \equiv 1 \pmod{N}$.
- (iv) If the test fails, l and r shared a factor. Take the least common multiple of the denominators from *two* runs, or simply repeat.

The probability that a uniformly random $l \in \{0, \dots, r-1\}$ is coprime to r is $\phi(r)/r$, where Euler’s totient $\phi(r)$ counts the integers below r coprime to r , and a classical estimate gives $\phi(r)/r \geq c/\log \log r$ for a constant c . So the expected number of runs of the quantum step before one returns r outright is $O(\log \log r)$: fewer than fourteen for any r below 2^{2048} , and two or three for the examples in this chapter. The two-run least common multiple of step (iv) makes it a constant.

5.1.3 Continued fractions

The chapter uses continued fractions repeatedly, so here is the algorithm.

Given a rational $m/2^t$, we want the fraction l'/r' with the smallest denominator that approximates it to within $1/2^{t+1}$. Write

$$\frac{m}{2^t} = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \dots}}}$$

by repeatedly taking the integer part and inverting the remainder. Truncating after k terms gives the k -th *convergent* p_k/q_k , computed by the recurrence

$$p_k = a_k p_{k-1} + p_{k-2}, \quad q_k = a_k q_{k-1} + q_{k-2}, \quad (5.8)$$

with $p_{-1} = 1$, $p_{-2} = 0$, $q_{-1} = 0$, $q_{-2} = 1$.

Example 5.2

Recover r from the measurement $m = 12$ with $t = 4$.

Solution. $m/2^t = 12/16 = 3/4$. Expanding:

$$\frac{3}{4} = 0 + \frac{1}{1 + \frac{1}{3}}, \quad \text{so} \quad a_0 = 0, \ a_1 = 1, \ a_2 = 3.$$

The convergents are $p_0/q_0 = 0/1$, $p_1/q_1 = 1/1$, $p_2/q_2 = 3/4$. The last has denominator 4, so the candidate is $r' = 4$. Verify with the oracle: check $f(0) = f(4)$. It holds, so $r = 4$.

Example 5.3

A run of Shor's algorithm on $N = 55$ with $a = 2$ and $t = 12$ returns $m = 1843$. Find r .

Solution. $m/2^t = 1843/4096$. The continued-fraction expansion is $[0; 2, 4, 2, 50, 1, 3]$, and the recurrence (5.8) gives convergents

$$\frac{0}{1}, \quad \frac{1}{2}, \quad \frac{4}{9}, \quad \frac{9}{20}, \quad \frac{454}{1009}, \quad \dots$$

We take the last convergent whose denominator is smaller than $N = 55$: that is $9/20$, so the candidate is $r' = 20$. Testing, $2^{20} \equiv 1 \pmod{55}$ ✓, so $r = 20$. The stopping rule matters: *stop before the denominator reaches N* . Continuing to the full expansion would return 4096, which is useless.

For this instance $t = 12$ and $r = 20$, so l ranges over $0, \dots, 19$. Running the numbers for every l , eight of the twenty outcomes return $r' = 20$ directly; these are the l coprime to 20. The rest return a proper divisor such as 10, 5, 4, 2 or 1, which fails the test $a^{r'} \equiv 1$. So roughly 40% of runs succeed on the first attempt, consistent with $\phi(20)/20 = 8/20$.

The classical cost of this step is negligible next to the quantum step. The expansion of $m/2^t$ has fewer than $2t$ quotients, because the denominators q_k in (5.8) grow at least as fast as the Fibonacci numbers and must stop before 2^t . Each quotient is one integer division of numbers with at most t bits. With $t = 2n$ and $n = \lceil \log_2 N \rceil$ the number of bits of N , that is $O(n)$ classical divisions on $O(n)$ -bit integers. It takes microseconds on a laptop even for a 2048-bit modulus.

5.1.4 Implementation notes

Registers and measurements

Register sizes. The input register needs t qubits with $2^t \geq r^2$. In Shor's case $t = 2\lceil \log_2 N \rceil$ is the standard choice, and it is what guarantees the continued-fraction step succeeds [24]. The output register needs $n = \lceil \log_2 N \rceil$ qubits. The input register plays exactly the role of the c counting qubits of Chapter 4; the Shor literature writes t for it, and this chapter follows that, with $t = 2n$.

The measurement in step (4) is optional. Tracing out the second register has the same effect on the first register's statistics as measuring it, so in code you can simply not measure it. Drawing the measurement makes the collapse explicit, which is why we show it.

Order of operations. Deferring the second register's measurement to after the inverse QFT is equally legitimate: the principle of deferred measurement says the statistics are unchanged. On hardware this can save a mid-circuit measurement.

Bit ordering, and how to test for it

Throughout this book, Chapter 4's QFT and phase estimation included, qubit 0 is on the top wire and carries the *most* significant bit. A measurement of the counting register therefore gives $m = m_0 2^{t-1} + \dots + m_{t-1} 2^0$, and the top counting qubit is the one controlling the *largest* power of V .

Qiskit and most other libraries use the opposite (little-endian) convention, in which the top wire is the *least* significant bit. A register printed by such a library therefore reads back-to-front relative to the circuits drawn here. Neither convention is wrong, but mixing the two is fatal: the recovered r comes out wrong with no warning. If that happens, reverse the counting register, either before the inverse QFT or when interpreting the measured string. In my experience this is the single most common bug in implementations of this algorithm.

Testing strategy. Verify your oracle classically first: build the full truth table of f and check (5.2) holds. Most bugs are here, not in the quantum part. Then test the whole circuit on a case with a known answer, $f(x) = x \bmod 4$ on $t = 3$ qubits, and confirm the outcome histogram is concentrated on $m \in \{0, 2, 4, 6\}$. Other outcomes mean your oracle is wrong or your bit ordering is reversed.

5.2 Number Theory for Factoring

Everything in this section is classical, and all of it is needed to turn order finding into factoring.

The letters inside this section's proofs are local to those proofs. Elsewhere in the chapter a is the base of $a^x \bmod N$, r is its order, s is the discrete logarithm, n is a qubit count and m is a measurement outcome. Inside a proof box below, m , n , x , y , s and R are whatever that proof says they are, and nothing carries over.

5.2.1 Primes and factorizations

Prime. An integer $p > 1$ divisible only by 1 and p .

Composite. An integer > 1 that is not prime.

Fundamental theorem of arithmetic

Every integer $N > 1$ can be written as

$$N = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}$$

with p_i distinct primes and $\alpha_i \geq 1$, and this representation is *unique* up to the order of the factors.

Existence follows by strong induction: if N is prime we are done; otherwise $N = mn$ with $1 < m, n < N$, and both factor by the inductive hypothesis.

Uniqueness needs Euclid's lemma: if a prime p divides ab then p divides a or b . Suppose $p_1 \cdots p_n = q_1 \cdots q_m$ are two factorizations. Then p_1 divides the right-hand side, so by Euclid's lemma p_1 equals some q_j ; cancel and induct. (An argument that merely cancels common factors and concludes that the leftovers must all equal 1 is not sufficient. Primes are > 1 by definition, so " $q_k \cdots q_m = 1$ " shows the list is *empty*, not that its entries are 1. Euclid's lemma is what does the real work.)

Every composite N has a prime factor $\leq \sqrt{N}$

If $N = ab$ with $1 < a, b < N$, then a and b cannot both exceed $\sqrt{N}$, since that would give $ab > N$. So one of them is $\leq \sqrt{N}$, and it is either prime or has a prime factor smaller still.

This is why classical trial division, which divides N by $2, 3, 4, \dots$ until something goes in exactly, needs at most $\sqrt{N}$ classical divisions. Written in terms of $n = \lceil \log_2 N \rceil$, the number of bits of N , that is about $2^{n/2}$ divisions: every two extra bits in N double the work, which is exponential in the number of digits.

Trial division is not the bound Shor's algorithm has to beat. The best classical

factoring algorithm, the general number field sieve, runs in about

$$\exp(c(\ln N)^{1/3}(\ln \ln N)^{2/3})$$

classical operations with $c \approx 1.9$. Since $\ln N$ is proportional to n , the number of bits of N , this is sub-exponential in n , far better than $2^{n/2}$, but it still grows faster than any polynomial in n . For a 2048-bit modulus the formula gives about 2^{117} operations, against 2^{1024} for trial division. Shor's algorithm, as we will count in §5.3.5, needs $O(n^3)$ quantum gates, a polynomial. That difference in growth, not any particular constant, is the whole story.

5.2.2 Greatest common divisors

The *greatest common divisor* $\gcd(a, b)$ is the largest integer dividing both a and b .

Example 5.4

Find $\gcd(18, 36)$.

Solution. The positive common divisors of 18 and 36 are 1, 2, 3, 6, 9, 18. The largest is 18.

Here 18 itself divides 36, so it is a common divisor. It is easy to stop the list at 9 and miss it. Whenever $a \mid b$, the answer is simply a .

The Euclidean algorithm

Factorizing to find a gcd is hopelessly slow. Euclid's algorithm rests on one lemma.

Lemma. $\gcd(a, b) = \gcd(b, R)$, where $R = a \bmod b$.

Proof. Write $a = bQ + R$. If d divides a and b it divides $a - bQ = R$, so it is a common divisor of b and R . Conversely if d divides b and R it divides $bQ + R = a$. The two pairs have the same common divisors, hence the same greatest one. $\square$

Iterating with $r_0 = a$, $r_1 = b$:

$$r_0 = r_1q_1 + r_2, \quad r_1 = r_2q_2 + r_3, \quad \dots, \quad r_{k-1} = r_kq_k + 0,$$

the remainders strictly decrease, so the process terminates, and $\gcd(a, b) = r_k$, the last nonzero remainder.

Example 5.5

Find $\gcd(21, 6)$.

Solution. $21 = 6 \cdot 3 + 3$, so $\gcd(21, 6) = \gcd(6, 3)$. Then $6 = 3 \cdot 2 + 0$, so $\gcd(6, 3) = 3$. Hence $\gcd(21, 6) = 3$.

5.2.3 Bézout's identity and modular inverses

Bézout's theorem

For any integers a, b not both zero there exist $x, y \in \mathbb{Z}$ with

$$\gcd(a, b) = ax + by. \quad (5.9)$$

For example $\gcd(6, 14) = 2$ and $2 = 6(-2) + 14(1) \checkmark$.

Proof. Let s be the smallest *positive* integer of the form $ax + by$. Since $\gcd(a, b)$ divides both a and b , it divides s , so $\gcd(a, b) \leq s$. Conversely, divide: $a = ks + R$ with $0 \leq R < s$. Then

$$R = a - ks = a(1 - kx) + b(-ky),$$

which is again of the form $ax' + by'$. Since $R < s$ and s was minimal, $R = 0$, so $s \mid a$; the same argument gives $s \mid b$. Hence s is a common divisor, so $s \leq \gcd(a, b)$. Therefore $s = \gcd(a, b)$. $\square$

Modular inverses. If $\gcd(a, N) = 1$, Bézout gives $ax + Ny = 1$, so $ax \equiv 1 \pmod{N}$ and x is the multiplicative inverse of a modulo N . Conversely, if a shares a factor with N it has no inverse.

Example 5.6

Find the inverse of $a = 2$ modulo $N = 5$.

Solution. $2 \cdot 3 = 6 \equiv 1 \pmod{5}$, so the inverse is 3.

You cannot divide to get a modular inverse

The inverse of e modulo ϕ is *not* $d = \frac{1 \bmod \phi}{e}$. That expression is ordinary division: with $\phi = 72$ and $e = 5$ it reads $d = 1/5$, which is not an integer and not the answer.

The inverse is computed with the *extended* Euclidean algorithm, which tracks the Bézout coefficients alongside the remainders. Equivalently, solve $de = 1 + k\phi$ for the smallest k making the right-hand side divisible by e .

5.2.4 From order finding to factoring

Here is the reduction that makes the whole chapter work.

The square-root trick

Suppose $x^2 \equiv 1 \pmod{N}$ with $x \not\equiv \pm 1$. Then

$$(x-1)(x+1) = x^2 - 1 \equiv 0 \pmod{N},$$

so N divides $(x-1)(x+1)$ but, by $x \not\equiv \pm 1$, neither factor alone. Its prime factors are therefore split between the two, and $\gcd(x-1, N)$ and $\gcd(x+1, N)$ are non-trivial factors of N . The condition $x \not\equiv \pm 1$ is essential: if $x \equiv 1$ then $\gcd(x-1, N) = N$ and $\gcd(x+1, N)$ is usually 1, both trivial.

Now let a be coprime to N and let r be its *order*, the smallest positive integer with

$$a^r \equiv 1 \pmod{N}. \quad (5.10)$$

Example 5.7

Find the order of 2 modulo 5.

Solution. $2^1 = 2$, $2^2 = 4$, $2^3 = 3$, $2^4 = 1 \pmod{5}$. The order is $r = 4$. Continuing, $2^5 = 2$, $2^6 = 4$: the sequence repeats with period 4, which is the same statement.

If r is *even*, set $x = a^{r/2}$. Then $x^2 = a^r \equiv 1$, and provided $x \not\equiv -1$ the square-root trick applies:

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N) \quad (5.11)$$

are non-trivial factors of N .

How often does this work?

Two things can go wrong: r can be odd, or $a^{r/2}$ can be $\equiv -1$. For N with at least two distinct odd prime factors, a classical theorem says a uniformly random a coprime to N avoids both failures with probability at least $1/2$. So the expected number of values of a you must try is at most two, and each value costs the runs of the quantum order-finding step needed to learn its r . The case $a^{r/2} \equiv +1$ cannot occur: it would contradict the minimality of r .

5.2.5 Implementation notes

The classical routines you need

All of these are a few lines and all are needed.

gcd: Euclidean algorithm. The remainder at least halves every two steps, so $\text{gcd}(a, N)$ takes at most about $2 \log_2 N$ classical divisions. Available as `math.gcd` in Python.

Extended gcd: returns (g, x, y) with $ax + by = g$. Needed for modular inverses, hence for RSA key generation. Python has `pow(e, -1, phi)` for the inverse directly.

Modular exponentiation: $a^x \bmod N$ by repeated squaring: one squaring per bit of x and one extra multiplication per bit that is set, so at most $2 \log_2 x$ classical modular multiplications. Use `pow(a, x, N)`, never `a**x % N`, which builds an astronomically large integer first.

Continued fractions: as in §5.1.3. Python's `fractions.Fraction` gives exact rational arithmetic, and `limit_denominator(N)` returns the closest fraction with denominator at most N , which for a good measurement outcome is the same fraction as the “last convergent with denominator below N ” step.

A classical order-finding reference implementation. Before writing any quantum code, write the brute-force version: loop $k = 1, 2, \dots$ until $a^k \equiv 1$. It is exponentially slow but correct, and it is the only way to know whether your quantum circuit is right on small instances.

5.3 Shor's Algorithm

5.3.1 The complete procedure

Shor's algorithm[26] is a classical wrapper around one quantum subroutine. Here is the whole thing.

Shor's factoring algorithm

Input: a composite integer N . **Output:** a non-trivial factor of N .

1. If N is even, return 2.
2. If $N = p^k$ for a prime p and $k \geq 2$, return p . (Classical: test $\sqrt[k]{N}$ for $k \leq \log_2 N$.)
3. Choose a uniformly at random from $\{2, \dots, N-1\}$.
4. Compute $\gcd(a, N)$. If it exceeds 1, return it; that is a lucky guess.
5. [QUANTUM] Find the order r of a modulo N by period finding.
6. If r is odd, go to step 3.
7. Compute $x = a^{r/2} \bmod N$. If $x \equiv -1 \pmod{N}$, go to step 3.
8. Return $\gcd(x-1, N)$ and $\gcd(x+1, N)$.

Steps 1 and 2 are not optional

Without step 1 the algorithm can loop forever on even N . Without step 2 it fails on prime powers: if $N = p^k$ there is only one prime, so the factors cannot be split between $x-1$ and $x+1$. Both tests are cheap, both are needed for correctness, and both are frequently omitted from expositions, including circuit-level tutorials.

5.3.2 The quantum subroutine as phase estimation

Define the modular-multiplication operator

$$V |y\rangle = |ay \bmod N\rangle. \quad (5.12)$$

This is a permutation of the residues coprime to N , hence unitary. Because $a^r \equiv 1$, applying V exactly r times returns every state to itself: $V^r = I$. So the eigenvalues of V are r -th roots of unity.

Example 5.8

Show that

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |a^k \bmod N\rangle \quad (5.13)$$

is an eigenvector of V with eigenvalue $e^{2\pi i s/r}$.

Solution.

$$\begin{aligned} V|u_s\rangle &= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |a^{k+1}\rangle = \frac{1}{\sqrt{r}} \sum_{m=1}^r e^{-2\pi i s(m-1)/r} |a^m\rangle \\ &= e^{2\pi i s/r} \left(\frac{1}{\sqrt{r}} \sum_{m=0}^{r-1} e^{-2\pi i s m/r} |a^m\rangle \right) = e^{2\pi i s/r} |u_s\rangle, \end{aligned}$$

where the sum was re-indexed using $a^r = a^0 = 1$ to wrap $m = r$ back to $m = 0$. Verified numerically for $N = 15$, $a = 2$, $r = 4$: all four s satisfy the eigenvalue equation exactly.

So phase estimation on V returns s/r , exactly what we want. But it requires us to prepare the eigenvector $|u_s\rangle$, which needs r , which is what we are trying to find. The escape is a small miracle:

Example 5.9

Show that $\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle = |1\rangle$.

Solution. Summing (5.13) over s and exchanging the order,

$$\sum_{s=0}^{r-1} |u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \left(\sum_{s=0}^{r-1} e^{-2\pi i s k/r} \right) |a^k\rangle.$$

The inner sum is a complete geometric series over the r -th roots of unity: it equals r when $k = 0$ and 0 otherwise. Only the $k = 0$ term survives, and $a^0 = 1$, so

$$\sum_{s=0}^{r-1} |u_s\rangle = \frac{r}{\sqrt{r}} |1\rangle = \sqrt{r} |1\rangle.$$

Verified numerically for $N = 15$, $a = 2$.

Why this is the crux of the algorithm

The trivially preparable state $|1\rangle$ is an equal superposition of *all* the eigenvectors we cannot construct. By §4.2.7, running phase estimation on $|1\rangle$ therefore returns a uniformly random s/r . One such sample, after continued fractions, gives r .

We never learn s , never construct $|u_s\rangle$, and never need r in advance. I think this is the single cleverest step in the algorithm.

5.3.3 The circuit

By the same argument as Chapter 4, the counting register controls successive squarings. With qubit 0 the most significant bit, $x = x_0 2^{t-1} + x_1 2^{t-2} + \cdots + x_{t-1} 2^0$, so

$$a^x = \left(a^{2^{t-1}}\right)^{x_0} \cdot \left(a^{2^{t-2}}\right)^{x_1} \cdots \left(a^2\right)^{x_{t-2}} \cdot a^{x_{t-1}}, \quad (5.14)$$

so controlled- $V^{2^{t-1-k}}$ applied from counting qubit k , for $k = 0, \dots, t-1$, builds $|a^x \bmod N\rangle$ in the second register. Look at the direction: the *top* counting qubit carries the most significant bit and therefore controls the *largest* power, $V^{2^{t-1}}$, while the bottom one controls $V^{2^0} = V$. Reversing this is the same mistake as reversing the readout, and it is equally silent.

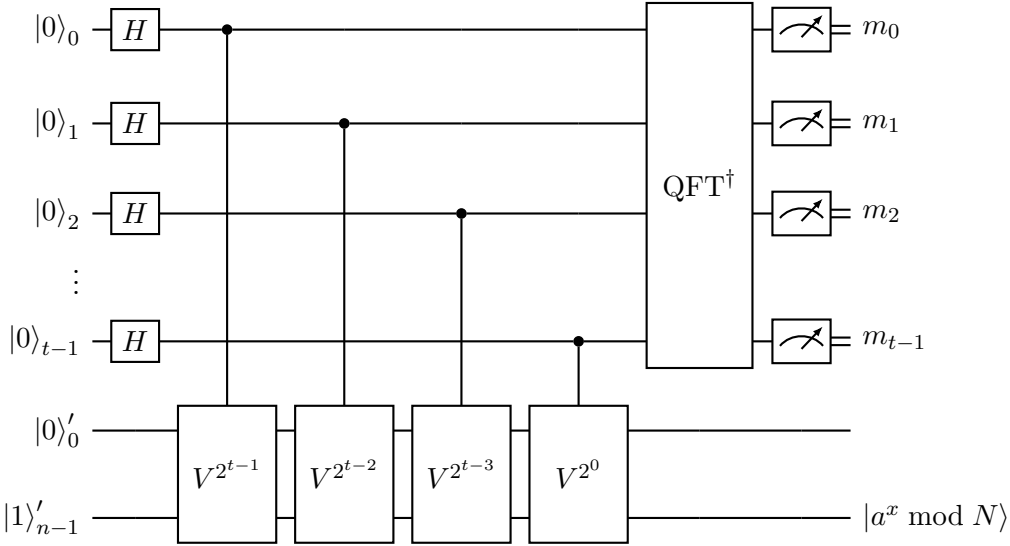


Figure 5.4: Shor's order-finding circuit. This is exactly the phase-estimation circuit of Fig. 4.9 with $U = V$ and the eigenvector register prepared in $|1\rangle$. By Example 5.9, that state is an equal superposition of all eigenvectors of V . Counting qubit 0 is the top wire and the most significant bit, so it controls the largest power $V^{2^{t-1}}$ and the bottom counting qubit controls V^{2^0} , as required by (5.14). The second register is drawn with its first and last wires, and holds $n = \lceil \log_2 N \rceil$ qubits. Holding the integer 1 means every wire starts in $|0\rangle$ except the bottom one, the least significant bit.

5.3.4 Worked example: factoring $N = 15$

Example 5.10

Factor $N = 15$ using $a = 2$.

Solution.

Step 1. 15 is odd. *Step 2.* 15 is not a prime power. *Steps 3–4.* Take $a = 2$; $\gcd(2, 15) = 1$, so no lucky factor.

Step 5 (quantum). The operator $V|y\rangle = |2y \bmod 15\rangle$ acts as $V|1\rangle = |2\rangle$, $V|2\rangle = |4\rangle$, $V|4\rangle = |8\rangle$, $V|8\rangle = |1\rangle$, so the orbit of $|1\rangle$ is $\{1, 2, 4, 8\}$ and $r = 4$. In binary this reads $|0001\rangle \rightarrow |0010\rangle \rightarrow |0100\rangle \rightarrow |1000\rangle \rightarrow |0001\rangle$, with qubit 0 the leftmost symbol on the top wire: the cyclic shift of the box below.

Step 6. $r = 4$ is even. ✓ *Step 7.* $x = 2^2 = 4 \bmod 15$, and $4 \not\equiv -1 \equiv 14$. ✓

Step 8. $\gcd(4 - 1, 15) = \gcd(3, 15) = 3$ and $\gcd(4 + 1, 15) = \gcd(5, 15) = 5$, so $15 = 3 \times 5$. All steps verified numerically.

Multiplication by 2 mod 15 is a cyclic bit shift

For $N = 15 = 2^4 - 1$ and $a = 2$ there is a shortcut. Doubling a 4-bit number shifts its bits left, and reducing mod 15 wraps the overflow bit back into the least significant place: a *cyclic* left shift. With y_0 the most significant bit on the top wire,

$$V |y_0 y_1 y_2 y_3\rangle = |y_1 y_2 y_3 y_0\rangle,$$

which three SWAP gates implement. Verified: the three-swap chain reproduces $2y \bmod 15$ for every $y = 0, \dots, 14$.

Two caveats. The identity fails at $y = 15$ and nowhere else, where the shift fixes 1111 but $2 \cdot 15 \equiv 0 \pmod{15}$. It does *not* fail at $y = 0$, where both sides give 0. Neither 0 nor 15 lies in the group generated by a , so the algorithm never visits either. And the shortcut is special to $N = 2^n - 1$; general N needs a real modular multiplier, where nearly all the gate cost lives.

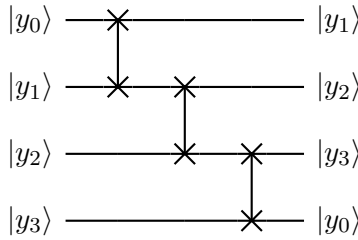


Figure 5.5: $V |y\rangle = |2y \bmod 15\rangle$ as a cyclic shift, built from three SWAPs. The top wire is qubit 0, the most significant bit; the three SWAPs carry its bit down to the bottom wire while every other bit moves up one place. Each SWAP costs three CNOTs (Chapter 1), so V costs nine CNOTs. The controlled version, which is what the algorithm actually needs, costs three Fredkin gates.

Exercise 5.1

Draw the circuit implementing $V |y\rangle = |7y \bmod 15\rangle$.

Answer: the orbit of 1 under multiplication by 7 is $1 \rightarrow 7 \rightarrow 4 \rightarrow 13 \rightarrow 1$, so $r = 4$. Since $7 = 2^3 - 1$, one route is $7y = 8y - y \bmod 15$: apply the cyclic shift three times, then subtract. In practice you would build it as a permutation circuit on the four basis states of the orbit.

5.3.5 Implementation notes

Register sizes and where the cost lives

Counting register: $t = 2n$ qubits, where $n = \lceil \log_2 N \rceil$. The factor of two is what makes the continued-fraction step provably succeed: it guarantees $m/2^t$ is within $1/2r^2$ of s/r , which uniquely determines s/r among fractions with denominator $< N$.

Work register: n qubits, initialized to $|1\rangle$, that is, $|0 \cdots 01\rangle$, not $|0 \cdots 0\rangle$. Forgetting this is a common bug: the state $|0\rangle$ is a fixed point of V and the algorithm returns nothing.

Total: $3n$ qubits in these two registers for the textbook version. The modular multiplier of the next box needs a further n -qubit scratch register, so the straightforward construction uses about $4n$; optimized modular arithmetic brings the whole thing down to about $2n + 3$. Counting only the two main registers, a 2048-bit RSA modulus needs roughly 6000 logical qubits. With error correction that becomes of the order of a million physical ones, and the exact figure moves every year.

Where the gates go. The inverse QFT on the $t = 2n$ counting qubits costs $O(n^2)$ gates and is negligible. The modular exponentiation costs $O(n^3)$ gates and dominates completely: $t = 2n$ controlled modular multiplications, each a full modular multiplier circuit of $O(n^2)$ gates. The counts are assembled in the text below. Any effort spent optimizing Shor's algorithm is effort spent on modular arithmetic.

Here is where those two counts come from, with $n = \lceil \log_2 N \rceil$ the number of bits of the number N being factored, and $t = 2n$ the number of counting qubits. Start with the inverse QFT. Counting qubit j receives one Hadamard and $t - 1 - j$ controlled phase gates, so the whole transform is t Hadamards and $t(t - 1)/2$ controlled phases, followed by $\lfloor t/2 \rfloor$ SWAPs. With $t = 2n$ that is $2n$ Hadamards and $n(2n - 1)$ controlled phases, about $2n^2$ gates in all.

Now the modular exponentiation. Each controlled multiplication by the classical constant $a_k = a^{2^k} \bmod N$ is built from additions. Writing the work register's value as $y = \sum_j y_j 2^j$, the product $a_k y \bmod N$ is the sum of the n classical constants $a_k 2^j \bmod N$, each added only when the bit y_j is set. So one multiplication is n controlled modular additions. A modular addition of an n -bit constant with a ripple-carry adder is an addition, a comparison with N and a conditional subtraction, each a chain of $O(n)$ Toffoli and CNOT gates. One controlled multiplication therefore costs $n \times O(n) = O(n^2)$ gates, and the ladder of $t = 2n$ of them in Fig. 5.4 costs $2n \times O(n^2) = O(n^3)$ gates, with the uncomputation of Fig. 5.6 doubling the constant. In plain words: doubling the number of bits in N costs about eight times as many gates. The QFT's share, at about $2n^2$, is n times smaller than the exponentiation's and can be ignored. Shor pointed out in the original paper [26] that

fast multiplication algorithms bring the exponentiation down to $O(n^2 \log n \log \log n)$ gates. The constant factors are large, though, and the resource estimates you will see quoted start from the schoolbook $O(n^3)$ count.

Building controlled- V^{2^k}

Do **not** build V and then raise it to a power with repeated application. That costs 2^k applications of V , which for the top counting wire, $k = t - 1 = 2n - 1$, is of the order of N^2 applications: exponential in n , and the speed-up is gone. Instead, precompute the classical constant $a_k = a^{2^k} \bmod N$ on your laptop (one call to `pow`), then build a single controlled multiplier for that constant:

$$c\text{-}V^{2^k} : |c\rangle |y\rangle \mapsto |c\rangle |a_k^c y \bmod N\rangle.$$

Each is one circuit, not 2^k circuits. This is the step that turns exponential work into polynomial work, and in my experience it is the single most common place implementations go wrong.

The standard construction of a modular multiplier runs: modular adder $\rightarrow$ controlled modular adder $\rightarrow$ modular multiply-accumulate $\rightarrow$ modular multiplier. It uses one extra n -qubit scratch register, which is uncomputed at the end (Chapter 2, §2.5.6).

Figure 5.6 shows the shape of that construction at register level. The scratch register must be back in $|0\rangle$ before the next controlled multiplication begins. If it is not, it stays entangled with the counting register, and the interference the algorithm depends on is destroyed.

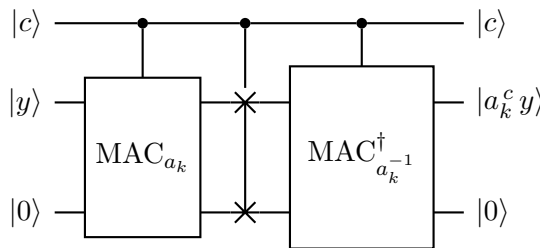


Figure 5.6: The controlled modular multiplier $c\text{-}V^{2^k}$, drawn at register level: $|y\rangle$ and the scratch register hold n qubits each, and $a_k = a^{2^k} \bmod N$ is a classical constant. MAC_{a_k} is the modular multiply-accumulate $|y\rangle |w\rangle \mapsto |y\rangle |w + a_k y \bmod N\rangle$, so acting on $|y\rangle |0\rangle$ it writes $a_k y$ into the scratch register. The controlled swap exchanges the two registers. Running $\text{MAC}_{a_k}^\dagger$ backwards then subtracts $a_k^{-1}(a_k y) = y$ and returns the scratch register to $|0\rangle$; the inverse a_k^{-1} exists because $\gcd(a, N) = 1$. With the control unset every block acts as the identity. Without the final uncomputation the scratch register would stay entangled with the counting register, and the phase estimation would fail.

Classical post-processing, in order

Given a measurement outcome m :

1. If $m = 0$, discard and rerun. It carries no information.
2. Form the exact rational $m/2^t$.
3. Compute the convergents of its continued fraction; take the last one whose denominator is $< N$. Call it r' .
4. Test $a^{r'} \equiv 1 \pmod{N}$. If yes, $r = r'$ and continue to step 6 of the main algorithm.
5. If no, try small multiples $2r', 3r', \dots$; often l and r shared a small factor. Failing that, combine with the denominator from another run via the least common multiple, or simply rerun.

Expect roughly $\phi(r)/r$ of runs to succeed immediately, where $\phi(r)$ is the number of integers below r coprime to r . For $N = 55$, $a = 2$, $r = 20$ that fraction is $8/20 = 40\%$.

Everything in that box is classical arithmetic on integers of at most $t = 2n$ bits, and all of it together takes far less time than a single shot of the quantum circuit. The expensive resource is the number of shots. Putting the pieces together: a random a is usable with probability at least $1/2$, and a run with a usable a returns r outright with probability $\phi(r)/r$. So the expected number of runs of the quantum circuit for one factorization is at most about $2r/\phi(r)$. That is a small constant for the examples in this chapter, and it stays below thirty for every r up to 2^{2048} .

Testing your implementation

Test 1: classical order finding. Brute-force r by looping. Every later test compares against this.

Test 2: the operator. Build V as an explicit $2^n \times 2^n$ matrix and check it is a permutation matrix, that $V^r = I$, and that $V|u_s\rangle = e^{2\pi i s/r}|u_s\rangle$ for each s .

Test 3: the full circuit on $N = 15$. Simulate and histogram the outcomes. With $t = 8$ and $r = 4$ expect sharp peaks at $m \in \{0, 64, 128, 192\}$ and essentially nothing elsewhere. Peaks in the wrong places mean bit ordering: check that first.

Test 4: a case where r does not divide 2^t . $N = 21$, $a = 2$ has $r = 6$, which divides no power of two, so the peaks broaden. This is the realistic case and the one that exercises your continued-fraction code.

A warning about tutorials. Many published $N = 15$ demonstrations hard-code the answer: their gates were chosen knowing $r = 4$, so the circuit factors 15 and nothing else. If your modular multiplier does not take a and N as parameters, you have not implemented Shor's algorithm.

5.4 Breaking RSA

5.4.1 The RSA scheme

RSA is the reason anyone outside physics cares about Shor's algorithm.

RSA key generation, encryption, decryption

1. Choose two large primes p and q .
2. Compute the modulus $N = pq$.
3. Compute the totient $\phi = (p - 1)(q - 1)$.
4. Choose e with $\gcd(e, \phi) = 1$. **Public key** = (N, e) .
5. Compute $d = e^{-1} \bmod \phi$ using the extended Euclidean algorithm. **Private key** = (d, N) .

Encrypt: $C = M^e \bmod N$. **Decrypt:** $M = C^d \bmod N$.

The RSA modulus is written N here, the same letter this chapter has used for the number being factored since §5.1. Cryptography papers usually write n for it, but in this book n is reserved for a qubit count, $n = \lceil \log_2 N \rceil$. The quantity ϕ is the Euler totient $\phi(N)$, abbreviated to ϕ throughout this section as is usual for RSA.

Decryption works because $ed \equiv 1 \pmod{\phi}$, so $ed = 1 + k\phi$ and

$$C^d = M^{ed} = M^{1+k\phi} = M \cdot (M^\phi)^k \equiv M \pmod{N},$$

by Euler's theorem, $M^\phi \equiv 1 \pmod{N}$ for $\gcd(M, N) = 1$.

What Shor's algorithm actually breaks

Shor's algorithm factors N , giving p and q . From those, $\phi = (p-1)(q-1)$ follows immediately, and then $d = e^{-1} \bmod \phi$, the private key, is a few lines of classical code. The entire security of RSA rests on N being hard to factor, and nothing else.

Two clarifications I always make to students:

- Shor's algorithm does *not* break symmetric cryptography (AES) or hash functions. Grover's algorithm (Chapter 6) gives only a quadratic speed-up there, met by doubling the key length.
- It *does* also break Diffie–Hellman and elliptic-curve cryptography, via the discrete-logarithm variant in §5.5. ECC is in some ways *more* vulnerable: a 256-bit elliptic-curve key needs fewer logical qubits than a 2048-bit RSA modulus.

Example 5.11

Encrypt and decrypt $M = 6$ with $p = 7$, $q = 13$.

Solution.

$$N = 7 \times 13 = 91, \quad \phi = 6 \times 12 = 72.$$

The smallest $e > 1$ with $\gcd(e, 72) = 1$ is $e = 5$ (since 2, 3, 4 all share a factor with 72). For d , solve $5d = 1 + 72k$:

k	$1 + 72k$	divisible by 5?
1	73	no
2	145	yes $\Rightarrow d = 29$

So $d = 29$. Then

$$C = 6^5 \bmod 91 = 7776 \bmod 91 = 41, \quad M = 41^{29} \bmod 91 = 6. \checkmark$$

All four numbers verified numerically.

5.4.2 Implementation notes

Coding RSA, and breaking it

Never write $M^{e} \% N$.** For realistic key sizes M^e has millions of digits. Use `pow(M,e,N)`, which does modular exponentiation by repeated squaring.

Computing d . `pow(e,-1,phi)` in Python 3.8+, or write the extended Euclidean algorithm yourself. Not division.

Choosing e . Real implementations use $e = 65537$: it is prime, so the gcd test almost always passes, and its binary form 10000000000000001 has only two ones, making encryption fast.

The attack, end to end. Given a public key (N, e) : run Shor on N to get p ; set $q = N/p$; compute $\phi = (p-1)(q-1)$; compute $d = e^{-1} \bmod \phi$; decrypt everything. Steps 2–5 are microseconds of classical arithmetic. Only step 1 is hard, and only classically.

A demonstration you can actually run. Take $p = 7$, $q = 13$ as above, factor $N = 91$ with the classical order-finding routine standing in for the quantum subroutine, and recover $d = 29$ from the public key alone. This makes the reduction concrete without needing a quantum computer, and is a good assignment.

5.5 The Discrete Logarithm Algorithm

5.5.1 The problem

Given a and b in a group where $b = a^s$, find s . Writing it additively,

$$b \equiv a^s \pmod{N}, \quad \text{find } s = \log_a b. \quad (5.15)$$

This is the problem underlying Diffie–Hellman key exchange and elliptic-curve cryptography. It is believed classically hard for the same kind of reason factoring is.

5.5.2 The trick: a two-dimensional period

Consider the function of *two* variables

$$f(x, y) = a^{sx+y} \bmod N = b^x a^y \bmod N. \quad (5.16)$$

The second form is what makes it computable: it needs only the known quantities a and b , never the unknown s .

This function is periodic, but with a two-dimensional period:

$$f(x+l, y-ls) = a^{s(x+l)+(y-ls)} = a^{sx+y} = f(x, y)$$

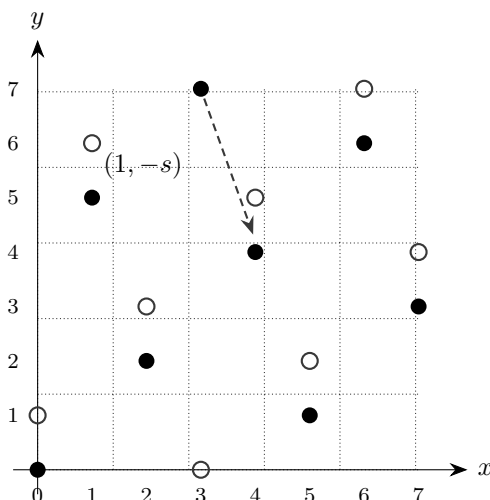


Figure 5.7: The two-dimensional period of $f(x, y) = b^x a^y \bmod N$, drawn for the small illustrative case $r = 8$, $s = 3$. (This is not the chapter's $N = 11$ example, which is too large to draw.) Filled dots mark the pairs with $sx + y \equiv 0 \pmod{r}$, on all of which f takes one value; open circles mark $sx + y \equiv 1$, a second value. Every level set is closed under the displacement $(1, -s)$ marked by the arrow, read modulo r in the vertical direction. So finding the period means finding the slope of these lines. The two measured registers return sk/r and k/r , and the ratio of the two is that slope.

for any integer l . So the period is the pair $(l, -ls)$, and finding it reveals s . Figure 5.7 draws this two-dimensional period for a small case.

Example 5.12

Verify the periodicity for $N = 11$, $a = 2$, $s = 6$.

Solution. $b = 2^6 = 64 \equiv 9 \pmod{11}$. Then $f(x, y) = 9^x 2^y \bmod 11$, and one checks $f(x + l, y - 6l) = f(x, y)$ for all tested x, y, l . This was verified numerically over a range of values. The order of 2 modulo 11 is $r = 10$, so all exponent arithmetic is modulo 10.

5.5.3 The circuit

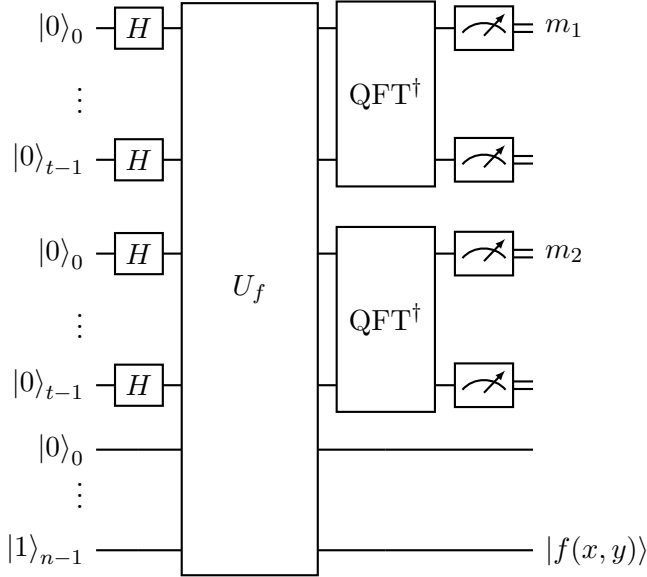


Figure 5.8: The discrete-logarithm circuit. Two t -qubit registers hold x and y ; the third holds $f(x, y) = b^x a^y \bmod N$ and is initialized to $|1\rangle$. Each of the first two registers gets its own inverse QFT and its own measurement. The third register is initialized to the integer $|1\rangle$, not $|0\rangle$, because, as in Shor’s algorithm, $|0\rangle$ is a fixed point of multiplication. With qubit 0 on the top wire as the most significant bit, the integer 1 means every wire of that register starts in $|0\rangle$ except the bottom one, qubit $n - 1$, the least significant bit. In all three registers the top wire is the most significant bit, so $x = x_0 2^{t-1} + \cdots + x_{t-1}$ and likewise for y and for the measured m_1 and m_2 .

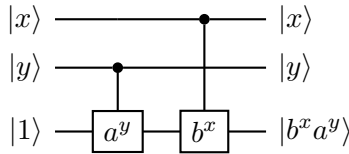


Figure 5.9: The oracle U_f . Each block is a modular exponentiation built exactly as in §5.3.5: controlled multiplications by the precomputed constants a^{2^k} and b^{2^k} , one per control qubit. As in Fig. 5.4, the top qubit of each argument register is the most significant bit. It therefore controls the largest power, $a^{2^{t-1}}$ or $b^{2^{t-1}}$.

5.5.4 The derivation

(1)–(2) After Hadamards on both argument registers,

$$|\psi_2\rangle = \frac{1}{2^t} \sum_{x=0}^{2^t-1} \sum_{y=0}^{2^t-1} |x\rangle |y\rangle |1\rangle.$$

(3) Query:

$$|\psi_3\rangle = \frac{1}{2^t} \sum_{x,y} |x\rangle |y\rangle |f(x,y)\rangle.$$

Write $z = sx + y$, so $f(x,y) = f(0,z)$, and expand in the Fourier basis over the group generated by a , of order r :

$$|f(0,z)\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{2\pi i z k/r} |\hat{f}(0,k)\rangle.$$

Substituting and separating the x and y sums:

$$|\psi_3\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \left(\frac{1}{\sqrt{2^t}} \sum_x e^{2\pi i s x k/r} |x\rangle \right) \left(\frac{1}{\sqrt{2^t}} \sum_y e^{2\pi i y k/r} |y\rangle \right) |\hat{f}(0,k)\rangle. \quad (5.17)$$

The second bracket carries $|y\rangle$

It is easy to write $|x\rangle$ in both brackets of (5.17). The second sum runs over y and must therefore carry $|y\rangle$, since it lives in the second register. With $|x\rangle$ in both, the expression is not even a state of the three registers.

Each bracket is a QFT of a basis state:

$$|\psi_3\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \text{QFT} \left| \frac{2^t s k}{r} \right\rangle \text{QFT} \left| \frac{2^t k}{r} \right\rangle |\hat{f}(0,k)\rangle.$$

(4) Apply $\text{QFT}^\dagger$ to both argument registers:

$$|\psi_4\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \left| \frac{2^t s k}{r} \right\rangle \left| \frac{2^t k}{r} \right\rangle |\hat{f}(0,k)\rangle.$$

(5) Measure both. Dividing by 2^t ,

$$\frac{m_1}{2^t} = \frac{s k}{r} \pmod{1}, \quad \frac{m_2}{2^t} = \frac{k}{r} \pmod{1}. \quad (5.18)$$

5.5.5 Recovering s

Continued fractions on $m_2/2^t$ give k and r . Then from $m_1/2^t = sk/r$,

$$s \equiv \frac{m_1}{2^t} \cdot \frac{r}{k} \equiv (s k) \cdot k^{-1} \pmod{r}. \quad (5.19)$$

You need $\gcd(k, r) = 1$

Equation (5.19) inverts k modulo r , which is only possible when $\gcd(k, r) = 1$. Since k is uniformly random in $\{0, \dots, r-1\}$, that happens with probability $\phi(r)/r$. The outcome $k = 0$ is useless outright.

So, exactly as in Shor's algorithm, discard degenerate outcomes and repeat. The expected number of runs of the quantum circuit is $r/\phi(r)$, which is $10/4 = 2.5$ for the $N = 11$ example below and fewer than fourteen for any r below 2^{2048} . Derivations often stop at (5.18) without mentioning this condition. That leaves the final step looking like a simple division when it is a modular inversion that can fail.

5.5.6 Implementation notes

Building the discrete-logarithm circuit

Register sizes. Both argument registers need $t \geq 2\lceil \log_2 r \rceil$ qubits; taking $t = 2n$ with $n = \lceil \log_2 N \rceil$ is safe. The value register needs n qubits. Total $2t + n \approx 5n$ qubits, not counting the multiplier's scratch register, against Shor's $3n$ counted the same way. That is one reason discrete-log resource estimates are larger for the same modulus size.

The oracle is two Shor oracles. To compute $b^x a^y$, run the modular-exponentiation construction of §5.3.5 twice into the same target register. The first pass uses base b controlled on the x register, the second base a controlled on the y register. Precompute $a^{2^k} \bmod N$ and $b^{2^k} \bmod N$ classically.

Order matters not at all. The two exponentiations commute, since both are multiplications in the same abelian group. Either order works.

You need r first. The post-processing divides by r , so run the order-finding algorithm on a before attempting the discrete log. In practice the two are run together.

Testing. Use $N = 11$, $a = 2$ (a primitive root, $r = 10$), $s = 6$, so $b = 9$. Brute-force the answer classically first: loop over $s' = 0, \dots, 9$ until $2^{s'} \equiv 9$. Then check your circuit's (m_1, m_2) pairs satisfy (5.18) for some k , and that (5.19) returns $s = 6$ whenever $\gcd(k, 10) = 1$, i.e. for $k \in \{1, 3, 7, 9\}$.

5.6 Summary

- Period finding requires f to be injective *within one period*; a single-bit output cannot satisfy this for $r > 2$.
- Measuring the output register collapses the input onto one coset; the inverse QFT moves the unknown offset into an unobservable phase and leaves $m/2^t =$

l/r .

- Recover r by continued fractions: take the last convergent with denominator $< N$, then *test* it. Discard $m = 0$. Do not take an LCM of raw outcomes.
- Order finding is phase estimation on $V|y\rangle = |ay \bmod N\rangle$, whose eigenvalues are $e^{2\pi i s/r}$.
- $|1\rangle$ is an equal superposition of all eigenvectors $|u_s\rangle$, so it can be used without knowing r . This is the key step.
- Shor's algorithm: handle even N and prime powers classically, pick random a , find r , require r even and $a^{r/2} \not\equiv -1$, then $\gcd(a^{r/2} \pm 1, N)$ are factors. Success probability per attempt $\geq 1/2$.
- Cost: $3n$ qubits in the counting and work registers, plus the multiplier's scratch; modular exponentiation at $O(n^3)$ gates dominates, the QFT at $O(n^2)$ gates does not.
- Build controlled- V^{2^k} from the precomputed constant $a^{2^k} \bmod N$, never by repeated application of V .
- RSA falls immediately once its modulus N is factored: ϕ , then $d = e^{-1} \bmod \phi$.
- The discrete logarithm uses a two-dimensional period: $f(x, y) = b^x a^y$ is invariant under $(x, y) \rightarrow (x + l, y - ls)$. Two measurements give sk/r and k/r ; recovering s needs $\gcd(k, r) = 1$.

5.7 End-of-Chapter Problems

- 5.1** Show that the function $f = (0, 1, 1, 0, 0, 1, 1, 0)$ on three bits satisfies $f(x+4) = f(x)$ but is not usable for period finding. Compute the state after the output register is measured and confirm it is a union of two cosets.
- 5.2** For $t = 3$ and $f(x) = x \bmod 4$, compute the state after each step of Fig. 5.2 and confirm the outcomes are $m \in \{0, 2, 4, 6\}$.
- 5.3** Prove that the phase $e^{-2\pi i l x_0/r}$ in (5.6) has no effect on the measurement statistics, and explain why this is essential.
- 5.4** Recover r from $m = 3891$ with $t = 12$ and $N = 55$ using continued fractions. Show all convergents.
- 5.5** Show that if $\gcd(l, r) = k > 1$, the continued-fraction step returns r/k rather than r . What fraction of outcomes is affected when $r = 12$?
- 5.6** Compute $\gcd(1071, 462)$ by the Euclidean algorithm, and find Bézout coefficients x, y with $1071x + 462y = \gcd$.

- 5.7** Prove Euclid's lemma from Bézout's identity, and hence complete the uniqueness half of the fundamental theorem of arithmetic.
- 5.8** Factor $N = 21$ by hand using $a = 2$. What is r ? Does the algorithm succeed? Now try $a = 5$.
- 5.9** Show that $a^{r/2} \equiv +1 \pmod{N}$ is impossible, and give an example where $a^{r/2} \equiv -1$ so that the attempt fails.
- 5.10** For $N = 15$, tabulate r and the outcome of Shor's algorithm for every $a \in \{2, 4, 7, 8, 11, 13, 14\}$. What fraction of choices succeed?
- 5.11** Verify that $|1\rangle = \frac{1}{\sqrt{r}} \sum_s |u_s\rangle$ explicitly for $N = 21$, $a = 2$, by writing out all six eigenvectors.
- 5.12** Estimate the number of logical qubits and Toffoli gates needed to factor a 2048-bit RSA modulus, using the counts in §5.3.5.
- 5.13** Explain why building V^{2^k} by applying V repeatedly destroys the exponential speed-up, and compute the resulting runtime.
- 5.14** Given the RSA public key $(N, e) = (91, 5)$ and ciphertext $C = 41$, recover the plaintext without being told d . List every step.
- 5.15** For $N = 11$, $a = 2$, $b = 9$: verify $s = 6$, and check that (5.19) recovers s for $k \in \{1, 3, 7, 9\}$ but fails for $k \in \{0, 2, 4, 5, 6, 8\}$. Note that k runs over $0, \dots, r-1$ with $r = 10$, so those ten values are the whole range.
- 5.16** Compare the qubit requirements of Shor's factoring algorithm on a 2048-bit modulus with the discrete-logarithm algorithm on a 256-bit elliptic curve. Which is easier to attack, and why is this surprising?

Research Problems and Projects

The problems above are exercises with settled answers. The three below are open-ended projects. Each is meant to take weeks rather than hours, and each ends in something you can write up: a measured curve, a table of resource counts, or working code with a benchmark attached. Where a project is a reproduction rather than genuinely new work, it says so.

- R5.1 How much counting register does order finding actually need?** The choice $t = 2n$ used throughout §5.3.5 is a sufficient condition. It is picked so that the continued-fraction step provably recovers $r[24]$. It is not known to be necessary, and on most instances it is visibly generous. Simulate the order-finding circuit for every odd composite N your machine can reach and

every a coprime to N , sweeping $t = n, n + 1, \dots, 2n$. For each pair, compute the exact one-shot success probability of the post-processing pipeline of §5.3.5: discard $m = 0$, run continued fractions, test $a^{r'} \equiv 1$, retry with small multiples. *Deliverable*: a scaling curve of the expected number of repetitions against t/n , split by whether r divides 2^t . Add a statement of the smallest t that keeps the expected repetition count bounded over your sample. Exact state-vector simulation caps you at roughly $t + n \leq 30$ qubits, so choose N accordingly and say so. Be clear about which of your conclusions are empirical and which are proved.

R5.2 Is elliptic-curve cryptography really the softer target? Section 5.4 asserts that a 256-bit elliptic-curve key needs fewer logical qubits than a 2048-bit RSA modulus. Section 5.5 gives the crude $\approx 5n$ against $3n$ count, which appears to say the opposite. Both claims are in the literature and neither is wrong; they are simply not stated on the same assumptions. Take Shor's two original algorithms [26] as the common starting point. Collect the modern resource estimates for factoring and for elliptic-curve discrete logarithms, and reduce them to a single set of assumptions: one error model, one set of surface-code parameters, one family of arithmetic primitives. *Deliverable*: one table giving logical qubits, Toffoli count and estimated running time for RSA-2048 and for a 256-bit curve under identical assumptions. Add a written account of exactly which modeling choices produce the spread between published numbers. This is a reproduction and reconciliation exercise rather than an open problem, and its value is in making explicit what other authors leave implicit.

R5.3 A modular multiplier that is not a demonstration. The warning that closes §5.3.5 is that most published $N = 15$ circuits hard-code the answer. Write one that does not. Build a generator that takes a and N as arguments and emits a circuit for $c \cdot V^{2^k}$ over a fixed gate set, structured as in Fig. 5.6. Implement at least two of the standard arithmetic strategies; a ripple-carry adder, a Fourier-basis adder and a windowed variant are the usual choices. Verify each against the classical order-finding routine of §5.2. *Deliverable*: the code, together with a benchmark table of Toffoli count, CNOT count, depth and ancilla count against n for $n = 4, \dots, 16$. Include end-to-end simulated runs on $N \in \{15, 21, 33, 35\}$ confirming that the outcome histograms peak where §5.3.5 says they should. Report the fitted exponent of each strategy and compare it against the $O(n^3)$ claimed in the text.

Chapter 6

Grover Search and Amplitude Amplification

Chapter 5 gave an exponential speed-up, but only for problems with a hidden periodic structure. Most problems have no such structure. This chapter covers the other great family of quantum algorithms: those that give a *quadratic* speed-up for problems with no structure at all.

The quadratic gain is more modest than the exponential one, but it applies far more widely. It covers database search, satisfiability, collision finding and Monte Carlo estimation, and it serves as a subroutine inside larger algorithms. And unlike Shor's algorithm, Grover's[15] is provably optimal. Any quantum algorithm that searches an unstructured space of N candidates needs $\Omega(\sqrt{N})$ queries to the oracle, a constant times $\sqrt{N}$ and no fewer.

What you should be able to do after this chapter

- Build the Grover oracle and diffuser, and write both in outer-product form.
- Derive the rotation picture and the optimal number of iterations.
- Explain why running *too many* iterations makes things worse.
- Handle multiple solutions, and count them with quantum counting.
- Generalize to amplitude amplification with an arbitrary state preparation $\mathcal{A}$.
- Implement all of it: each algorithm ends with an implementation guide.

6.1 The Search Problem

We are given a function

$$f : \{0, 1\}^n \rightarrow \{0, 1\}, \quad f(x) = \begin{cases} 1 & x \text{ is a solution,} \\ 0 & \text{otherwise,} \end{cases} \quad (6.1)$$

and nothing else. There is no structure to exploit: the only way to learn anything about f is to query it.

With $N = 2^n$ candidates (n input bits) and one solution w , a classical algorithm has nothing to go on but the answers it has already seen. Each query rules out one candidate, so on average it takes about $N/2$ queries to hit w , and N in the worst case. Grover's algorithm needs $O(\sqrt{N})$ queries to the same f , evaluated as a quantum circuit. For a million candidates that is about eight hundred queries instead of half a million. Quadrupling N doubles Grover's query count, where the classical count quadruples.

What “database search” really means

Grover's algorithm is usually described as searching a database. That description misleads, and I want to be precise about it.

The algorithm does not search stored data. It searches the space of *inputs to a function*, and it needs that function as a quantum circuit, evaluated in superposition. If your data really is an unsorted list in memory, you must first load all N items into a quantum-accessible structure. That loading step by itself costs $\Omega(N)$ elementary operations, since every item has to be touched at least once. No speed-up.

Where it genuinely helps is when f is *computed*, not looked up: does this assignment satisfy my Boolean formula? does this key decrypt this ciphertext? does this configuration have energy below E ? There the circuit for f is small, and searching 2^n inputs with $O(2^{n/2})$ evaluations of that circuit is a real gain. This is why Grover's algorithm halves the effective key length of AES but does nothing for a phone book.

6.2 Grover's Algorithm

6.2.1 The circuit

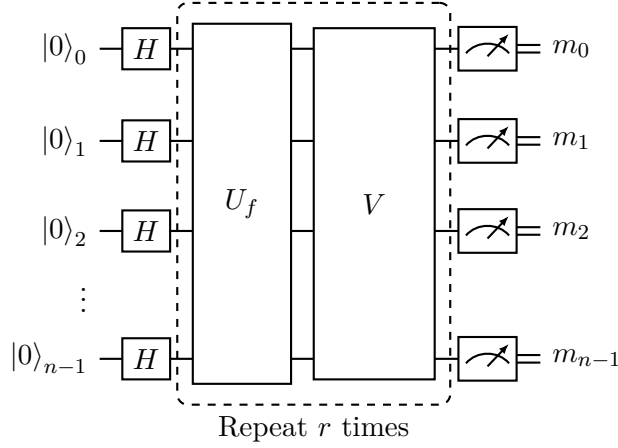


Figure 6.1: Grover's algorithm. After a layer of Hadamards, the *Grover operator* $G = VU_f$ is applied $r \approx \frac{\pi}{4}\sqrt{N}$ times, with $N = 2^n$ the number of candidates, and then every qubit is measured. There is no ancilla in this drawing: the phase oracle U_f is assumed to include its own $|-\rangle$ ancilla internally, as built in Chapter 2.

The Grover operator has two parts:

$$G = VU_f, \quad U_f = \text{phase oracle}, \quad V = H^{\otimes n} U_{f_0} H^{\otimes n} = \text{diffuser}. \quad (6.2)$$

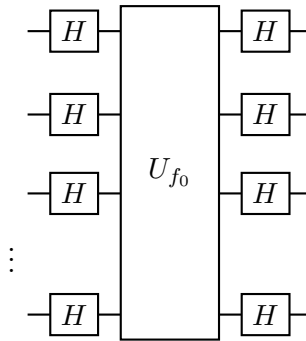


Figure 6.2: The diffuser $V = H^{\otimes n} U_{f_0} H^{\otimes n}$, drawn with individual wires. Every qubit gets its own Hadamard on both sides.

V is a product, not a tensor product

The diffuser is the *matrix product* $V = H^{\otimes n} U_{f_0} H^{\otimes n}$: three operators applied in sequence to the same n qubits. Writing $V = H^{\otimes n} \otimes (2|0\rangle\langle 0| - \mathbb{I}) \otimes H^{\otimes n}$ with tensor-product symbols would describe an operator on $3n$ qubits, which is a different object entirely. The $\otimes$ belongs *inside* $H^{\otimes n}$, not between the factors.

6.2.2 The oracle

The phase oracle flips the sign of the solution:

$$U_f |x\rangle = (-1)^{f(x)} |x\rangle = \begin{cases} -|x\rangle & x = w, \\ |x\rangle & x \neq w. \end{cases}$$

For $n = 3$ and $w = 101$:

Table 6.1: The oracle for $w = 101$. As a matrix it is $\text{diag}(1, 1, 1, 1, 1, -1, 1, 1)$, with the -1 sitting at index $5 = 101_2$.

$ x\rangle$	output	$ x\rangle$	output
$ 000\rangle$	$+ 000\rangle$	$ 100\rangle$	$+ 100\rangle$
$ 001\rangle$	$+ 001\rangle$	$ 101\rangle$	$- 101\rangle$
$ 010\rangle$	$+ 010\rangle$	$ 110\rangle$	$+ 110\rangle$
$ 011\rangle$	$+ 011\rangle$	$ 111\rangle$	$+ 111\rangle$

In outer-product form, start from $U_f = \sum_x |x\rangle\langle x| - 2|w\rangle\langle w|$ and use the completeness relation $\sum_x |x\rangle\langle x| = \mathbb{I}$ to get

$$U_f = \mathbb{I} - 2|w\rangle\langle w|. \quad (6.3)$$

This is a *reflection about the hyperplane orthogonal to $|w\rangle$* .

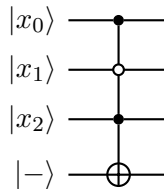


Figure 6.3: Concrete oracle marking $w = 101$ on three qubits, built exactly as in Chapter 2. Filled dots sit where w has a 1 and open circles where it has a 0. A $|-\rangle$ ancilla converts the bit flip into a phase; it returns unchanged and factors out.

6.2.3 The diffuser

Take U_{f_0} to flip the sign of everything *except* $|0\rangle$:

$$U_{f_0} |0\rangle = |0\rangle, \quad U_{f_0} |x\rangle = -|x\rangle \quad (x \neq 0),$$

so that

$$U_{f_0} = 2|0\rangle\langle 0| - \mathbb{I}. \quad (6.4)$$

Then, using $H^{\otimes n} |0\rangle = |s\rangle$ and $(H^{\otimes n})^2 = \mathbb{I}$,

$$V = H^{\otimes n} (2|0\rangle\langle 0| - \mathbb{I}) H^{\otimes n} = 2|s\rangle\langle s| - \mathbb{I}, \quad (6.5)$$

a *reflection about* $|s\rangle$. All three identities verified numerically for $n = 3$.

Two reflections make a rotation

U_f reflects about the hyperplane orthogonal to $|w\rangle$; V reflects about $|s\rangle$. The composition of two reflections is a *rotation* by twice the angle between their axes. That single geometric fact is the whole algorithm, and it is why the analysis below reduces to two-dimensional trigonometry.

6.2.4 The two-dimensional picture

Everything happens in the plane spanned by $|w\rangle$ and

$$|w^\perp\rangle = \frac{1}{\sqrt{N-1}} \sum_{x \neq w} |x\rangle.$$

Splitting the uniform superposition,

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle = \sqrt{\frac{N-1}{N}} |w^\perp\rangle + \frac{1}{\sqrt{N}} |w\rangle,$$

and since the two coefficients square to 1 we may write

$$|s\rangle = \cos \theta |w^\perp\rangle + \sin \theta |w\rangle, \quad \sin \theta = \frac{1}{\sqrt{N}}. \quad (6.6)$$

Here θ is the half-angle throughout this chapter; every full rotation is written 2θ . The initial success probability is $\sin^2 \theta = 1/N$, as it must be.

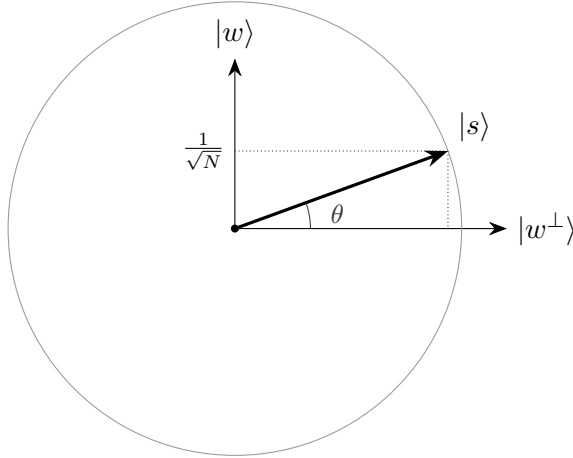


Figure 6.4: The initial state $|s\rangle$ sits at angle θ from the “no-solution” axis, with $\sin \theta = 1/\sqrt{N}$. For large N this angle is tiny. The solution then has almost no amplitude, which is exactly the problem the algorithm has to fix.

6.2.5 One Grover iteration

Step 1: the oracle reflects. Applying (6.3),

$$|s'\rangle = U_f |s\rangle = \cos \theta |w^\perp\rangle - \sin \theta |w\rangle, \quad (6.7)$$

a reflection in the horizontal axis: the state moves from $+\theta$ to $-\theta$.

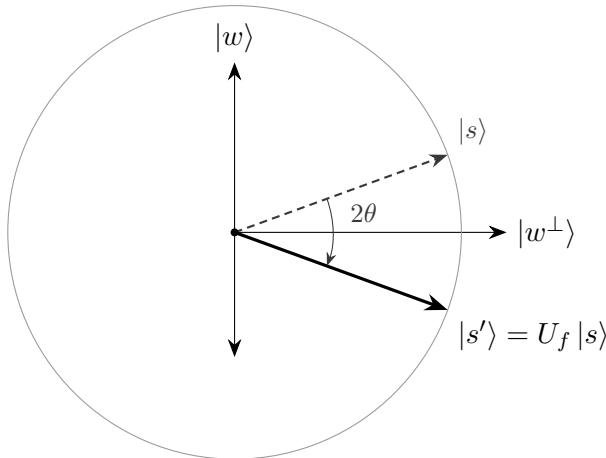


Figure 6.5: The oracle reflects the state in the $|w^\perp\rangle$ axis, turning it clockwise by 2θ .

Step 2: the diffuser reflects. Applying (6.5),

$$V |s'\rangle = 2 |s\rangle \langle s|s'\rangle - |s'\rangle, \quad \langle s|s'\rangle = \cos^2 \theta - \sin^2 \theta = \cos 2\theta.$$



Figure 6.6: The same step as an amplitude bar chart. The oracle flips the sign of the marked amplitude and leaves everything else alone. I want you to notice that the *magnitudes* are unchanged. Measuring now would still find w with probability only $1/N$. The oracle alone accomplishes nothing measurable; it is the diffuser that converts this sign into amplitude.

Expanding and using the double-angle identities,

$$G|s\rangle = VU_f|s\rangle = \cos(3\theta)|w^\perp\rangle + \sin(3\theta)|w\rangle. \quad (6.8)$$

Verified numerically for $n = 3$.

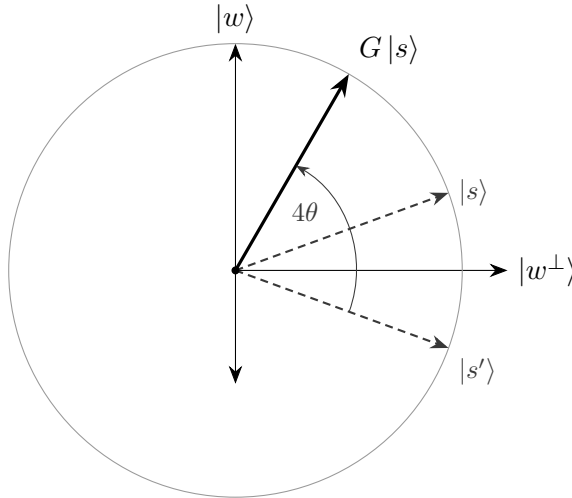


Figure 6.7: The diffuser reflects $|s'\rangle$ about $|s\rangle$, turning it counterclockwise by 4θ . Net effect of one Grover iteration: a rotation by 2θ toward $|w\rangle$, from θ to 3θ .

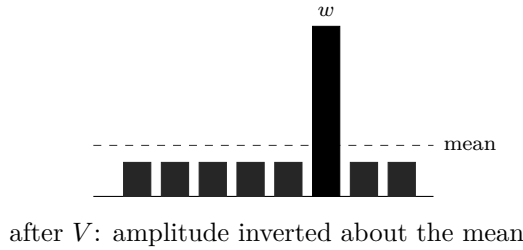


Figure 6.8: The diffuser as “inversion about the mean”. Because the marked amplitude was negative, it sits far below the average; reflecting every amplitude about the mean sends it far above, while the others drop. As drawn, with $N = 8$, the marked amplitude grows by a factor of 2.5 and each of the others falls by exactly half; for large N that fall is slight. This is the same operation as Fig. 6.7, seen in the computational basis.

6.2.6 How many iterations?

Each iteration adds 2θ , so after r of them

$$G^r |s\rangle = \cos((2r+1)\theta) |w^\perp\rangle + \sin((2r+1)\theta) |w\rangle. \quad (6.9)$$

Verified numerically for $r = 1, \dots, 6$ at $n = 3$. The success probability is

$$\Pr(w) = \sin^2((2r+1)\theta), \quad (6.10)$$

which is maximal when $(2r+1)\theta = \pi/2$, i.e.

$$r = \frac{\pi}{4\theta} - \frac{1}{2} \approx \frac{\pi}{4}\sqrt{N} \quad \text{using } \theta \approx \frac{1}{\sqrt{N}} \text{ for large } N. \quad (6.11)$$

Let me spell out where the approximation comes from. Here $N = 2^n$ is the number of candidates and r is the number of Grover iterations. Each iteration applies the oracle exactly once, so r is also the number of oracle queries. For large N the half-angle is small, so $\theta \approx \sin \theta = 1/\sqrt{N}$ and $\pi/(4\theta) \approx (\pi/4)\sqrt{N}$. The $-1/2$ is a fixed offset against a count that grows with $\sqrt{N}$, and for the order-of-magnitude statement it does not matter. So Grover’s algorithm makes $O(\sqrt{N})$ oracle queries, where a classical search makes $O(N)$. In plain words: quadrupling the search space doubles the number of queries. Figure 6.10 plots (6.10) against r for $N = 64$. The probability climbs to almost 1 at $r = 6$ and then falls away again. That falling half of the curve is the behavior the next box warns about.

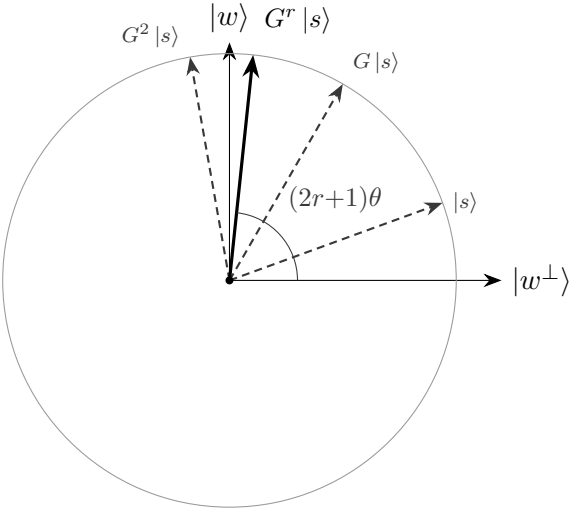


Figure 6.9: Successive Grover iterations rotate the state toward $|w\rangle$ in steps of 2θ . We stop when $(2r + 1)\theta$ is as close to $\pi/2$ as an integer r allows.

More iterations is not better

Equation (6.10) is periodic. Once the state passes $|w\rangle$ it keeps rotating and the success probability *falls*. For $N = 8$:

r	1	2	3	4	5	6
$\text{Pr}(w)$	0.781	0.945	0.330	0.012	0.548	1.000

At $r = 4$ the algorithm succeeds barely 1% of the time. That is far worse than the 12.5% you would get by guessing. All values verified numerically. I regard this as the single most important practical point in the chapter. Grover’s algorithm is not a process you run until convergence; it is a rotation you must stop at the right moment. Overshooting is a real failure mode, and it is why knowing μ , the number of solutions, matters; see §6.3.

Residual error

Since r must be an integer, the final angle overshoots or undershoots $\pi/2$ by at most θ , so the failure probability is bounded by $\text{Pr}(w^\perp) \leq \sin^2 \theta = 1/N$. Verified: for $N = 8$ the actual failure probability at optimal r is 0.0547 against the bound 0.125; for $N = 1024$ it is 0.00054 against 0.00098. The bound always holds and both sides vanish as N grows, but it is a bound, not an equality.

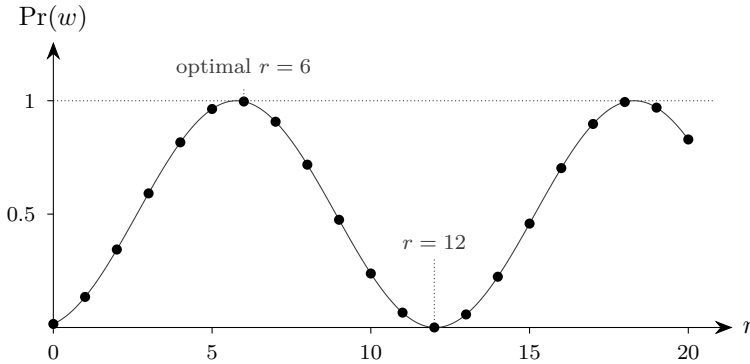


Figure 6.10: Success probability against the number of Grover iterations, for $N = 64$ with a single solution, from (6.10). The dots are the integer values of r that are actually available; the curve is the underlying $\sin^2((2r + 1)\theta)$. The best choice is $r = 6$. Running past it is not merely wasteful: by $r = 12$ the success probability has fallen to about 10^{-4} , well below the $1/64$ of a random guess.

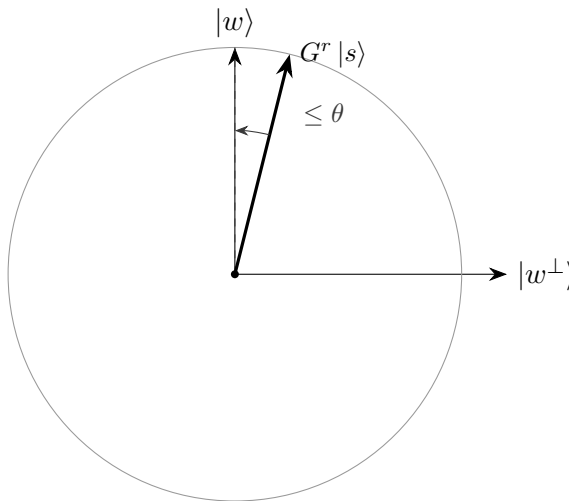


Figure 6.11: In general the final state misses $|w\rangle$ by at most θ , because r is an integer. The failure probability is at most $\sin^2 \theta = 1/N$.

6.2.7 Multiple solutions

If there are μ solutions $w_1, \dots, w_\mu$, define

$$|w\rangle = \frac{1}{\sqrt{\mu}} \sum_{i=1}^{\mu} |w_i\rangle, \quad |w^\perp\rangle = \frac{1}{\sqrt{N-\mu}} \sum_{x \notin \{w_i\}} |x\rangle.$$

Then $|s\rangle = \cos \theta |w^\perp\rangle + \sin \theta |w\rangle$ with

$$\sin \theta = \sqrt{\frac{\mu}{N}}, \quad (6.12)$$

and everything goes through unchanged, giving

$$r = \frac{\pi}{4\theta} - \frac{1}{2} \approx \frac{\pi}{4} \sqrt{\frac{N}{\mu}}. \quad (6.13)$$

Here μ is the number of solutions among the $N = 2^n$ candidates, and r again counts Grover iterations, one oracle query each. The approximation uses $\theta \approx \sin \theta = \sqrt{\mu/N}$ for small μ/N . The query count therefore falls as $1/\sqrt{\mu}$: four solutions instead of one halve the number of oracle calls. Verified: for $N = 16$, $\mu = 3$, the optimal $r = 1$ gives $\Pr(\text{some solution}) = 0.949$.

You need μ before you can choose r

Equation (6.13) requires μ . Without knowing how many solutions there are you cannot choose the number of iterations, and choosing wrongly can leave you worse off than random guessing. Two standard fixes:

- Run quantum counting first (§6.3) to estimate μ .
- Use the *exponential-search* trick [4]: pick r uniformly at random from $\{0, \dots, R-1\}$, and let R grow by a fixed factor each round; the analysis in [4] uses the factor 6/5. This finds a solution in $O(\sqrt{N/\mu})$ expected oracle queries, the same scaling as (6.13), with no prior knowledge of μ .

6.2.8 Implementation guide

Building Grover's algorithm

Qubits. n for the search register, plus one $|-\rangle$ ancilla for the oracle, plus whatever ancillas your multi-controlled gates need. With the clean-ancilla Toffoli chain of Chapter 2, a gate with k controls needs $k - 2$ ancillas, so the oracle's $C^n(X)$ needs $n - 2$ and the diffuser's $C^{n-1}(Z)$ fits inside the same $n - 2$. Dirty-ancilla constructions need fewer.

The oracle. For a known target w : a multi-controlled X onto the $|-\rangle$ ancilla, with control polarities read off the bits of w (Fig. 6.3). The ancilla turns that bit flip into a sign by phase kickback. For a *computed* predicate, build the circuit for f , apply a $C^k(X)$ onto the ancilla conditioned on the output bit, then uncompute f (Chapter 2, §2.5.6). The uncomputation is not optional.

The diffuser decomposes as $V = -H^{\otimes n} X^{\otimes n} C^{n-1}(Z) X^{\otimes n} H^{\otimes n}$, because conjugating by $X^{\otimes n}$ turns “mark $|0 \cdots 0\rangle$ ” into “mark $|1 \cdots 1\rangle$ ”, and marking $|1 \cdots 1\rangle$ is what a multi-controlled Z does. This is the form to code, and Figure 6.12 draws it gate by gate. The minus sign is real, and Problem 6.5 asks you to check it. In plain Grover it is a harmless global phase, but not in quantum counting, so read the warning in §6.3 before controlling this circuit.

Choosing r . Compute $r = \lfloor \frac{\pi}{4} \sqrt{N/\mu} \rfloor$ and *stop there*. Do not loop until some convergence criterion. There is none, and overshooting destroys the answer.

Let me also count gates, so that you know what you are asking of the hardware. Take n search qubits, $N = 2^n$ candidates, and a known target w . The diffuser of Figure 6.12 is n Hadamards, n X gates, one $C^{n-1}(Z)$, n more X gates and n more Hadamards. The $C^{n-1}(Z)$ is a $C^{n-1}(X)$ between two Hadamards on its target, and by the count of Chapter 2 a Toffoli chain with $n - 1$ controls uses $2(n - 1) - 3 = 2n - 5$ Toffoli gates. So the diffuser costs $4n + 2$ single-qubit gates and $2n - 5$ Toffolis. The oracle for a known w is a $C^n(X)$ with all n search qubits as controls: $2n - 3$ Toffolis, plus the X gates that set the control polarities. One Grover iteration is therefore about $4n$ Toffolis and $O(n)$ single-qubit gates, and because each Toffoli in a chain waits for the one before it, the depth is also $O(n)$. Multiply by $r \approx (\pi/4)\sqrt{N}$ iterations and the whole circuit is $O(n\sqrt{N}) = O(n2^{n/2})$ Toffoli gates, and $O(n2^{n/2})$ in depth as well. In plain words: every qubit you add to the search register multiplies the circuit by about $\sqrt{2}$ and adds a few Toffolis to each iteration. A computed predicate replaces the $C^n(X)$ with the circuit for f , its $C^k(X)$ onto the ancilla, and its uncomputation, and that oracle cost usually dominates.

Testing Grover, and what to expect from hardware

Testing, in order.

1. Build U_f as an explicit $2^n \times 2^n$ matrix and check it is diagonal with -1 exactly at the solutions.
2. Check $V = 2|s\rangle\langle s| - \mathbb{I}$ numerically.
3. Simulate $\Pr(w)$ against r for $n = 3$ and confirm it matches $\sin^2((2r + 1)\theta)$, including the fall after the peak. If your probability rises monotonically, your diffuser is wrong.
4. Only then run on hardware.

The honest expectation. Grover’s algorithm has deep circuits: r iterations, each containing two multi-controlled gates of depth $O(n)$. On present hardware, even $n = 4$ with a handful of iterations is dominated by noise, and published demonstrations rarely beat classical guessing. In my view it is a fault-tolerant-era algorithm.

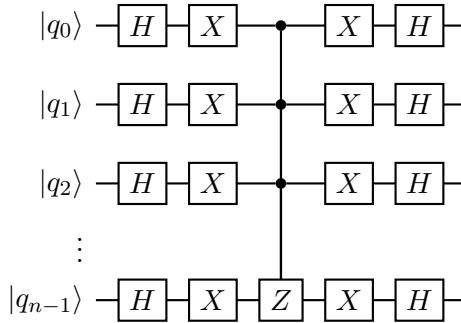


Figure 6.12: The diffuser as it is actually coded. The drawn circuit is $H^{\otimes n} X^{\otimes n} C^{n-1}(Z) X^{\otimes n} H^{\otimes n} = \mathbb{I} - 2|s\rangle\langle s| = -V$, so it realizes V up to a global sign. The X layers convert the condition “mark $|0 \cdots 0\rangle$ ” into “mark $|1 \cdots 1\rangle$ ”, and a single multi-controlled Z marks that. Compare Fig. 6.2, which shows V itself, with U_{f_0} still drawn as a black box.

6.3 Quantum Counting

6.3.1 The idea

Grover’s algorithm needs μ . Quantum counting finds it, by running phase estimation (Chapter 4) on the Grover operator G instead of on some external unitary.

The reason this works is that G is, in the two-dimensional $\{|w^\perp\rangle, |w\rangle\}$ plane, exactly a rotation matrix. One Grover step turns the state by the full angle 2θ , so in that plane

$$G = \begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}, \quad |s\rangle = \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}. \quad (6.14)$$

A rotation matrix has eigenvectors

$$\begin{pmatrix} i \\ 1 \end{pmatrix} \text{ with } e^{2i\theta}, \quad \begin{pmatrix} -i \\ 1 \end{pmatrix} \text{ with } e^{-2i\theta}. \quad (6.15)$$

Both verified numerically. Phase estimation therefore returns $2\theta/2\pi$, the *full* rotation angle divided by 2π , and not θ itself. I want you to watch this factor of two. It is the one place in the chapter where it is easy to drop. Halving the estimate and using (6.12) gives

$$\mu = N \sin^2 \theta. \quad (6.16)$$

Example 6.1

Show that $|s\rangle = \frac{1}{2i}e^{i\theta}\begin{pmatrix} i \\ 1 \end{pmatrix} - \frac{1}{2i}e^{-i\theta}\begin{pmatrix} -i \\ 1 \end{pmatrix}$.

Solution. Expanding the right-hand side, the first components give $\frac{1}{2i}(ie^{i\theta} + ie^{-i\theta}) = \cos \theta$, and the second components give $\frac{1}{2i}(e^{i\theta} - e^{-i\theta}) = \sin \theta$. Together these are exactly the $|s\rangle$ of (6.14). Verified numerically.

Why this matters: $|s\rangle$ is an equal-weight superposition of the two eigenvectors of G , so by §4.2.7 phase estimation on $|s\rangle$ returns one of the two phases at random. We never construct an eigenvector, exactly as in Shor's algorithm.

6.3.2 The circuit

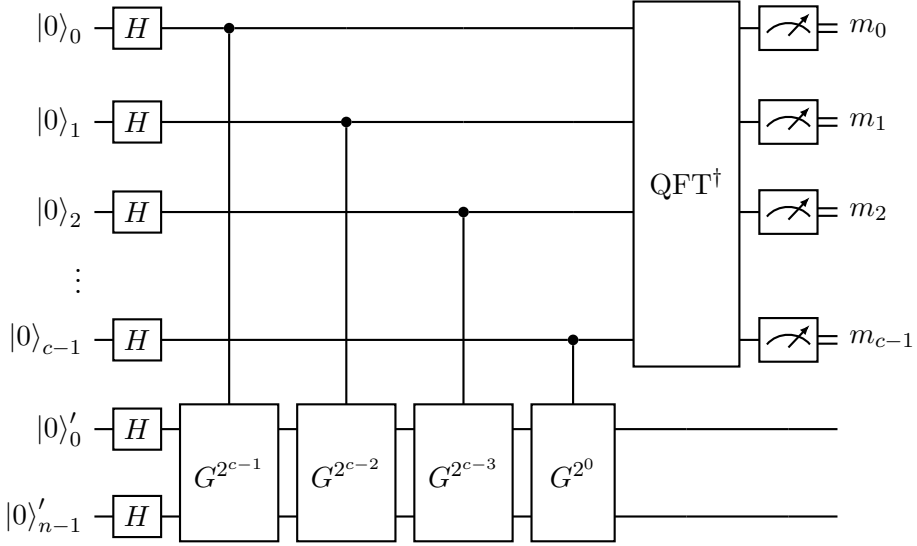


Figure 6.13: Quantum counting: phase estimation applied to the Grover operator. The lower register is prepared in $|s\rangle = H^{\otimes n} |0\rangle$, which is an equal superposition of G 's two eigenvectors. As in Fig. 4.9, counting qubit q controls $G^{2^{c-1-q}}$: the top wire controls the largest power and the bottom wire controls G itself. The measured bits then form $m = m_0 m_1 \cdots m_{c-1}$ with m_0 most significant, and the upper register returns an estimate of $2^c(2\theta)/2\pi$. Verified numerically: with the powers assigned the other way up the histogram peaks at the wrong m .

Since the eigenvalues are $e^{\pm 2i\theta}$, and $e^{-2i\theta} = e^{2\pi i(1-2\theta/2\pi)}$, the measurement returns one of

$$\left| 2^c \frac{2\theta}{2\pi} \right\rangle \quad \text{or} \quad \left| 2^c \left(1 - \frac{2\theta}{2\pi} \right) \right\rangle,$$

each with probability $\frac{1}{2}$. Either outcome gives the same μ , because the two are related by $2\theta \mapsto 2\pi - 2\theta$, that is $\theta \mapsto \pi - \theta$, and $\sin^2 \theta$ is unchanged under that map.

Example 6.2

A counting run on $N = 256$ with $c = 8$ returns $m = 21$. Estimate μ .

Solution. Phase estimation returns the full angle, so $2\theta = 2\pi m/2^c = 2\pi \cdot 21/256 = 0.5154$ rad and $\theta = 0.2577$. Then

$$\mu = N \sin^2 \theta = 256 \times \sin^2(0.2577) = 256 \times 0.0650 = 16.6,$$

so $\mu \approx 17$. Since μ is an integer, the true value is most likely 16 or 17. Verified: with $\mu = 16$ exactly, $2\theta = 0.50536$, which maps to $m = 20.6$, consistent with the measured $m = 21$ after rounding.

6.3.3 Implementation guide**Building the counting circuit**

Qubits. c counting qubits plus the n -qubit search register plus the oracle ancillas. With c counting qubits, phase estimation returns $2\theta/2\pi$ to within 2^{-c} , so $\delta(2\theta) = 2\pi/2^c$ and $\delta\theta = \pi/2^c$. Differentiating $\mu = N \sin^2 \theta$ gives $\delta\mu = N \sin(2\theta) \delta\theta$, and since $N \sin 2\theta = 2\sqrt{\mu(N - \mu)}$,

$$\delta\mu \approx \sqrt{\mu(N - \mu)} \frac{2\pi}{2^c}.$$

Dropping the halving from 2θ to θ inflates $\delta\mu$ by two. Each extra counting qubit halves $\delta\mu$, and for small μ the resolution degrades as $\sqrt{\mu}$: at $N = 1024$ and $c = 8$ the measured $\delta\mu$ runs 0.785, 1.568, 3.117, 6.084 for $\mu = 1, 4, 16, 64$. For $\mu = O(1)$, resolving $\delta\mu < 1$ needs $2^c > 2\pi\sqrt{N}$, so take $c \approx n/2 + 3$ as a starting point.

Controlled- G^{2^k} . Unlike Shor, you cannot precompute a shortcut: G is a circuit, not a classical constant, so G^{2^k} genuinely means 2^k controlled repetitions of G . Summing the geometric series over $k = 0, \dots, c-1$, the total is $\sum_k 2^k = 2^c - 1$ Grover iterations, one oracle query each. Counting is *not* cheaper than searching. With $c \approx n/2 + 3$ it makes about $8\sqrt{N}$ oracle queries, roughly ten times the $(\pi/4)\sqrt{N}$ of the search itself and the same $O(\sqrt{N})$ scaling. That is why the exponential-search trick of §6.2 is often preferred in practice.

Controlling G . You need a controlled version of the whole Grover operator. It is enough to control the oracle and the diffuser's central multi-controlled gate: the Hadamard layers can stay uncontrolled, because $H^{\otimes n} H^{\otimes n} = \mathbb{I}$ when the control is off.

The two statements in that box combine into a cost as a function of precision,

and I want you to see how. The resolution formula says that 2^c must be about $2\pi\sqrt{\mu(N-\mu)}/\delta\mu$ to resolve μ to within $\pm\delta\mu$, and the cost is $2^c - 1$ oracle queries. Ask for a relative error ϵ , meaning $\delta\mu = \epsilon\mu$, and the query count becomes about $(2\pi/\epsilon)\sqrt{(N-\mu)/\mu}$, which is $O\left(\frac{1}{\epsilon}\sqrt{N/\mu}\right)$ oracle queries. Here $N = 2^n$ is the number of candidates, μ the number of solutions and ϵ the relative error in the estimate of μ . In plain words: halving the error doubles the number of queries, and the $\sqrt{N/\mu}$ in front is the cost of one search. The classical alternative is to sample inputs at random and count hits; that needs $O\left(\frac{1}{\epsilon^2} N/\mu\right)$ evaluations of f , so quantum counting is quadratically better in both factors.

The diffuser's sign, and reading the output

The sign will silently ruin the count. The circuit of Figure 6.12 is $-V$, not V . In plain Grover that is a global phase, but a *controlled* $-G$ turns it into a relative phase on the control qubit. The estimated full angle 2θ then shifts by π , so θ shifts by $\pi/2$, and $N \sin^2(\theta + \pi/2) = N \cos^2 \theta = N - \mu$: you recover the complement. Simulated at $n = 3$ with $c = 6$, an uncorrected sign returns 7 for $\mu = 1$ and 6 for $\mu = 2$. Fix it with a Z on the control qubit once per controlled diffuser, since $(Z \otimes \mathbb{I}) C(-G) = C(G)$. Step 3 of the Grover checklist cannot catch this, because $\Pr(w)$ against r is insensitive to the sign.

Interpreting the output. Compute $2\theta = 2\pi m/2^c$, then $\mu = N \sin^2 \theta$, then round to the nearest integer. If $m > 2^c/2$, you got the $e^{-2i\theta}$ branch; use $\theta \rightarrow \pi - \theta$, which gives the same μ .

Testing. Build G as an explicit matrix for $n = 3$ with $\mu = 1$ and $\mu = 2$, diagonalize, and check the eigenvalues are $e^{\pm 2i\theta}$ with $\theta = \arcsin \sqrt{\mu/N}$. Then simulate the full circuit and confirm the histogram peaks at the two predicted values of m .

6.4 Amplitude Amplification

6.4.1 Generalizing Grover

Nothing in the Grover analysis required the initial state to be the uniform superposition. Suppose instead we have any state-preparation unitary $\mathcal{A}$ with

$$\mathcal{A}|0\rangle = \sqrt{1-g}|w^\perp\rangle + \sqrt{g}|w\rangle, \quad (6.17)$$

where $|w\rangle$ collects the “good” states and g is the probability that measuring $\mathcal{A}|0\rangle$ gives a good outcome. Define

$$\mathcal{Q} = \mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_g, \quad \mathcal{S}_g = \mathbb{I} - 2|w\rangle\langle w|, \quad \mathcal{S}_0 = 2|0\rangle\langle 0| - \mathbb{I}. \quad (6.18)$$

Exactly as in (6.5),

$$\mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger = 2\mathcal{A}|0\rangle\langle 0|\mathcal{A}^\dagger - \mathbb{I} = 2|s\rangle\langle s| - \mathbb{I},$$

so $\mathcal{Q}$ is again two reflections, hence a rotation. Setting $\sin \theta = \sqrt{g}$,

$$\mathcal{Q}^r |s\rangle = \cos((2r+1)\theta) |w^\perp\rangle + \sin((2r+1)\theta) |w\rangle. \quad (6.19)$$

Verified numerically for $r = 1, \dots, 4$ with $N = 16$, one solution. This is (6.9) with $\mathcal{Q}$ written in place of G . The derivation in §6.2 used only two facts: $|s\rangle$ sits at angle θ from $|w^\perp\rangle$, and the operator is a product of two reflections whose axes are θ apart. Both hold here, so the algebra is the same line for line. The whole procedure is one state preparation followed by r rounds of $\mathcal{Q}$, and Figure 6.14 draws it as a circuit. Reading that circuit left to right reverses the order of the symbols in (6.18), which is the usual convention clash between operator products and circuit diagrams.

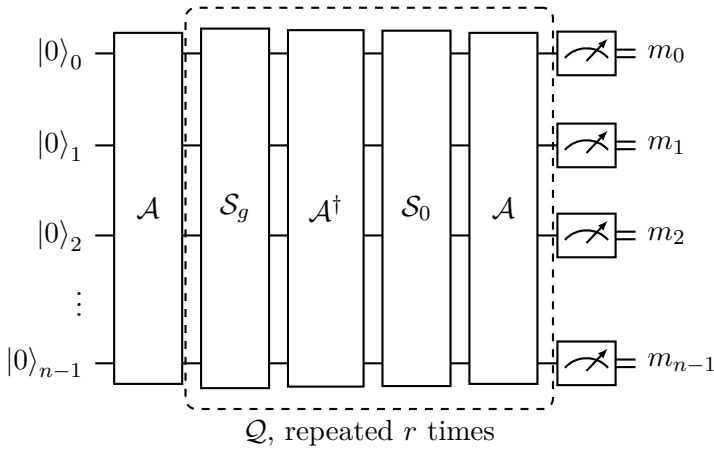


Figure 6.14: Amplitude amplification. The state preparation $\mathcal{A}$ replaces the Hadamard layer of Fig. 6.1, and the operator $\mathcal{Q} = \mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_g$ of (6.18) is applied r times. Grover's algorithm is the special case $\mathcal{A} = H^{\otimes n}$; there $\mathcal{A}^\dagger = \mathcal{A}$, and the three boxes $\mathcal{A}^\dagger, \mathcal{S}_0, \mathcal{A}$ collapse into the diffuser V .

Why this generalization matters

Grover's algorithm is the special case $\mathcal{A} = H^{\otimes n}$, $g = \mu/N$. The general statement is far more useful. *Any* quantum procedure that succeeds with probability g can be boosted to succeed with probability near 1 using $O(1/\sqrt{g})$ rounds of $\mathcal{Q}$, each round one $\mathcal{A}$, one $\mathcal{A}^\dagger$ and two reflections. The count comes from (6.19): with $\sin \theta = \sqrt{g}$ and g small, $\theta \approx \sqrt{g}$, so the optimal $r \approx \pi/(4\theta)$ is about $\pi/(4\sqrt{g})$. Classical repetition, running $\mathcal{A}$ and checking, succeeds with probability g each time and so needs about $1/g$ tries. For a one-percent success rate that is eight rounds against a hundred tries.

That quadratic saving is the engine behind amplitude estimation, quantum Monte Carlo methods, and the speed-ups quoted for quantum algorithms in finance and optimization. It applies whenever you can write your problem as “prepare a state, check whether it is good”.

6.4.2 Amplitude estimation

Running phase estimation on $\mathcal{Q}$, exactly as quantum counting does on G , returns 2θ and hence

$$g = \sin^2 \theta. \quad (6.20)$$

The factor of two is easy to lose. $\mathcal{Q}$ is a product of two reflections whose axes are θ apart, so it rotates by 2θ and its non-trivial eigenvalues are $e^{\pm 2i\theta}$. Diagonalizing $\mathcal{Q}$ for $N = 16$ and $\mu = 1$ gives eigen-phases ± 0.505361 , which is twice $\theta = 0.252680$. This matches §6.3 exactly, where G has eigenvalues $e^{\pm 2i\theta}$ for the same reason. Grover's algorithm is the case $\mathcal{A} = H^{\otimes n}$ and $g = \mu/N$, so the two derivations are the same calculation written twice.

Now the cost. Let ϵ be the target additive error in g , so that the estimate lies within $g \pm \epsilon$. With c counting qubits, phase estimation pins 2θ to within $2\pi/2^c$, so $\delta\theta = \pi/2^c$. Since $dg/d\theta = \sin 2\theta \leq 1$, the error in g is at most $\pi/2^c$. Reaching ϵ therefore needs $2^c \approx \pi/\epsilon$, and the circuit applies $\mathcal{Q}$ a total of $2^c - 1 \approx \pi/\epsilon$ times. That is $O(1/\epsilon)$ applications of $\mathcal{Q}$, each containing one $\mathcal{A}$, one $\mathcal{A}^\dagger$ and one call to the check. Classical Monte Carlo runs $\mathcal{A}$ and the check S times and takes the fraction of successes; the standard error is $\sqrt{g(1-g)/S}$, so error ϵ needs $S \approx g(1-g)/\epsilon^2$, which is $O(1/\epsilon^2)$ samples. In plain words: one more decimal digit costs ten times as many applications of $\mathcal{Q}$, but a hundred times as many classical samples. This quadratic speed-up in estimation[5] is one of the most frequently cited near-to-medium-term applications of quantum computing.

6.4.3 Derandomizing Grover

Grover's algorithm usually fails with small probability, because $r = \frac{\pi}{4\theta} - \frac{1}{2}$ is rarely an integer. Amplitude amplification offers a fix: adjust the *initial amplitude* so that the required r becomes exactly an integer.

Start from

$$|s\rangle = \sqrt{1 - g_0} |w^\perp\rangle + \sqrt{g_0} |w\rangle, \quad g_0 = \frac{\mu}{N}. \quad (6.21)$$

Check your normalization

It is easy to write this state as $\sqrt{\frac{N-\mu}{N}} |w^\perp\rangle + \sqrt{\frac{\mu}{N}} |w\rangle$, with the second fraction inverted. That is not a state: for $N = 16$, $\mu = 3$ the squared coefficients sum to 6.15, not 1. The amplitude of the good subspace is $\sqrt{\mu/N}$, which is *small*; $\sqrt{N/\mu}$ is large and cannot be an amplitude at all.

Now append one ancilla qubit and a rotation

$$R|0\rangle = \sqrt{1 - \frac{g'_0}{g_0}} |0\rangle + \sqrt{\frac{g'_0}{g_0}} |1\rangle, \quad (6.22)$$

and take the new state-preparation operator to be $\mathcal{A} = H^{\otimes n} \otimes R$. Then

$$\mathcal{A}|0\rangle^{\otimes n}|0\rangle = \left(\sqrt{1 - g_0} |w^\perp\rangle + \sqrt{g_0} |w\rangle\right) \otimes \left(\sqrt{1 - \frac{g'_0}{g_0}} |0\rangle + \sqrt{\frac{g'_0}{g_0}} |1\rangle\right).$$

Defining the new good state as $|u\rangle = |w\rangle |1\rangle$, the amplitude on $|u\rangle$ is $\sqrt{g_0} \cdot \sqrt{g'_0/g_0} = \sqrt{g'_0}$, so

$$|s_{\text{new}}\rangle = \sqrt{1 - g'_0} |u^\perp\rangle + \sqrt{g'_0} |u\rangle.$$

Choosing g'_0 so that $\frac{\pi}{4 \arcsin \sqrt{g'_0}} - \frac{1}{2}$ is an integer makes the algorithm *exact*.

The trick can only reduce the amplitude

Equation (6.22) requires $g'_0/g_0 \leq 1$, so the ancilla can only *lower* the good-state amplitude, never raise it. Pick the smallest integer r with $r \geq \frac{\pi}{4 \arcsin \sqrt{g_0}} - \frac{1}{2}$, then set $g'_0 = \sin^2\left(\frac{\pi}{2(2r+1)}\right)$, which is automatically $\leq g_0$.

Worked check for $N = 16$, $\mu = 1$: $g_0 = 0.0625$ and the exact r is 2.608. Rounding *up* to $r = 3$ gives $g'_0 = \sin^2(\pi/14) = 0.04952$, so $g'_0/g_0 = 0.792 \leq 1$ ✓ and R is a valid rotation. Rounding *down* to $r = 2$ would demand $g'_0 = \sin^2(\pi/10) = 0.0955 > g_0$, and no such R exists. Always round up.

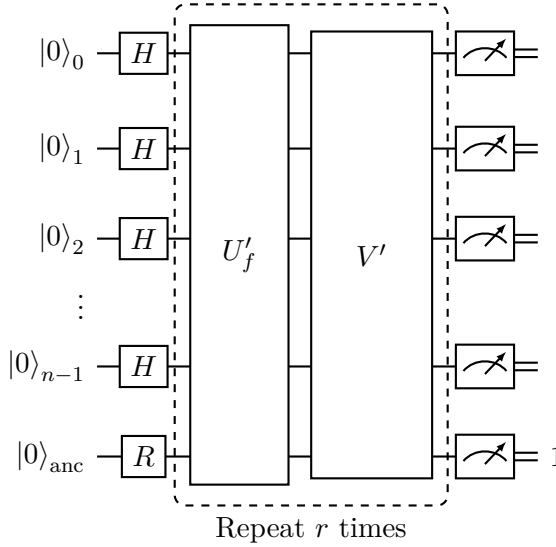


Figure 6.15: Derandomized Grover. One extra ancilla is rotated by R instead of H , and the oracle U'_f is modified to mark $|u\rangle = |w\rangle|1\rangle$: the original marking *and* the ancilla being $|1\rangle$. With g'_0 chosen as above, r iterations succeed with probability exactly 1.

6.4.4 Implementation guide

Building amplitude amplification

What you must supply. Three circuits: $\mathcal{A}$ (state preparation), $\mathcal{A}^\dagger$ (its inverse, obtained by reversing the gate order and daggering each gate), and $\mathcal{S}_g$ (mark the good states). The reflection $\mathcal{S}_0$ is the same multi-controlled- Z sandwich as Grover’s diffuser.

$\mathcal{A}$ must be unitary, not a measurement. A common mistake is to implement “prepare and check” with a mid-circuit measurement, which collapses the superposition and destroys the amplification. The check must be computed coherently into an ancilla.

$\mathcal{A}^\dagger$ is where the cost doubles. Each $\mathcal{Q}$ contains one $\mathcal{A}$ and one $\mathcal{A}^\dagger$, so an iteration costs twice the state preparation plus the two reflections.

Derandomizing, and testing what you built

For derandomization: compute $r = \lceil \frac{\pi}{4 \arcsin \sqrt{g_0}} - \frac{1}{2} \rceil$, then $g'_0 = \sin^2\left(\frac{\pi}{2(2r+1)}\right)$, then $R = R_y(2 \arcsin \sqrt{g'_0/g_0})$ on the ancilla, and modify the oracle to require the ancilla be $|1\rangle$: one extra control line.

Testing. Simulate $\Pr(\text{good})$ against r and check it matches $\sin^2((2r+1)\theta)$ with $\theta = \arcsin \sqrt{g}$. For the derandomized version, confirm the probability at your chosen r is 1 to numerical precision; if it is not, your g'_0 or your modified oracle is wrong.

6.5 Quantum Search as Hamiltonian Simulation

Grover's algorithm is discrete: reflect, reflect, repeat. There is a continuous alternative that reaches the answer *deterministically*, and it is instructive because it connects search to the Hamiltonian simulation of Chapter 7.

6.5.1 Guessing the Hamiltonian

Write the uniform superposition in the two-dimensional picture as before, using the same $|w\rangle$ and $|w^\perp\rangle$ as §6.2:

$$|\psi\rangle = a|w\rangle + b|w^\perp\rangle, \quad a = \frac{1}{\sqrt{N}}, \quad b = \sqrt{\frac{N-1}{N}}. \quad (6.23)$$

Here a and b are amplitudes, not probabilities and not angles. The state $|\psi\rangle$ is the $|s\rangle$ of (6.6), renamed because it is about to evolve in time, and in that notation $a = \sin \theta$ and $b = \cos \theta$. That is why t^* below carries the same $\sqrt{N}$ as the Grover count. We want a Hamiltonian that drives $|\psi\rangle$ to $|w\rangle$. The simplest guess that knows about both is

$$H = |w\rangle\langle w| + |\psi\rangle\langle\psi|. \quad (6.24)$$

In the $\{|w\rangle, |w^\perp\rangle\}$ basis, taken in that order,

$$H = \begin{pmatrix} 1+a^2 & ab \\ ab & b^2 \end{pmatrix} = \begin{pmatrix} 1+a^2 & ab \\ ab & 1-a^2 \end{pmatrix} = \mathbb{I} + a(aZ + bX),$$

using $a^2 + b^2 = 1$. Verified numerically. Watch the ordering. Equation (6.14) put $|w^\perp\rangle$ first, and here $|w\rangle$ comes first. That is only a labeling of the two axes, but the sign of the Z term depends on it.

6.5.2 Evolving

Since $(aZ + bX)^2 = (a^2 + b^2)\mathbb{I} = \mathbb{I}$ (the cross terms cancel because the Pauli matrices satisfy $ZX = -XZ$), the exponential closes in the usual way:

$$e^{-iHt} = e^{-it} [\cos(at)\mathbb{I} - i\sin(at)(aZ + bX)].$$

Acting on $|\psi\rangle$ and using $(aZ + bX)|\psi\rangle = \frac{H-\mathbb{I}}{a}|\psi\rangle = |w\rangle$,

$$|\psi(t)\rangle = e^{-it} [\cos(at)|\psi\rangle - i\sin(at)|w\rangle]. \quad (6.25)$$

Discarding the global phase, at

$$t^* = \frac{\pi}{2a} = \frac{\pi}{2}\sqrt{N} \quad (6.26)$$

we have $\cos(at^*) = 0$ and the state is exactly $|w\rangle$. Verified: for $N = 16$, evolving to $t^* = 6.2832$ gives $\Pr(w) = 1.000000$.

Deterministic, and still $O(\sqrt{N})$

Unlike Grover's algorithm, this reaches the solution with probability exactly 1, with no residual error and no overshoot. But the evolution time is $t^* = \frac{\pi}{2}\sqrt{N}$, with $N = 2^n$ the number of candidates, and simulating it with a well-chosen time step costs $O(\sqrt{N})$ oracle calls (the implementation guide below counts them). The quadratic speed-up is the same, as it must be: the $\Omega(\sqrt{N})$ lower bound on oracle queries applies to any quantum algorithm, continuous or discrete.

6.5.3 Simulating the Hamiltonian

H is a sum of two non-commuting projectors, $H_1 = |w\rangle\langle w|$ and $H_2 = |\psi\rangle\langle\psi|$. We simulate it by Trotter splitting: apply $e^{-iH_1\Delta t}e^{-iH_2\Delta t}$ a total of K times, with $K\Delta t = t$. The letter is K because r counts Grover iterations everywhere else in this chapter, and L counts Hamiltonian terms in Chapter 7.

The split is an approximation, not an identity

Because the commutator $[H_1, H_2] \neq 0$,

$$e^{-i(H_1+H_2)\Delta t} \neq e^{-iH_1\Delta t}e^{-iH_2\Delta t}.$$

The two differ at order Δt^2 . Measured for $N = 16$:

Δt	$\ \text{exact} - \text{split}\ $	Δt^2
0.5	0.0425	0.25
0.1	0.00171	0.01
0.01	0.0000171	0.0001

The error falls as Δt^2 , confirming first-order Trotter. Writing the split as an equality is misleading, especially in the same paragraph as “ H_1 and H_2 do not commute”. It hides the fact that K must be chosen large enough for the accumulated Trotter error to be acceptable: $K = t/\Delta t$ steps, each off by $O(\Delta t^2)$, add up to $O(t\Delta t)$. Chapter 7 develops this properly.

Each factor has a closed form. For any normalized $|\phi\rangle$,

$$e^{-i|\phi\rangle\langle\phi|\Delta t} = \mathbb{I} + |\phi\rangle\langle\phi| \left(e^{-i\Delta t} - 1 \right), \quad (6.27)$$

which follows because $|\phi\rangle\langle\phi|^k = |\phi\rangle\langle\phi|$ for all $k \geq 1$, so the series $\sum_k \frac{(-i\Delta t)^k}{k!} |\phi\rangle\langle\phi|$ sums to $(e^{-i\Delta t} - 1) |\phi\rangle\langle\phi|$. Verified numerically. Keep the $1/k!$ factors. Dropping them gives a divergent series, not the stated answer.

6.5.4 Circuits

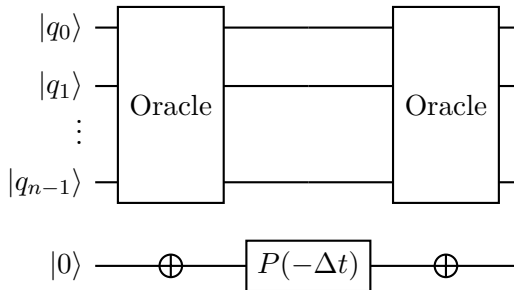


Figure 6.16: Circuit for $e^{-i|w\rangle\langle w|\Delta t}$. The oracle flips the ancilla if and only if the register holds the solution $|w\rangle$. The phase gate $P(-\Delta t) = \text{diag}(1, e^{-i\Delta t})$ then attaches the phase to that branch alone. The second oracle call uncomputes the ancilla back to $|0\rangle$.

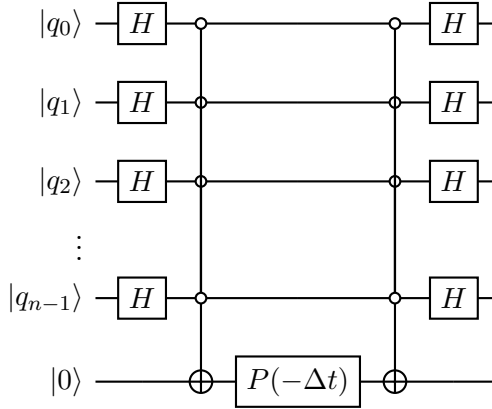


Figure 6.17: Circuit for $e^{-i|\psi\rangle\langle\psi|\Delta t}$. Conjugating by $H^{\otimes n}$ turns $|\psi\rangle$ into $|0\cdots 0\rangle$, so the middle section only has to mark the all-zeros string, which is what the open circles do. This is the same conjugation trick as the Grover diffuser.

Example 6.3

Verify that the circuit of Fig. 6.17 acts correctly on $|w\rangle$ and on $|w^\perp\rangle$.

Solution. By (6.27) the circuit implements $\mathbb{I} + |\psi\rangle\langle\psi|(e^{-i\Delta t} - 1)$. Since $\langle\psi|w\rangle = a$ and $\langle\psi|w^\perp\rangle = b$,

$$\begin{aligned} |w\rangle &\mapsto |w\rangle + a(e^{-i\Delta t} - 1)|\psi\rangle, \\ |w^\perp\rangle &\mapsto |w^\perp\rangle + b(e^{-i\Delta t} - 1)|\psi\rangle. \end{aligned}$$

Both verified numerically for $N = 16$, $\Delta t = 0.7$.

I want you to notice that *both* lines carry $e^{-i\Delta t}$. A $e^{+i\Delta t}$ in the second would not correspond to any evolution under H_2 . The operator does not distinguish $|w\rangle$ from $|w^\perp\rangle$ except through the overlap.

6.5.5 Implementation guide

Building the simulation-based search

Qubits. n for the register, one ancilla for the phase kickback, plus multi-controlled-gate ancillas. Same as Grover.

Choose Δt and K together. Here K is the number of Trotter steps, $N = 2^n$, and each step calls the oracle twice (Fig. 6.16: once to compute, once to uncompute). You need $K\Delta t = t^* = \frac{\pi}{2}\sqrt{N}$, and the first-order Trotter error accumulates as $O(t^*\Delta t)$ in the norm distance between the exact and the split evolution. For a target error ϵ take $\Delta t \sim \epsilon/t^*$ and hence $K \sim (t^*)^2/\epsilon$, which is $O(N/\epsilon)$ steps and $O(N/\epsilon)$ oracle calls: halving the error doubles the count, and so does doubling N . That is worse than the $r = O(\sqrt{N})$ oracle calls of (6.11), unless you use a smarter splitting.

The $\Delta t = \pi$ shortcut. Taking $\Delta t = \pi$ maximizes the rotation angle per step: the resulting operator rotates the Bloch vector by φ with $\cos(\varphi/2) = 1 - 2/N$, so $\varphi \approx 4/\sqrt{N}$. The Bloch vector has to turn through about π , from $|w^\perp\rangle$ to $|w\rangle$, so $\pi/\varphi \approx (\pi/4)\sqrt{N}$ steps suffice, the Grover count. The separate letter marks a Bloch-sphere angle, not the half-angle θ of the rest of the chapter. The two are related exactly by $\varphi = 4\theta$, the Bloch angle of one Grover step. The algorithm has become Grover's algorithm in disguise, which is the real lesson of this section.

Testing. Build H as a $2^n \times 2^n$ matrix, exponentiate it directly with a dense solver, and confirm $\Pr(w) = 1$ at $t^* = \frac{\pi}{2}\sqrt{N}$. Then compare against your Trotterized circuit and plot the error against Δt : it should fall as Δt^2 . If it falls as Δt , you have a bug in the splitting order.

6.6 Summary

- Grover searches an unstructured space of size N in $O(\sqrt{N})$ oracle queries, and this is optimal.
- It searches inputs to a *computed* function, not stored data; loading a database would itself cost $\Omega(N)$.
- $U_f = \mathbb{I} - 2|w\rangle\langle w|$ reflects about the hyperplane orthogonal to $|w\rangle$; $V = 2|s\rangle\langle s| - \mathbb{I}$ reflects about $|s\rangle$. Two reflections make a rotation, and that is the whole algorithm.
- V is a matrix *product* $H^{\otimes n}U_{f_0}H^{\otimes n}$, not a tensor product.
- θ is the half-angle throughout, with $\sin \theta = \sqrt{\mu/N}$, and every full rotation is written 2θ . Each iteration rotates by 2θ .
- After r iterations, $\Pr(w) = \sin^2((2r+1)\theta)$; the optimum is $r \approx \frac{\pi}{4}\sqrt{N/\mu}$.

- **Overshooting makes things worse.** At $N = 8$, $r = 4$ gives $\Pr(w) = 0.012$, below random guessing.
- Residual failure probability is *bounded by* $1/N$, not equal to it.
- Quantum counting is phase estimation on G , whose eigenvalues are $e^{\pm 2i\theta}$; it returns 2θ , and then $\mu = N \sin^2 \theta$.
- Amplitude amplification replaces $H^{\otimes n}$ by any $\mathcal{A}$: success probability g is boosted in $O(1/\sqrt{g})$ rounds instead of $O(1/g)$.
- Amplitude estimation gives g to accuracy ϵ in $O(1/\epsilon)$ rounds versus $O(1/\epsilon^2)$ classically. Chapter 10 removes the phase-estimation register from it entirely.
- Grover's operator is a Chebyshev polynomial of a block-encoded projector; Chapter 10 makes that precise and explains the overshoot.
- Derandomization appends an ancilla rotated by R ; it can only *lower* the good amplitude, so always round r up.
- Continuous-time search under $H = |w\rangle\langle w| + |\psi\rangle\langle\psi|$ reaches $|w\rangle$ exactly at $t = \frac{\pi}{2}\sqrt{N}$; Trotterizing it recovers Grover.

6.7 End-of-Chapter Problems

- 6.1** Write out U_f and V as explicit 4×4 matrices for $n = 2$ with $w = 11$, and verify $G = VU_f$ is a rotation by 2θ .
- 6.2** For $n = 2$, $\mu = 1$, show that $r = 1$ gives $\Pr(w) = 1$ exactly. Why is $N = 4$ special?
- 6.3** Prove $\langle s|s' \rangle = \cos 2\theta$ and complete the derivation of (6.8) using the double-angle identities.
- 6.4** Tabulate $\Pr(w)$ against $r = 0, \dots, 10$ for $N = 16$, $\mu = 1$. Identify the r that (6.11) selects, the r with the highest probability in this range, and the worst r . The first two differ, because the tabulated range covers more than one revolution of the state.
- 6.5** Show that $V = H^{\otimes n} X^{\otimes n} C^{n-1}(Z) X^{\otimes n} H^{\otimes n}$ implements $2|s\rangle\langle s| - \mathbb{I}$ up to a global sign.
- 6.6** Explain why applying U_f alone, without the diffuser, leaves all measurement probabilities unchanged. Which quantity does it change?
- 6.7** For $N = 1024$ and μ unknown, describe the exponential-search procedure and bound its expected number of oracle calls.

- 6.8** Derive the eigenvalues and eigenvectors of the rotation matrix (6.14) from scratch.
- 6.9** A quantum-counting run with $c = 6$ on $N = 64$ returns $m = 8$. Estimate μ . What is the resolution in μ at this value of c ?
- 6.10** Show that the two possible counting outcomes, $2^c(2\theta)/2\pi$ and $2^c(1 - 2\theta/2\pi)$, give the same estimate of μ .
- 6.11** Show that $\mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger = 2|s\rangle\langle s| - \mathbb{I}$ for any unitary $\mathcal{A}$ with $\mathcal{A}|0\rangle = |s\rangle$.
- 6.12** Prove (6.19) by induction on r .
- 6.13** For $N = 32$, $\mu = 1$, compute the exact r , round it up, and find g'_0 and the rotation angle of R . Verify $g'_0 \leq g_0$.
- 6.14** Show that $[|w\rangle\langle w|, |\psi\rangle\langle\psi|] \neq 0$ unless $|\psi\rangle = |w\rangle$ or $\langle w|\psi\rangle = 0$.
- 6.15** Verify (6.27) by summing the exponential series explicitly, keeping the $1/k!$ factors.
- 6.16** Compute the Trotter error $\|e^{-i(H_1+H_2)\Delta t} - e^{-iH_1\Delta t}e^{-iH_2\Delta t}\|$ for $N = 16$ at $\Delta t = 0.5, 0.1, 0.01$ and confirm the $O(\Delta t^2)$ scaling.
- 6.17** Grover's algorithm halves the effective key length of a symmetric cipher. Explain why AES-256 is therefore considered quantum-safe while AES-128 is marginal.

Research Problems and Projects

The problems below are open-ended. Each is meant to occupy several weeks and to end in a written report with figures and tables that someone else could check. None of them can be answered by a single calculation.

- R6.1 How much does a wrong μ cost?** Section 6.2 states that overshooting is a real failure mode, but the chapter never quantifies the damage as a function of how wrong your estimate of μ is. Build a state-vector simulator for the Grover operator (no circuit compilation needed; the two-dimensional rotation picture of (6.9) is enough to check your full 2^n simulator against). Then, for $n = 8, 10, 12, 14$ and true solution counts $\mu_{\text{true}} \in \{1, 2, 4, 8, 16\}$, run the algorithm with r computed from a wrong assumed μ_{est} and record the success probability over a grid of $\mu_{\text{est}}/\mu_{\text{true}}$. Compare the resulting expected number of oracle calls (repeat until success) against the exponential-search schedule of [4], which needs no estimate at all. The deliverable is a family of measured curves plus a stated rule of thumb: how accurate must μ_{est} be

before a fixed- r run beats exponential search? Be careful to report expected oracle calls including repetitions, not single-shot success probability. The two orderings can disagree.

R6.2 Reproduce a published amplitude-estimation claim. Amplitude estimation as presented in §6.4 runs phase estimation on $\mathcal{Q}$, which needs a large ancilla register and a QFT. A line of later work removes that register and claims to keep the $O(1/\epsilon)$ scaling. Maximum-likelihood amplitude estimation [28] is the best known example. Read it against the original treatment in [5]. Reproduce the central scaling figure of [28] from scratch: implement the likelihood function, the Grover-power schedule, and the estimator, and recover the reported error versus oracle-call curve. Then extend it in one direction the paper does not cover, for instance depolarizing noise on each application of $\mathcal{Q}$, or a $\mathcal{Q}$ whose rotation angle carries a small systematic error. The deliverable is a reproduction plot placed beside the published one, a table of fitted scaling exponents, and an honest statement of where your reproduction disagrees with the paper and why. Reproductions of this kind often fail on the choice of Grover powers rather than on the estimator, so report that schedule explicitly.

R6.3 A fault-tolerant resource count for a real predicate. The implementation guide in §6.2.8 calls Grover [15] a fault-tolerant-era algorithm but gives no numbers. Pick one concrete computed predicate, for example a random 3-SAT instance at the satisfiability threshold, or preimage search on a reduced-round hash. Implement the oracle in a real framework and compile it to Clifford+ T . Then produce a resource table running from $n = 8$ up to as large an n as your tooling allows. Record logical qubit count, T -count, T -depth, and total oracle calls at the optimal r . Compare at least two constructions of $C^{n-1}(X)$, the clean-ancilla version assumed in §6.2.8 and a dirty-ancilla or relative-phase alternative, and report how much each saves. Then combine your T -counts with a published surface-code cycle time to estimate the wall-clock cost, and compare against a classical SAT solver on the same instances. The deliverable is the resource table, a fitted T -count versus n curve, and a defensible crossover estimate. The expected conclusion, that the crossover is far away, is already widely believed. The contribution here is the numbers for your specific predicate. Document your gate-set and cost assumptions carefully enough that someone can disagree with them.

Chapter 7

Hamiltonian Simulation

Simulating quantum systems was Feynman's original motivation for quantum computers, and it remains the application with the clearest physical case. This chapter builds the machinery.

The dynamics of a closed quantum system obey the Schrödinger equation

$$i\hbar \frac{d}{dt} |\psi\rangle = H |\psi\rangle, \quad (7.1)$$

whose solution for a time-independent Hamiltonian is

$$|\psi(t)\rangle = e^{-iHt} |\psi(0)\rangle. \quad (7.2)$$

For n qubits, e^{-iHt} is a $2^n \times 2^n$ matrix with 4^n entries. Storing it classically is already impossible for $n \approx 50$, where 4^n is about 10^{30} complex numbers. Exponentiating it is worse: by the usual methods that costs about 8^n classical arithmetic operations. That exponential wall is the problem this chapter solves. It is not solved in general. It is solved for the structured Hamiltonians that physics and chemistry actually produce.

What you should be able to do after this chapter

- State the Trotter formulas and their error scaling, per step and in total.
- Choose the number of Trotter steps for a target accuracy.
- Simulate a 1-sparse Hamiltonian given oracles for its structure.
- Implement a linear combination of unitaries and know its success probability.
- Write any Hamiltonian as a sum of Pauli strings.
- Draw the circuit for $e^{-i\theta P}$ for any Pauli string P , from memory.

Where this chapter leads

Section 7.4 builds the *Pauli exponential* circuit: given a Pauli string $P = \sigma_1 \otimes \cdots \otimes \sigma_n$, implement $e^{-i\theta P}$ exactly, using CNOTs, single-qubit basis changes and one R_z .

That one construction is the foundation of the next two chapters:

- **Chapter 8** maps fermionic Hamiltonians onto sums of Pauli strings and then exponentiates each one exactly this way.
- **Chapter 9** builds QAOA as a Trotterized adiabatic evolution, exponentiating cost and mixer Pauli strings with angles set by a schedule instead of by a single fixed time.

If you read only one section of this chapter, read §7.4.

7.1 The Trotter–Suzuki Method

7.1.1 Locality

The naive first-order step

$$|\psi(\Delta t)\rangle = \left(\mathbb{I} - iH\Delta t + \mathcal{O}(\Delta t^2)\right) |\psi(0)\rangle$$

is not useful, because the operator in brackets is not even unitary. The way forward uses a structural fact about physical Hamiltonians: they are *local*. Almost every Hamiltonian of interest decomposes as

$$H = \sum_{k=1}^L H_k, \tag{7.3}$$

where each H_k acts non-trivially on at most $f \ll n$ qubits and L , the number of terms, is polynomial in n . Locality is what makes efficient quantum simulation possible at all, a point first made rigorous by Lloyd [19]. Each $e^{-iH_k t}$ is then an f -qubit gate. By Chapter 2 we can build it from a number of CNOTs and single-qubit gates that depends on f but not on n , so implementing all L of them costs L times a constant, which is polynomial in n .

The difficulty is that the H_k generally do not commute, so

$$e^{-iHt} \neq \prod_k e^{-iH_k t}. \tag{7.4}$$

7.1.2 First-order Trotter

Lie product formula

For Hermitian A , B , the Lie product formula states that

$$\lim_{r \rightarrow \infty} \left(e^{iAt/r} e^{iBt/r} \right)^r = e^{i(A+B)t}. \quad (7.5)$$

For finite r the single-step error is $e^{iAt}e^{iBt} = e^{i(A+B)t} + \mathcal{O}(t^2)$, the leading correction being $-\frac{1}{2}[A, B]t^2$.

The $\mathcal{O}(t^2)$ is the *non-commuting* case

Deriving the $\mathcal{O}(t^2)$ error from “if A and B commute, then ...” gets it backwards. **If A and B commute the identity is exact**, with no error term at all: $e^{iAt}e^{iBt} = e^{i(A+B)t}$ identically. Verified numerically. The error term exists *because* they do not commute, and it is proportional to the commutator. Getting this the wrong way round makes the rest of the chapter look unmotivated.

The Trotter–Suzuki decomposition splits the total time into r steps of duration t/r , which gives the first-order formula:

$$e^{i(A_1 + \dots + A_L)t} = \left(e^{iA_1 t/r} e^{iA_2 t/r} \dots e^{iA_L t/r} \right)^r + \mathcal{E}_{\text{tot}}, \quad (7.6)$$

where L is the number of terms, as in (7.3), t is the total evolution time and r is the number of Trotter steps.

Here is how the error bound is assembled. Within one step, every pair of terms A_j , A_k contributes a commutator error of at most $\|[A_j, A_k]\| (t/r)^2/2$, by the Lie product formula. There are fewer than L^2 pairs, and if every $\|A_k\|$ is of order one, each commutator norm is too, so one step carries an error of order $L^2 t^2 / r^2$. Errors of successive unitary steps add at worst, and there are r steps, so

$$\mathcal{E}_{\text{tot}} = \mathcal{O}\left(\frac{L^2 t^2}{r}\right) \implies r = \mathcal{O}\left(\frac{L^2 t^2}{\epsilon}\right). \quad (7.7)$$

Here ϵ is the target error, the operator norm of the difference between the product on the right of (7.6) and the exact e^{iHt} , and the second statement follows from the first by setting $\mathcal{E}_{\text{tot}} = \epsilon$ and solving for r . In plain words: halving the tolerated error doubles the number of steps, and doubling the evolution time quadruples it. What is being counted is exponentials. Each step applies L of them, one $e^{iA_k t/r}$ per term, so the whole circuit uses $rL = \mathcal{O}(L^3 t^2 / \epsilon)$ exponentials of the individual terms, each of which is a few-qubit gate of fixed cost.

Counting terms this way assumes each $\|A_k\|$ is $\mathcal{O}(1)$. The norm-aware form, used in the implementation guide below, replaces L by $\Lambda = \sum_k \|H_k\|$, the sum of the operator norms of the terms, which reduces to the same thing when every term has unit norm.

Per-step error versus total error

The per-step Trotter error is $\mathcal{O}(\Delta t^2)$ with $\Delta t = t/r$. The *accumulated* error over r steps is $r \cdot \mathcal{O}(\Delta t^2) = \mathcal{O}(t \Delta t)$, which is first order in Δt . Quoting $\mathcal{O}(\Delta t^2)$ as the error of the whole simulation is a factor of r too optimistic. Table 7.1 measures it.

Table 7.1: Total first-order Trotter error for $H = X \otimes \mathbb{I} + Z \otimes Z$ at $t = 1$, in the Frobenius norm. The clean $1/r$ scaling confirms that the total error is $\mathcal{O}(\Delta t)$, not $\mathcal{O}(\Delta t^2)$.

r	total error	$1/r$
1	1.598	1
2	0.725	0.5
10	0.140	0.1
100	0.0140	0.01

7.1.3 Second-order (symmetric) Trotter

Symmetrizing the product cancels the leading error:

$$e^{i(A+B)t} = e^{iAt/2} e^{iBt} e^{iAt/2} + \mathcal{O}(t^3). \quad (7.8)$$

Verified numerically:

Table 7.2: Single-step errors for $H = X + Z$, with $A = X$ and $B = Z$, in the Frobenius norm. Halving t divides the first-order error by 4 and the symmetric error by 8. Those are the signatures of $\mathcal{O}(t^2)$ and $\mathcal{O}(t^3)$ respectively.

t	first-order error	symmetric error
0.50	3.34×10^{-1}	6.31×10^{-2}
0.20	5.61×10^{-2}	4.19×10^{-3}
0.10	1.41×10^{-2}	5.26×10^{-4}
0.05	3.53×10^{-3}	6.59×10^{-5}

I want you to see the merging written out. Figure 7.1 shows the symmetric product for $r = 3$ steps on the top row and the merged form underneath. Each adjacent pair of A half-steps fuses into one full A step. Three steps therefore need four exponentials of A rather than six, and in general r steps need $r + 1$ rather

Why symmetric Trotter is nearly free

The symmetric formula costs one extra exponential per step, but consecutive steps share their end-points: $\dots e^{iAt/2} e^{iAt/2} \dots = \dots e^{iAt} \dots$. Over r steps the overhead is a single extra exponential in total. An order of accuracy for essentially nothing, which is why practical implementations always use it.

than $2r$. First-order Trotter would have used r of them, so the entire price of an extra order of accuracy is one exponential.

The step count changes too, and for the better. For the symmetric formula the error of one step of duration t/r is of order $(\Lambda t/r)^3$, where $\Lambda = \sum_k \|H_k\|$ is the sum of the operator norms of the terms and t is the total evolution time. The r steps accumulate at worst additively, giving a total error of order $\Lambda^3 t^3 / r^2$. Setting that equal to the target error ϵ and solving gives $r \approx (\Lambda^3 t^3 / \epsilon)^{1/2}$, the second-order rule quoted in the implementation guide below. In plain words: tightening the tolerance by a factor of one hundred now costs only ten times as many steps, where the first-order formula would have cost a hundred times as many. The circuit still applies L exponentials per step, one per term of H , so the whole simulation uses about rL exponentials of the individual terms, each a few-qubit gate of fixed cost.

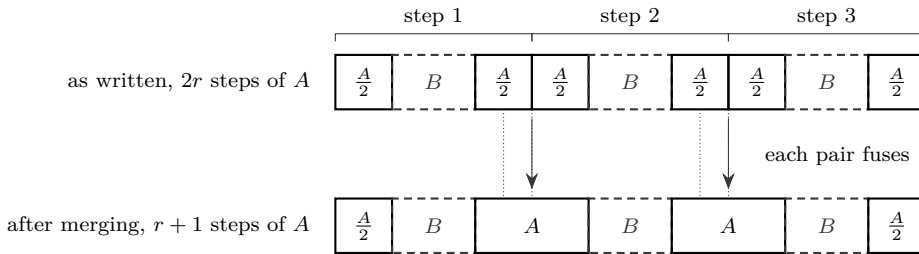


Figure 7.1: Why the symmetric formula is nearly free, drawn for $r = 3$. Solid boxes are exponentials of A and dashed boxes are exponentials of B ; among the solid boxes the width is proportional to the time evolved, so a full A step is drawn twice as wide as a half-step. The dashed B boxes all carry the same time and are drawn to their own scale. The two half-steps of A that meet at a step boundary carry the same generator and add their times, so they fuse into one full step. The total evolution time of each generator is unchanged. Only the number of circuit blocks drops, from $2r$ to $r + 1$.

Exercise 7.1

Prove (7.8) by expanding both sides to third order in t .

Hint: expand each factor to $\mathcal{O}(t^3)$, multiply, and check that the t^2 terms match $\frac{(i)^2}{2}(A + B)^2 t^2$. The commutator terms cancel by symmetry.

7.1.4 Implementation guide

Choosing the Trotter parameters

Decide the splitting first. Group the terms of H into sets that commute internally: all Z -type terms, all X -type terms, and so on. Terms within a commuting group can be exponentiated in any order with no error, so only the number of *groups* enters the error, not the number of terms.

Choose r from the target accuracy, using $r \approx \Lambda^2 t^2 / \epsilon$ for first order or $r \approx (\Lambda^3 t^3 / \epsilon)^{1/2}$ for second order, where $\Lambda = \sum_k \|H_k\|$ is the sum of the operator norms, t the evolution time and ϵ the target error. In my experience these worst-case bounds are very pessimistic. In practice, halve r until the answer visibly changes, then double back.

Always use the symmetric formula and merge the shared half-steps, as in Figure 7.1.

Testing a Trotter circuit

For small n , build H densely and compare a library matrix exponential e^{-iHt} against your circuit's unitary. Plot $\log(\text{error})$ against $\log(\Delta t)$ for a *single* step of duration Δt : the slope must be 2 for first order and 3 for symmetric. A slope of 1 means a bug in the ordering. Repeat for the whole r -step circuit at fixed t , where the slopes drop by one to 1 and 2 because the per-step errors accumulate.

7.2 Simulating Sparse Hamiltonians

7.2.1 Sparsity and oracles

A matrix is *d-sparse* if every row has at most d non-zero entries. It is *row-computable* if we have two oracles:

$$\begin{aligned} O_f |x\rangle |j\rangle |0\rangle &= |x\rangle |j\rangle |f(x, j)\rangle, \\ O_H |x\rangle |y\rangle |0\rangle &= |x\rangle |y\rangle |H_{xy}\rangle, \end{aligned} \tag{7.9}$$

where $f(x, j)$ is the column index of the j -th non-zero entry in row x , and H_{xy} is that entry's value.

Example 7.1

For

$$A = \begin{pmatrix} 0 & 1 & 0 & a \\ 1 & b & 0 & 2 \\ 0 & 0 & c & 0 \\ a & 2 & 0 & 0 \end{pmatrix},$$

row 0 has non-zeros in columns 1 and 3, so $f(0,0) = 1$ and $f(0,1) = 3$, and the value oracle gives $O_H |0\rangle |1\rangle |0\rangle = |0\rangle |1\rangle |1\rangle$ and $O_H |0\rangle |3\rangle |0\rangle = |0\rangle |3\rangle |a\rangle$: $A_{01} = 1$, $A_{03} = a$. The matrix is 3-sparse (row 1 has three non-zeros).

7.2.2 The 1-sparse case

A d -sparse Hermitian H decomposes into a sum of 1-sparse Hermitian pieces, each simulable separately and recombined by Trotter. Think of the non-zero off-diagonal entries as the edges of a graph on the basis states. Every vertex has at most d edges, so the edges can be colored with at most $d + 1$ colors such that no two edges of the same color share a vertex, and each color class is then a 1-sparse Hermitian matrix, with the diagonal entries added to a class in which their row is unmatched. (Exactly d colors do not always suffice: three states coupled in a triangle are 2-sparse but need three colors.) When each piece must be computable from the oracles alone, without seeing the whole matrix, the coloring has to be chosen locally and the number of pieces grows, to order d^2 in the best constructions. Either way, it suffices to handle $d = 1$. The reduction and the 1-sparse subroutine below are due to Aharonov and Ta-Shma [2].

Consider

$$H = \begin{pmatrix} 0 & 0 & 0 & a \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ a & 0 & 0 & 0 \end{pmatrix}, \quad f(0) = 3, \quad f(1) = 1, \quad f(2) = 2, \quad f(3) = 0. \quad (7.10)$$

Verified: H is Hermitian, exactly one non-zero per row, and f is an *involution*, $f(f(x)) = x$. That involution property is forced by Hermiticity, and it is the key structural fact.

The action on basis states splits into two kinds:

$$\begin{aligned} H |0\rangle &= a |3\rangle, & H |3\rangle &= a |0\rangle && \text{(a swapped pair, } x \neq f(x)), \\ H |1\rangle &= b |1\rangle, & H |2\rangle &= c |2\rangle && \text{(eigenstates, } x = f(x)). \end{aligned}$$

Reordering the basis to $(0, 3, 1, 2)$ makes H block-diagonal:

$$H \longrightarrow \begin{pmatrix} 0 & a \\ a & 0 \end{pmatrix} \oplus \begin{pmatrix} b & 0 \\ 0 & c \end{pmatrix} = aX \oplus \text{diag}(b, c),$$

so the evolution is

$$e^{-iHt} = e^{-iaXt} \oplus \text{diag}(e^{-ibt}, e^{-ict}). \quad (7.11)$$

Both verified numerically.

Getting the rotation angles right

The off-diagonal block is

$$e^{-iaXt} = R_x(2at),$$

not $R_x(2a)$. The time must appear in the angle. Verified: with $a = 0.7$ and $t = 1.7$, $e^{-iaXt} = R_x(2at)$ holds and $R_x(2a)$ fails.

The diagonal block is more subtle. The quantities e^{-ibt} and e^{-ict} are *scalars*, not gates; calling them “ $R_z(b)$ and $R_z(c)$ ” conflates a phase with a rotation. What is true is

$$\text{diag}(e^{-ibt}, e^{-ict}) = e^{-i(b+c)t/2} R_z((b-c)t),$$

verified numerically. That is a z -rotation by the *difference* of the eigenvalues, times a global phase given by their mean.

7.2.3 The algorithm

The circuit must decide, coherently, which of the two cases applies. It must do so without measuring, since x is generally in superposition.

Case I: $x \neq f(x)$ (a swapped pair). Take $|\psi\rangle = \alpha|x\rangle + \beta|f(x)\rangle$.

Step 1. Apply O_f : $\alpha|x\rangle|f(x)\rangle + \beta|f(x)\rangle|x\rangle$.

Step 2. Apply the comparison circuit, writing an ancilla bit that records which of the two registers is larger: $\alpha|x\rangle|f(x)\rangle|0\rangle + \beta|f(x)\rangle|x\rangle|1\rangle$ (assuming $f(x) > x$).

Step 3. Controlled-SWAP on the first two registers, controlled by that bit. Both branches now have the *same* first two registers: $|x\rangle|f(x)\rangle$ ($\alpha|0\rangle + \beta|1\rangle$). *This is the crucial step:* the pair $\{x, f(x)\}$ has been sorted into a canonical order, so the two-dimensional subspace is now carried entirely by the ancilla.

Step 4. Apply O_H to fetch $H_{x,f(x)}$ into a further register.

Step 5. Apply R_x to the ancilla, with angle controlled by $H_{x,f(x)}$.

Step 6. Uncompute everything except the rotation, leaving $e^{-iH_{x,f(x)}Xt}(\alpha|0\rangle + \beta|1\rangle)$.

Case II: $x = f(x)$ (an eigenstate). The comparison returns 0, the controlled-SWAP does nothing, O_H fetches H_{xx} , and a controlled R_z applies the diagonal phase. Figure 7.2 assembles both cases into a single circuit.

The comparison circuit must define $x = y$

The natural specification is

$$\text{COMP } |x\rangle |y\rangle |0\rangle = \begin{cases} |x\rangle |y\rangle |0\rangle & x < y, \\ |x\rangle |y\rangle |1\rangle & x > y. \end{cases}$$

This leaves $x = y$ undefined, and Case II uses exactly that input. The specification must be

$$\text{COMP } |x\rangle |y\rangle |0\rangle = \begin{cases} |x\rangle |y\rangle |0\rangle & x \leq y, \\ |x\rangle |y\rangle |1\rangle & x > y, \end{cases} \quad (7.12)$$

so that the diagonal case falls cleanly into the “0” branch and the controlled-SWAP correctly does nothing.

Exercise 7.2

Draw the circuit for O_f realizing (7.10), i.e. $f(0) = 3$, $f(1) = 1$, $f(2) = 2$, $f(3) = 0$ on two qubits.

Answer: f swaps $|00\rangle \leftrightarrow |11\rangle$ and fixes $|01\rangle, |10\rangle$, so with $x = x_0x_1$ (x_0 the most significant bit, on the top wire) it is an X on both qubits controlled on $x_0 = x_1$. Three two-qubit gates suffice, with no ancilla: a CNOT from q_0 into q_1 , then an X on q_0 controlled on $q_1 = 0$, then that same CNOT again. Verified as an explicit 4×4 permutation matrix; a Toffoli would need a third qubit, which this exercise does not have. This is the *in-place* map $|x\rangle \rightarrow |f(x)\rangle$, while Step 1 of the algorithm needs the out-of-place oracle $O_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$ of (7.9): copy x into the second register with two CNOTs first, then run the permutation there.

What does the whole sparse simulation cost? Let me assemble it from the pieces, counting oracle queries, since those are the expensive operations here and everything else is arithmetic on n -bit registers. Figure 7.2 makes four queries, two to compute $f(x)$ and $H_{x,f(x)}$ and two to uncompute them, and it simulates one 1-sparse piece for any time t in one pass, because the rotation angle simply scales

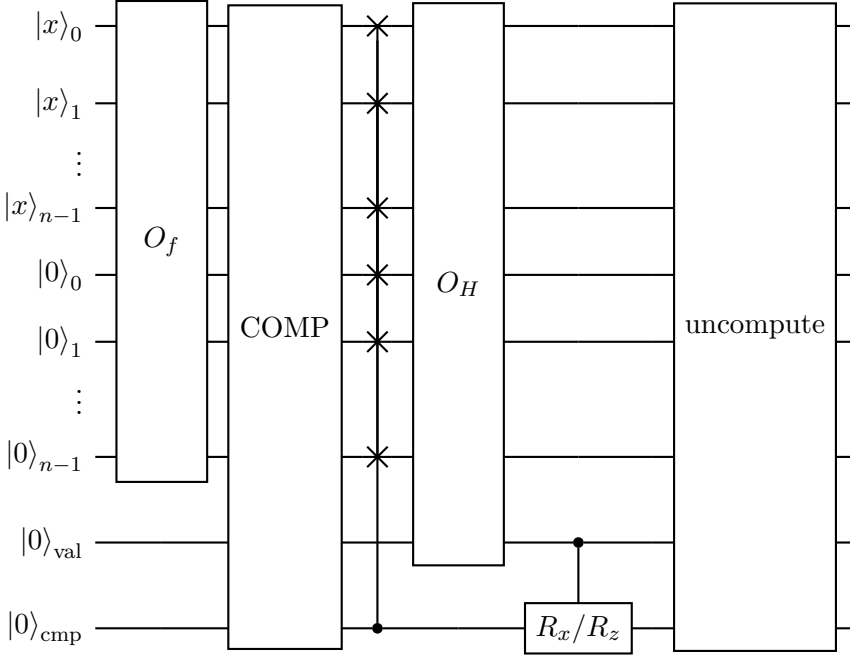


Figure 7.2: Circuit simulating a 1-sparse Hamiltonian. The top register holds x , the second register receives $f(x)$ from O_f , the value register receives $H_{x,f(x)}$ from O_H , and the bottom qubit records the comparison. The COMP box is drawn across the value register only because that register lies between the wires it acts on; COMP touches the two index registers and the comparison qubit alone. After the controlled-SWAP, the whole two-dimensional dynamics lives on that single ancilla. There an R_x (off-diagonal case) or R_z (diagonal case) rotation, with its angle controlled by the fetched value, implements the evolution. Everything except the rotation is then uncomputed. Four oracle queries in total.

with t . A d -sparse H has been split into D such pieces, where D is $d + 1$ for a global coloring and of order d^2 for a local one. The pieces do not commute, so they are recombined by first-order Trotter with r steps, and each step runs all D circuits once. That is $4Dr$ queries in total. The step count comes from (7.7) with the norm-aware Λ : every 1-sparse piece has operator norm at most $\|H\|_{\max}$, the largest $|H_{xy}|$, so $\Lambda \leq D\|H\|_{\max}$ and $r = \mathcal{O}(D^2\|H\|_{\max}^2 t^2/\epsilon)$, where ϵ is the target error in the final unitary. Altogether the simulation makes $\mathcal{O}(D^3\|H\|_{\max}^2 t^2/\epsilon)$ oracle queries, which for a local coloring is $\mathcal{O}(d^6\|H\|_{\max}^2 t^2/\epsilon)$. In plain words: the cost is polynomial in the sparsity and in the evolution time, and it never sees the dimension 2^n except through the n -bit arithmetic inside each query. The qubit count is $2n$ for the two index registers plus a constant-size value register and a few ancillas. The polynomial in d is poor. Qubitization, which Chapter 10 builds, brings the query count down to linear in $d\|H\|_{\max}t$ plus a term logarithmic in $1/\epsilon$ [21]. What I want you to take from this section is the structure, not the exponent.

7.2.4 Implementation guide

Building the sparse simulator

Registers. Two n -qubit registers (x and $y = f(x)$), one comparison ancilla, one more ancilla flagging $x = y$ so that the rotation can be R_z for an eigenstate and R_x for a pair, and a register wide enough to hold H_{xy} to your chosen precision (typically 8–16 qubits for a fixed-point value).

The comparison circuit is a standard ripple-borrow comparator: compute $x - y$ into a scratch register and keep the borrow-out bit. One borrow cell per bit, so n cells and $O(n)$ Toffolis for n -bit registers; the equality flag costs another $O(n)$ Toffolis. Do not forget to uncompute the scratch.

The controlled rotation by a register value. H_{xy} arrives as a bit string, not a classical angle. To implement $e^{-i\theta X}$ with θ held in a register, apply a controlled $R_x(2^{-k} \cdot \text{scale})$ for each bit k : one controlled single-qubit rotation per bit of the value register. This is the binary-decomposition trick used for controlled- U^{2^k} in Chapter 4.

Uncomputing and testing the sparse simulator

Uncomputation. Steps 1–4 must all be reversed. As in Chapter 2, an un-uncomputed ancilla stays entangled with x and destroys the interference the outer algorithm depends on.

Testing. For a small H like (7.10), build the circuit's unitary densely and compare against `expm(-1j*H*t)`. Check both cases separately: prepare $|0\rangle$ (diagonal, should pick up a phase) and $|0\rangle + |3\rangle$ (off-diagonal, should rotate).

7.3 Linear Combination of Unitaries

7.3.1 The problem

Trotter is not the only route. Suppose we want to apply an operator

$$A = \sum_{i=0}^{m-1} c_i U_i, \quad c_i > 0, \quad U_i \text{ unitary}, \quad (7.13)$$

to a state. A sum of unitaries is rarely unitary, so A is generally not unitary and we cannot simply build a circuit for it. The linear combination of unitaries (LCU) technique implements A probabilistically, using an ancilla register to hold the index i in superposition. The technique is due to Childs and Wiebe [8].

A note on the letter. From here to the end of the chapter, a subscripted c_i or c_j is always an expansion coefficient, matching Chapters 8 and 10 and Appendix A. The bare a , b , c of §7.2 were entries of one specific matrix and do not carry over.

Why bother, given Trotter?

The number of Trotter steps, hence the gate count, grows polynomially in $1/\epsilon$: as $1/\epsilon$ for first order and as $1/\sqrt{\epsilon}$ for second. In LCU-based methods the gate count grows only *logarithmically* in $1/\epsilon$, so each extra digit of accuracy costs a fixed additional amount rather than a multiplicative factor. Quantum chemistry demands high accuracy, and there that difference dominates everything else. LCU is also the gateway to qubitization [21] and quantum signal processing [20], the current state of the art. The price is ancillas and post-selection: for near-term work Trotter usually wins, for fault-tolerant resource estimates LCU-family methods do.

7.3.2 The construction

Let $\|c\|_1 = \sum_i c_i$ and let O be the “prepare” unitary, acting on an index register of $\lceil \log_2 m \rceil$ qubits, the smallest number whose $2^{\lceil \log_2 m \rceil}$ basis states can label the m terms, with

$$O|0\rangle = \frac{1}{\sqrt{\|c\|_1}} \sum_i \sqrt{c_i} |i\rangle. \quad (7.14)$$

Let V be the “select” unitary

$$V = \sum_i |i\rangle\langle i| \otimes U_i, \quad (7.15)$$

which applies U_i to the system conditioned on the ancilla holding $|i\rangle$.

1. Start in $|0\rangle |\psi\rangle$.
2. Apply O to the ancilla: $\frac{1}{\sqrt{\|c\|_1}} \sum_i \sqrt{c_i} |i\rangle |\psi\rangle$.
3. Apply V : $\frac{1}{\sqrt{\|c\|_1}} \sum_i \sqrt{c_i} |i\rangle U_i |\psi\rangle$.
4. Apply $O^\dagger$ to the ancilla. Projecting onto $|0\rangle$ picks out $\langle 0|O^\dagger|i\rangle = \sqrt{c_i}/\sqrt{\|c\|_1}$, giving

$$|\Psi_4\rangle = \frac{1}{\|c\|_1} |0\rangle \otimes A|\psi\rangle + (\text{terms orthogonal to } |0\rangle). \quad (7.16)$$

5. Measure the ancilla. Outcome 0 leaves the system in $A|\psi\rangle / \|A|\psi\rangle\|$.

The success probability is

$$p = \frac{\|A|\psi\rangle\|^2}{\|c\|_1^2}. \quad (7.17)$$

It is the squared norm of the $|0\rangle$ component in (7.16). In plain words: the larger the coefficients are relative to what A actually does to $|\psi\rangle$, the more amplitude leaks into the discarded branches. All of this verified numerically for $A = c_0X + c_1Z$ with $c_0 = 0.6$ and $c_1 = 1.4$. The ancilla- $|0\rangle$ block equals $A|\psi\rangle / \|c\|_1$ exactly, and $p = 0.580$ matches (7.17) for every $|\psi\rangle$, because here $A^2 = (c_0^2 + c_1^2)\mathbb{I}$ and so $\|A|\psi\rangle\|^2 = 2.32$ regardless of the state.

Figure 7.3 draws the five steps for general m . The three blocks O , V , $O^\dagger$ are the whole method. I name their composition U_{LCU} now, because the amplitude amplification of the next subsection treats U_{LCU} as a single black box.

Let me count what one application of U_{LCU} costs. The select unitary V applies each of the m unitaries U_i once, controlled on the index register, so it costs m controlled- U_i operations; when the U_i are Pauli strings each of those is a handful of controlled Paulis. The prepare unitary O and its inverse are state preparations on $\lceil \log_2 m \rceil$ qubits, which cost $\mathcal{O}(m)$ controlled single-qubit rotations each, as the implementation guide explains. The qubit overhead is the $\lceil \log_2 m \rceil$ index qubits. In plain words: the select step dominates, and it costs about one controlled application of each term, so adding terms to A costs a linear amount of circuit.

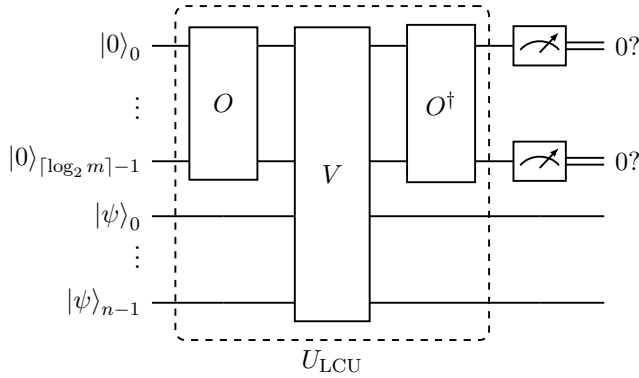


Figure 7.3: The general LCU circuit for $A = \sum_i c_i U_i$. The prepare unitary O loads the coefficient amplitudes $\sqrt{c_i / \|c\|_1}$ into the index register. The select unitary V applies U_i to the system conditioned on that index. Then $O^\dagger$ interferes the branches back together. Measuring every ancilla qubit and accepting only the all-zeros outcome leaves the system in $A|\psi\rangle / \|A|\psi\rangle\|$, with probability given by (7.17). The dashed block is the operator U_{LCU} used in Figure 7.5.

Example 7.2

Construct the circuit for $A = c_0 U_0 + c_1 U_1$ and give O explicitly.

Solution. One ancilla qubit suffices. Take

$$O = \frac{1}{\sqrt{c_0 + c_1}} \begin{pmatrix} \sqrt{c_0} & -\sqrt{c_1} \\ \sqrt{c_1} & \sqrt{c_0} \end{pmatrix}, \quad O^\dagger = \frac{1}{\sqrt{c_0 + c_1}} \begin{pmatrix} \sqrt{c_0} & \sqrt{c_1} \\ -\sqrt{c_1} & \sqrt{c_0} \end{pmatrix}.$$

Both verified: O is unitary, $O|0\rangle$ gives the required amplitudes, and the printed $O^\dagger$ is the true adjoint. Observe that O is simply $R_y(2 \arctan \sqrt{c_1/c_0})$. The select operator factors into U_0 controlled on the ancilla being $|0\rangle$ then U_1 controlled on $|1\rangle$, whose product is verified to equal V . Figure 7.4 draws the result.

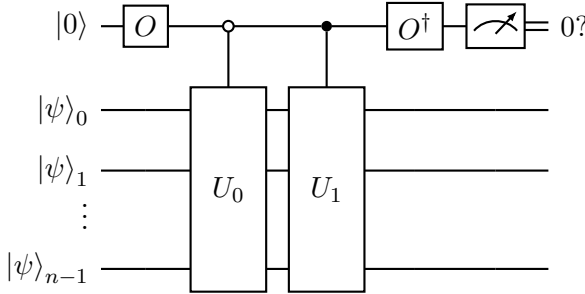


Figure 7.4: LCU circuit for $A = c_0 U_0 + c_1 U_1$. The single ancilla selects which unitary acts; $O^\dagger$ interferes the two branches so that the $|0\rangle$ component carries $A|\psi\rangle$. The system register is drawn with individual wires, and U_0 and U_1 act on all n of them.

Exercise 7.3

Build the LCU circuit for $H = X + Z$ and give O .

Answer: $c_0 = c_1 = 1$, so $O = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} = R_y(\pi/2)$, and the select operator is X controlled on $|0\rangle$ followed by Z controlled on $|1\rangle$. Success probability $p = \|(X + Z)|\psi\rangle\|^2/4$, which is $1/2$ for $|\psi\rangle = |0\rangle$.

7.3.3 Oblivious amplitude amplification

The success probability (7.17) may be small. Amplitude amplification (Chapter 6) boosts it in $O(1/\sqrt{p})$ rounds, where p is the success probability of a single unamplified attempt. The count is assembled exactly as in Chapter 6: with $\sin \theta = \sqrt{p}$, r rounds rotate the success amplitude to $\sin((2r + 1)\theta)$, and driving that to 1 needs $(2r + 1)\theta \approx \pi/2$, that is $r \approx \pi/(4\sqrt{p})$ when p is small. In plain

words, quartering the success probability doubles the number of rounds, where plain repetition would have quadrupled it. Each round costs one U_{LCU} , one $U_{\text{LCU}}^\dagger$ and two cheap reflections, so the price of amplification is about $2r + 1$ applications of the LCU block, each of which is dominated by its m controlled unitaries. It can be done *obliviously*, without knowing $|\psi\rangle$ [8], and I find that a remarkable fact.

Write the whole LCU circuit as U_{LCU} , so that

$$U_{\text{LCU}} |0\rangle |\psi\rangle = \sqrt{p} |\text{good}\rangle + \sqrt{1-p} |\text{bad}\rangle.$$

The hypothesis that makes it oblivious

Obliviousness is not free. It requires the good component to be $|0\rangle \otimes U|\psi\rangle$ for one fixed unitary U . That holds exactly when A is proportional to a unitary, so that $A/\|c\|_1 = \sqrt{p}U$ with p independent of $|\psi\rangle$. Only then do r rounds give $\sin^2((2r+1)\theta)$ with $\sin\theta = \sqrt{p}$. Verified at $p = 1/5$, away from the special value $1/4$: one round gives exactly $0.968000 = \sin^2(3\theta)$.

When A is far from proportional to a unitary the rotation law fails. For $A = \mathbb{I} + X$, where $A/2$ is a projector, and $|\psi\rangle = (0.6, 0.8)$, one round gives 0.980000 while $\sin^2(3\theta) = 0.829472$. Truncated Taylor series of e^{-iHt} meet the hypothesis up to the truncation error, which is why the method applies to them.

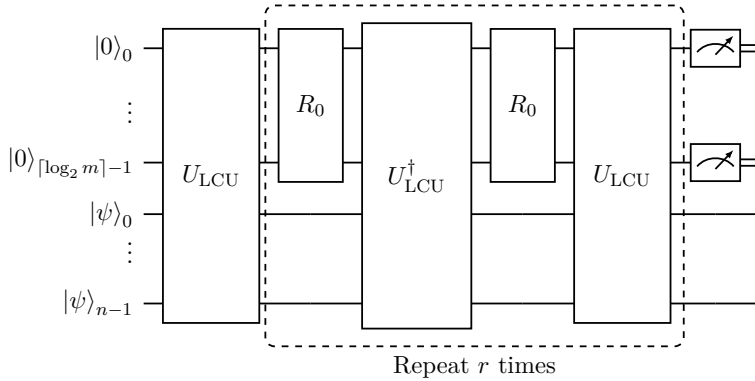


Figure 7.5: Oblivious amplitude amplification. The reflection $R_0 = (\mathbb{I} - 2|0\rangle\langle 0|) \otimes \mathbb{I}$ acts only on the ancilla register, marking the good subspace; the sandwich $U_{\text{LCU}} R_0 U_{\text{LCU}}^\dagger$ is the reflection about $U_{\text{LCU}} |0\rangle |\psi\rangle$. Repeating $r = O(1/\sqrt{p})$ times drives the success probability to near 1, provided A is proportional to a unitary.

The two reflections are not the same operator

It is tempting to write $U_f = U_{f_0} = (\mathbb{I} - 2|0\rangle\langle 0|) \otimes \mathbb{I}$ for both reflections. They are different objects, as in Chapter 6:

- $R_0 = (\mathbb{I} - 2|0\rangle\langle 0|) \otimes \mathbb{I}$ marks the good subspace, acting on the *ancilla* register only.
- $U_{\text{LCU}} R_0 U_{\text{LCU}}^\dagger$ is the reflection about the prepared state, acting on *everything*.

The Grover operator is their product, exactly as in §6.2. Writing both as one expression hides which register each acts on, the one thing a reader needs in order to code it.

7.3.4 Hamiltonian simulation via LCU

To simulate e^{-iHt} with $H = \sum_j c_j H_j$ and each H_j unitary (true for Pauli strings), truncate the Taylor series:

$$e^{-iHt} = \sum_{k=0}^{\infty} \frac{(-it)^k}{k!} \left(\sum_j c_j H_j \right)^k \approx \sum_{k=0}^K \frac{t^k}{k!} \sum_{j_1 \dots j_k} c_{j_1} \dots c_{j_k} (-i)^k H_{j_1} \dots H_{j_k}. \quad (7.18)$$

Every term is a product of unitaries, hence unitary, and every coefficient is positive. So the truncated series is itself an LCU and can be implemented by the construction above. Truncating at $K = O\left(\frac{\log(1/\epsilon)}{\log \log(1/\epsilon)}\right)$ gives error ϵ . That is the logarithmic dependence promised earlier [8].

Here is where that K comes from. Write $\|c\|_1 = \sum_j c_j$ for the sum of the coefficients and ϵ for the target error in the operator norm. The terms dropped by the truncation are bounded by the tail of the exponential series, $\sum_{k>K} (t\|c\|_1)^k / k!$, and once $t\|c\|_1$ is held at order one, as the box below insists, that tail is about $1/(K+1)!$. Requiring $1/(K+1)! \leq \epsilon$ and using $\log K! \approx K \log K$ gives $K \log K \approx \log(1/\epsilon)$, whose solution is the expression above. In plain words: because factorials grow so fast, each extra digit of accuracy costs less than one extra term of the series. The resource this buys is circuit size. Each order k of the series is a product of k selected H_j , so the select step for the whole truncated series applies at most K controlled Pauli strings per branch, each a handful of controlled Paulis, and the total number of controlled operations per segment is $\mathcal{O}(Km)$, with m the number of terms in H . Compare Trotter, where the number of exponentials grows like $1/\epsilon$ or $1/\sqrt{\epsilon}$.

Segment the evolution, or the post-selection fails

The truncated series has one-norm $\|c'\|_1 \approx e^{t\|c\|_1}$, where $\|c\|_1$ is the sum of the coefficients of H , so (7.17) gives $p \approx e^{-2t\|c\|_1}$: the success probability falls exponentially in the evolution time. Measured values match to four digits, $p = 1.353 \times 10^{-1}$ at $t\|c\|_1 = 1$, and by $t\|c\|_1 = 10$ the value is about 2×10^{-9} , which is unusable. So apply the construction only to short segments with $t\|c\|_1 = O(1)$: for total time T , split it into $T\|c\|_1/\ln 2$ segments and simulate each in turn. The standard choice $t\|c\|_1 = \ln 2$ gives $\|c'\|_1 = 2$ and $p = 1/4$, the value at which one round of oblivious amplitude amplification succeeds exactly.

The total cost now assembles as segments times cost per segment. For a total evolution time T there are $T\|c\|_1/\ln 2$ segments, where $\|c\|_1$ is the sum of the coefficients of H . Each segment is one truncated-series LCU followed by one round of oblivious amplitude amplification, so three applications of the LCU block, each costing $\mathcal{O}(Km)$ controlled Pauli operations with m the number of terms and K the truncation order. Since the errors of the segments add, each segment must be accurate to $\epsilon \ln 2/(T\|c\|_1)$ for a final error ϵ , which only shifts K by a logarithm. Altogether the simulation uses $\mathcal{O}\left(T\|c\|_1 m \frac{\log(T\|c\|_1/\epsilon)}{\log \log(T\|c\|_1/\epsilon)}\right)$ controlled Pauli operations. In plain words: the cost is linear in the evolution time and in the number of terms, and grows only logarithmically as the accuracy is tightened, which is the whole point of the method.

7.3.5 Implementation guide

Building an LCU

Ancillas. $\lceil \log_2 m \rceil$ for the index register. For the Taylor-series version the index must label a product of up to K of the U_j , so you need K copies of that register, $K \lceil \log_2 m \rceil$ qubits, plus $\lceil \log_2(K+1) \rceil$ more to hold the order k .

Preparing O . For general c_i this is a state preparation problem: build the state $\sum_i \sqrt{c_i/\|c\|_1} |i\rangle$. For m up to a few dozen, a cascade of controlled R_y rotations is fine. Beyond that use the standard binary-tree preparation: one controlled R_y per internal node of a binary tree with m leaves, so $m-1$ controlled single-qubit rotations, which is $O(m)$ gates.

The select operator is the expensive part: m controlled unitaries. For Pauli strings each is a controlled Pauli, which is cheap. Unary encoding of the index register makes the controls single-qubit rather than multi-controlled.

Coefficients must be positive. If some c_i is negative or complex, absorb the phase into U_i : write $c_i = |c_i|e^{i\phi_i}$ and use $U'_i = e^{i\phi_i}U_i$, which is still unitary.

Testing. Verify each piece separately: O unitary and producing the right amplitudes; V equal to $\sum_i |i\rangle\langle i| \otimes U_i$; then the full circuit's ancilla- $|0\rangle$ block equal to $A|\psi\rangle/\|c\|_1$. Compare p against (7.17).

7.4 Simulating Pauli Strings

This section is the one Chapters 8 and 9 depend on.

7.4.1 Every Hamiltonian is a sum of Pauli strings

Any Hermitian operator on n qubits expands uniquely as

$$H = \sum_j c_j P_j, \quad P_j = \sigma_1^{(j)} \otimes \cdots \otimes \sigma_n^{(j)}, \quad \sigma \in \{\mathbb{I}, X, Y, Z\}, \quad (7.19)$$

with real coefficients

$$c_j = \frac{1}{2^n} \text{Tr}[H P_j]. \quad (7.20)$$

The 4^n Pauli strings form an orthogonal basis for Hermitian matrices under the trace inner product, which is why the expansion exists and is unique.

Since each P_j is both Hermitian and unitary, Trotterizing H reduces to implementing $e^{-i\theta P}$ for a single Pauli string. That is what we now do, and we do it exactly.

7.4.2 The building block: $e^{-i\theta Z}$

For one qubit,

$$e^{-i\theta Z} = R_z(2\theta). \quad (7.21)$$

Verified. Note the factor of two: $R_z(\phi) = \text{diag}(e^{-i\phi/2}, e^{i\phi/2})$, so the rotation angle is twice the exponent coefficient. In my experience this factor of two is the most common source of wrong answers in Chapters 8 and 9.

7.4.3 $e^{-i\theta Z \otimes Z}$: the parity trick

Expanding, and using $(Z \otimes Z)^2 = \mathbb{I}$,

$$e^{-i\theta Z \otimes Z} = \cos \theta \mathbb{I} - i \sin \theta Z \otimes Z,$$

so on a basis state

$$e^{-i\theta Z \otimes Z} |q_0 q_1\rangle = e^{-i\theta(-1)^{q_0 \oplus q_1}} |q_0 q_1\rangle. \quad (7.22)$$

I want you to notice that the phase depends only on the *parity* $q_0 \oplus q_1$:

Table 7.3: Action of $e^{-i\theta Z \otimes Z}$. Verified numerically.

input	output	input	output
$ 00\rangle$	$e^{-i\theta} 00\rangle$	$ 10\rangle$	$e^{+i\theta} 10\rangle$
$ 01\rangle$	$e^{+i\theta} 01\rangle$	$ 11\rangle$	$e^{-i\theta} 11\rangle$

A CNOT computes the parity into the second qubit; an R_z applies the phase; a second CNOT undoes the parity computation.

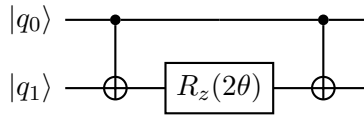


Figure 7.6: $e^{-i\theta Z \otimes Z}$. Verified numerically to equal the exact matrix exponential.

7.4.4 The general recipe

Circuit for $e^{-i\theta P}$, any Pauli string P

1. **Rotate into the Z basis.** Apply H on each qubit where P has X , and $HS^\dagger$ where it has Y (or $R_x(\pi/2)$, a different gate that is an equally valid Y basis change). Qubits carrying Z or $\mathbb{I}$ are left alone.
2. **Compute the parity** of all qubits on which P acts non-trivially, using a ladder of CNOTs into the last such qubit.
3. **Apply $R_z(2\theta)$** to that qubit.
4. **Uncompute the parity** (reverse the CNOT ladder).
5. **Rotate back** (apply the inverse of step 1).

Qubits where P acts as $\mathbb{I}$ are skipped entirely, taking no gates at all.

The basis changes work because

$$X = HZH, \quad Y = (HS^\dagger)^\dagger Z (HS^\dagger), \quad S^\dagger = \text{diag}(1, -i). \quad (7.23)$$

Both verified numerically.

Two conventions to pin down

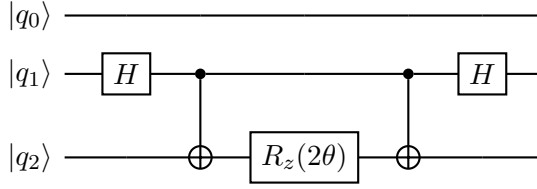
The Y basis change. Both $R_z(\pi/2)XR_z(-\pi/2)$ and $(HS^\dagger)^\dagger Z (HS^\dagger)$ equal Y , and both are verified. Only the second puts Y in the form $B^\dagger Z B$ that the recipe needs; the first expresses Y through X and then needs a further Hadamard gate. Implement $HS^\dagger$ directly. And $R_z(-\pi/2)XR_z(\pi/2)$ gives $-Y$, not Y : the sign of the rotation matters.

The factor of two. $R_z(2\theta)$, not $R_z(\theta)$. For a Hamiltonian term $c_j P_j$ evolved for time t , the angle is $2c_j t$.

Example 7.3

Simulate $H = \mathbb{I} \otimes X \otimes Z$.

Solution. Qubit 0 is skipped. Qubit 1 carries X , so conjugate it with H ; qubit 2 carries Z , so leave it. Then compute the parity of qubits 1 and 2 with one CNOT, rotate, and undo. Verified numerically: $\mathbb{I} \otimes X \otimes Z = (\mathbb{I} \otimes H \otimes \mathbb{I})(\mathbb{I} \otimes Z \otimes Z)(\mathbb{I} \otimes H \otimes \mathbb{I})$, and the circuit below reproduces $e^{-i\theta \mathbb{I} \otimes X \otimes Z}$ exactly.


Example 7.4

Simulate $H = X \otimes Y \otimes Z$ using $\{H, S, R_z, \text{CNOT}\}$.

Solution. Conjugate qubit 0 with H (for X), qubit 1 with $HS^\dagger$ (for Y), and leave qubit 2. Then a CNOT ladder $q_0 \rightarrow q_2, q_1 \rightarrow q_2$ collects the parity onto q_2 , where $R_z(2\theta)$ acts. Verified numerically: with $B = H \otimes HS^\dagger \otimes \mathbb{I}$,

$$B^\dagger (Z \otimes Z \otimes Z) B = X \otimes Y \otimes Z, \quad e^{-i\theta X \otimes Y \otimes Z} = B^\dagger e^{-i\theta Z \otimes Z \otimes Z} B,$$

and the CNOT-ladder circuit implements $e^{-i\theta Z \otimes Z \otimes Z}$ exactly. Figure 7.7 is the resulting circuit.

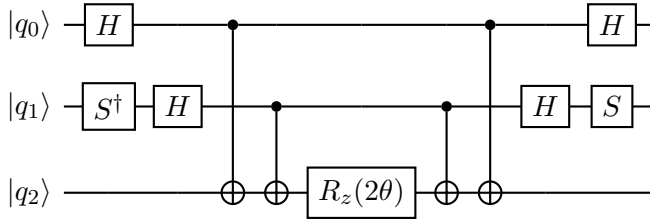


Figure 7.7: $e^{-i\theta X \otimes Y \otimes Z}$. The two CNOTs accumulate the parity of all three qubits onto q_2 , where the single $R_z(2\theta)$ acts; the mirror-image CNOTs and basis changes then undo everything. Verified numerically against $\exp(-i\theta X \otimes Y \otimes Z)$.

Figure 7.8 draws the recipe once and for all, for a Pauli string that acts non-trivially on every qubit. **This circuit is the workhorse of Chapters 8 and 9.**

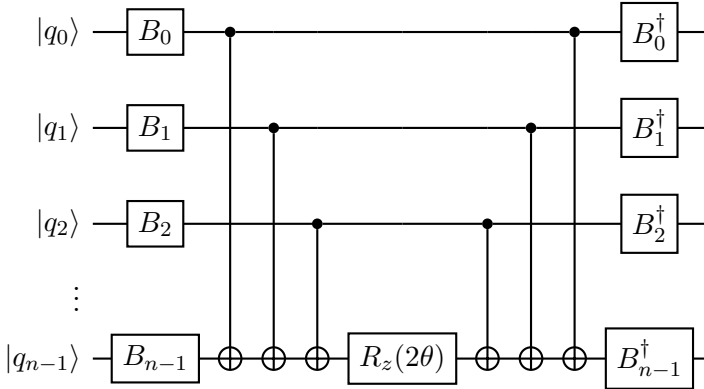


Figure 7.8: The general Pauli-exponential circuit $e^{-i\theta P}$ on n qubits. B_k is H if P has X on qubit k , $HS^\dagger$ if Y , and identity if Z or $\mathbb{I}$. The CNOT ladder accumulates the parity of all non-identity qubits onto the last one, where a single $R_z(2\theta)$ applies the phase; the second ladder undoes it. It is the workhorse of the two chapters that follow.

Cost

A Pauli string with weight w (the number of non-identity factors) costs $2(w-1)$ CNOTs, one R_z , and at most $4w$ single-qubit gates. The CNOTs are the ladder of $w-1$ into the last active qubit, run once and then undone. The exact single-qubit count is $2 \times (\text{number of } X \text{ factors}) + 4 \times (\text{number of } Y \text{ factors})$, since a Y position costs $S^\dagger$ and H going in and H and S coming out; the worst case $4w$ is attained by an all- Y string. The cost depends on the *weight*, not on n : a weight-two term in a thousand-qubit system still costs two CNOTs. That is why local Hamiltonians are cheap to simulate and why fermionic encodings that keep Pauli weight low (Chapter 8) matter so much.

7.4.5 Putting it together: a sum of Pauli strings

Example 7.5

Simulate $H = X \otimes \mathbb{I} + Z \otimes Z$ for time t .

Solution. The two terms do not commute (verified), so Trotterize:

$$e^{-iHt} \approx \left(e^{-i(X \otimes \mathbb{I})\Delta t} e^{-i(Z \otimes Z)\Delta t} \right)^r, \quad \Delta t = t/r.$$

Each factor is a Pauli exponential from §7.4: the first is a single H - $R_z(2\Delta t)$ - H on qubit 0, the second the CNOT sandwich of Fig. 7.6. Figure 7.9 assembles the two into one Trotter step. This is the Hamiltonian measured in Table 7.1, whose clean $1/r$ error scaling is what §7.1 predicts.

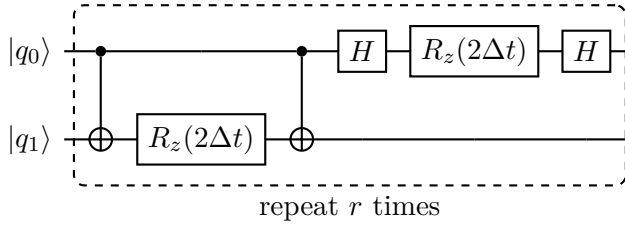


Figure 7.9: One first-order Trotter step for $H = X \otimes \mathbb{I} + Z \otimes Z$. The CNOT sandwich on the left is $e^{-i(Z \otimes Z)\Delta t}$ and the H - R_z - H block on the right is $e^{-i(X \otimes \mathbb{I})\Delta t}$. They appear in that order because a circuit is read left to right while the operator product is read right to left. Repeating the dashed block r times gives the whole simulation, with $\Delta t = t/r$. Nothing cancels at the joint between consecutive steps here, because a Hadamard on q_0 separates the two CNOT sandwiches.

7.4.6 Implementation guide

Building a Pauli-sum simulator: the core routine for Chapters 8 and 9

Write one function and reuse it everywhere:

```
    pauli_evolution(circuit, pauli_string, angle, qubits)
```

Step by step.

1. Collect the indices where the string is not $\mathbb{I}$. If empty, the term is a global phase, so skip it (but record it if you need energies).
2. Apply H on X positions, and the phase gate $S^\dagger$ then H on Y positions.
3. CNOT from each active qubit into the last active qubit, in order.
4. $R_z(2\theta)$ on the last active qubit.
5. Reverse steps 3 and 2.

The angle. For a term $c_j P_j$ evolved for Δt , pass $\theta = c_j \Delta t$, so the gate is $R_z(2c_j \Delta t)$. Half the bugs in this area are a missing or spurious factor of two.

Ordering the terms. Group commuting terms together; within a group the order is irrelevant and no Trotter error is incurred. Sorting by qubit support also lets consecutive CNOT ladders cancel, often halving the two-qubit gate count.

Testing a Pauli-sum simulator, and where it reappears

Do all four tests.

1. Single Pauli, $n = 1$: compare against `expm`.
2. $Z \otimes Z$: check the four phases of Table 7.3.
3. A weight-3 string with a Y in it: this catches S versus $S^\dagger$ errors, which the X/Z -only cases cannot.
4. A non-commuting sum: plot error against Δt and confirm the slope.

Looking ahead. In Chapter 8 the Pauli strings come from a fermionic encoding and can have weight $O(n)$, making the CNOT ladders the dominant cost, $O(n)$ CNOTs per term. In Chapter 9 the angle θ is set by an adiabatic schedule that changes from one layer to the next rather than by a single fixed $c_j \Delta t$, and the circuit is identical. That is the QAOA layer of Chapter 9. The same circuit with freely optimized angles is the variational ansatz of the near-term literature, which this book sets aside.

7.5 Summary

- $|\psi(t)\rangle = e^{-iHt} |\psi(0)\rangle$; the challenge is that e^{-iHt} is a $2^n \times 2^n$ matrix.
- Physical Hamiltonians are local, $H = \sum_k H_k$, and each $e^{-iH_k t}$ is a few-qubit gate.
- If the H_k commute, the product formula is *exact*. The Trotter error exists because they do not, and is proportional to the commutator.
- First-order Trotter: per-step error $\mathcal{O}(\Delta t^2)$, total error $\mathcal{O}(t\Delta t)$, requiring $r = \mathcal{O}(L^2 t^2 / \epsilon)$ steps for L terms, or $r \approx \Lambda^2 t^2 / \epsilon$ with $\Lambda = \sum_k \|H_k\|$; the circuit applies rL exponentials of the individual terms.
- Symmetric Trotter $e^{iAt/2} e^{iBt} e^{iAt/2}$ is $\mathcal{O}(t^3)$ per step and costs almost nothing extra.
- Trotter error is polynomial in $1/\epsilon$. Chapter 10 achieves $\mathcal{O}(\log(1/\epsilon))$ by qubitization, at the cost of ancillas and classical pre-processing.
- A d -sparse row-computable Hamiltonian splits into 1-sparse pieces, at most $d + 1$ of them for a global edge coloring, and each piece is simulated with four oracle queries; the controlled-SWAP sorts each pair into canonical order so the dynamics lands on a single ancilla.
- The comparison circuit must define the $x = y$ case, since the diagonal terms use it.
- LCU implements $A = \sum_i c_i U_i$ probabilistically. The success probability is $p = \|A|\psi\rangle\|^2 / \|c\|_1^2$, boosted by oblivious amplitude amplification. Its error scales logarithmically in $1/\epsilon$, unlike Trotter's polynomial scaling.
- Any Hermitian H is a sum of Pauli strings with $c_j = 2^{-n} \text{Tr}[HP_j]$.
- $e^{-i\theta P}$: **basis-change, CNOT ladder, $R_z(2\theta)$, undo**. Cost $2(w-1)$ CNOTs for weight w , one R_z , and $2 \times (\text{number of } X) + 4 \times (\text{number of } Y)$ single-qubit gates, at most $4w$. Independent of n .

7.6 End-of-Chapter Problems

- 7.1** Show that if $[A, B] = 0$ then $e^{iAt} e^{iBt} = e^{i(A+B)t}$ exactly, and identify the leading error term when they do not commute.
- 7.2** Prove the symmetric Trotter formula (7.8) to $\mathcal{O}(t^3)$.
- 7.3** For $H = X + Z$ and $t = 1$, compute the first-order and symmetric Trotter errors at $r = 1, 2, 5, 10$ and confirm the predicted scalings.

- 7.4** Estimate r for simulating a 10-qubit nearest-neighbor Ising model to accuracy 10^{-3} for $t = 10$, using both first-order and symmetric formulas.
- 7.5** Show that a d -sparse Hermitian matrix decomposes into at most $d + 1$ 1-sparse Hermitian matrices. Why must each piece be Hermitian?
- 7.6** For the Hamiltonian (7.10), verify that e^{-iHt} is block-diagonal in the reordered basis and compute both blocks explicitly for $t = 1$.
- 7.7** Show that $\text{diag}(e^{-ibt}, e^{-ict}) = e^{-i(b+c)t/2} R_z((b-c)t)$.
- 7.8** Design a comparison circuit for two 3-bit registers satisfying (7.12), and count its Toffoli gates.
- 7.9** Verify that O in Example 7.2 is unitary, and express it as $R_y(\phi)$ for an explicit ϕ .
- 7.10** For $A = X + Z$ and $|\psi\rangle = |0\rangle$, compute the LCU success probability. Then use $(X + Z)^2 = 2\mathbb{I}$ to show that every $|\psi\rangle$ gives the same value.
- 7.11** Show that if some c_i in (7.13) is complex, the phase can be absorbed into U_i without loss of generality.
- 7.12** Expand $H = \begin{pmatrix} 1 & 2 \\ 2 & -1 \end{pmatrix}$ in the Pauli basis using (7.20).
- 7.13** Expand the two-qubit Heisenberg Hamiltonian $H = X \otimes X + Y \otimes Y + Z \otimes Z$ and draw the circuit for e^{-iHt} using symmetric Trotter. Do the three terms commute?
- 7.14** Draw the circuit for $e^{-i\theta Y \otimes Z \otimes \mathbb{I} \otimes X}$ on four qubits. How many CNOTs does it use?
- 7.15** Show that a Pauli string of weight w needs $2(w - 1)$ CNOTs, and explain why two consecutive strings with overlapping support can often share ladders.
- 7.16** Verify numerically that $Y = (HS^\dagger)^\dagger Z (HS^\dagger)$ but $Y \neq R_z(-\pi/2) X R_z(\pi/2)$. What is the latter equal to?
- 7.17** (*Looking ahead to Chapter 9.*) Replace the fixed angle $2c_j\Delta t$ in Fig. 7.8 by a free parameter ϑ_j . Explain why the resulting circuit is the natural parameterized form that the variational literature builds on, which this book sets aside, and how many parameters a Hamiltonian with L Pauli terms gives.

Research Problems and Projects

The problems above have answers. The three below do not, at least not ones you can look up. Each is sized for several weeks of work by a strong MS student or a beginning PhD student, and each ends in something you could write up: a figure with error bars, a resource table, or a piece of reusable software. Read the cited papers before you start, and be prepared to discover that your first measurement disagrees with the bound you expected.

R7.1 How pessimistic is the Trotter bound, and why? The worst-case estimate $r \approx \Lambda^2 t^2 / \epsilon$ used in §7.1 depends only on $\Lambda = \sum_k \|H_k\|$. It ignores the commutator structure entirely. Build a numerical study that measures the *actual* number of Trotter steps needed to reach a fixed ϵ , and compare it against that bound. Suggested family: the one-dimensional transverse-field Ising and Heisenberg chains at $n = 4, 6, 8, 10, 12$ qubits, with t ranging over at least two decades, using both the first-order and the symmetric formula. Vary the term ordering as well: interleaved, grouped by Pauli type, and grouped by qubit support. *Deliverable*: a scaling figure of measured r against n and against t , with fitted exponents, plus a table of the ratio (measured r)/(bound r). Then attempt the harder part: find an empirical predictor built from $\sum_{j < k} \|[H_j, H_k]\|$ that tracks the measured r better than Λ does, and report where your predictor fails. Commutator-based Trotter bounds are an active research area, so a careful negative result here is publishable as a benchmark.

R7.2 Trace the crossover from Trotter to LCU in the published record. Lloyd's product-formula construction [19], the sparse-oracle simulation of Aharonov and Ta-Shma [2], and the linear combination of unitaries of Childs and Wiebe [8] give three different answers to the same question. Reconstruct, from the papers themselves, exactly what each one assumes about the input Hamiltonian and what each one proves about query and gate complexity. Then follow the thread forward to qubitization [21] and quantum signal processing [20]. *Deliverable*: a single comparison table with one row per method. Each row should give the input model, the asymptotic query complexity in n , t and $1/\epsilon$, and the ancilla count. Record for each claim the precise place in the paper where it is proved. The interesting part is not the table but the footnotes: identify at least two places where the methods are compared in the literature using incompatible input assumptions, and say what a fair comparison would require. Be careful to distinguish what is proved from what is conjectured or merely observed in small numerical experiments.

R7.3 A Pauli-exponential compiler, and how much the ordering buys. The recipe of §7.4 compiles one Pauli string into $2(w - 1)$ CNOTs. A Hamiltonian with L terms naively costs L independent ladders, but consecutive strings

with overlapping support can share ladder segments. Write a compiler that takes a list of (c_j, P_j) pairs and emits a circuit. Then implement at least two term-ordering heuristics: sorting by support, and a greedy nearest-neighbor ordering under Hamming distance between the strings' support masks. Add a peephole pass that cancels adjacent identical CNOTs. Benchmark against the naive ordering on molecular Hamiltonians of increasing size, whose Pauli decompositions you can take from the electronic structure literature [23]. *Deliverable:* working code, plus a resource table of CNOT count and circuit depth for each molecule under each ordering, with the percentage saved. State clearly which savings come from the ordering and which from the peephole pass, since the two are easy to confound. Optimal term ordering for this problem is not known to be tractable, so aim to characterize your heuristics rather than to claim optimality.

Chapter 8

Quantum Chemistry on a Quantum Computer

Simulating molecules is the application most often cited as quantum computing’s first genuine advantage. It is also the natural destination for everything we have built so far. Chapter 7 showed how to implement $e^{-i\theta P}$ for a Pauli string P . This chapter shows where those Pauli strings come from. For a survey of the wider field, including the variational and error-mitigation methods I have left out, see the review [23].

The route is a chain of three translations:

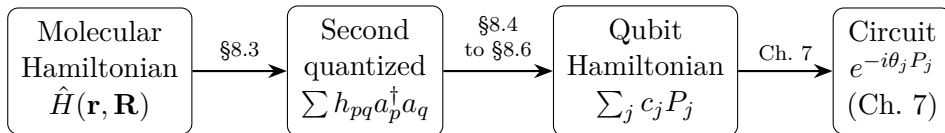


Figure 8.1: The pipeline of this chapter. The middle arrow is the *fermion-to-qubit encoding*. That is the step specific to chemistry, and the step where the interesting design choices live.

What you should be able to do after this chapter

- Write down the molecular Hamiltonian and apply the Born–Oppenheimer approximation.
- Explain why fermionic antisymmetry forces a nontrivial encoding.
- Convert a second-quantized Hamiltonian into a sum of Pauli strings by Jordan–Wigner, parity, or Bravyi–Kitaev.
- Compute the $U(j)$, $P(j)$ and $F(j)$ sets from the Bravyi–Kitaev matrix.
- Prepare a Hartree–Fock reference state on a quantum register.
- Assemble the whole pipeline into runnable code.

8.1 The Molecular Hamiltonian

8.1.1 Many-particle Schrödinger equation

The state of N particles is a wavefunction $\Psi(\mathbf{r}_1, \dots, \mathbf{r}_N, t)$ obeying

$$i\hbar \frac{\partial}{\partial t} \Psi = \hat{H} \Psi, \quad \hat{H} = - \sum_{i=1}^N \frac{\hbar^2}{2m_i} \nabla_i^2 + \hat{V}(\mathbf{r}_1, \dots, \mathbf{r}_N). \quad (8.1)$$

For a molecule with N electrons and N_{nuc} nuclei,

$$\hat{H} = \underbrace{- \sum_i \frac{\hbar^2}{2m_e} \nabla_i^2}_{\text{electron KE}} - \underbrace{\sum_A \frac{\hbar^2}{2M_A} \nabla_A^2}_{\text{nuclear KE}} - \underbrace{\sum_{i,A} \frac{\zeta_A e^2}{4\pi\epsilon_0 r_{iA}}}_{\text{e-n attraction}} + \underbrace{\sum_{j>i} \frac{e^2}{4\pi\epsilon_0 r_{ij}}}_{\text{e-e repulsion}} + \underbrace{\sum_{B>A} \frac{\zeta_A \zeta_B e^2}{4\pi\epsilon_0 R_{AB}}}_{\text{n-n repulsion}}. \quad (8.2)$$

Here m_e is the electron mass, while M_A and ζ_A are the mass and the charge number of nucleus A . The distances are r_{iA} from electron i to nucleus A , r_{ij} between two electrons, and R_{AB} between two nuclei.

Two symbols to fix before going further

The nuclear charge is written ζ_A , not Z_A . From §8.4 onward Z is reserved for the Pauli operator, and Z_i will always mean Pauli Z on qubit i . Chemistry papers write Z_A here, but they do not also carry a qubit Pauli algebra; this chapter does.

Likewise the nuclear mass M_A always carries a nucleus label. The bare M , from §8.3 onward, is the number of spin-orbitals.

8.1.2 Born–Oppenheimer approximation

Nuclei are thousands of times heavier than electrons, so they move far more slowly. Freezing them turns the nuclear kinetic term to zero and the nuclear repulsion into a constant. What is left is the *electronic* Hamiltonian

$$\hat{H}_e = -\sum_i \frac{\nabla_i^2}{2} - \sum_{i,A} \frac{\zeta_A}{r_{iA}} + \frac{1}{2} \sum_{i \neq j} \frac{1}{r_{ij}}, \quad (8.3)$$

written in atomic (Hartree) units, where $\hbar = m_e = e = 4\pi\epsilon_0 = 1$.

What Born–Oppenheimer buys and costs

It converts one impossible problem into a family of hard ones: solve (8.3) at each fixed nuclear geometry $\mathbf{R}$, then treat $E(\mathbf{R})$ as a potential energy surface for the nuclei. Every point on a reaction-energy curve is a separate quantum computation.

The approximation fails near conical intersections and for light nuclei (hydrogen tunneling). Keep that in mind before quoting chemical accuracy.

8.2 Identical Particles and Antisymmetry

8.2.1 Exchange symmetry

Define the exchange operator $\hat{P}_{ij}$ that swaps particles i and j . Since swapping twice returns the original state, $\hat{P}_{ij}^2 = \hat{I}$, so its eigenvalues satisfy $\lambda^2 = 1$, giving $\lambda = \pm 1$:

$$\hat{P}_{ij}\Psi = \pm\Psi. \quad (8.4)$$

The $+$ case describes **bosons** (integer spin); the $-$ case describes **fermions** (half-odd-integer spin). Electrons are fermions, so

$$\Psi(\dots, \mathbf{x}_i, \dots, \mathbf{x}_j, \dots) = -\Psi(\dots, \mathbf{x}_j, \dots, \mathbf{x}_i, \dots).$$

8.2.2 Slater determinants and the Pauli principle

For two fermions in orbitals ψ_{k_1}, ψ_{k_2} ,

$$\Psi_f(\mathbf{x}_1, \mathbf{x}_2) = \frac{1}{\sqrt{2}} [\psi_{k_1}(\mathbf{x}_1)\psi_{k_2}(\mathbf{x}_2) - \psi_{k_2}(\mathbf{x}_1)\psi_{k_1}(\mathbf{x}_2)] = \frac{1}{\sqrt{2}!} \begin{vmatrix} \psi_{k_1}(\mathbf{x}_1) & \psi_{k_1}(\mathbf{x}_2) \\ \psi_{k_2}(\mathbf{x}_1) & \psi_{k_2}(\mathbf{x}_2) \end{vmatrix}. \quad (8.5)$$

Putting both fermions in the same orbital, $k_1 = k_2$, makes the determinant vanish identically, because two rows are then equal. That is the **Pauli exclusion principle**. I want you to see it as a consequence of antisymmetry rather than an extra postulate.

For N fermions the generalization is the *Slater determinant*

$$\Psi_f(\mathbf{x}_1, \dots, \mathbf{x}_N) = \frac{1}{\sqrt{N!}} \begin{vmatrix} \psi_{k_1}(\mathbf{x}_1) & \cdots & \psi_{k_1}(\mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ \psi_{k_N}(\mathbf{x}_1) & \cdots & \psi_{k_N}(\mathbf{x}_N) \end{vmatrix}. \quad (8.6)$$

Why this forces a nontrivial encoding

An N -electron state built from M spin-orbitals is a combination of $\binom{M}{N}$ Slater determinants, one for each way of choosing the N occupied spin-orbitals. Each determinant carries a sign that depends on the *ordering* of the spin-orbitals. Qubits, by contrast, are distinguishable and commute: $|01\rangle$ and $|10\rangle$ are simply different states, with no relative sign.

So we cannot just assign one qubit per electron. The whole content of §8.4–8.6 is a bookkeeping device that makes commuting qubit operators reproduce anticommuting fermionic ones. Everything else in this chapter is standard quantum chemistry.

8.3 Second Quantization

8.3.1 Fock space

Tracking which electron is in which orbital is meaningless for indistinguishable particles. We therefore track only *how many* electrons occupy each orbital. With M spin-orbitals, a basis state is

$$|k_0 k_1 \cdots k_{M-1}\rangle, \quad k_i \in \{0, 1\}, \quad (8.7)$$

with $k_i = 1$ meaning orbital i is occupied. Pauli exclusion is now automatic: k_i cannot exceed 1. Throughout this chapter M counts *spin*-orbitals, never spatial orbitals. Each spatial orbital contributes two of them, so a minimal-basis H_2 has $M = 4$, not $M = 2$.

This is already a qubit register, one qubit per spin-orbital. What remains is to map the operators.

8.3.2 Creation and annihilation operators

On a single spin-orbital, the creation and annihilation operators act by

$$a^\dagger |0\rangle = |1\rangle, \quad a^\dagger |1\rangle = 0, \quad a |1\rangle = |0\rangle, \quad a |0\rangle = 0. \quad (8.8)$$

The two zeros have different origins: $a |0\rangle = 0$ because there is nothing to remove, and $a^\dagger |1\rangle = 0$ because Pauli exclusion forbids a second electron in the same spin-orbital. From here on a subscript names the spin-orbital acted on, so $a_i^\dagger$ creates

an electron in spin-orbital i . I use the letter a for nothing else in this chapter; expansion coefficients are written c_j .

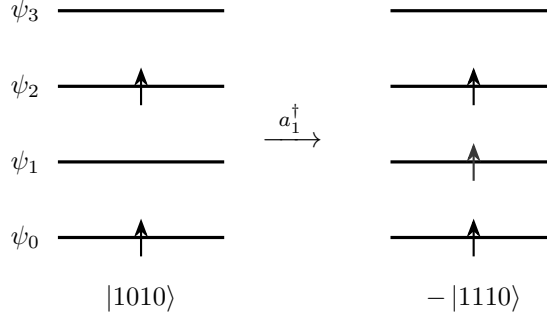


Figure 8.2: Occupation-number picture. Creating an electron in orbital 1 of $|1010\rangle$ gives $|1110\rangle$ with a *minus sign*, because one occupied orbital ($k_0 = 1$) lies to its left. Tracking that sign is the entire difficulty of fermionic encoding.

8.3.3 Anticommutation and the ordering sign

Because $|k_i, k_j\rangle = a_i^\dagger a_j^\dagger |0\rangle$ must be antisymmetric under exchange,

$$\{a_i^\dagger, a_j^\dagger\} = 0, \quad \{a_i, a_j\} = 0, \quad \{a_i, a_j^\dagger\} = \delta_{ij}. \quad (8.9)$$

Acting on a general occupation state,

$$\begin{aligned} a_i^\dagger |\dots k_i \dots\rangle &= (-1)^{\sum_{j<i} k_j} (1 - k_i) |\dots k_i + 1 \dots\rangle, \\ a_i |\dots k_i \dots\rangle &= (-1)^{\sum_{j<i} k_j} k_i |\dots k_i - 1 \dots\rangle. \end{aligned} \quad (8.10)$$

The factor $(1 - k_i)$ or k_i enforces Pauli exclusion; the sign $(-1)^{\sum_{j<i} k_j}$ counts how many anticommutations are needed to move the operator into position i .

Example 8.1

Apply $a_1^\dagger$ to $|1011\rangle$.

Solution. Orbital 1 is empty, so the state is created, with sign $(-1)^{k_0} = (-1)^1 = -1$: thus $a_1^\dagger |1011\rangle = -|1111\rangle$, verified numerically. By contrast $a_2^\dagger |1011\rangle = 0$, since orbital 2 is already occupied.

8.3.4 The second-quantized Hamiltonian

One- and two-body operators become

$$\hat{T} = \sum_{pq} t_{pq} a_p^\dagger a_q, \quad \hat{F} = \frac{1}{2} \sum_{pqrs} f_{pqrs} a_p^\dagger a_q^\dagger a_r a_s,$$

so the electronic Hamiltonian (8.3) becomes

$$\hat{H} = \sum_{pq} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s, \quad (8.11)$$

with integrals

$$h_{pq} = \int d\mathbf{x} \chi_p^*(\mathbf{x}) \left[-\frac{\nabla^2}{2} - \sum_A \frac{\zeta_A}{r_{A\mathbf{x}}} \right] \chi_q(\mathbf{x}), \quad (8.12)$$

$$h_{pqrs} = \int d\mathbf{x}_1 d\mathbf{x}_2 \frac{\chi_p^*(\mathbf{x}_1) \chi_q^*(\mathbf{x}_2) \chi_r(\mathbf{x}_2) \chi_s(\mathbf{x}_1)}{|\mathbf{r}_1 - \mathbf{r}_2|}.$$

Here $\chi_p(\mathbf{x})$ is spin-orbital p , a function of position and spin. Look at which coordinate each factor carries in h_{pqrs} . The operator string in (8.11) is $a_p^\dagger a_q^\dagger a_r a_s$, with r and s in reverse order, and to match it the integral pairs χ_s with χ_p at $\mathbf{x}_1$ and χ_r with χ_q at $\mathbf{x}_2$. Pairing them the other way, with χ_r at $\mathbf{x}_1$, flips the sign of every two-body term and turns the electron repulsion into an attraction. I checked this on H_2 : the wrong pairing gives a ground-state energy near -2.5 Hartree instead of the correct -1.14 .

Where the integrals come from

The h_{pq} and h_{pqrs} are ordinary numbers, computed *classically* before any quantum circuit is written; every standard chemistry package (PySCF, Psi4) will produce them for a given molecule and basis set. Count them. With M spin-orbitals the pair (p, q) takes M^2 values and the quadruple (p, q, r, s) takes M^4 , so there are $O(M^2)$ one-body and $O(M^4)$ two-body integrals. Symmetries of the integrals cut the M^4 by a constant factor, but the fourth power stays: at $M = 20$ there are already tens of thousands of two-body terms, and each becomes several Pauli strings after encoding. Managing that term count is the dominant practical problem in quantum chemistry algorithms.

The classical cost of this step is modest by comparison. For the Gaussian basis functions every package uses, each integral has a closed form, so producing the whole set takes polynomial classical time, of order M^4 to M^5 arithmetic operations depending on how the package transforms to molecular orbitals, and it is done once per nuclear geometry. Nothing quantum happens here. The quantum computer never sees a basis function, only the numbers h_{pq} and h_{pqrs} .

8.4 Jordan–Wigner Transformation

8.4.1 The mapping

The oldest of the three encodings [17] is also the most transparent. Identify occupation number with qubit state: $|k_i\rangle \rightarrow |q_i\rangle$. The occupation part of (8.10) is handled by

$$A^+ = \frac{X - iY}{2} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \quad A^- = \frac{X + iY}{2} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad (8.13)$$

which satisfy $A^+|0\rangle = |1\rangle$, $A^+|1\rangle = 0$, $A^-|1\rangle = |0\rangle$, $A^-|0\rangle = 0$. All four verified.

The sign $(-1)^{\sum_{j<i} k_j}$ is supplied by a string of Z operators, since $Z|k\rangle = (-1)^k|k\rangle$:

$$a_i^\dagger \mapsto Z_0 Z_1 \cdots Z_{i-1} A_i^+, \quad a_i \mapsto Z_0 Z_1 \cdots Z_{i-1} A_i^-. \quad (8.14)$$

Verified: these satisfy all three anticommutation relations of (8.9) exactly, for every pair i, j on four qubits.

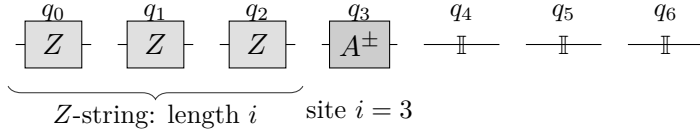


Figure 8.3: Jordan–Wigner image of $a_i^\pm$ on seven modes. Occupation information is *local* (one qubit), but the parity sign requires a Z on every qubit to the left. The image of $a_i^\pm$ therefore has weight $i + 1$, counting non-identity factors, and the worst case $i = M - 1$ has weight M : the Pauli weight grows as $O(M)$.

8.4.2 Useful special cases

Two combinations occur constantly and are worth memorizing:

$$a_i^\dagger a_i \mapsto \frac{\mathbb{I} - Z_i}{2} \quad (\text{the number operator}), \quad (8.15)$$

verified numerically, and for *adjacent* modes the Z -strings cancel:

$$a_i^\dagger a_{i+1} + a_{i+1}^\dagger a_i \mapsto \frac{X_i X_{i+1} + Y_i Y_{i+1}}{2}. \quad (8.16)$$

The hopping term (8.16) is the one that dominates a chemistry Hamiltonian at short range, so I want to follow it all the way to a circuit. The two strings $X_i X_{i+1}$ and $Y_i Y_{i+1}$ commute, so the exponential of their sum factorizes exactly:

$$e^{-i\theta(X_i X_{i+1} + Y_i Y_{i+1})} = e^{-i\theta X_i X_{i+1}} e^{-i\theta Y_i Y_{i+1}}, \quad (8.17)$$

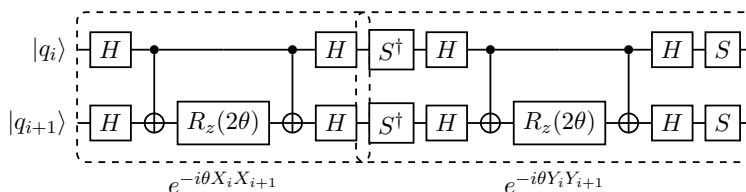


Figure 8.4: Circuit for the nearest-neighbor hopping exponential (8.17). Reading left to right, the first dashed block is $e^{-i\theta X_i X_{i+1}}$ and the second is $e^{-i\theta Y_i Y_{i+1}}$. Each one is the basis-change, CNOT, $R_z(2\theta)$, undo pattern of the general Pauli-exponential circuit of Chapter 7, with H on both qubits for an X factor and $S^\dagger$ followed by H for a Y factor. Because the two strings commute, the product is exact rather than a Trotter approximation.

with no Trotter error. Each factor is one copy of the Chapter 7 template. Figure 8.4 shows the assembled circuit. The two halves differ only in their basis-change gates.

Example 8.2

Map $\hat{H} = h_{00} a_0^\dagger a_0 + h_{11} a_1^\dagger a_1 + h_{01} (a_0^\dagger a_1 + a_1^\dagger a_0)$ with $h_{00} = 0.4$, $h_{11} = -0.7$, $h_{01} = 0.25$.

Solution. Verified numerically, the Pauli expansion is

$$\hat{H} \mapsto -0.15 \mathbb{I}\mathbb{I}\mathbb{I} - 0.20 Z\mathbb{I} + 0.35 \mathbb{I}Z + 0.125 XX + 0.125 YY.$$

Five terms. Look at the identity term $-0.15 \mathbb{I}\mathbb{I}\mathbb{I}$. It is a constant energy offset, it contributes only a global phase to any circuit, and it must be *added back classically* when reporting the energy. In my experience, forgetting it is the commonest source of energies that are right up to a constant.

8.4.3 The cost

The Z-string is the problem

A hopping term $a_p^\dagger a_q$ with $|p - q| = d$ maps to Pauli strings of weight $d + 1$: the two Z -strings cancel on the qubits below both p and q , and what survives acts on the $d + 1$ qubits from p to q . For a molecule with $M = 100$ spin-orbitals, a term connecting the lowest and highest orbitals is a weight-100 string. By the cost formula of Chapter 7, a string of weight w costs $2(w - 1)$ CNOTs, so each such term needs 198 two-qubit gates before its single-qubit gates are counted.

This is what motivates the alternatives below. It is *not* a correctness problem: Jordan–Wigner is exact, simple, and remains the default choice for small systems.

8.5 Parity Transformation

Jordan–Wigner stores occupation locally and parity non-locally. The parity encoding [25] does the opposite: qubit j stores the cumulative parity

$$p_j = \sum_{i \leq j} k_i \bmod 2, \quad (8.18)$$

which as a matrix over $\mathbb{F}_2$ is

$$|\mathbf{p}\rangle = \pi |\mathbf{k}\rangle, \quad \pi = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{pmatrix} \quad (\text{lower triangular of ones}). \quad (8.19)$$

Example 8.3

Transform $|11100101\rangle$ to the parity basis.

Solution. Taking running sums mod 2 of $k = (1, 1, 1, 0, 0, 1, 0, 1)$:

$$p = (1, 0, 1, 1, 1, 0, 0, 1) \implies |10111001\rangle.$$

Verified numerically.

The matrix π is lower triangular with ones on the diagonal, so over $\mathbb{F}_2$ it is invertible and linear. A linear reversible map on bits is realized by CNOTs alone. Figure 8.5 draws the resulting staircase. One CNOT from qubit $j - 1$ into qubit j , applied for $j = 1, 2, \dots, M - 1$ in that order, overwrites each k_j by the running sum p_j in place. The cost is $M - 1$ CNOTs, one for every qubit after the first. Each CNOT must wait for the one before it, because its control is the qubit the previous CNOT just wrote, so the depth is also $M - 1$.

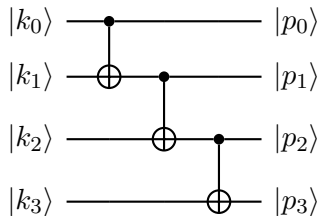


Figure 8.5: The occupation-to-parity basis change of (8.19) as a circuit, drawn for $M = 4$. Each CNOT adds the running sum already sitting on qubit $j - 1$ into qubit j , so the register ends holding $p_j = \sum_{i \leq j} k_i \bmod 2$. Running the same three CNOTs in the reverse order undoes the map and returns the occupation basis.

Now the parity sign is read off a *single* qubit. The price is that occupation is no longer local. Whether orbital j is occupied is $p_j \oplus p_{j-1}$, so creating or annihilating depends on qubit $j - 1$:

$$\mathcal{P}_j^\pm \equiv |0\rangle\langle 0|_{j-1} \otimes A_j^\pm - |1\rangle\langle 1|_{j-1} \otimes A_j^\mp = \frac{1}{2} (Z_{j-1} X_j \mp i Y_j). \quad (8.20)$$

Both signs verified numerically.

Read the tensor structure carefully

In (8.20) the term $Z_{j-1} X_j$ acts on *two* qubits, while Y_j acts on *one*, implicitly $\mathbb{I}_{j-1} \otimes Y_j$. Written without subscripts the expression looks like a sum of a two-qubit and a one-qubit operator, which is ill-formed. The subscripts are doing essential work here.

Changing occupation j also changes every cumulative sum above it, so an X -string is needed:

$$a_j^\dagger \mapsto \mathcal{P}_j^+ X_{j+1} X_{j+2} \cdots X_{M-1}, \quad a_j \mapsto \mathcal{P}_j^- X_{j+1} \cdots X_{M-1}. \quad (8.21)$$

The Z -string of Jordan–Wigner has become an X -string of the same $O(M)$ length, running the other way. Jordan–Wigner puts Z on the j qubits below mode j , for a weight of $j + 1$; the parity encoding puts X on the $M - 1 - j$ qubits above it and touches qubit $j - 1$ as well, for a weight of up to $M - j + 1$. The two are mirror images, and the worst case of each is a string of weight M , one non-identity factor on every qubit. Parity encoding is therefore not an improvement in weight. It does, however, enable the two-qubit reduction of §8.6.6.

8.6 Bravyi–Kitaev Transformation

8.6.1 The idea

The Bravyi–Kitaev encoding [6] splits the difference. Jordan–Wigner is local in occupation, non-local in parity; the parity encoding is the reverse. Bravyi–Kitaev instead stores partial sums over nested blocks of size 2, 4, 8, $\dots$, so that *both* pieces of information cost $O(\log M)$.

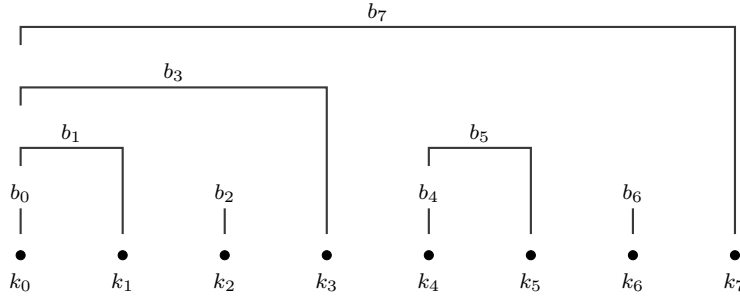


Figure 8.6: Which orbitals each Bravyi–Kitaev qubit stores, for $M = 8$. Even-indexed qubits store a single occupation number; odd-indexed qubits store partial sums over blocks of size 2, 4, 8, $\dots$. The tree structure is why both occupation and parity cost only $O(\log M)$.

8.6.2 The matrix

The transformation $|\mathbf{b}\rangle = \beta |\mathbf{k}\rangle$ is built recursively:

$$\beta_1 = \begin{pmatrix} 1 \end{pmatrix}, \quad \beta_{2m} = \begin{pmatrix} \beta_m & 0 \\ A & \beta_m \end{pmatrix}, \quad (8.22)$$

where A is the $m \times m$ matrix of zeros except a bottom row of ones. This A is a block of a bit matrix and has nothing to do with the single-qubit ladder operators $A^\pm$ of (8.13). For $M = 8$ (indices ascending, b_0 first):

$$\beta_8 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}. \quad (8.23)$$

Verified against the recursive construction.

Invert over $\mathbb{F}_2$, not over $\mathbb{R}$

β^{-1} must be computed by Gaussian elimination *modulo 2*. Computing a floating-point inverse and rounding happens to work for $M = 8$, but not in general: as M grows β becomes badly conditioned, and the rounding then silently produces a matrix that is not the inverse. Always verify $\beta\beta^{-1} \equiv \mathbb{I} \pmod{2}$.

8.6.3 The three sets

Three index sets determine the qubit operators; the definitions and notation follow [25].

Definitions

- **Update set** $U(j)$: qubits, other than j , whose stored partial sums include orbital j and so must be flipped when its occupation changes. Read from column j of β , entries *below* the diagonal.
- **Parity set** $P(j)$: a set of qubits whose joint parity equals the parity of all orbitals with index $< j$. Read from **row** $j - 1$ of $\pi\beta^{-1}$.
- **Flip set** $F(j)$: qubits that, together with j , determine whether b_j has the same parity as k_j . Read from row j of β^{-1} , entries left of the diagonal.

$P(j)$ comes from row $j - 1$, not row j

This is the single easiest thing to get wrong, and it fails silently. Row j of $\pi\beta^{-1}$ gives the parity of orbitals with index $\leq j$, including orbital j itself. What the operator needs is the parity of orbitals *strictly below* j , which is row $j - 1$.

Using row j produces $R(j) \cup \{j\}$, which for $j < M - 1$ is $P(j + 1)$: the parity set of the *next* orbital, not of this one. At $M = 8$ and $j = 2$, row 2 gives $\{1, 2\} = P(3)$ where the operator needs $P(2) = \{1\}$. The result is a set of the right kind and roughly the right size, so the mistake yields a plausible table rather than an obvious contradiction, and that is what makes it silent. I made exactly this error while checking this chapter, and it took a β_8 -scale example to expose it.

Example 8.4

Compute all three sets for $M = 8$.

Solution. Read them from β_8 , β_8^{-1} and $\pi_8\beta_8^{-1}$:

Table 8.1: Bravyi–Kitaev sets for $M = 8$, all three families verified numerically against the matrices. Observe that $F(j) = \emptyset$ for every even j , as it must be: even-indexed qubits store a single occupation number.

j	$U(j)$	$P(j)$	$F(j)$	$R(j) = P(j) \setminus F(j)$
0	$\{1, 3, 7\}$	$\emptyset$	$\emptyset$	$\emptyset$
1	$\{3, 7\}$	$\{0\}$	$\{0\}$	$\emptyset$
2	$\{3, 7\}$	$\{1\}$	$\emptyset$	$\{1\}$
3	$\{7\}$	$\{1, 2\}$	$\{1, 2\}$	$\emptyset$
4	$\{5, 7\}$	$\{3\}$	$\emptyset$	$\{3\}$
5	$\{7\}$	$\{3, 4\}$	$\{4\}$	$\{3\}$
6	$\{7\}$	$\{3, 5\}$	$\emptyset$	$\{3, 5\}$
7	$\emptyset$	$\{3, 5, 6\}$	$\{3, 5, 6\}$	$\emptyset$

8.6.4 The operator mapping

Define the even/odd projectors on a set S ,

$$E_S = \frac{\mathbb{I} + Z_S}{2}, \quad O_S = \frac{\mathbb{I} - Z_S}{2}, \quad Z_S = \prod_{i \in S} Z_i, \quad (8.24)$$

which return the state if S has even (respectively odd) parity and zero otherwise. For $S = \emptyset$ the product is empty, so $Z_\emptyset = \mathbb{I}$, $E_\emptyset = \mathbb{I}$ and $O_\emptyset = 0$.

Three objects wear the letter P

Their forms distinguish them reliably. $\hat{P}_{ij}$ (hat, two indices) is the exchange operator of §8.2; $P(j)$ (parenthesized argument) is the Bravyi–Kitaev parity set of this section; P_j (plain subscript) is a Pauli *string*, as in Fig. 8.1 and throughout §8.7. So in $Z_{P(j)}$ below the subscript is a set of qubit indices, and $Z_{P(j)}$ means $\prod_{i \in P(j)} Z_i$ in the sense of (8.24), not the Pauli string P_j .

Even j . Qubit j stores only k_j , so the mapping mirrors Jordan–Wigner with the strings replaced by the sets:

$$a_j^\dagger \mapsto X_{U(j)} A_j^+ Z_{P(j)}, \quad a_j \mapsto X_{U(j)} A_j^- Z_{P(j)}. \quad (8.25)$$

Odd j . Qubit j stores a partial sum, so whether we raise or lower depends on the parity of $F(j)$:

$$\Pi_j^\pm \equiv A_j^\pm E_{F(j)} - A_j^\mp O_{F(j)}, \quad (8.26)$$

and since $\Pi^\pm$ already accounts for the part of $P(j)$ lying in $F(j)$, only $R(j) = P(j) \setminus F(j)$ remains:

$$a_j^\dagger \mapsto X_{U(j)} \Pi_j^+ Z_{R(j)}, \quad a_j \mapsto X_{U(j)} \Pi_j^- Z_{R(j)}. \quad (8.27)$$

Writing

$$\rho(j) = \begin{cases} P(j) & j \text{ even,} \\ R(j) & j \text{ odd,} \end{cases} \quad (8.28)$$

both cases collapse to

$$a_j^\dagger \mapsto X_{U(j)} \Pi_j^+ Z_{\rho(j)}, \quad a_j \mapsto X_{U(j)} \Pi_j^- Z_{\rho(j)}. \quad (8.29)$$

The even case really is contained in this single form. For even j we have $F(j) = \emptyset$, so $E_{F(j)} = \mathbb{I}$ and $O_{F(j)} = 0$, and (8.26) collapses to $\Pi_j^\pm = A_j^\pm$. Since $\rho(j) = P(j)$ there, (8.29) reproduces (8.25) term for term. Table 8.1 confirms $F(j) = \emptyset$ at every even j for $M = 8$.

Verified

The operators (8.29) were built explicitly for $M = 4$ and checked against all three anticommutation relations (8.9) for every pair i, j . They pass exactly. The measured set sizes scale as claimed:

M	max set size	$\log_2 M$
4	2	2
8	3	3
16	4	4
32	5	5
64	6	6

8.6.5 Comparing the three encodings

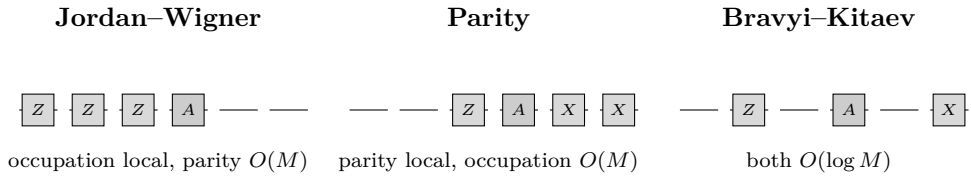


Figure 8.7: Schematic Pauli support of a single fermionic operator under the three encodings. Jordan–Wigner and parity each pay $O(M)$ for one of the two pieces of information; Bravyi–Kitaev pays $O(\log M)$ for both.

Table 8.2: Comparison of the three encodings. All use M qubits; they differ only in the weight of the resulting Pauli strings.

	Jordan–Wigner	Parity	Bravyi–Kitaev
Occupation info	local	$O(M)$	$O(\log M)$
Parity info	$O(M)$	local	$O(\log M)$
Pauli weight	$O(M)$	$O(M)$	$O(\log M)$
Qubits	M	M	M
Ease of reasoning	highest	moderate	lowest
Two-qubit reduction	no	yes	yes

Which to use in practice

For small molecules the weight difference is negligible and Jordan–Wigner’s transparency wins: you can read off what every term means. Bravyi–Kitaev pays off once M is large enough that $O(M)$ versus $O(\log M)$ matters, in practice a few tens of spin-orbitals upward. The asymptotic advantage is partly cosmetic for *chemistry* Hamiltonians, where the number of terms is $O(M^4)$ regardless: the weight reduction helps the gate count per term, not the term count.

Let me assemble the gate count of one Trotter step, since that is where the three encodings are really compared. Write L for the number of Pauli strings in the encoded Hamiltonian and w for the weight of a string, its number of non-identity factors. By the recipe of Chapter 7 each string costs $2(w - 1)$ CNOTs plus at most $4w$ single-qubit gates, so one first-order Trotter step costs about $2 \sum_j (w_j - 1)$ CNOTs, which is at most $2L(w_{\max} - 1)$. For a chemistry Hamiltonian L grows as M^4 , because each of the $O(M^4)$ integrals produces a bounded number of strings. Under Jordan–Wigner $w_{\max} = M$, so a Trotter step costs $O(M^5)$ two-qubit gates. Under Bravyi–Kitaev $w_{\max} = O(\log M)$, so it costs $O(M^4 \log M)$. In plain words,

doubling the number of spin-orbitals multiplies the two-qubit gate count of one step by about thirty-two under Jordan–Wigner and by a little over sixteen under Bravyi–Kitaev. The full simulation repeats the step r times, with r chosen from the target accuracy as in Chapter 7. Every encoding uses exactly M qubits, one per spin-orbital, and none of them changes the classical cost of the integrals, which was paid before the encoding began. The M^4 is common to all three, which is why I called the advantage partly cosmetic: the encoding chooses between M^5 and $M^4 \log M$, not between a polynomial and something better.

8.6.6 Qubit tapering

A short but useful addition. Both the parity and Bravyi–Kitaev encodings place the *total* electron parity on a single qubit, and molecular Hamiltonians conserve the number of spin-up and spin-down electrons separately. Two qubits therefore hold values that never change, and can be replaced by their classical values ± 1 .

For a system with M spin-orbitals and a fixed electron number, this reduces the register from M to $M - 2$ qubits at no cost in accuracy. Every Pauli string that touched one of the two frozen qubits did so with a Z or an identity, so the string survives with its coefficient multiplied by ± 1 . For the canonical H_2 example in a minimal basis, $M = 4$ becomes 2 qubits. That is why the calculation can be run on almost any device.

8.7 From Qubit Hamiltonian to Circuit

Most treatments stop once the fermionic operators have been mapped. I want to close the loop and show how the resulting qubit Hamiltonian is turned into something a device can run.

8.7.1 Preparing the reference state

Every method needs a starting state. The natural choice is the *Hartree–Fock* state: fill the N lowest-energy spin-orbitals,

$$|\Psi_{\text{HF}}\rangle = |\underbrace{1 \cdots 1}_N \underbrace{0 \cdots 0}_{M-N}\rangle. \quad (8.30)$$

Under Jordan–Wigner this is trivial to prepare: an X gate on each occupied orbital. Under Bravyi–Kitaev the pattern differs: apply X to qubit j if and only if $b_j = \sum_i \beta_{ji} k_i \bmod 2$ equals 1 for the occupation vector $\mathbf{k}$. It is still a product state and still a single layer of X gates. Only *which* qubits get flipped changes.

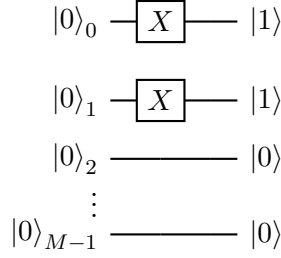


Figure 8.8: Hartree–Fock state preparation under Jordan–Wigner, for $N = 2$ electrons in M spin-orbitals: one X gate per occupied orbital.

8.7.2 Evolving under the Hamiltonian

After encoding, the Hamiltonian is a sum of Pauli strings,

$$\hat{H} = \sum_j c_j P_j, \quad (8.31)$$

with real c_j . Chapter 7 gives the circuit for each $e^{-ic_j P_j \Delta t}$, the general Pauli-exponential construction of Fig. 7.8, and Trotterization assembles them. Nothing new is required. That is why I put Chapter 7 first.

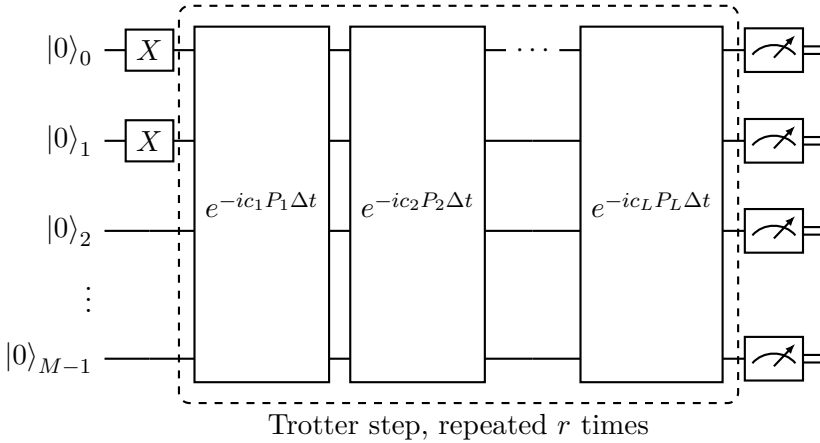


Figure 8.9: The full pipeline. Hartree–Fock preparation, then r Trotter steps each containing one general Pauli-exponential circuit of Chapter 7 per Hamiltonian term, then measurement. Every block in the dashed region is the basis-change / CNOT-ladder / $R_z(2c_j \Delta t)$ / undo construction of Chapter 7.

8.7.3 Measuring the energy

The energy is

$$E = \langle \Psi | \hat{H} | \Psi \rangle = \sum_j c_j \langle \Psi | P_j | \Psi \rangle, \quad (8.32)$$

so each Pauli string is measured separately and the results combined classically. Measuring $\langle \Psi | P | \Psi \rangle$ takes three steps. Rotate each qubit into the Z basis exactly as in Chapter 7, using H for an X factor and $HS^\dagger$ for a Y factor. Measure every qubit in the support of P . Then average $(-1)^{\text{parity of outcomes}}$ over the shots. Figure 8.10 draws the circuit for one representative string.

How many shots does this take? Each shot returns $+1$ or -1 , so a single outcome has variance at most 1, and the average of S shots has a standard error of at most $1/\sqrt{S}$. To pin $\langle P_j \rangle$ down to a target precision ϵ therefore takes $S \approx 1/\epsilon^2$ shots, where a shot means one repetition of the prepare-and-measure circuit. Halving ϵ costs four times as many shots. That is the price of one term. There are L terms, and for a chemistry Hamiltonian L grows as M^4 , where M is the number of spin-orbitals, so the total is $O(M^4/\epsilon^2)$ circuit repetitions before any of the mitigations below. Here ϵ is the precision on each dimensionless expectation value $\langle P_j \rangle$. The error in the energy is then of order ϵ times the sum of the $|c_j|$, so ϵ must be set below chemical accuracy divided by that sum.

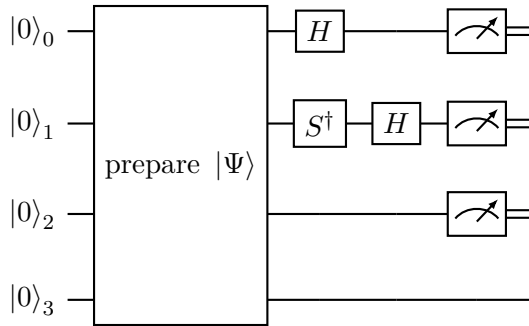


Figure 8.10: Measuring $\langle \Psi | P | \Psi \rangle$ for $P = X_0 Y_1 Z_2$ on a four-qubit register. Qubit 0 carries an X factor and is rotated by H ; qubit 1 carries a Y factor and is rotated by $S^\dagger$ then H ; qubit 2 carries a Z factor and needs no rotation. Qubit 3 lies outside the support of P , so it is neither rotated nor measured. Each shot returns three bits $b_0 b_1 b_2$, and the estimate of $\langle \Psi | P | \Psi \rangle$ is the average of $(-1)^{b_0 \oplus b_1 \oplus b_2}$ over the shots.

The measurement cost is the real bottleneck

Estimating each $\langle P_j \rangle$ to precision ϵ needs $O(1/\epsilon^2)$ shots, meaning circuit repetitions, and there are $O(M^4)$ terms for M spin-orbitals. For chemical accuracy (1.6×10^{-3} Hartree) on a modest molecule this runs to 10^8 repetitions or more. Two standard mitigations, treated at length in the review [23]:

- **Commuting groups.** Partitioning the terms into qubit-wise commuting groups, which can be measured simultaneously, typically cuts the distinct measurement settings by an order of magnitude.
- **Weighted sampling.** Allocate shots in proportion to $|c_j|$ rather than uniformly; the large-coefficient terms dominate the variance.

The identity term $c_0 \mathbb{I}$ needs no measurement. Add it classically.

8.7.4 Implementation guide

Building the whole pipeline

Step 1: get the integrals. Use a classical chemistry package to produce h_{pq} and h_{pqrs} for your molecule and basis. Sanity-check by recomputing the Hartree–Fock energy and comparing with the package’s own value.

Step 2: build the encoding matrices. For Bravyi–Kitaev, construct β by the recursion (8.22) and invert it *over* $\mathbb{F}_2$ by Gaussian elimination mod 2, asserting $\beta\beta^{-1} \equiv \mathbb{I} \pmod{2}$ before going further.

Step 3: compute the sets. For each j :

- $U(j)$: column j of β , rows $> j$;
- $P(j)$: row $j - 1$ of $\pi\beta^{-1}$ (empty for $j = 0$);
- $F(j)$: row j of β^{-1} , columns $< j$;
- $R(j) = P(j) \setminus F(j)$.

Assert $F(j) = \emptyset$ for even j , a cheap check that catches most indexing errors.

Step 4: map the operators. Represent each Pauli string as a pair of bit vectors (x, z) and multiply in that symplectic representation. A dense $2^M \times 2^M$ complex matrix occupies $16 \cdot 4^M$ bytes of classical memory, a few hundred megabytes at $M = 12$, so dense expansion is viable only for $M \lesssim 12$ and is for testing only.

Step 5: collect terms. Expanding $a_p^\dagger a_q^\dagger a_r a_s$ generates many Pauli strings, most of which cancel or combine. Group identical strings, sum their coefficients, then *drop terms with $|c_j|$ below a threshold* (typically 10^{-8} Hartree), which routinely removes half the terms with no measurable effect on the energy.

Step 6: build circuits. Call the `pauli_evolution` routine from Chapter 7 for each term. Nothing new to write.

Testing the pipeline, and the sign trap

Run these tests in order:

1. *Anticommutators.* For $M = 4$, build every a_j and $a_j^\dagger$ as a dense matrix and verify all three relations (8.9). This single test catches almost every encoding bug.
2. *Number operator.* Check $\sum_j a_j^\dagger a_j$ is diagonal with integer eigenvalues from 0 to M .
3. *Hermiticity.* The assembled qubit Hamiltonian must have real coefficients on every Pauli string. A stray imaginary part means a dropped conjugate or a sign error.
4. *Ground-state energy.* Diagonalize the qubit Hamiltonian densely for small M and compare against a classical full-CI calculation. The encoding is exact, so they must agree to machine precision; any discrepancy is a bug rather than an approximation.
5. *Encoding agreement.* Jordan–Wigner and Bravyi–Kitaev must give the *same* spectrum, an excellent independent check.

A warning about signs. The commonest failure is a sign convention: whether $|0\rangle$ means occupied or empty, whether A^+ raises or lowers, and the ordering of $a_p^\dagger a_q^\dagger a_r a_s$ against the coordinates in h_{pqrs} (r and s are reversed relative to p, q , so χ_s shares $\mathbf{x}_1$ with χ_p in (8.12)). Fix your conventions in one place, write them in a comment, and test against the anticommutators.

Example 8.5

The canonical worked example: H_2 in the STO-3G basis at bond length 0.735 Å.

Setup. Two hydrogen atoms, one spatial orbital each, two spin-orbitals per spatial orbital: $M = 4$ spin-orbitals, $N = 2$ electrons.

Step 1. The classical package returns a handful of nonzero h_{pq} and h_{pqrs} .

Step 2. Jordan–Wigner encoding gives a qubit Hamiltonian on 4 qubits with 15 Pauli terms, counting the identity, of the form

$$\hat{H} = c_0 \mathbb{I} + c_1 Z_0 + c_2 Z_1 + c_3 Z_0 Z_1 + c_4 Y_0 Y_1 X_2 X_3 + \cdots$$

Step 3. Parity encoding plus the two-qubit tapering of §8.6.6 reduces this to **2 qubits** and six terms: $\mathbb{I}$, Z_0 , Z_1 , $Z_0 Z_1$, $X_0 X_1$ and $Y_0 Y_1$.

Step 4. Diagonalizing the 4×4 matrix gives the ground-state energy, which must match the classical full-CI value exactly. With the nuclear repulsion added back it comes to about -1.137 Hartree.

This example is worth doing by hand once. It is small enough to check every step and large enough to exercise every part of the pipeline.

8.8 Summary

- Born–Oppenheimer freezes the nuclei, leaving an electronic Hamiltonian to be solved at each geometry.
- Fermionic antisymmetry forces Slater determinants and gives the Pauli principle for free; it is also what makes encoding nontrivial, since qubits commute and fermions anticommute.
- Second quantization replaces wavefunctions with occupation numbers, which *is* a qubit register. The work is in mapping the operators.
- $\hat{H} = \sum_{pq} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s$, with $O(M^2)$ and $O(M^4)$ classically computed integrals.
- Jordan–Wigner: $a_i \mapsto Z_0 \cdots Z_{i-1} A_i^-$. Occupation local, parity $O(M)$. $a_i^\dagger a_i \mapsto (\mathbb{I} - Z_i)/2$.
- Parity encoding: the reverse trade. It gives an $O(M)$ X -string instead of an $O(M)$ Z -string, so there is no weight advantage, but it enables tapering.
- Bravyi–Kitaev: partial sums over nested blocks of size 2, 4, 8, $\dots$; both occupation and parity cost $O(\log M)$. Verified: max set size equals $\log_2 M$ exactly for $M = 4, \dots, 64$.

- $U(j)$ from β ; $P(j)$ from **row** $j-1$ of $\pi\beta^{-1}$; $F(j)$ from β^{-1} ; $R(j) = P(j) \setminus F(j)$. $F(j) = \emptyset$ for even j .
- Invert β over $\mathbb{F}_2$, never in floating point.
- Tapering removes two qubits using conserved parities.
- Hartree–Fock reference is a product state, made by one layer of X gates.
- Energy measurement, not circuit depth, is usually the bottleneck: $O(M^4)$ terms $\times O(1/\epsilon^2)$ shots, each shot one repetition of the circuit.

8.9 End-of-Chapter Problems

- 8.1** Show that the Slater determinant (8.6) vanishes if any two orbitals coincide, and hence derive the Pauli exclusion principle.
- 8.2** Verify that $\hat{P}_{ij}^2 = \hat{I}$ implies eigenvalues ± 1 , and explain why this argument does not by itself establish the spin–statistics connection.
- 8.3** Show that $A^+ = (X - iY)/2$ and $A^- = (X + iY)/2$ act as raising and lowering operators with $|0\rangle$ denoting an empty orbital.
- 8.4** Compute $a_2^\dagger a_0 |1100\rangle$ including the sign, both directly from (8.10) and via the Jordan–Wigner matrices.
- 8.5** Prove (8.15), that $a_i^\dagger a_i \mapsto (\mathbb{I} - Z_i)/2$ under Jordan–Wigner.
- 8.6** Show that the Z -strings cancel in $a_i^\dagger a_{i+1} + a_{i+1}^\dagger a_i$ and derive (8.16).
- 8.7** Map $a_0^\dagger a_3 + a_3^\dagger a_0$ under Jordan–Wigner on four modes. What is the Pauli weight, and how many CNOTs does its exponential need by the Chapter 7 formula?
- 8.8** Transform $|10110010\rangle$ into the parity basis, and back.
- 8.9** Prove the identity $\mathcal{P}_j^\pm = \frac{1}{2} (Z_{j-1} X_j \mp i Y_j)$ from the definition (8.20).
- 8.10** Construct β_4 and β_4^{-1} by hand over $\mathbb{F}_2$, and verify $\beta_4 \beta_4^{-1} \equiv \mathbb{I}$.
- 8.11** Compute $U(j)$, $P(j)$, $F(j)$ and $R(j)$ for all j with $M = 4$, and check $F(j) = \emptyset$ for even j .
- 8.12** Show that $P(j)$ must be read from row $j-1$ of $\pi\beta^{-1}$ by computing both rows for $j = 5$, $M = 8$, and identifying which gives the parity of orbitals with index < 5 .

- 8.13** Verify numerically that the Bravyi–Kitaev operators for $M = 4$ satisfy all three anticommutation relations.
- 8.14** For $M = 16$, compute the maximum Pauli weight of a single fermionic operator under Jordan–Wigner and under Bravyi–Kitaev. Compare with $\log_2 M$.
- 8.15** Explain why tapering removes exactly two qubits, and identify the two conserved quantities involved.
- 8.16** Draw the Hartree–Fock preparation circuit for $N = 4$ electrons in $M = 8$ spin-orbitals, under Jordan–Wigner and under Bravyi–Kitaev.
- 8.17** Estimate the number of measurements needed to reach chemical accuracy for a molecule with $M = 20$ spin-orbitals, assuming uniform shot allocation. How much does qubit-wise commuting grouping help?
- 8.18** (*Project.*) Implement the Jordan–Wigner encoding for H_2 in a minimal basis, diagonalize the resulting qubit Hamiltonian, and compare the ground-state energy with a classical full-CI calculation. Then repeat with Bravyi–Kitaev and confirm the spectra agree.

Research Problems and Projects

The problems above all have answers you can check. The three below are open at the level of measurement rather than proof: the mathematics is settled, but what actually happens for a real molecule, a real compiler and a real device is not. Each is sized for several weeks of work by a strong MS student or a beginning PhD student, and each ends in an artifact you can show someone: a scaling figure, a resource table, or a piece of software other people can run. Read the cited papers first, and expect your first measurement to disagree with the asymptotic statement you thought you were testing.

- R8.1 Where, if anywhere, does Bravyi–Kitaev actually win?** Table 8.2 states the standard asymptotic claim: Pauli weight $O(M)$ under Jordan–Wigner and the parity encoding, $O(\log M)$ under Bravyi–Kitaev [6, 25]. That claim is about a *single* fermionic operator. What a chemist cares about is the two-qubit gate count of one Trotter step of the *assembled* Hamiltonian, which mixes $O(M^4)$ terms of many different supports. Measure the difference. Take a family of molecules of increasing size in a fixed minimal basis, for example the hydrogen chains H_n for $n = 2, 4, 6, 8, 10$, plus one non-trivial case such as LiH or BeH_2 . For each, build the qubit Hamiltonian under all three encodings, with and without the tapering of §8.6.6, and record the full distribution of Pauli weights, the term count after thresholding, and the CNOT count of one first-order Trotter step compiled by the recipe of

Chapter 7. *Deliverable:* scaling curves of mean weight, maximum weight and total CNOT count against M , with fitted exponents and error bars over basis choices, plus a table giving the value of M at which Bravyi–Kitaev first beats Jordan–Wigner on total CNOT count. Then push on the interesting part. The chapter warns that the asymptotic advantage may be partly cosmetic for chemistry Hamiltonians. Decide whether your data support that warning, and if the crossover is far larger than $\log_2 M$ reasoning suggests, explain which structural feature of the molecular Hamiltonian is responsible.

R8.2 Audit the published resource estimates. The box on measurement cost in §8.7 gives one costing of quantum chemistry: $O(M^4)$ Pauli terms times $O(1/\epsilon^2)$ shots. The review literature [23] gives several others, and the qubitization and linear-combination-of-unitaries lines [21, 8] give costings in a different currency again, counting queries and Toffoli gates rather than shots. Published resource estimates for the same molecules have moved by several orders of magnitude over the last decade. Reconstruct why. Pick two or three molecules that appear repeatedly in the literature, collect every resource estimate you can find for them, and normalize the estimates onto a common footing: the same basis, the same target error, the same definition of a gate. *Deliverable:* one table with a row per published estimate, giving the method, the input model, the accuracy target, the qubit count, the gate or shot count, and the precise place in the paper where each number is stated. Attach to each row a note saying which assumption differs from the row above it. The point is the notes, not the table. Identify at least two pairs of estimates that are routinely compared in the literature but that rest on assumptions incompatible enough that the comparison is not meaningful, and say what a fair comparison would require. Distinguish carefully throughout between what is proved, what is extrapolated, and what is asserted.

R8.3 An encoding library, tested to destruction, and the real shot cost. Implement all three encodings of this chapter as a library, using the symplectic (x, z) representation of Step 4 in §8.7.4 rather than dense matrices, and put the whole test ladder of that section into a test suite that runs on every commit. Then use it to answer a question the chapter states but does not quantify. Simulate energy estimation for H_2 , LiH and BeH_2 with realistic shot noise and measure how many shots are actually needed to reach chemical accuracy under three strategies: uniform allocation, allocation weighted by $|c_j|$, and qubit-wise commuting grouping combined with weighted allocation [23]. *Deliverable:* the library, with the test suite and a documented public interface, plus a table of measured shots-to-chemical-accuracy for every molecule and strategy, each entry an average over independent runs with a reported spread. The $1/\epsilon^2$ scaling is not in doubt and you should confirm it as a sanity check. The constant in front of it is what decides whether a molecule is reachable on

hardware, and it is what your table should pin down. A careful negative result, showing that grouping helps far less than the order of magnitude usually quoted, would itself be worth writing up.

Chapter 9

Optimization and Linear Algebra

The two algorithms in this chapter look unrelated. One approximates the ground state of a classical cost function; the other solves a linear system. I have placed them together because they are the same kind of object:

The unifying idea

Both algorithms *apply a function to the spectrum of an operator*.

- Adiabatic optimization and QAOA apply $\lambda \mapsto e^{-i\gamma\lambda}$ to the cost Hamiltonian, arranging for interference to concentrate amplitude on its lowest eigenvalue.
- HHL applies $\lambda \mapsto 1/\lambda$ to the matrix A , which is exactly matrix inversion.

Chapter 10 makes this observation into a technique. The quantum singular value transformation implements $\lambda \mapsto f(\lambda)$ for almost any polynomial f . It recovers both of these algorithms as special cases, along with Grover and phase estimation.

A note on scope

I present both algorithms in *fault-tolerant* form: fixed schedules, fixed rotation angles, no classical optimization loop. The variational versions (VQE, variational QAOA, variational linear solvers) are the subject of a large and fast-moving near-term literature which this book does not cover.

I have left them out not because they are unimportant, but because their analysis is empirical rather than mathematical. Convergence, barren plateaus and optimizer choice are open questions, whereas the algorithms below have provable resource counts. Everything here can be costed exactly; that is what makes it teachable.

What you should be able to do after this chapter

- Encode a combinatorial problem as a QUBO and then as an Ising Hamiltonian.
- State the adiabatic theorem and explain what sets the runtime.
- Derive QAOA as a Trotterization of adiabatic evolution, and draw its circuit.
- Use the Hadamard test to estimate $\langle \Phi | U | \Phi \rangle$.
- Trace every register through the HHL algorithm.
- State the four caveats that qualify HHL's "exponential speed-up".

9.1 QUBO and the Ising Hamiltonian

9.1.1 Quadratic unconstrained binary optimization

A QUBO problem asks for the binary string minimizing (or maximizing)

$$C(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}, \quad \mathbf{x} \in \{0, 1\}^n, \quad (9.1)$$

with $Q \in \mathbb{R}^{n \times n}$ and $\mathbf{c} \in \mathbb{R}^n$. A surprising number of problems reduce to this form: scheduling, portfolio selection, graph partitioning and satisfiability. Only the sum $Q_{ij} + Q_{ji}$ ever enters $C(\mathbf{x})$, so I will take Q to be symmetric throughout; that costs nothing and simplifies the algebra below.

9.1.2 MaxCut

Given a weighted graph $G = (V, E, W)$, partition the vertices into two sets so as to maximize the total weight of edges crossing between them.

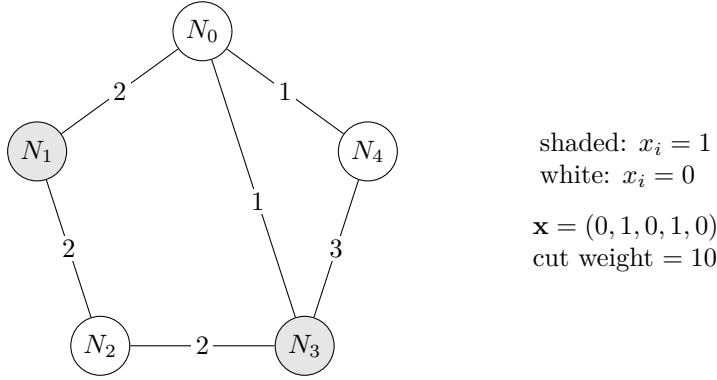


Figure 9.1: A weighted MaxCut instance on five vertices and six edges. The partition shown puts N_1, N_3 in one group and N_0, N_2, N_4 in the other. All the edges except N_0-N_4 cross the partition, giving weight $2 + 2 + 2 + 3 + 1 = 10$. Verified by exhaustive search over all 2^5 assignments: 10 is the maximum, and the total weight of all edges is 11.

With W_{ij} the weight of edge (i, j) (and 0 if absent),

$$W = \begin{pmatrix} 0 & 2 & 0 & 1 & 1 \\ 2 & 0 & 2 & 0 & 0 \\ 0 & 2 & 0 & 2 & 0 \\ 1 & 0 & 2 & 0 & 3 \\ 1 & 0 & 0 & 3 & 0 \end{pmatrix}, \quad (9.2)$$

the cut weight for an assignment $\mathbf{x}$ is

$$C(\mathbf{x}) = \sum_{i,j} W_{ij} x_i (1 - x_j). \quad (9.3)$$

9.1.3 Converting to QUBO form

Expanding (9.3),

$$C(\mathbf{x}) = \sum_{i,j} W_{ij} x_i - \sum_{i,j} W_{ij} x_i x_j = \mathbf{x}^T (-W) \mathbf{x} + \left(\sum_j W_{ij} \right) \mathbf{x},$$

so

$$Q = -W, \quad c_i = \sum_j W_{ij}. \quad (9.4)$$

Verified numerically: (9.1) with these Q and $\mathbf{c}$ reproduces (9.3) for all 32 assignments.

9.1.4 Converting to an Ising Hamiltonian

We need an operator whose eigenvalues are the cost values. Since $Z_i |\mathbf{x}\rangle = (-1)^{x_i} |\mathbf{x}\rangle = (1 - 2x_i) |\mathbf{x}\rangle$,

$$x_i \longleftrightarrow \frac{\mathbb{I} - Z_i}{2}. \quad (9.5)$$

Substituting into (9.1) and collecting terms, with Q symmetric,

$$H_C = \sum_{i,j} \frac{Q_{ij}}{4} Z_i Z_j - \sum_i \frac{1}{2} \left(c_i + \sum_j Q_{ij} \right) Z_i + \left(\sum_{i,j} \frac{Q_{ij}}{4} + \sum_i \frac{c_i}{2} \right) \mathbb{I}. \quad (9.6)$$

The $i = j$ terms of the first sum are constants, since $Z_i^2 = \mathbb{I}$. Verified: for the MaxCut instance above, the diagonal entry of H_C at basis state $|\mathbf{x}\rangle$ equals $C(\mathbf{x})$ for every one of the 32 strings.

Everything downstream minimizes

Adiabatic evolution and QAOA drive the register toward the *ground* state of H_C , which is the *smallest* value of $C(\mathbf{x})$. MaxCut asks for the *largest* cut, so H_C as built above is the wrong way up.

The remedy is one sign. Hand the algorithm $-H_C$, which by (9.6) means replacing Q by $-Q$ and $\mathbf{c}$ by $-\mathbf{c}$. For MaxCut that is $Q = W$ and $c_i = -\sum_j W_{ij}$. I keep (9.4) in its natural sign so that $C(\mathbf{x})$ is the cut weight itself, and flip only where the Hamiltonian enters §9.2 and §9.3. Cut weights are then read off as $-\langle \mathbf{x} | H_C | \mathbf{x} \rangle$.

Verified numerically on the instance of Fig. 9.1. With the flipped sign the fixed schedule of (9.12) reaches approximation ratio 1.000 at $T = 40$, $p = 64$. Without it the same run converges on the *minimum* cut, of weight 0.

Three things to notice

H_C is **diagonal** in the computational basis. It encodes a classical cost function, so it has no off-diagonal terms and creates no superposition on its own.

It is a **sum of Pauli strings of weight at most 2**, so by the cost formula of Chapter 7 each term costs at most two CNOTs and one R_z . The whole of $e^{-i\gamma H_C}$ is therefore cheap.

The **identity term** contributes only a global phase to $e^{-i\gamma H_C}$ and can be dropped from the circuit. It must be added back when reporting the cost value.

The corresponding evolution operator factorizes into the Pauli-exponential circuits

of Chapter 7:

$$U_C(\gamma) = e^{-i\gamma H_C} = \prod_{i < j} R_{Z_i Z_j}(\gamma Q_{ij}) \prod_i R_{Z_i}(-\gamma(c_i + \sum_j Q_{ij})), \quad (9.7)$$

exactly, up to the global phase of the identity term of (9.6), which is dropped from the circuit as the box above says. The terms all commute, since they are all diagonal, so there is no Trotter error here. The unordered pair $\{i, j\}$ collects both the Q_{ij} and the Q_{ji} term of (9.6), so its coefficient is $Q_{ij}/2$ and the rotation angle is twice that, namely γQ_{ij} . Each $R_{Z_i Z_j}$ is the CNOT- R_z -CNOT sandwich of Chapter 7, Fig. 7.6. I want you to notice one thing about MaxCut in particular: there $c_i + \sum_j Q_{ij} = \sum_j W_{ij} - \sum_j W_{ij} = 0$, so every single-qubit R_{Z_i} angle vanishes and the cost layer is nothing but $R_{Z_i Z_j}$ gates, one per edge.

9.2 Adiabatic Quantum Computation

9.2.1 The adiabatic theorem

Adiabatic quantum computation [12] rests on the following.

Adiabatic theorem

A system prepared in the ground state of $H(0)$ and evolved under a time-dependent $H(t)$ remains in the instantaneous ground state, provided the evolution is slow compared with

$$T \sim \mathcal{O}\left(\frac{1}{\Delta_{\min}^2}\right), \quad (9.8)$$

where $\Delta_{\min} = \min_t [E_1(t) - E_0(t)]$ is the smallest gap between the ground and first excited states along the path.

Read (9.8) as a statement about a resource. The quantity T is the total evolution time, in the units in which $H(t)$ is measured. Once we discretize in §9.3 it becomes a gate count. The number of Trotter slices is $p = T/\Delta t$ at a fixed slice length Δt , and each slice is one layer of two-qubit and single-qubit rotations, so the circuit depth grows in proportion to T . The constant hidden in the $\mathcal{O}$ depends on how fast $H(t)$ changes; for the linear ramp below that rate is $\|H_f - H_i\|/T$, and the condition becomes $T \gg \|H_f - H_i\|/\Delta_{\min}^2$. In plain words: halving the smallest gap quadruples the time you must spend. For hard optimization problems the gap typically closes exponentially in the number of variables n , and then T , and with it the circuit depth, grows exponentially in n .

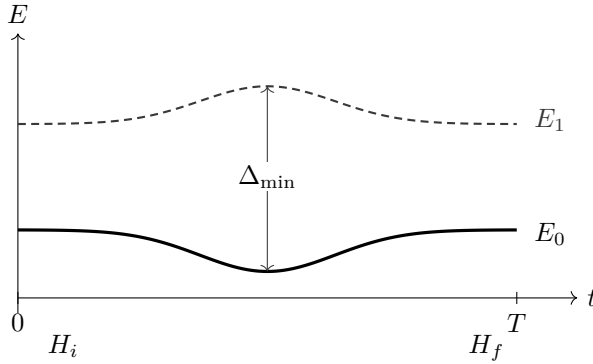


Figure 9.2: Adiabatic evolution. The runtime is set by the *smallest* gap between the ground and first excited state along the interpolation path. For hard optimization problems that gap typically closes exponentially in the problem size. That is why adiabatic computation is not a general-purpose exponential speed-up.

9.2.2 The interpolation

Start from a Hamiltonian whose ground state is trivial to prepare,

$$H_i = - \sum_i X_i, \quad (9.9)$$

whose ground state is $|+\rangle^{\otimes n} = H^{\otimes n} |0\rangle^{\otimes n}$. Interpolate linearly to the problem Hamiltonian $H_f = H_C$:

$$H(t) = \frac{t}{T} H_f + \left(1 - \frac{t}{T}\right) H_i. \quad (9.10)$$

At $t = 0$ this is H_i ; at $t = T$ it is H_f . If the evolution is slow enough, the system ends in the ground state of H_f , which is the solution.

9.2.3 Discretizing

The evolution operator for a time-dependent Hamiltonian is a time-ordered product. We approximate it by slicing $[0, T]$ into p steps of length $\Delta t = T/p$ and holding H constant on each:

$$U \approx e^{-iH(t_p)\Delta t} \dots e^{-iH(t_2)\Delta t} e^{-iH(t_1)\Delta t}. \quad (9.11)$$

Within each slice, $H(t_j)$ is a sum of two *non-commuting* pieces, so a further Trotter step is needed:

$$e^{-iH(t_j)\Delta t} \approx e^{-i\beta_j H_i} e^{-i\gamma_j H_f}, \quad \gamma_j = \frac{t_j}{T} \Delta t, \quad \beta_j = \left(1 - \frac{t_j}{T}\right) \Delta t. \quad (9.12)$$

Two approximations, not one

Equations (9.11) and (9.12) are both approximations, and I want you to keep them distinct:

- (9.11) discretizes the *time dependence*, with error from holding H fixed across each slice.
- (9.12) is ordinary Trotter error from $[H_i, H_f] \neq 0$, at $\mathcal{O}(\Delta t^2)$ per step by Chapter 7.

Writing either with an equals sign hides a source of error that determines how large p must be. And $H_i = -\sum_i X_i$ and H_f (diagonal) never commute, so the second approximation is always active.

9.3 QAOA

9.3.1 The algorithm

Equation (9.12) is the quantum approximate optimization algorithm[11]. Applying it for $j = 1, \dots, p$ to the state $|+\rangle^{\otimes n}$:

$$|\gamma, \beta\rangle = U_M(\beta_p) U_C(\gamma_p) \cdots U_M(\beta_1) U_C(\gamma_1) H^{\otimes n} |0\rangle^{\otimes n}, \quad (9.13)$$

with the *mixer*

$$H_M = H_i = -\sum_i X_i, \quad U_M(\beta) = e^{-i\beta H_M} = \prod_i R_x(-2\beta). \quad (9.14)$$

The mixer Hamiltonian is the initial Hamiltonian (9.9) itself, sign included, and that sign is forced. The schedule fixes $\beta_j > 0$, and (9.12) evolves under H_i , not under $-H_i$. Taking $H_M = +\sum_i X_i$ with the same positive β_j would apply the inverse mixer in every layer.

Once p and T are chosen, both angle sequences are fixed; nothing is left to tune. Figure 9.3 plots them. The cost angle γ_j grows linearly from layer to layer while the mixer angle β_j shrinks. The early layers are therefore dominated by the mixer and the late ones by the cost Hamiltonian. That ramp is the adiabatic interpolation (9.10), read off one slice at a time.

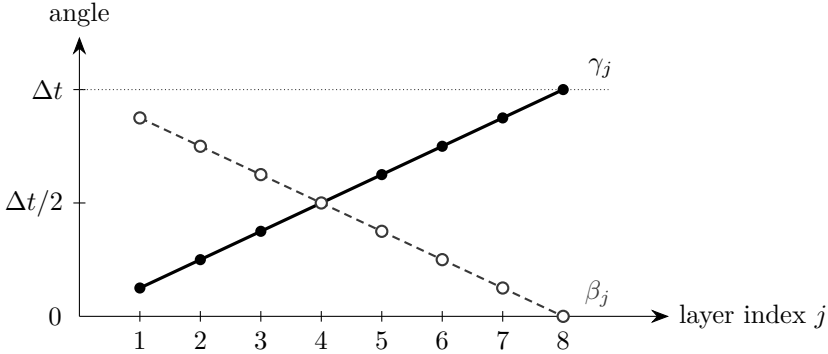


Figure 9.3: The fixed adiabatic schedule of (9.12), drawn for $p = 8$ layers, in units of the slice length $\Delta t = T/p$. Filled circles are the cost angles γ_j and open circles the mixer angles β_j ; they satisfy $\gamma_j + \beta_j = \Delta t$ at every layer. With $t_j = j \Delta t$ the last mixer angle is $\beta_p = 0$, so the final mixer does nothing.

Fixed schedule versus optimized angles

QAOA is usually presented as a *variational* algorithm in which (γ, β) are tuned by a classical optimizer. That is not what we do here. Taking the angles from the adiabatic schedule (9.12) gives a fully specified circuit with no optimization loop, no barren plateaus and a provable guarantee: as $p \rightarrow \infty$ with $\Delta t \rightarrow 0$, the output converges to the exact ground state by the adiabatic theorem. The cost is that p must be large.

Optimizing the angles typically achieves a given approximation quality at much smaller p , which is why the variational form dominates near-term work. But it comes with no guarantee, and the optimization itself is the hard part. The fixed-schedule version is what can be analyzed, so it is what I present.

9.3.2 The circuit

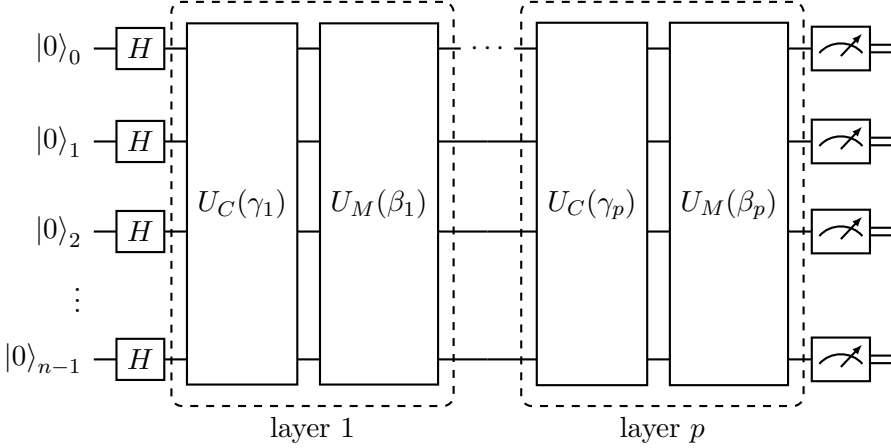


Figure 9.4: The QAOA circuit. Each layer alternates the cost unitary $U_C(\gamma_j)$ with the mixer $U_M(\beta_j)$. Measuring in the computational basis returns a candidate bit string; repeating and keeping the best is the output.

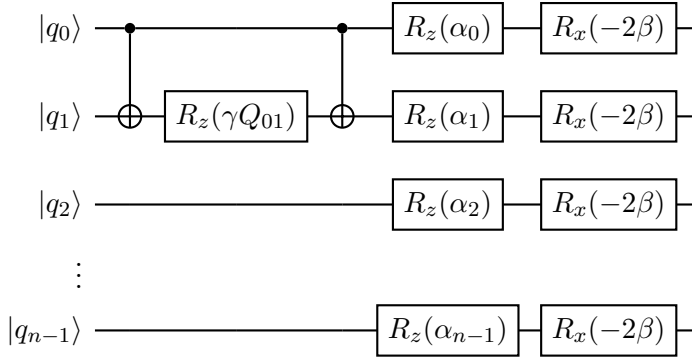


Figure 9.5: One QAOA layer expanded into elementary gates, showing the Z_0Z_1 coupling term. Each Z_iZ_j term of H_C is the CNOT– R_z –CNOT sandwich of Chapter 7, §7.4. Each Z_i term is a single R_z with $\alpha_i = -\gamma(c_i + \sum_j Q_{ij})$, and the mixer is one $R_x(-2\beta)$ per qubit. All the U_C terms commute, so their order is free.

Let us count the gates in Fig. 9.5. Take a QUBO whose quadratic part couples m pairs of variables, so that Q has m non-zero entries above the diagonal; for MaxCut, m is the number of edges and n the number of vertices. The cost layer has one $R_{Z_iZ_j}$ per coupled pair, each two CNOTs and one R_z : $2m$ CNOTs and m rotations. The linear terms add at most n more R_z gates, and for MaxCut none, since their angles vanish as noted after (9.7). The mixer is n gates $R_x(-2\beta)$. One layer therefore costs $2m$ CNOTs and at most $m + 2n$ single-qubit rotations; p layers

cost $2mp$ CNOTs and at most $(m + 2n)p$ rotations, on n qubits with no ancillas. These are gate counts, not depth: $R_{Z_i Z_j}$ gates on disjoint edges run in parallel, so the depth of a cost layer is set by the largest number of edges meeting at one vertex, not by m . In plain words, the circuit is linear in the size of the graph and in the number of layers, and the only thing that can make it expensive is the p that the adiabatic condition demands.

Exercise 9.1

Draw circuits implementing (i) $\prod_{i=1}^3 R_{Z_i}(\alpha)$, (ii) $\prod_{i=1}^3 R_x(\vartheta)$, (iii) $R_{Z_1 Z_2}(\beta)$.

Answer: (i) and (ii) are single-qubit gates in parallel: one layer, depth 1. (iii) is CNOT, $R_z(\beta)$, CNOT, using the parity trick of Chapter 7. Only (iii) needs two-qubit gates.

9.3.3 Implementation guide

Building QAOA

Qubits. Exactly n , one per binary variable. No ancillas. This is one of the few algorithms in this book with no ancilla overhead.

Step 1: build Q and $\mathbf{c}$. Do this classically and test it: for small n , enumerate all 2^n strings and check $\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$ against your original cost function, since bugs here are invisible later. For a maximization, negate Q and $\mathbf{c}$ after this test, once and only once, as the box in §9.1 explains.

Step 2: build H_C via (9.6). Store it as a list of (Pauli string, coefficient) pairs. Test that the diagonal of the assembled matrix reproduces $C(\mathbf{x})$, or $-C(\mathbf{x})$ if you negated in Step 1; for $n \leq 12$ that is a direct check and it catches sign errors immediately.

Step 3: the cost layer. For each $Z_i Z_j$ term emit $\text{CNOT}(i, j)$, $R_z(2\gamma Q_{ij}/2)$ on j , $\text{CNOT}(i, j)$; for each Z_i term emit one R_z . Reuse the `pauli_evolution` routine from Chapter 7; nothing new is needed. *Watch the factor of two:* a term cP evolved for γ needs $R_z(2c\gamma)$. Here $c = Q_{ij}/2$, because (9.6) sums over ordered pairs and the unordered pair $\{i, j\}$ appears in it twice.

Step 4: the mixer. $R_x(-2\beta)$ on every qubit. The sign matters: $H_M = -\sum_i X_i$ and β_j is positive.

Step 5: the schedule. For the fixed-schedule version, set $\gamma_j = (j/p)\Delta t$ and $\beta_j = (1 - j/p)\Delta t$ with $\Delta t = T/p$, and choose T large enough that the adiabatic condition holds. In practice, scan T and watch the approximation ratio saturate.

Step 6: sample and post-process. Run the circuit N_{shots} times, evaluate $C(\mathbf{x})$ *classically* on each measured string, and report the best. You do not need the expectation value $\langle H_C \rangle$ at all; you need the best sample, and reporting the mean understates performance considerably.

Testing QAOA

1. $p = 1, n = 2$: compute the output state by hand and compare.
2. Check that $U_C(\gamma)$ is diagonal with entries $e^{-i\gamma C(\mathbf{x})}$, up to the global phase of the dropped identity term.
3. At fixed T , check the approximation ratio settles down as p grows, and that it rises toward 1 as T grows. Do *not* expect it to increase monotonically in p . On the instance of Fig. 9.1 at $T = 40$ it runs 0.550, 0.667, 0.531, 0.618, 0.660, 0.491 for $p = 1, 2, 4, 8, 16, 32$ and only then reaches 1.000 at $p = 64$. A dip is not by itself a sign that the angles are wrong.
4. Compare against brute force for $n \leq 20$.

The first number in that list is worth understanding rather than just checking. At $p = 1$ the schedule gives $\beta_1 = 0$, so the mixer does nothing, and $U_C(\gamma_1)$ is diagonal, so it changes no measurement probability. The circuit is then $|+\rangle^{\otimes n}$ measured, and the approximation ratio is the average cut of a uniformly random string over the maximum cut. Every edge is cut with probability $1/2$, so the average is half the total weight, 5.5, and $5.5/10 = 0.550$. If your $p = 1$ run gives anything else, the bug is in the schedule, not the circuit.

What QAOA does not promise

There is no proof that QAOA beats the best classical algorithms for MaxCut. For 3-regular graphs, $p = 1$ QAOA achieves an approximation ratio of 0.6924 [11], against 0.8786 for the classical Goemans–Williamson algorithm [14]. QAOA improves with p , but so does the classical competition, and no crossover has been demonstrated.

The honest statement is that QAOA is a well-defined heuristic with a clean adiabatic limit, not a proven speed-up. That is how I present it, and I ask you to present it the same way.

9.4 The Hadamard Test

Before HHL we need a primitive that recurs throughout the rest of the book: estimating $\langle \Phi | U | \Phi \rangle$ for a unitary U and a state $|\Phi\rangle$ we can prepare.

Let us trace it through. The first Hadamard gives $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|\Phi\rangle$. The controlled- U gives $\frac{1}{\sqrt{2}}(|0\rangle|\Phi\rangle + |1\rangle U|\Phi\rangle)$. The second Hadamard gives

$$\frac{1}{2} \left[|0\rangle (\mathbb{I} + U) |\Phi\rangle + |1\rangle (\mathbb{I} - U) |\Phi\rangle \right].$$

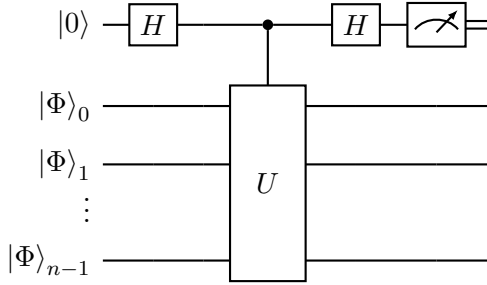


Figure 9.6: The Hadamard test. The ancilla is measured; the system register is untouched. Inserting an $S^\dagger$ on the ancilla between the first Hadamard and the control (the empty column above) yields the imaginary part instead.

Hence

$$\Pr(0) - \Pr(1) = \operatorname{Re} \langle \Phi | U | \Phi \rangle. \quad (9.15)$$

No factor of two, and only the real part

The normalization. Expanding $\Pr(0) = \frac{1}{4} \langle \Phi | (\mathbb{I} + U^\dagger)(\mathbb{I} + U) | \Phi \rangle = \frac{1}{2} (1 + \operatorname{Re} \langle \Phi | U | \Phi \rangle)$ and similarly for $\Pr(1)$ gives (9.15) with *no* factor of 2. Verified numerically on random unitaries: $\Pr(0) - \Pr(1) = 0.25767$ against $\operatorname{Re} \langle \Phi | U | \Phi \rangle = 0.25767$. Writing $\Pr(0) - \Pr(1) = \frac{1}{2} \langle \Phi | U | \Phi \rangle$ is off by a factor of two, an easy slip since the intermediate expressions do carry factors of $\frac{1}{4}$.

Only the real part. The circuit as drawn gives $\operatorname{Re} \langle \Phi | U | \Phi \rangle$. For the imaginary part, apply $S^\dagger$ to the ancilla after the first Hadamard, as in Figure 9.7; then $\Pr(0) - \Pr(1) = \operatorname{Im} \langle \Phi | U | \Phi \rangle$. Two runs are needed for the full complex value.

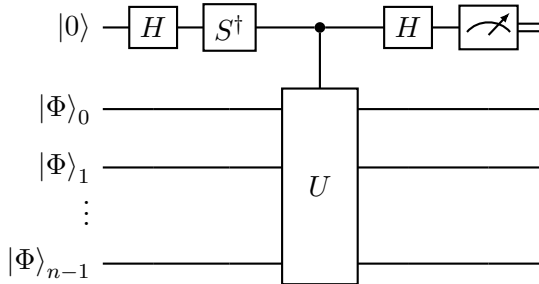


Figure 9.7: The imaginary-part Hadamard test. The only change from Fig. 9.6 is the $S^\dagger$ on the ancilla, sitting between the first Hadamard and the control. The same measurement now returns $\Pr(0) - \Pr(1) = \operatorname{Im} \langle \Phi | U | \Phi \rangle$, so a full complex $\langle \Phi | U | \Phi \rangle$ costs two circuits and two sets of shots.

How many times must the circuit run? Each shot returns 0 or 1; score it as +1 or −1 and the score has mean $\Pr(0) - \Pr(1) = \text{Re} \langle \Phi | U | \Phi \rangle$ by (9.15), and variance $1 - (\text{Re} \langle \Phi | U | \Phi \rangle)^2 \leq 1$. The average of M such scores therefore has standard error at most $1/\sqrt{M}$. To bring that below a target precision ϵ , meaning the additive error in the estimate of $\text{Re} \langle \Phi | U | \Phi \rangle$, we need $M \approx 1/\epsilon^2$ shots. Each shot is one full run of Fig. 9.6: one preparation of $|\Phi\rangle$, one controlled- U , one measurement. Doubling the precision costs four times as many runs.

Cost

Estimating $\text{Re} \langle \Phi | U | \Phi \rangle$ to additive precision ϵ needs $\mathcal{O}(1/\epsilon^2)$ shots, each a full run of the circuit, by the sampling bound just derived. Chapter 10 shows how amplitude estimation reduces this to $\mathcal{O}(1/\epsilon)$ uses of the state preparation and of controlled- U , a quadratic saving that matters whenever the Hadamard test sits inside a loop.

9.5 The HHL Algorithm

9.5.1 The problem

The algorithm of Harrow, Hassidim and Lloyd [16] solves linear systems in a specific and carefully qualified sense.

Given a Hermitian matrix A and a vector $\mathbf{b}$, solve $A\mathbf{x} = \mathbf{b}$. In quantum form: given a circuit preparing $|b\rangle$, produce

$$|x\rangle = \frac{A^{-1} |b\rangle}{\|A^{-1} |b\rangle\|}. \quad (9.16)$$

Singular means *not* unique

If A is **non-singular** the solution is unique. If A *is* singular, the system either has no solution or infinitely many. In that case A^{-1} does not exist, so (9.16) is meaningless. HHL requires A to be invertible, and its runtime degrades as A approaches singularity; that dependence is the condition number κ below.

Writing A in its eigenbasis, $A = \sum_j \lambda_j |u_j\rangle\langle u_j|$, and expanding $|b\rangle = \sum_j b_j |u_j\rangle$,

$$|x\rangle \propto A^{-1} |b\rangle = \sum_j \frac{b_j}{\lambda_j} |u_j\rangle. \quad (9.17)$$

So the task is: find the λ_j , invert them, and reassemble. Those three steps are the algorithm.

9.5.2 The three steps

$$\begin{aligned}
 P\Lambda P^\dagger |x\rangle = |b\rangle &\xrightarrow{\text{QPE}} \Lambda P^\dagger |x\rangle = P^\dagger |b\rangle \\
 &\xrightarrow{\text{rotate}} P^\dagger |x\rangle = \Lambda^{-1} P^\dagger |b\rangle \\
 &\xrightarrow{\text{QPE}^\dagger} |x\rangle = P\Lambda^{-1} P^\dagger |b\rangle.
 \end{aligned}$$

Eigenvalues are extracted by phase estimation (Chapter 4) applied to $U = e^{iAt_0}$, whose eigenvectors are those of A and whose eigenvalue phases encode λ_j . Implementing e^{iAt_0} is Hamiltonian simulation (Chapter 7). *HHL is built entirely from parts we already have.*

The clock register holds c qubits, written with the same letter as the counting register of Chapter 4 because it plays the same role. That scalar c is a register width. It has nothing to do with the QUBO coefficients c_i of §9.1, nor with the Pauli coefficient c in the factor-of-two rule of §9.3.3. The ket subscripts a , c and s below name the ancilla, clock and system registers.

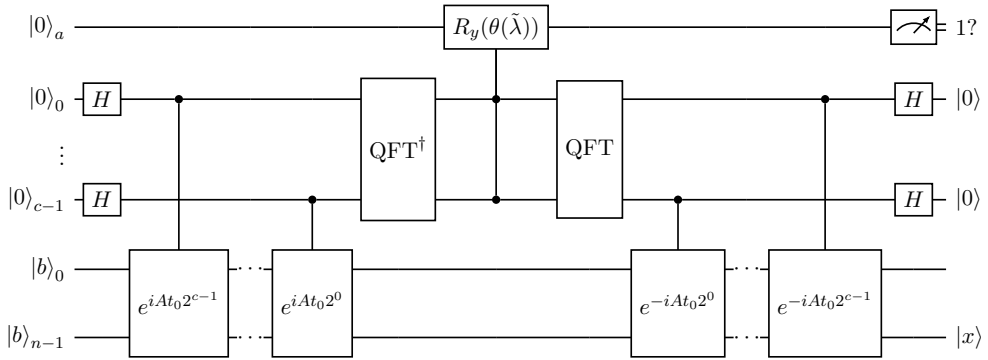


Figure 9.8: The HHL circuit. Top: the ancilla whose measurement heralds success on outcome 1. Middle: the c -qubit clock register, shown with its first and last wires. Bottom: the n -qubit register holding $|b\rangle$ and finally $|x\rangle$. Phase estimation applies a ladder of controlled powers of e^{iAt_0} , one rung per clock wire. Following the convention of Chapter 4, clock qubit q controls $e^{iAt_0 2^k}$ with $k = c - 1 - q$, so the top wire carries the largest power and the bottom wire carries e^{iAt_0} itself. The top wire is therefore the most significant bit of $\tilde{\lambda}_j$, as it is in Fig. 9.9. The two elided columns stand for the remaining $c - 2$ rungs. The inverse QFT turns the accumulated phases into the binary string $\tilde{\lambda}_j$. The controlled rotation writes $1/\lambda$ into the ancilla amplitude, and the mirrored ladder of e^{-iAt_0} powers uncomputes the clock register back to $|0\rangle$.

Step 1: phase estimation

$$|0\rangle_a |0\rangle_c |b\rangle_s \mapsto \sum_j b_j |0\rangle_a |\tilde{\lambda}_j\rangle_c |u_j\rangle_s.$$

Here $|\tilde{\lambda}_j\rangle_c$ is the clock register holding an integer ℓ_j that encodes the eigenvalue. The unitary e^{iAt_0} has eigenvalue $e^{i\lambda_j t_0} = e^{2\pi i \varphi_j}$ with $\varphi_j = \lambda_j t_0 / 2\pi$, and phase estimation with c clock qubits writes $\ell_j = 2^c \varphi_j$, rounded to an integer, into the clock. So the estimate is $\tilde{\lambda}_j = 2\pi \ell_j / (2^c t_0)$, exact when $2^c \varphi_j$ is an integer. The worked examples in this chapter choose t_0 so that it is.

Step 2: controlled rotation

Rotate the ancilla by an angle depending on the clock register:

$$|0\rangle_a \mapsto \sqrt{1 - \frac{C^2}{\tilde{\lambda}_j^2}} |0\rangle_a + \frac{C}{\tilde{\lambda}_j} |1\rangle_a, \quad (9.18)$$

where C is a constant chosen so that $C \leq \lambda_{\min}$, ensuring the argument stays in $[-1, 1]$. This is $R_y(\theta) |0\rangle = \cos(\theta/2) |0\rangle + \sin(\theta/2) |1\rangle$ with $\sin(\theta/2) = C/\tilde{\lambda}_j$, that is, a rotation by the angle $\theta(\tilde{\lambda}_j) = 2 \arcsin(C/\tilde{\lambda}_j)$ written in the ancilla of Fig. 9.8. The state becomes

$$\sum_j b_j \left(\sqrt{1 - \frac{C^2}{\tilde{\lambda}_j^2}} |0\rangle_a + \frac{C}{\tilde{\lambda}_j} |1\rangle_a \right) |\tilde{\lambda}_j\rangle_c |u_j\rangle_s.$$

Step 3: uncompute and measure

Apply $\text{QPE}^\dagger$, uncomputing the clock register back to $|0\rangle$:

$$\sum_j b_j \left(\sqrt{1 - \frac{C^2}{\tilde{\lambda}_j^2}} |0\rangle_a + \frac{C}{\tilde{\lambda}_j} |1\rangle_a \right) |0\rangle_c |u_j\rangle_s.$$

Measure the ancilla. On outcome $|1\rangle$, which occurs with probability $\sum_j |b_j|^2 C^2 / \lambda_j^2$, the system register collapses to

$$|x\rangle \propto \sum_j \frac{b_j}{\lambda_j} |u_j\rangle,$$

which is (9.17). On outcome $|0\rangle$ the run has failed and must be repeated.

How often does a run fail? Since $C \leq \lambda_{\min} \leq \lambda_j$, every term $|b_j|^2 C^2 / \lambda_j^2$ is at most $|b_j|^2$, and the sum is at most 1. Taking $C = \lambda_{\min}$, the largest value allowed, every term is at least $|b_j|^2 \lambda_{\min}^2 / \lambda_{\max}^2$, so the success probability is at least $1/\kappa^2$, where

$\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number of A . Plain repetition therefore needs $\mathcal{O}(\kappa^2)$ runs on average before the ancilla reads 1; amplitude amplification, Chapter 6, brings that down to $\mathcal{O}(\kappa)$ repetitions of the whole circuit. Keep this factor in mind; it is one of the two powers of κ in the runtime below.

Which outcome heralds success?

The useful amplitude is the one carrying C/λ_j . Whether that sits on $|0\rangle$ or $|1\rangle$ is a convention, fixed by whether the rotation angle is θ or $\pi - \theta$. The two must be consistent between the rotation formula and the post-selection rule. It is easy to write $\cos \theta = C/\lambda$ and then post-select on $|1\rangle$, which discards exactly the branch you wanted. I use (9.18) and post-select on $|1\rangle$ throughout.

9.5.3 Implementing the $1/\lambda$ rotation

The clock register holds $\tilde{\lambda}$ as a bit string; we need an R_y rotation by an angle depending on $1/\tilde{\lambda}$. The standard trick uses a cascade of controlled rotations with binary-weighted angles.

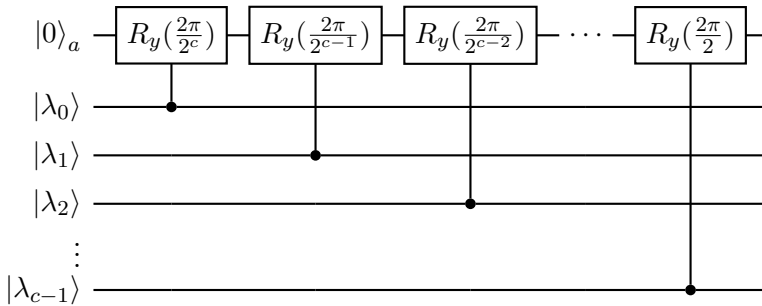


Figure 9.9: Binary-weighted controlled rotations. The clock register holds $\tilde{\lambda} = \lambda_0\lambda_1\cdots\lambda_{c-1}$ with λ_0 the most significant bit on the top wire, the same ordering as Fig. 9.8. Clock wire q controls a rotation of $2\pi/2^{c-q}$, so the bit of place value 2^k controls $2\pi/2^{k+1}$ and the accumulated angle is $\theta = \pi \left(\lambda_{c-1} + \frac{\lambda_{c-2}}{2} + \frac{\lambda_{c-3}}{4} + \cdots \right)$. The *least* significant bit, on the bottom wire, is weighted most heavily.

This circuit is exact only for powers of two

Writing $\lambda = \lambda_0 2^{c-1} + \lambda_1 2^{c-2} + \cdots + \lambda_{c-1} 2^0$ with λ_0 the most significant bit, the cascade produces $\theta = \pi(\lambda_{c-1} + \lambda_{c-2}/2 + \lambda_{c-3}/4 + \cdots)$, that is, π times the digits of λ read backwards. Verified numerically for $c = 4$:

λ	θ	$\theta\lambda$	λ	θ	$\theta\lambda$
1	3.1416	3.1416	3	4.7124	14.14
2	1.5708	3.1416	5	3.9270	19.64
4	0.7854	3.1416	6	2.3562	14.14
8	0.3927	3.1416	7	5.4978	38.48

First, for λ a power of two, $\theta\lambda$ is exactly constant, so $\theta \propto 1/\lambda$. For every other λ it is not. The mapping is a bit-reversal, which coincides with reciprocal only on powers of two.

Second, even where $\theta \propto 1/\lambda$ holds, that is the *small-angle approximation* to the angle required, which is $\theta = 2 \arcsin(C/\lambda)$. At $C/\lambda = 0.5$ the error is 4.5%; at $C/\lambda = 0.125$ it is 0.26%.

Practical implementations therefore do not use this cascade. They compute $1/\tilde{\lambda}$ into a second register using a quantum arithmetic circuit (Newton–Raphson division), then apply a controlled R_y using the standard binary decomposition of Chapter 4. That is more expensive and more accurate. The cascade is worth showing because it explains the idea; it should not be presented as the implementation.

For the small demonstrations that fill the literature there is a third option, and it is the one they almost always use. Enumerate the clock values and attach the exact angle to each. Figure 9.10 shows that form: one multi-controlled R_y per clock value $\tilde{\lambda}$, with the control pattern matching the bits of $\tilde{\lambda}$ and the angle set to $\theta_{\tilde{\lambda}} = 2 \arcsin(C/\tilde{\lambda})$. It is exact for every clock value. Its cost is one gate per non-zero clock value, so $2^c - 1$ multi-controlled rotations for a clock of c qubits, and each of those has c controls, which by §2.3.4 of Chapter 2 unfolds into $2c - 2$ Toffolis and one controlled- R_y , with $c - 1$ ancillas that return to $|0\rangle$. The gate count doubles with every clock qubit added, so this form is practical only for a clock register of two or three qubits.

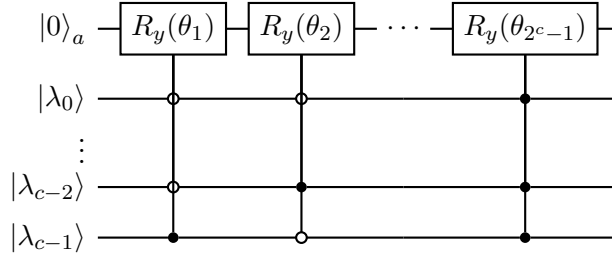


Figure 9.10: Exact rotation by enumeration. Each column applies $R_y(\theta_{\tilde{\lambda}})$ with $\theta_{\tilde{\lambda}} = 2 \arcsin(C/\tilde{\lambda})$, controlled on the clock register holding precisely the bit pattern of $\tilde{\lambda}$. Filled circles are controls on 1, open circles controls on 0. The bit ordering is that of Fig. 9.9, with λ_0 most significant on the top wire, so the wires drawn are the most significant one and the two least significant ones. The three columns shown are $\tilde{\lambda} = 1$, $\tilde{\lambda} = 2$ and $\tilde{\lambda} = 2^c - 1$. Unlike the cascade of Fig. 9.9 this is exact for every clock value, but it needs one multi-controlled rotation per value and so does not scale.

9.5.4 Cost, and four caveats

Runtime

For a d -sparse Hermitian A (at most d non-zero entries per row) of dimension $N = 2^n$, with condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ (largest over smallest eigenvalue in absolute value) and target accuracy ϵ in the output state:

conjugate gradient : $\mathcal{O}(Nd\kappa \log(1/\epsilon))$ classical arithmetic operations,

HHL : $\mathcal{O}(\log(N) d^2 \kappa^2 / \epsilon)$ gates and oracle queries.

The dependence on N drops from linear to logarithmic. That is the “exponential speed-up”. The $1/\epsilon$ factor in the second line is not the last word: Childs, Kothari and Somma give a linear-system algorithm whose dependence on the target accuracy is $\text{poly} \log(1/\epsilon)$, using the polynomial machinery of Chapter 10 in place of the rotation above [7].

I want to assemble both lines rather than quote them, and the classical one first, because the textbook bound for conjugate gradient is not the one in the box. For a symmetric positive-definite system, conjugate gradient shrinks the error by a constant factor every $\mathcal{O}(\sqrt{\kappa})$ iterations, so it needs $\mathcal{O}(\sqrt{\kappa} \log(1/\epsilon))$ iterations to reach relative error ϵ . Each iteration is one product of A with a vector, which for a d -sparse matrix is $\mathcal{O}(Nd)$ multiplications and additions. The total is $\mathcal{O}(Nd\sqrt{\kappa} \log(1/\epsilon))$ arithmetic operations, with $\sqrt{\kappa}$, not κ . But HHL assumes only that A is Hermitian. Its eigenvalues may have either sign, and on such an indefinite system conjugate gradient has no convergence guarantee. The standard remedy is to solve the normal equations $A^\dagger A \mathbf{x} = A^\dagger \mathbf{b}$ instead, which for Hermitian A read

$A^2\mathbf{x} = A\mathbf{b}$. The matrix A^2 is positive definite, so conjugate gradient applies, but its eigenvalues are the λ_j^2 and its condition number is κ^2 . The iteration count becomes $\mathcal{O}(\sqrt{\kappa^2} \log(1/\epsilon)) = \mathcal{O}(\kappa \log(1/\epsilon))$, each iteration now costing two sparse products, and that is the κ in the box: the figure Harrow, Hassidim and Lloyd quote as the like-for-like classical comparison [16]. If you know your A is positive definite, the classical side improves to $\sqrt{\kappa}$ and the gap narrows by that much.

The quantum line is assembled from the three steps. Phase estimation must resolve the smallest eigenvalue, of size $\lambda_{\min} = \lambda_{\max}/\kappa$, to relative precision ϵ , because an absolute error δ in λ becomes a relative error δ/λ in $1/\lambda$ and that is worst for the smallest eigenvalue. Resolving a phase to that precision takes a total simulation time $t_0 = \mathcal{O}(\kappa/\epsilon)$ in units where $\lambda_{\max} = 1$. Simulating a d -sparse Hamiltonian for time t_0 with the sparse-matrix oracles of Chapter 7 costs $\mathcal{O}(\log(N) d^2 t_0)$ gates and oracle queries, up to logarithmic factors, which is $\mathcal{O}(\log(N) d^2 \kappa/\epsilon)$. Every run succeeds with probability at least $1/\kappa^2$, as shown after Step 3, so amplitude amplification repeats the whole circuit $\mathcal{O}(\kappa)$ times. Multiplying the two gives $\mathcal{O}(\log(N) d^2 \kappa^2/\epsilon)$. What the count includes: elementary gates and queries to the oracles that return the entries of A , with each call to the circuit preparing $|b\rangle$ counted as one unit. What it hides: polylogarithmic factors, the arithmetic for $1/\tilde{\lambda}$, and everything in the four caveats below. In plain words: doubling the condition number quadruples the cost, doubling the precision doubles it, and doubling N adds one qubit and a constant to the cost.

The four caveats: read before quoting the speed-up

The comparison above is not like-for-like, and every serious treatment of HHL says so[1].

1. State preparation. HHL assumes $|b\rangle$ can be prepared efficiently. For a general vector this costs $\Omega(N)$, destroying the speed-up on its own. It works when $|b\rangle$ has structure, or comes from another quantum process.

2. Hamiltonian simulation. You must implement e^{iAt_0} efficiently, which needs A sparse and row-computable (Chapter 7) or given as a block encoding (Chapter 10).

3. Condition number. The runtime carries κ^2 . If κ grows with N , as it commonly does, the advantage erodes. Preconditioning is as necessary here as classically.

4. Readout. HHL does not output $\mathbf{x}$; it outputs the *state* $|x\rangle$, and reading all N components costs $\Omega(N)$ samples. The algorithm is useful only when you want a scalar summary such as $\langle x|M|x\rangle$ for some observable M , which is where the Hadamard test of §9.4 comes in.

None of this makes HHL uninteresting. It makes HHL a *subroutine* with preconditions, not a drop-in replacement for a linear solver.

9.5.5 Implementation guide

Building HHL

Registers. One ancilla; c clock qubits with $c \approx \log_2 \kappa + \log_2(1/\epsilon)$, the first term to separate $\lambda_{\max}$ from $\lambda_{\min}$ and the second to give each eigenvalue $\log_2(1/\epsilon)$ further bits; $n = \log_2 N$ system qubits. Plus whatever the arithmetic for $1/\lambda$ needs.

Step 0: rescale. Choose t_0 so that the eigenvalues of $At_0/2\pi$ lie in $[0, 1)$; otherwise phase estimation wraps around and returns garbage. In practice set $t_0 = 2\pi/\lambda_{\max}$ times a safety factor, bounding $\lambda_{\max}$ cheaply with the Gershgorin circle theorem.

Step 1: build e^{iAt_0} . Decompose A into Pauli strings (Chapter 7, §7.4) and Trotterize. For the canonical 2×2 test cases A is already a sum of one or two Paulis and the exponential is exact.

Step 2: phase estimation exactly as in Chapter 4, with controlled $e^{iAt_0 2^k}$. Remember to precompute the powers rather than repeating the gate 2^k times.

Step 3: the rotation. For teaching, hard-code the angles: for each of the 2^c clock values, precompute $\theta = 2 \arcsin(C/\lambda)$ and apply a multi-controlled R_y , as in Fig. 9.10. That is $2^c - 1$ gates, exponential in the clock width, but exact, and it is what almost every published small-scale demonstration does. Say so.

Step 4: uncompute with QPE[†] and measure the ancilla.

Testing HHL, and what the demonstrations really show

Testing.

1. Pick A with known eigenvalues that are exact powers of two. Then phase estimation is exact and any error is yours.
2. Check the clock register holds the right $\tilde{\lambda}_j$ before adding the rotation.
3. Check the clock register returns to $|0\rangle$ after $\text{QPE}^\dagger$. If it does not, the state is still entangled and $|x\rangle$ will be wrong.
4. Compare the post-selected state against $A^{-1}|b\rangle$ normalized, computed classically. Compare *up to global phase*.
5. Check the success probability against $\sum_j |b_j|^2 C^2 / \lambda_j^2$.

A warning about demonstrations. The widely reproduced 2×2 HHL example uses an A whose eigenvalues are 1 and 2. They are chosen so that a 2-qubit clock register represents them exactly and the rotation angles can be hard-coded. It is a useful teaching circuit and not an implementation of the algorithm, much like the $N = 15$ Shor demonstrations of Chapter 5.

9.6 Summary

- Both algorithms in this chapter apply a function to the spectrum of an operator: $e^{-i\gamma\lambda}$ for optimization, $1/\lambda$ for linear systems. Chapter 10 generalizes this.
- A QUBO $C(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$ becomes an Ising Hamiltonian via $x_i \rightarrow (\mathbb{I} - Z_i)/2$; the result is diagonal, weight-2, and cheap to exponentiate.
- MaxCut: $Q = -W$, $c_i = \sum_j W_{ij}$. Verified on a five-vertex instance.
- Adiabatic evolution and QAOA both seek a ground state, so they minimize. A maximization such as MaxCut is handed $-H_C$, which means negating Q and $\mathbf{c}$ once.
- The adiabatic theorem gives an evolution time $T = \mathcal{O}(1/\Delta_{\min}^2)$, hence a circuit depth proportional to T ; for hard problems the gap typically closes exponentially.
- Discretizing adiabatic evolution involves *two* approximations: time-slicing and Trotter.
- QAOA is that discretization. With the adiabatic schedule it needs no classical optimizer and converges provably as $p \rightarrow \infty$; with optimized angles it does better at small p but loses the guarantee.

- One QAOA layer costs $2m$ CNOTs and at most $m + 2n$ single-qubit rotations for m coupled pairs on n qubits, with no ancillas.
- QAOA has no proven advantage over classical MaxCut algorithms: $p = 1$ gives 0.6924 against Goemans–Williamson’s 0.8786.
- Hadamard test: $\Pr(0) - \Pr(1) = \text{Re} \langle \Phi | U | \Phi \rangle$, with no factor of two, real part only, $S^\dagger$ for the imaginary part; $\mathcal{O}(1/\epsilon^2)$ shots for additive precision ϵ .
- HHL is phase estimation, a $1/\lambda$ rotation, and uncomputation. Runtime $\mathcal{O}(\log(N)d^2\kappa^2/\epsilon)$ gates and oracle queries, against $\mathcal{O}(Nd\kappa \log(1/\epsilon))$ classical arithmetic operations for conjugate gradient on the normal equations.
- The binary cascade for $1/\lambda$ is exact only when λ is a power of two, and is a small-angle approximation to $2 \arcsin(C/\lambda)$ even then.
- HHL’s speed-up is conditional: it needs efficient state preparation, sparse simulation, bounded κ , and a scalar readout.

9.7 End-of-Chapter Problems

- 9.1** For the graph of Fig. 9.1, enumerate all 2^5 assignments and confirm the maximum cut weight is 10. How many distinct optimal partitions are there?
- 9.2** Derive (9.4) from (9.3), and verify $\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$ reproduces the cut weight for three different assignments.
- 9.3** Derive the Ising form (9.6) from (9.1) using $x_i \rightarrow (\mathbb{I} - Z_i)/2$, keeping track of the constant term.
- 9.4** Show that H_C is diagonal in the computational basis, and explain why this means all its terms commute.
- 9.5** Write the number-partitioning problem (split a set of integers into two subsets with equal sums) as a QUBO, and give the corresponding Ising Hamiltonian.
- 9.6** Show that $|+\rangle^{\otimes n}$ is the ground state of $H_i = -\sum_i X_i$, and compute its energy.
- 9.7** For a two-level system with $H(t) = (1-s)X + sZ$, $s = t/T$, compute the gap $\Delta(s)$ and find $\Delta_{\min}$. What runtime does (9.8) predict?
- 9.8** Explain why (9.11) and (9.12) are separate approximations, and estimate the total error in terms of p .
- 9.9** Draw the full $p = 1$ QAOA circuit for a triangle graph with unit weights, in elementary gates. How many CNOTs?

- 9.10** For the triangle graph, compute $\langle H_C \rangle$ analytically as a function of (γ, β) at $p = 1$, and find the optimum.
- 9.11** Show that the Hadamard test gives $\text{Re} \langle \Phi | U | \Phi \rangle$ by computing $\text{Pr}(0)$ and $\text{Pr}(1)$ explicitly, and confirm there is no factor of two.
- 9.12** Show that inserting $S^\dagger$ on the ancilla changes the output to $\text{Im} \langle \Phi | U | \Phi \rangle$.
- 9.13** For $A = \begin{pmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{pmatrix}$ and $|b\rangle = |0\rangle$, compute the eigenvalues, $|x\rangle$, and the HHL success probability with $C = \lambda_{\min}$.
- 9.14** Show that the binary cascade of Fig. 9.9 gives $\theta\lambda = \pi$ exactly when λ is a power of two, and compute $\theta\lambda$ for $\lambda = 3, 5, 6, 7$.
- 9.15** Compute the relative error between the exact angle $2 \arcsin(C/\lambda)$ and its small-angle form $2C/\lambda$, for $C/\lambda = 0.5, 0.25$ and 0.125 .
- 9.16** Explain why the clock register must return to $|0\rangle$ after $\text{QPE}^\dagger$, and what goes wrong if it does not.
- 9.17** For each of the four HHL caveats, give a concrete example of a problem where it is not a difficulty.
- 9.18** (*Looking ahead to Chapter 10.*) Both algorithms in this chapter apply a function of a matrix: $e^{-i\gamma H_C}$ and A^{-1} . Write each as $f(\lambda)$ for an explicit f , and say which is a polynomial.

Research Problems and Projects

The problems above have settled answers. The three below do not. Each is sized for several weeks of work by a strong MS student or a beginning PhD student. Each ends in something you could show other people: a scaling figure with error bars, a resource table, or a piece of software somebody else can run. Read the cited papers first, and expect your first measurement to disagree with the bound you were expecting.

- R9.1 How much depth does the fixed schedule actually cost?** The fixed-schedule QAOA of §9.3 converges as $p \rightarrow \infty$ because the adiabatic theorem says it must [12]. Nobody can tell you in advance how large p has to be for a given instance. Measure it. Take random 3-regular MaxCut instances at $n = 6, 8, 10, 12, 14$ and compute $\Delta_{\min}$ for each by exact diagonalization of $H(t)$ along the path. Then run the fixed schedule of (9.12) at increasing p , and record the smallest p that reaches an approximation ratio of 0.8, 0.9 and 0.95. *Deliverable:* a plot of that p against n and against $1/\Delta_{\min}^2$, with fitted exponents and instance-to-instance spread shown honestly. Repeat the

measurement with angles optimized numerically at each p , so that the two curves sit on one axis. The interesting question is not which curve is lower, since the optimized one will be, but whether the ratio between them grows with n or settles down. Report whichever you find. Statevector simulation to $n = 14$ is comfortable on a laptop; $n = 16$ and beyond needs care with memory.

R9.2 Audit one published HHL speed-up claim against the four caveats.

The caveats of §9.5 are stated in the abstract [1]; applying them to a specific paper is a different and more instructive exercise. Start from the original complexity claim [16] and the improved precision dependence of Childs, Kothari and Somma [7]. Reconstruct from the papers themselves exactly what each one assumes about the input model, the condition number, and the readout. Then pick one downstream application paper that quotes an exponential speed-up from a linear-solver subroutine, and work out which of the four caveats its claim actually rests on. *Deliverable:* a comparison table with one row per paper, giving the input access model, the dependence on κ , ϵ and d , the ancilla count, and the exact quantity that is read out at the end. Add a short verdict on the application paper: does the speed-up survive, does it survive under an extra assumption you can state precisely, or does it not survive. Several such claims have been withdrawn or weakened by classical algorithms that imitate the same access model, so check whether that has already happened to the paper you chose before you write your verdict.

R9.3 Three ways to rotate by $1/\lambda$, costed properly. Section 9.5 gives three constructions for the controlled rotation: the binary cascade of Fig. 9.9, the exact enumeration of Fig. 9.10, and a reciprocal computed by quantum arithmetic followed by a controlled R_y . Implement all three against a common interface, in a simulator that reports gate counts. Test them inside a complete HHL run [16] on a family of Hermitian A with condition number $\kappa = 2, 4, 8, 16$ and clock size $c = 3, \dots, 8$. *Deliverable:* a resource table giving CNOT count, T count after decomposition, and ancilla count for each construction at each c . Alongside it, plot the fidelity between the post-selected output and the exact $A^{-1}|b\rangle$ as a function of c . State the crossover: the value of c above which the arithmetic circuit is cheaper than enumeration, and whether the cascade is ever the right choice at all. This project is engineering rather than open science, but the numbers are not collected anywhere convenient, and a clean released implementation with its resource table is genuinely useful to other people.

Chapter 10

Quantum Singular Value Transformation Algorithms

Nine chapters have produced a collection of algorithms that look unrelated [13, 22]. Grover search reflects; phase estimation Fourier transforms; Hamiltonian simulation Trotterizes; HHL inverts eigenvalues. In this chapter I want to show you that they are one algorithm.

The claim

Take a unitary that contains a matrix A as a sub-block. One technique then implements $P(A)$ for almost any polynomial P of definite parity with $|P| \leq 1$ on $[-1, 1]$. That technique is the *quantum singular value transformation*, and it costs $\deg(P)$ uses of that unitary, plus one single-qubit rotation per use. Then:

Choose $P =$	and you recover
sign function	Grover search / amplitude amplification
e^{-ixt} (Jacobi–Anger)	Hamiltonian simulation
$1/x$	linear systems (HHL, and better)
rectangle function	phase estimation, eigenvalue filtering
$\sqrt{x}$, $x^{-1/2}$, ...	Gibbs states, ground-state projection

What you should be able to do after this chapter

- State what a block encoding is and construct one for a Hermitian matrix.
- Explain quantum signal processing on a single qubit and why it produces Chebyshev polynomials.
- Explain qubitization: why the walk operator decomposes into two-dimensional invariant subspaces.
- State the QSVT theorem and use it to recover Grover and HHL.
- Implement amplitude estimation without a phase-estimation register.
- Explain why qubitization beats Trotter for high-accuracy simulation.

10.1 Block Encoding

Quantum computers apply unitaries. Most matrices we care about are not unitary: Hamiltonians, data matrices, the A of a linear system. Block encoding is the bridge.

Definition

A unitary U acting on $a + n$ qubits is an (α, a) -block encoding of an n -qubit operator A if

$$A = \alpha \left(\langle 0|^{\otimes a} \otimes \mathbb{I} \right) U \left(|0\rangle^{\otimes a} \otimes \mathbb{I} \right), \quad (10.1)$$

that is, if A/α sits in the top-left block of U :

$$U = \begin{pmatrix} A/\alpha & \cdot \\ \cdot & \cdot \end{pmatrix}.$$

The unspecified blocks are whatever unitarity requires; they are never used.

Because U is unitary, $\|A/\alpha\| \leq 1$. So α is a normalization, and I want you to treat it as a cost from the start. Every complexity in this chapter is counted in uses of U , and α enters each of them: the circuit sees A/α rather than A , so simulating e^{-iAt} means simulating A/α for the longer time αt , and inverting A means inverting a matrix whose smallest eigenvalue has shrunk by α . Part of the design problem is keeping α small. Figure 10.1 shows the layout of U , and Fig. 10.2 shows how the encoded matrix is actually applied to a state.

Example 10.1

Construct a block encoding of a Hermitian A with $\|A\| \leq 1$.

Solution. Take one ancilla qubit and

$$U = \begin{pmatrix} A & \sqrt{\mathbb{I} - A^2} \\ \sqrt{\mathbb{I} - A^2} & -A \end{pmatrix}. \quad (10.2)$$

Verified numerically for $A = \begin{pmatrix} 0.4 & 0.2 \\ 0.2 & -0.3 \end{pmatrix}$: U is unitary, U is Hermitian, $U^2 = \mathbb{I}$, and $(\langle 0| \otimes \mathbb{I})U(|0\rangle \otimes \mathbb{I}) = A$ exactly. That self-inverse property is not accidental and will matter in §10.3.

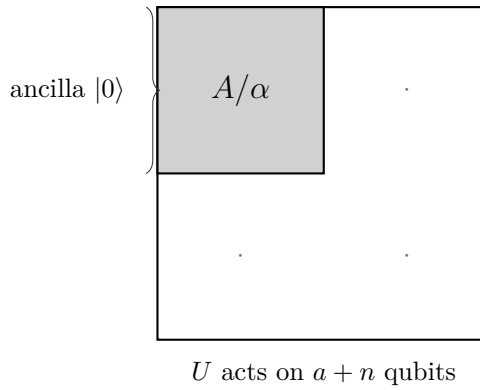


Figure 10.1: A block encoding. The matrix of interest, rescaled, occupies the block of U selected by preparing the ancillas in $|0\rangle$ and post-selecting on $|0\rangle$. Everything else in U is padding required by unitarity.

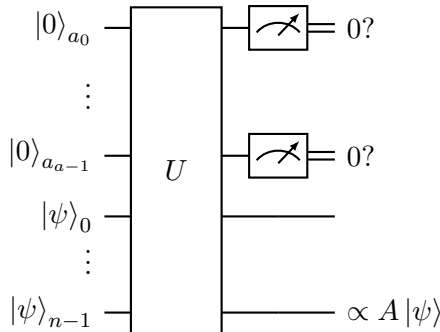


Figure 10.2: Using a block encoding. Preparing the ancillas in $|0\rangle$, applying U and post-selecting on all-zero leaves $A|\psi\rangle/\alpha$ on the system register. Compare the LCU circuit of Chapter 7. LCU is one recipe for *building* a block encoding.

Where block encodings come from

Three standard sources, all already in this book:

- **Linear combination of unitaries** (Chapter 7). For $A = \sum_i c_i U_i$, the circuit $O^\dagger V O$ is a block encoding with $\alpha = \|c\|_1$. This is the workhorse for Hamiltonians written as sums of Pauli strings.
- **Sparse-access oracles** (Chapter 7). A d -sparse row-computable matrix with entries of magnitude at most one admits a block encoding with $\alpha = d$, the row sparsity. Everywhere else in this chapter d is the degree of the QSP polynomial; where the sparsity is meant it is named.
- **Density matrices**. If you can prepare ρ , you can block-encode it with a swap-test-like circuit.

Everything downstream in this chapter treats the block encoding as a black box, so any of these plugs in unchanged.

10.2 Quantum Signal Processing

QSVT is easiest to understand from the one-qubit case first. There it is called *quantum signal processing*[20], and it is a statement about products of 2×2 rotations.

10.2.1 The signal operator

Fix $x \in [-1, 1]$ and define

$$W(x) = \begin{pmatrix} x & i\sqrt{1-x^2} \\ i\sqrt{1-x^2} & x \end{pmatrix}, \quad (10.3)$$

the *signal operator*, an X -rotation by angle $-2 \arccos x$. Verified unitary for all x tested.

Now interleave $W(x)$ with Z -rotations by *fixed* angles ϕ_k . Fixed here means independent of x . The result is the QSP sequence

$$U_\phi(x) = e^{i\phi_0 Z} \prod_{k=1}^d \left[W(x) e^{i\phi_k Z} \right]. \quad (10.4)$$

Figure 10.3 draws that sequence as a one-qubit circuit. Reading it left to right reverses the order of the symbols in (10.4), which is the usual convention clash between operator products and circuit diagrams. The same applies to Fig. 10.6 and (10.10) below.

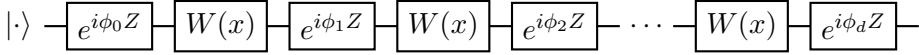


Figure 10.3: The quantum signal processing sequence: d uses of the signal operator $W(x)$ interleaved with $d + 1$ fixed Z -rotations. The ϕ_k do not depend on x ; that is the whole point.

QSP theorem

For any $\phi \in \mathbb{R}^{d+1}$,

$$U_\phi(x) = \begin{pmatrix} P(x) & i Q(x)\sqrt{1-x^2} \\ i Q^*(x)\sqrt{1-x^2} & P^*(x) \end{pmatrix}, \quad (10.5)$$

where P and Q are polynomials with

- (i) $\deg P \leq d$ and $\deg Q \leq d - 1$;
- (ii) P has parity $d \bmod 2$, and Q has parity $(d - 1) \bmod 2$;
- (iii) $|P(x)|^2 + (1 - x^2)|Q(x)|^2 = 1$ for all $x \in [-1, 1]$.

Conversely, *any* pair (P, Q) satisfying these conditions is achievable by some choice of ϕ . Both parities are needed. Dropping the one on Q makes the converse false: $(P, Q) = (x^2, 1 + ix)$ at $d = 2$ satisfies (i) and (iii), and no phase sequence produces it.

Verified

For random phase sequences at $d = 1, 2, 3, 5$, the top-left entry of (10.4) was sampled at 400 points in $[-0.99, 0.99]$ and fitted by a degree- d polynomial:

d	fit residual	parity	$ P(-x) - (-1)^d P(x) $	$ P ^2 + (1 - x^2) Q ^2 - 1 $
1	3.3×10^{-16}		0	6.7×10^{-16}
2	7.9×10^{-16}		0	6.7×10^{-16}
3	1.0×10^{-15}		0	1.3×10^{-15}
5	1.1×10^{-15}		0	1.3×10^{-15}

All three conditions hold to machine precision.

10.2.2 Why Chebyshev polynomials appear

Setting every $\phi_k = 0$ reduces (10.4) to $W(x)^d$, and

$$\left[W(x)^d\right]_{00} = T_d(x), \quad (10.6)$$

the degree- d Chebyshev polynomial of the first kind. Verified for $d = 1, \dots, 6$ at several values of x : for instance $d = 3$, $x = 0.3$ gives $-0.792 = T_3(0.3)$, and $d = 5$, $x = -0.7$ gives $0.6709 = T_5(-0.7)$.

The reason is immediate once you write $x = \cos \theta$. Then (10.3) reads $W = \cos \theta \mathbb{I} + i \sin \theta X = e^{i\theta X}$, so $W^d = e^{id\theta X}$, whose diagonal entry is $\cos(d\theta) = T_d(\cos \theta)$. This is the same operator as the X -rotation by $-2 \arccos x$ named above, because R_x carries a factor of $\frac{1}{2}$ in its exponent. *Chebyshev polynomials are what repeated rotation looks like in the computational basis.*

Why this matters for cost

Chebyshev polynomials are the optimal basis for uniform polynomial approximation on $[-1, 1]$, so “ d uses of the signal operator” buys exactly “degree- d Chebyshev approximation”. The degree d is the cost: it is the number of uses of the signal operator (later, of the block encoding) and, up to one, the number of single-qubit phase rotations. For the functions we care about, once the domain is fixed, the Chebyshev error falls geometrically with d , so error ϵ needs $d = \mathcal{O}(\log(1/\epsilon))$. Each extra digit of precision costs a fixed number of extra uses. That is why QSVT gets $\log(1/\epsilon)$ scaling where Trotter gets $\text{poly}(1/\epsilon)$.

10.2.3 Finding the phases

Given a target polynomial P , computing the ϕ that produces it is a classical problem. It is not trivial. Naive root-finding is numerically unstable beyond degree ~ 30 . But the problem is solved: modern optimization-based methods handle degrees in the thousands in double precision, in classical computation time that grows only polynomially with the degree d . None of that time is quantum.

Practical note

Phase-factor computation is done *once, classically, offline*, and the resulting angles are hard-coded into the circuit. Do not attempt to compute them inside a quantum algorithm.

If you implement QSVT, I would use an existing phase-finding package[10] rather than write your own. The numerical conditioning is the hard part and has been worked out. Verify the output by evaluating (10.4) classically at a few hundred points and comparing against your target polynomial. That is exactly the test in the box above.

10.3 Qubitization

QSP acts on one qubit. To lift it to a matrix we need the block encoding to behave, on each eigenvector of A , like the single-qubit signal operator. That is qubitization [21].

10.3.1 The walk operator

Let U be the Hermitian, self-inverse block encoding (10.2) of a Hermitian A , and let

$$Z_{\Pi} = (2|0\rangle\langle 0| - \mathbb{I}) \otimes \mathbb{I} \quad (10.7)$$

be the reflection about the ancilla being $|0\rangle$. Define the *walk operator*

$$W = Z_{\Pi} U. \quad (10.8)$$

Figure 10.4 draws the circuit. It is one call to the block encoding followed by one reflection on the ancilla register, and nothing else. Repeating that pair d times is the whole of the QSVT circuit at phases $\phi_k = \pi/2$, and it is what the numerical test in §10.4 checks.

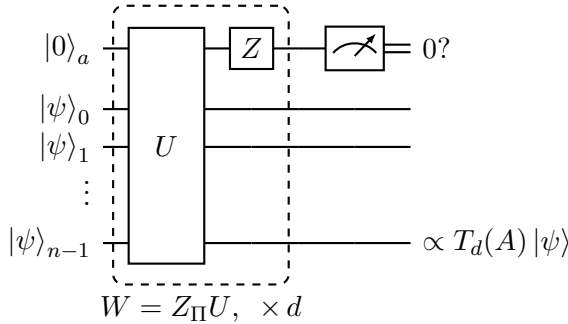


Figure 10.4: The walk operator, drawn for the one-ancilla encoding of (10.2). There Z_{Π} is a single Z gate on the ancilla. With a ancillas it becomes the reflection about $|0\rangle^{\otimes a}$, which is the $X^{\otimes a} C^{a-1}(Z) X^{\otimes a}$ core of the Chapter 6 diffuser up to an overall sign. Running the dashed block d times and post-selecting the ancilla on 0 leaves $T_d(A) |\psi\rangle$ on the system register, up to normalization.

Verified

For $A = \begin{pmatrix} 0.4 & 0.2 \\ 0.2 & -0.3 \end{pmatrix}$ with eigenvalues $\lambda = -0.35311, 0.45311$:

$$\text{eig}(W) = \{0.4531 \pm 0.8915i, -0.3531 \pm 0.9356i\},$$

matching $e^{\pm i \arccos \lambda_j}$ exactly. The real parts *are* the eigenvalues of A . That is (10.9) restated.

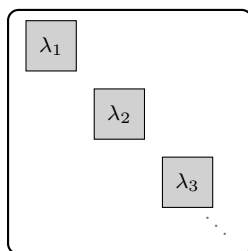
Qubitization

W block-diagonalizes into two-dimensional invariant subspaces, one for each eigenvalue λ_j of A . Within the subspace attached to λ_j , W acts as a rotation by $\arccos \lambda_j$, so its eigenvalues are

$$e^{\pm i \arccos \lambda_j}. \quad (10.9)$$

Figure 10.5 is the picture I want you to hold on to.

W on 2^{a+n} dimensions



each block is a
 2×2 rotation by
 $\arccos \lambda_j$

Figure 10.5: Qubitization. The walk operator is block-diagonal, with one two-dimensional block per eigenvalue of A . Inside each block it is the QSP signal operator $W(\lambda_j)$ up to a diagonal change of basis, so the two have the same eigenvalues. A QSP phase sequence applied to the ancilla therefore acts, simultaneously and independently, on every eigenvalue.

Why the name

The 2^{a+n} -dimensional problem has been reduced to a family of independent *qubits*, one per eigenvalue. That is what “qubitization” means: an arbitrary block-encoded matrix has been turned into a direct sum of single-qubit signal operators, to which the QSP theorem applies verbatim.

10.4 The Quantum Singular Value Transformation

QSVT theorem (Hermitian case)

Let U be a block encoding of a Hermitian A with $\|A\| \leq 1$, and let P be a polynomial with

- (i) $\deg P \leq d$;
- (ii) P of parity $d \bmod 2$;
- (iii) $|P(x)| \leq 1$ on $[-1, 1]$, and moreover there is a polynomial Q with $\deg Q \leq d - 1$, parity $(d - 1) \bmod 2$, and $|P(x)|^2 + (1 - x^2)|Q(x)|^2 = 1$ on $[-1, 1]$.

Then there is a sequence of phases $\phi \in \mathbb{R}^{d+1}$ such that the circuit

$$U_\phi = \Pi_{\phi_0} \prod_{k=1}^d \left[U^{(\dagger)} \Pi_{\phi_k} \right] \quad (10.10)$$

is a block encoding of $P(A)$, that is,

$$(\langle 0| \otimes \mathbb{I}) U_\phi (|0\rangle \otimes \mathbb{I}) = P(A).$$

Here $U^{(\dagger)}$ alternates U and $U^\dagger$ along the product, as Fig. 10.6 draws, and

$$\Pi_\phi = e^{i\phi(2\Pi - \mathbb{I})}, \quad \Pi = (|0\rangle\langle 0|)^{\otimes a} \otimes \mathbb{I}, \quad (10.11)$$

is the ancilla phase rotation. So the circuit is d uses of the block encoding interleaved with $d + 1$ phases. Note $\Pi_0 = \mathbb{I}$ and $\Pi_{\pi/2} = iZ_\Pi$, with Z_Π the reflection of (10.7).

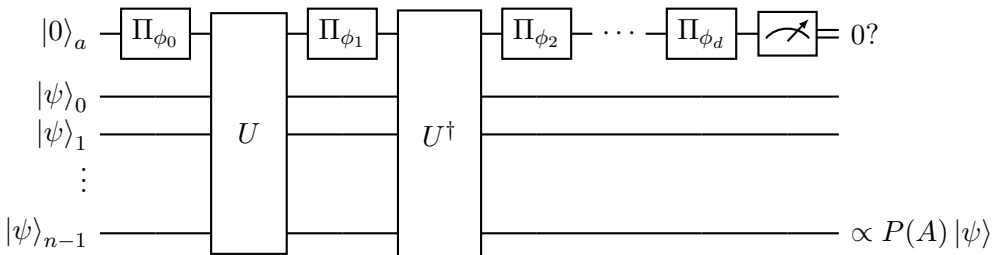


Figure 10.6: The QSVT circuit. Alternating uses of the block encoding U and its inverse, interleaved with fixed single-ancilla phase rotations Π_{ϕ_k} , implement a block encoding of $P(A)$. The alternation between U and $U^\dagger$ is what handles non-Hermitian A ; for the Hermitian, self-inverse encoding of (10.2) the two coincide.

Parity and $|P| \leq 1$ are not enough

Condition (iii) is a genuine extra hypothesis, not a restatement of $|P| \leq 1$. Setting $x = \pm 1$ kills the Q term and forces $|P(\pm 1)| = 1$, so $P(x) = x/2$, which is odd with $|P| \leq 1$, is achievable at *no* degree. Numerically, $|P(1)| = 1$ for every phase sequence tested, so no phase sequence at any degree gets closer than 0.5 to $x/2$ in the maximum norm; the gap is forced at $x = 1$.

Most polynomials one actually wants, including the filter and the rescaled inverse below, are *real* and do not satisfy (iii) on their own. For those, use the standard real-polynomial version. If P is real of degree d , has parity $d \bmod 2$ and $|P(x)| \leq 1$ on $[-1, 1]$, then some $\phi \in \mathbb{R}^{d+1}$ gives a block $\tilde{P}(A)$ whose real part is $P(A)$. One extra ancilla, two Hadamards and controlled applications of U_ϕ and $U_{-\phi}$ then block-encode $P(A)$ itself, because $(\tilde{P} + \tilde{P}^*)/2 = P$. Verified at $d \leq 3$ for $P = x/2, x^2, T_3, 0.8 - 1.6x^2$ and $0.9x - 0.4x^3$.

Verified: the Chebyshev case

With every phase set to $\phi_k = \pi/2$ we have $\Pi_{\pi/2} = iZ_\Pi$, and (10.10) collapses to $i^{d+1}W^dZ_\Pi$. The reflection acts as the identity on $|0\rangle$, so the projected block should be $T_d(A)$ up to the overall phase i^{d+1} . Measured error $\|(\langle 0|W^d|0\rangle) - T_d(A)\|$:

d	error	d	error	d	error
1	1.6×10^{-16}	3	5.0×10^{-16}	5	1.2×10^{-15}
2	2.7×10^{-16}	4	4.4×10^{-16}	6	7.4×10^{-16}

Exact to machine precision. Any other phase list replaces T_d by the polynomial that list encodes. Setting every phase to zero instead gives $\Pi_0 = \mathbb{I}$ and hence U^d , which for the self-inverse encoding of (10.2) is $\mathbb{I}$ or U , not W^d .

10.4.1 Recovering the algorithms of earlier chapters

The circuit never changes. Only P does. Figure 10.7 draws the three choices of P that this book has already used, on the same axes, so that the differences between Grover search, eigenvalue filtering and matrix inversion become differences between three curves. Each of the three boxes below reads one of those curves.

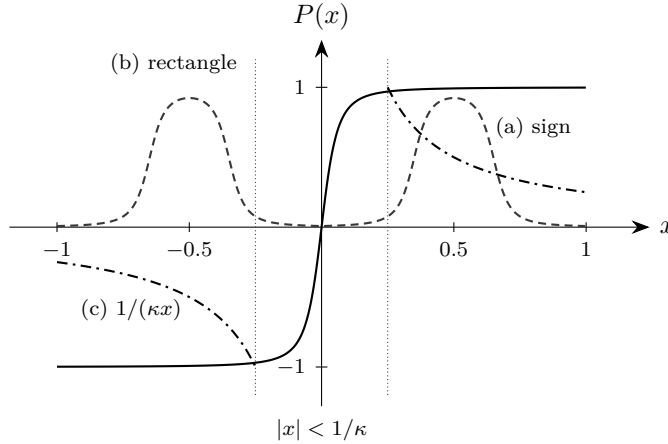


Figure 10.7: Three choices of P on $[-1, 1]$, drawn for $\kappa = 4$. Curve (a) is a smoothed sign function: it maps everything above a threshold to ≈ 1 and gives amplitude amplification. Curve (b) is a rectangle supported on the symmetric pair of windows $\pm[0.35, 0.65]$: it keeps the eigenvectors whose eigenvalues lie in either window and gives an eigenvalue filter. The pair is forced by the parity condition, since an even P cannot separate λ from $-\lambda$. Curve (c) is $1/x$ rescaled by $1/\kappa$ so that $|P| \leq 1$: it gives a linear solver, and it is defined only outside the dotted band $|x| < 1/\kappa$, where the true $1/x$ exceeds the bound. The same circuit implements all three.

Grover search

Amplitude amplification (Chapter 6) rotates the initial state toward the good subspace. In QSVT language, the Grover operator is the walk operator for a block encoding of the projector onto the good subspace. In that language r Grover iterations implement the Chebyshev polynomial T_{2r+1} , which is curve (a) of Fig. 10.7 in its raw, unsmoothed form.

The optimal polynomial for search is an approximation to the *sign function*, which amplifies everything above a threshold to ≈ 1 . That is why the success probability of Chapter 6 was $\sin^2((2r+1)\theta)$, a Chebyshev-like oscillation. It is also why overshooting made things worse: you were evaluating a polynomial past its useful range.

Phase estimation and eigenvalue filtering

Choosing P to approximate a rectangle function on the symmetric pair of windows $\pm[\mu - \delta, \mu + \delta]$ gives an *eigenvalue filter*, which is curve (b) of Fig. 10.7. Applying it to a state projects onto the eigenvectors whose eigenvalues lie in either window.

The window has to come in a symmetric pair. A one-sided rectangle has no definite parity, so it is not a legal target for the theorem above. An even P keeps λ and $-\lambda$ together; separating them needs a second filter or a shift of the spectrum.

This does the job of phase estimation (Chapter 4) without a clock register at all. It trades the c clock qubits of Chapter 4 for a longer sequence on fewer qubits. The trade is qubits for depth: a rectangle of half-width δ with edges sharp to ϵ needs a polynomial of degree about $(1/\delta) \log(1/\epsilon)$, and that is the number of uses of the block encoding. A bare c always means that register width in this chapter; the LCU coefficients c_j of §10.1 always carry a subscript. That trade is the basis of the amplitude-estimation method in §10.5.

Matrix inversion

Choosing $P(x) \approx 1/x$ on the domain $[1/\kappa, 1]$ gives a block encoding of A^{-1} , hence a linear solver. That is curve (c) of Fig. 10.7. This is §10.7, and it is asymptotically better than HHL.

10.5 Application 1: Qubit-Efficient Amplitude Estimation

10.5.1 The problem, and why it matters

Given a unitary $\mathcal{A}$ with

$$\mathcal{A}|0\rangle = \sqrt{1-g} |\psi_{\text{bad}}\rangle + \sqrt{g} |\psi_{\text{good}}\rangle, \quad (10.12)$$

estimate g . Chapter 6 showed this is phase estimation on the Grover operator. The symbols are the ones used there: g is the good-state *probability*, so $\sqrt{g}$ is the amplitude, and $|\psi_{\text{good}}\rangle$, $|\psi_{\text{bad}}\rangle$ are its $|w\rangle$ and $|w^\perp\rangle$.

Why this is the most practically important algorithm here

Amplitude estimation gives a *quadratic* speed-up for Monte Carlo integration. To learn g to additive error ϵ it needs $\mathcal{O}(1/\epsilon)$ applications of $\mathcal{A}$ and $\mathcal{A}^\dagger$, the queries, where classical sampling of the same circuit needs $\mathcal{O}(1/\epsilon^2)$ shots. Ten times the precision costs ten times the queries here and a hundred times the samples classically. Monte Carlo is everywhere: derivative pricing, risk measures, reliability analysis, high-dimensional integrals in physics. Unlike Shor, the speed-up is only quadratic. Unlike Grover on unstructured search, the problems are real and the input is generated rather than loaded. Most credible near-to-medium-term resource estimates for commercially relevant quantum advantage run through this algorithm.

10.5.2 The textbook version, and its cost

Canonical amplitude estimation applies phase estimation to the Grover operator $\mathcal{Q}$, which rotates by 2θ with $\sin^2 \theta = g$. It needs $c \approx \log_2(1/\epsilon)$ clock qubits, controlled powers $\mathcal{Q}^{2^k}$, and an inverse quantum Fourier transform. Here ϵ is the target additive error in g . The count is assembled as in Chapter 4: clock qubit k controls $\mathcal{Q}^{2^k}$, so the c clock qubits together apply $\mathcal{Q}$ a total of $2^c - 1 \approx 1/\epsilon$ times, every one of them in controlled form, plus $\mathcal{O}(c^2)$ gates for the inverse QFT.

That is expensive on early fault-tolerant hardware: controlled- $\mathcal{Q}$ is much costlier than $\mathcal{Q}$, and the clock register plus QFT is a large overhead for a single number.

10.5.3 Removing the phase estimation

The observation is simple. After r Grover iterations,

$$\Pr[\text{good}] = \sin^2((2r + 1)\theta). \quad (10.13)$$

Measure at several values of r , then fit θ by maximum likelihood[28]. There is no clock register, no controlled- $\mathcal{Q}$ and no QFT. There is only $\mathcal{Q}$ applied repeatedly, and a measurement. That is the whole circuit, and Fig. 10.8 is all of it.

Let me count what this costs, because the count is the whole argument for the method. The resource is queries: applications of $\mathcal{A}$ or $\mathcal{A}^\dagger$, each counted once. A circuit at depth r applies $\mathcal{A}$ once and then $\mathcal{Q}$ r times, and each $\mathcal{Q}$ contains one $\mathcal{A}$ and one $\mathcal{A}^\dagger$, so it makes $1 + 2r$ queries. With N_{shots} repetitions at $r = 0$ and at each $r = 2^k$ for $k = 0, \dots, M$, the total is

$$N_{\text{queries}} = N_{\text{shots}} \left[1 + \sum_{k=0}^M (1 + 2^{k+1}) \right] = N_{\text{shots}} (M + 2^{M+2}) \approx 4N_{\text{shots}} 2^M.$$

The deepest circuit multiplies θ by $2^{M+1} + 1$ before the sine is taken. So a shot-noise error of order $1/\sqrt{N_{\text{shots}}}$ in its success probability becomes an error

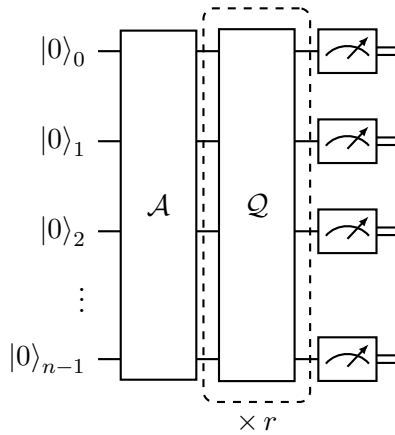


Figure 10.8: Maximum-likelihood amplitude estimation. The circuit is just state preparation followed by r Grover iterations and a measurement. Compare the quantum counting circuit of Chapter 6: the entire clock register and inverse QFT are gone. The schedule of r values and the classical likelihood fit replace them.

Verified

With $g = 0.13$ (so $\theta = 0.368863$), the success probabilities that (10.13) predicts at $r \in \{0, 1, 2, 4, 8, 16, 32\}$ are:

r	0	1	2	4	8	16	32
Pr[good] (exact)	0.130	0.800	0.927	0.031	0.0002	0.147	0.838

These are the exact values, not measured frequencies. Drawing 100 shots at each r from them and maximizing the likelihood returns $\hat{g} = 0.129036$ against a true 0.13. That is an absolute error of $\epsilon = 9.6 \times 10^{-4}$, using 13,300 queries in total, where a query is one application of $\mathcal{A}$ or $\mathcal{A}^\dagger$; the 6,300 Grover iterations account for 12,600 of them. Classical sampling would need $g(1 - g)/\epsilon^2 \approx 1.2 \times 10^5$ shots for the same error.

of order $2^{-M}/\sqrt{N_{\text{shots}}}$ in θ ; the shallower circuits mainly serve to pick the right branch. Reaching additive error ϵ in g therefore takes $M \approx \log_2(1/\epsilon)$ and $N_{\text{queries}} \approx 4N_{\text{shots}}/\epsilon$, with N_{shots} a constant of your choosing, of order 100. The error falls as $1/N_{\text{queries}}$: doubling the number of queries halves the error, where doubling the number of classical samples improves it only by $\sqrt{2}$. The qubit count is what $\mathcal{A}$ needs, and the depth of the longest circuit is about $2^{M+1} \approx 2/\epsilon$ queries. In the Verified box of this section, $M = 5$ and $N_{\text{shots}} = 100$ gave 13,300 queries and an error of 10^{-3} , consistent with this estimate.

The likelihood is multimodal

Equation (10.13) is periodic in θ , so a single r cannot determine g . Above, $r = 16$ gives $\text{Pr} = 0.147$, close to the $r = 0$ value, for a completely different reason. Figure 10.9 shows the mechanism.

The schedule must break that degeneracy. An exponential schedule $r = 0, 1, 2, 4, 8, \dots, 2^M$ works and is the standard choice. Here M is the largest exponent in the schedule. Each r resolves one more bit, sequentially rather than in superposition. A linear schedule is easier to analyze but needs more total queries.

Do not fit with a local optimizer started from a single point. Evaluate the log-likelihood on a grid over $\theta \in (0, \pi/2)$ first, then refine.

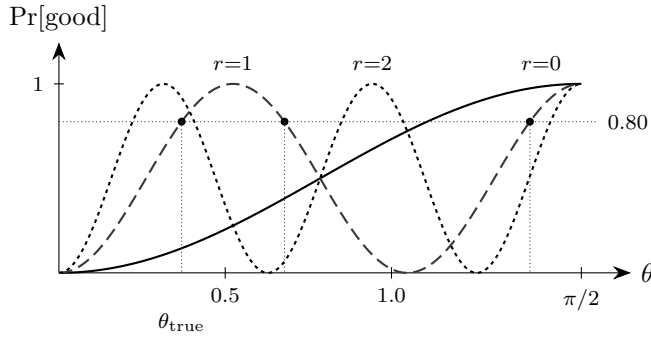


Figure 10.9: Why one value of r is not enough. The curves are $\sin^2((2r+1)\theta)$ for $r = 0, 1$ and 2 . A single run at $r = 1$ that returns a success rate of 0.80 is consistent with all three marked values of θ , only one of which is the truth. Measuring at a second value of r kills two of them, because the other curves take different values there. That is what the schedule of r values buys, and it is the sequential version of what the clock register did in superposition.

10.5.4 Implementation guide

Building maximum-likelihood amplitude estimation

Qubits. Exactly what $\mathcal{A}$ needs, plus the oracle ancillas. *No clock register.* This is the entire point.

What you must supply. $\mathcal{A}$ (state preparation), $\mathcal{A}^\dagger$, and $\mathcal{S}_{\text{good}}$ (mark the good subspace). Then $\mathcal{Q} = \mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_{\text{good}}$ exactly as in Chapter 6, §6.4. Unlike textbook amplitude estimation, you need no *controlled* version of it.

Schedule. Let r_k run over $0, 1, 2, 4, \dots, 2^M$ with $M \approx \log_2(1/\epsilon)$, ϵ the target error in g , and take N_{shots} per value (100 is a reasonable starting point). Keep the $r = 0$ point: it is the raw sample and costs one query.

The fit. With h_k successes out of N_{shots} at r_k ,

$$\log L(\theta) = \sum_k \left[h_k \log \sin^2((2r_k+1)\theta) + (N_{\text{shots}} - h_k) \log \cos^2((2r_k+1)\theta) \right],$$

maximized over $\theta \in (0, \pi/2)$; then $\hat{g} = \sin^2 \hat{\theta}$. Clip the probabilities away from 0 and 1 before taking logarithms.

Testing the amplitude estimator

1. Check $\mathcal{Q}$ rotates by 2θ : simulate $\Pr[\text{good}]$ against r and compare with (10.13).
2. Run the estimator on a known g and confirm the error falls as $1/N_{\text{queries}}$, not $1/\sqrt{N_{\text{queries}}}$. If you see the square-root rate, your schedule is not resolving the phase.
3. Try g near 0 and near 1; both are harder and expose bugs in the likelihood clipping.

10.6 Application 2: Trotter-Free Simulation via Qubitization

10.6.1 The idea

Chapter 7 simulated e^{-iHt} by Trotterization, with error $\mathcal{O}(t\Delta t)$ for time step Δt , and hence a step count polynomial in $1/\epsilon$, where ϵ is the operator-norm error of the simulated evolution. Qubitization does better: choose the polynomial

$$P(x) \approx e^{-ixt} \tag{10.14}$$

and apply QSVT to a block encoding of H .

One step is needed before that works. Since $e^{-ixt} = \cos(xt) - i \sin(xt)$ is neither even nor odd, it has no definite parity, and §10.4 demands one. The fix is to build the two halves separately. Approximate the even part $\cos(xt)$ by an even polynomial and the odd part $\sin(xt)$ by an odd one. Both halves are real, so each reaches the circuit through the real-polynomial version of §10.4 rather than through condition (iii) directly. Give each its own QSVT circuit, then add the two with a one-ancilla LCU. The sum comes out as $\frac{1}{2}e^{-ixt}$, since the LCU averages its two branches. One round of amplitude amplification removes the factor of two. Skipping the split is a common and silent bug: the best even polynomial misses e^{-2ix} by at least 0.909 on $[-1, 1]$ and the best odd one by at least 1.000.

The classical approximation theory is the *Jacobi–Anger expansion*

$$e^{-ixt} = J_0(t) + 2 \sum_{k=1}^{\infty} (-i)^k J_k(t) T_k(x), \quad (10.15)$$

with J_k the Bessel function of the first kind. It splits along the parity line for free: the even- k terms sum to $\cos(xt)$ and the odd- k terms to $-i \sin(xt)$, each verified to 10^{-13} . So the two circuits are read off the one expansion. Because the coefficients $J_k(t)$ decay super-exponentially once k exceeds roughly $e|t|/2$, truncating at

$$K = \mathcal{O}\left(t + \frac{\log(1/\epsilon)}{\log \log(1/\epsilon)}\right) \quad (10.16)$$

suffices.

Let me spell out what (10.16) is counting, since it is the central cost of the chapter. K is the truncation order of the series, hence the degree of the polynomial, hence the number of uses of the block encoding in one QSVT circuit; it is also, up to one, the number of single-ancilla phase rotations in that circuit. The symbol t is the evolution time and ϵ is the maximum error of the truncated polynomial on $[-1, 1]$, which becomes the operator-norm error of the simulated e^{-iHt} . The two terms come from the two regimes of the Bessel coefficients: up to $k \approx e|t|/2$ they are of order one and every term must be kept, which gives the t ; beyond that they fall like $(et/2k)^k$, so reaching ϵ needs $k \log k \approx \log(1/\epsilon)$ more terms, which gives the second term, and the $\log \log$ in its denominator is the small bonus from the k^k decay. I will mostly quote the simpler bound $K = \mathcal{O}(t + \log(1/\epsilon))$. In plain words, doubling the evolution time doubles the circuit, and each extra decimal digit of accuracy adds a fixed handful of uses of the block encoding, independent of t .

Verified

Truncating (10.15) at order K and measuring the maximum error over $x \in [-1, 1]$:

K	$t = 2$	$t = 5$	$t = 10$
5	2.5×10^{-3}	3.0×10^{-1}	1.3×10^0
10	4.6×10^{-8}	7.3×10^{-4}	3.1×10^{-1}
20	1.0×10^{-15}	6.8×10^{-12}	6.1×10^{-6}
30	1.0×10^{-15}	1.6×10^{-15}	5.3×10^{-13}

The order needed for 10^{-10} accuracy is 13, 19 and 28 for $t = 2, 5, 10$. It grows *additively* with t , and only logarithmically with $1/\epsilon$. Compare Chapter 7, where reaching 10^{-10} for $H = X + Z$ at $t = 10$ by first-order Trotter would need about 10^{12} steps.

Table 10.1 sets the two methods side by side. Read the first row carefully, because the two entries count different things. The Trotter entry counts Trotter steps for a product formula of order $2k$, with $k = 1$ the symmetric formula of Chapter 7; each step costs one exponential per term of H . The qubitization entry counts uses of the block encoding of H , and each use costs whatever the block encoding costs, which for an LCU of L Pauli terms is of order L controlled Pauli strings plus the state-preparation circuit. The symbols are the evolution time t and the operator-norm error ϵ . The qubitized query complexity is optimal: it matches the known lower bound for Hamiltonian simulation [21]. In plain words, Trotter pays a power of $1/\epsilon$ for accuracy, and a higher-order formula only makes that power smaller; qubitization pays a logarithm.

Table 10.1: Trotter versus qubitization for Hamiltonian simulation. Qubitization is asymptotically optimal, in the sense that it matches the known lower bound. But it carries ancilla and classical-precomputation overheads that make Trotter preferable for small problems.

	Trotter (Ch. 7)	Qubitization
Query complexity	$\mathcal{O}(t^{1+1/2k}/\epsilon^{1/2k})$	$\mathcal{O}(t + \log(1/\epsilon))$
Dependence on ϵ	polynomial	logarithmic
Ancillas	none	block-encoding ancillas
Needs	the terms of H	a block encoding of H
Classical pre-processing	term grouping	phase factors
Best for	short times, low accuracy	long times, high accuracy

α is not free

The query complexity above is in uses of the *block encoding*, and the relevant time is αt , not t , where α is the block-encoding normalization. For a Hamiltonian written as $\sum_j c_j P_j$ and encoded by LCU, $\alpha = \|c\|_1 = \sum_j |c_j|$. So the honest count is $\mathcal{O}(\alpha t + \log(1/\epsilon))$ uses of U : doubling α lengthens the circuit exactly as doubling t would.

For molecular Hamiltonians $\|c\|_1$ can be enormous, thousands of Hartree for a modest molecule, so a naive block encoding can wipe out the asymptotic advantage. Reducing $\|c\|_1$ by better factorizations of the Hamiltonian is an active and quite practical research area. In my view it is where most of the real engineering in this subject now sits.

10.6.2 Implementation guide**Building a qubitized simulator**

Step 1: block-encode H . If $H = \sum_j c_j P_j$ (Chapter 7, §7.4; Chapter 8 for chemistry), use LCU: O prepares $\sum_j \sqrt{|c_j|/\|c\|_1} |j\rangle$ and $V = \sum_j |j\rangle\langle j| \otimes \text{sign}(c_j) P_j$ selects. Chapter 7 assumed positive coefficients, but chemistry Hamiltonians have negative ones everywhere. Dropping the signs block-encodes $\sum_j |c_j| P_j / \|c\|_1$, a different operator. For complex c_j use $e^{i \arg c_j}$. Record $\alpha = \|c\|_1$; all costs below are in units of it.

Step 2: rescale. QSVT needs $\|H/\alpha\| \leq 1$, which the block encoding gives automatically; the evolution time to target is αt .

Step 3: get the phase factors. Compute the Jacobi–Anger coefficients $J_k(\alpha t)$ classically and truncate at K from (10.16). Split by parity: even- k terms give $\cos(\alpha t x)$, odd- k terms give $-i \sin(\alpha t x)$, with x standing for H/α . Run the phase-finding routine *twice* offline, once per half.

Step 4: build the circuit. Two QSVT circuits, each K uses of the block encoding alternating with ancilla phase rotations per Fig. 10.6, added by a one-ancilla LCU with a Hadamard either side; a single circuit cannot work, since e^{-ixt} has no definite parity. The LCU delivers $\frac{1}{2}e^{-iHt}$, so amplify or post-select. Total: a constant times K uses of U , the two halves, the real-part trick and the amplification round each contributing a factor of two or three, on $a + 2$ ancillas plus the system.

Testing a qubitized simulator

1. Verify the block encoding: $(\langle 0| \otimes \mathbb{I})U(|0\rangle \otimes \mathbb{I}) = H/\alpha$ as a dense matrix, for $n \leq 6$.
2. Verify the walk eigenvalues are $e^{\pm i \arccos(\lambda_j/\alpha)}$.
3. Verify W^d projects to $T_d(H/\alpha)$. This is the test in §10.4, and it isolates qubitization bugs from phase-factor bugs.
4. Only then compare the full circuit against `expm(-1j*H*t)`.

In my experience steps 1 to 3 are where the bugs are. Step 4 rarely fails on its own.

10.7 Application 3: Linear Systems Beyond HHL

10.7.1 Inversion as polynomial approximation

Chapter 9's HHL runs in $\mathcal{O}(\log(N)d^2\kappa^2/\epsilon)$ elementary gates and oracle calls. Here $N = 2^n$ is the dimension of A , d is its row sparsity, $\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number, and ϵ is the error in the normalized output state. The $\log(N)d^2$ is the price of one Hamiltonian simulation step. The κ^2 comes from combining phase estimation (which costs κ to resolve the smallest eigenvalue) with post-selection (which succeeds with probability $\sim 1/\kappa^2$, so needs κ rounds of amplification). The $1/\epsilon$ is the clock-register resolution.

QSVT removes both. Approximate $1/x$ directly by an odd polynomial on the domain where the eigenvalues live,

$$x \in \left[-1, -\frac{1}{\kappa}\right] \cup \left[\frac{1}{\kappa}, 1\right], \quad (10.17)$$

and apply it. No phase estimation, no clock register, no eigenvalue readout.

Verified

Chebyshev fits of $1/x$ on (10.17), to relative accuracy 10^{-3} :

κ	required odd degree	$\kappa \log(1/\epsilon)$
4	33	28
8	65	55
16	127	111

The degree grows *linearly* in κ , matching $\mathcal{O}(\kappa \log(1/\epsilon))$. That is the depth of one circuit. The query count also carries the amplification rounds of Step 4, as the next box records.

Here is where the degree comes from. The reciprocal is smooth on the domain (10.17) but its nearest singularity, at $x = 0$, sits only $1/\kappa$ away from the domain's inner edge. A Chebyshev-type series for such a function converges geometrically, with a ratio that approaches one as the singularity approaches, so the error after degree d is roughly $e^{-d/\kappa}$. Setting that equal to ϵ gives $d \approx \kappa \log(1/\epsilon)$; a more careful bookkeeping, which also accounts for the size of $1/x$ near the edge, gives $d = \mathcal{O}(\kappa \log(\kappa/\epsilon))$, and the measured degrees in the box track that slightly larger expression. Here d is the polynomial degree and hence the number of uses of the block encoding of A in one circuit, κ is the condition number and ϵ the relative error of the polynomial against $1/x$. In plain words, doubling the condition number doubles the circuit, while each extra digit of accuracy adds a fixed number of uses per unit of κ .

The improvement

The rows count calls to the routine that gives access to A , the block encoding for QSVT and the simulation step e^{iAt_0} for HHL, together with calls to the preparation of $|b\rangle$; the $\log(N)d^2$ cost of one such call is common to all rows and is stripped off.

HHL (Ch. 9)	$\mathcal{O}(\kappa^2/\epsilon)$
QSVT + amplitude amplification	$\mathcal{O}(\kappa^2 \log(1/\epsilon))$
QSVT + <i>variable-time</i> amplification	$\mathcal{O}(\kappa \log(1/\epsilon))$

Polynomial in $1/\epsilon$ becomes logarithmic [7]: halving the error doubles HHL's cost and adds a fixed number of queries to QSVT's. That gain is the one the recipe below delivers: Step 4 wraps a degree- $\mathcal{O}(\kappa \log(1/\epsilon))$ circuit in $\mathcal{O}(\kappa)$ rounds, so the queries multiply to $\kappa^2 \log(1/\epsilon)$. At $\kappa = 16$ that is degree 127 times 16 rounds, about 2000 block-encoding queries.

Removing the second factor of κ as well needs *variable-time* amplitude amplification, which this chapter does not develop.

The four caveats survive

QSVT improves the *algorithm*. It does not touch the four caveats of Chapter 9, §9.5. You still need efficient state preparation of $|b\rangle$ and an efficient block encoding of A . You still need a bounded condition number, and a scalar readout rather than the full solution vector.

A faster linear solver whose input you cannot load and whose output you cannot read is still not a drop-in replacement for a classical solver. Improving $1/\epsilon$ to $\log(1/\epsilon)$ is real and worth having; it does not change the preconditions.

10.7.2 Implementation guide

Building the QSVT linear solver

Step 1: bound κ . You need $\lambda_{\min}$ to set the approximation domain. Underestimating it makes the polynomial blow up between the two intervals; overestimating it silently truncates part of the spectrum. Gershgorin bounds are the usual cheap route.

Step 2: build the polynomial. Fit an odd polynomial to $1/x$ on (10.17), then *rescale* so that $|P| \leq 1$ on $[-1, 1]$. QSVT requires this. The rescaling costs a factor $1/\kappa$ in the output amplitude, which is what amplitude amplification later recovers.

Step 3: phase factors, offline. Degrees in the hundreds are routine; use an established package.

Step 4: amplify. Wrap the whole thing in $\mathcal{O}(\kappa)$ rounds of amplitude amplification (Chapter 6, §6.4) to raise the success probability from $\mathcal{O}(1/\kappa^2)$ to $\mathcal{O}(1)$. Each round runs the whole degree- d circuit and its inverse once, so the query count is the degree times the rounds.

Testing.

1. Evaluate your polynomial classically on (10.17) and confirm the relative error against $1/x$. Do this before any quantum code.
2. Confirm $|P(x)| \leq 1$ on all of $[-1, 1]$, including the forbidden gap $(-1/\kappa, 1/\kappa)$. This is the constraint people forget.
3. For $n \leq 6$, build the QSVT circuit densely and compare its projected block against A^{-1} rescaled.
4. Compare the final state against $A^{-1}|b\rangle$ normalized, up to global phase.

10.8 Summary

- A *block encoding* embeds A/α in the top-left block of a unitary. LCU and sparse-access oracles both produce them.
- *Quantum signal processing*: interleaving d uses of the signal operator $W(x)$ with $d + 1$ fixed Z -rotations produces a degree- d polynomial P of parity $d \bmod 2$, a Q of parity $(d - 1) \bmod 2$, and $|P|^2 + (1 - x^2)|Q|^2 = 1$. Both parities are needed. Verified to machine precision.
- With zero phases, $[W(x)^d]_{00} = T_d(x)$. Chebyshev polynomials are what repeated rotation looks like in the computational basis.
- *Qubitization*: the walk operator $W = Z_{\Pi}U$ decomposes into 2×2 blocks, one per eigenvalue, each a rotation by $\arccos \lambda_j$. Verified: $\text{eig}(W) = e^{\pm i \arccos \lambda_j}$.

- **QSVT**: d uses of a block encoding, interleaved with $d + 1$ phases, block-encode $P(A)$, provided P admits a complementary Q . Verified: phases $\pi/2$ give exactly $T_d(A)$.
- Phase factors are computed classically and offline; use an existing package.
- **Amplitude estimation** without phase estimation: measure $\Pr = \sin^2((2r + 1)\theta)$ at an exponential schedule of r and fit by maximum likelihood. Verified: $\hat{g} = 0.129$ against $g = 0.13$ using 13,300 queries, where classical sampling would need $g(1 - g)/\epsilon^2 \approx 1.2 \times 10^5$ shots.
- **Qubitized simulation**: Jacobi–Anger gives $\mathcal{O}(t + \log(1/\epsilon))$ queries against Trotter’s polynomial $1/\epsilon$. Verified: order 28 suffices for 10^{-10} at $t = 10$. But the relevant time is αt , and α can be large.
- **Linear systems**: approximating $1/x$ needs degree $\mathcal{O}(\kappa \log(1/\epsilon))$, and the $\mathcal{O}(\kappa)$ amplification rounds bring the query count to $\mathcal{O}(\kappa^2 \log(1/\epsilon))$ against HHL’s κ^2/ϵ . Verified: polynomial degree grows linearly in κ . The four caveats of Chapter 9 are unchanged.

Where this leaves the book

QSVT is not one more algorithm. It is the observation that most of the algorithms in this book are the same algorithm with different polynomials: Grover a sign function, simulation an exponential, HHL a reciprocal. That is worth knowing for its own sake. It is also the practical state of the art. Current resource estimates for fault-tolerant chemistry and optimization are almost all built on block encodings and qubitization rather than on the constructions of Chapters 4–9. Those chapters remain the right way to *learn* the subject; this one is the right way to *implement* it.

10.9 End-of-Chapter Problems

- 10.1** Verify that (10.2) is unitary when A is Hermitian with $\|A\| \leq 1$, and show $U^2 = \mathbb{I}$.
- 10.2** For $A = \begin{pmatrix} 0.4 & 0.2 \\ 0.2 & -0.3 \end{pmatrix}$, construct the block encoding explicitly and verify $(\langle 0| \otimes \mathbb{I})U(|0\rangle \otimes \mathbb{I}) = A$.
- 10.3** Show that the LCU circuit of Chapter 7 is a block encoding, and identify α .
- 10.4** Show that $W(x)$ in (10.3) is unitary for all $x \in [-1, 1]$, and identify the rotation axis and angle.
- 10.5** Prove $[W(x)^d]_{00} = T_d(x)$ by writing $x = \cos \theta$.

- 10.6** For $d = 2$ and $\phi = (0, \pi/4, 0)$, compute $U_\phi(x)$ explicitly and identify $P(x)$. Check its parity.
- 10.7** Verify the QSP constraint $|P|^2 + (1 - x^2)|Q|^2 = 1$ for the polynomial of the previous problem.
- 10.8** Explain why P must have definite parity, using the structure of (10.4).
- 10.9** Show that the walk operator $W = Z_\Pi U$ is unitary, and compute its eigenvalues for a 1×1 “matrix” $A = (\lambda)$.
- 10.10** Show that within the invariant subspace attached to λ_j , the walk operator equals the QSP signal operator $W(\lambda_j)$.
- 10.11** Verify numerically that $(\langle 0|W^d|0\rangle) = T_d(A)$ for $d = 1, \dots, 5$ on a 2×2 Hermitian A of your choice.
- 10.12** Express r Grover iterations as a Chebyshev polynomial, and explain the overshoot phenomenon of Chapter 6 in those terms.
- 10.13** Compute the Jacobi–Anger coefficients $J_k(5)$ for $k = 0, \dots, 20$ and find the smallest truncation order giving error below 10^{-8} .
- 10.14** Show that $J_k(t)$ decays super-exponentially for $k \gtrsim e|t|/2$, and explain why this gives (10.16).
- 10.15** For $H = X \otimes \mathbb{I} + Z \otimes Z$, compute $\alpha = \|c\|_1$ for the LCU block encoding. How does the required circuit depth change if H is rescaled by a factor of 10?
- 10.16** Fit an odd polynomial to $1/x$ on $[1/8, 1] \cup [-1, -1/8]$ to relative accuracy 10^{-3} and confirm the degree is about 65. Check that $|P| \leq 1$ on all of $[-1, 1]$ after rescaling.
- 10.17** Compare the query complexity of HHL and the QSVT linear solver for $\kappa = 10^3$ and $\epsilon = 10^{-6}$. By what factor do they differ?
- 10.18** Explain why amplitude amplification recovers one factor of κ in the QSVT linear solver but cannot recover two.
- 10.19** Implement maximum-likelihood amplitude estimation in simulation for $g = 0.3$, using the schedule $r = 0, 1, 2, 4, 8, 16$. Confirm the error scales as $1/N_{\text{queries}}$ rather than $1/\sqrt{N_{\text{queries}}}$.
- 10.20** (*Project.*) Take the H_2 qubit Hamiltonian from Chapter 8, build an LCU block encoding, verify the walk-operator eigenvalues against $\arccos(\lambda_j/\alpha)$, and compare α with $\|H\|$. Comment on the overhead.

Research Problems and Projects

The problems above all have answers at the back of somebody's notebook. The three below do not. Each is sized for several weeks of work by a strong MS student or a beginning PhD student, and each ends in an artifact you could show somebody: a scaling curve with error bars, a resource table, or a piece of software other people can run. This chapter is the newest material in the book and the closest to the current literature, so in all three you should expect to have to read the papers rather than only this chapter.

R10.1 Where does qubitization actually overtake Trotter? Table 10.1 states an asymptotic verdict. It does not say at which n , t and ϵ the crossover happens for a specific Hamiltonian, and the “ α is not free” box gives good reason to think the crossover is much later than the asymptotics suggest. Build the comparison honestly. Pick a family such as the one-dimensional transverse-field Ising chain at $n = 4, 6, 8, 10, 12$, and for each instance count the resources both ways: Trotter steps and two-qubit gates for the product formulas of Chapter 7 [19], against block-encoding queries and ancilla count for the qubitized construction [21]. Use the same target error ϵ in the same norm on both sides, which is the step where most published comparisons quietly cheat. Sweep ϵ over at least four decades and t over at least two. *Deliverable:* a crossover plot in the (t, ϵ) plane, one curve per system size, marking the region where qubitization wins on gate count rather than on query count. Then repeat with α computed from a smarter block encoding, such as one built from a grouped or factorized form of H , and report how far the crossover moves. A careful negative result, showing that qubitization loses on gate count over the whole range you can simulate, is a perfectly good outcome and worth writing up.

R10.2 Read the unification claim back against the original papers. This chapter asserts that Grover search, phase estimation, Hamiltonian simulation and HHL are one algorithm with four polynomials. That claim was made in a specific form by Martyn and coauthors [22], resting on the QSVT construction of Gilyén and coauthors [13], which in turn generalizes the quantum signal processing of Low and Chuang [20, 21]. Reconstruct the chain of results from the papers themselves rather than from review articles. For each of the four algorithms, write down precisely what the reduction to QSVT preserves and what it costs: the degree of the polynomial, the ancilla count, the normalization α , and the overhead relative to the original bespoke algorithm. *Deliverable:* a table with one row per algorithm, giving the original complexity, the QSVT complexity, and the ratio, with a citation to the exact theorem number that justifies each entry. The interesting part is the losses. Identify at least two places where the unified construction is

genuinely worse than the special-purpose algorithm it subsumes, and say whether the gap is believed to be inherent or is an artifact of the current analysis. Distinguish carefully between what is proved, what is conjectured, and what is folklore.

R10.3 A phase-factor toolchain, and the degree at which it breaks. The chapter tells you to use somebody else’s phase-finding package and does not say what happens when you push it. Find out. Implement or wrap a phase-factor routine [10] and build a test harness that, for a target polynomial and a computed phase sequence, evaluates (10.4) classically on a dense grid and reports the maximum deviation from the target. Then run it on the three families this chapter needs: the Jacobi–Anger truncation of e^{-ixt} , the odd approximation to $1/x$ on (10.17) used by the QSVT linear solver [7], and a rectangle filter of adjustable width. *Deliverable:* a measured curve of achieved accuracy against polynomial degree for each family, in double and in extended precision, together with the degree at which the routine stops converging and a diagnosis of why. Report wall-clock time as well, since for the linear-solver polynomial at $\kappa \sim 10^3$ the degree runs into the thousands. Package the harness so that somebody else can drop in a target function and get phases plus an error certificate. Conditioning of phase-factor evaluation at high degree is an active area, so a clean and reproducible failure boundary is a genuine contribution.

Appendix A

Notation and Conventions

In my experience, most errors in quantum algorithm implementations are not mathematical. They are convention mismatches: a factor of two in a rotation angle, a reversed bit order, a raising operator that lowers. I have collected every convention used in this book in one place here, and flagged the ones that most often go wrong.

A.1 Dirac notation

I use Dirac notation throughout, in the forms collected below.

The `\braket` macro takes ONE argument

This book loads the `physics` package and then the `braket` package. The load order is deliberate and must not be changed: `braket` overrides the `physics` definition, so that

$$\backslash\mathrm{braket}\{v|w\} \longrightarrow \langle v|w\rangle$$

Writing `\braket{v}{w}` instead produces $\langle v\rangle w$, silently. If you reorder the package loads, every inner product in the book doubles.

Table A.1: Dirac and operator notation used throughout.

Symbol	Meaning	Note
$ \psi\rangle$	column vector (ket)	normalized unless stated
$\langle\psi $	conjugate transpose (bra)	
$\langle\phi \psi\rangle$	inner product	
$ \phi\rangle\langle\psi $	outer product	
$\otimes$	tensor product	often omitted: $ 0\rangle 1\rangle = 01\rangle$
$A^\dagger$	conjugate transpose	
$\ A\ $	spectral norm	largest singular value
$\{A, B\}$	anticommutator $AB + BA$	
$[A, B]$	commutator $AB - BA$	

A.2 Scalars and counts

The table below fixes every scalar symbol that carries a book-wide meaning and names the chapter that establishes it. Where a letter genuinely carries more than one meaning, all of them are listed, so that you do not leave this page with the wrong one. Two meanings of one letter never share a formula.

Three reuses in that table are deliberate, and I want to state them plainly. Each is standard in its own literature. Changing it would put the book out of step with the papers you will go on to read.

- N . $N = 2^n$ everywhere except Chapter 5, where N is the number being factored, and §8.1–§8.2, where it is an electron count. None of the three ever appears in a formula with either of the others.
- θ . The rotation-gate reading and the phase-estimation reading differ by a factor of 2π . The explicit 2π in the exponent is what tells you which is meant: $e^{-i\theta Z/2}$ is radians, $e^{2\pi i\theta}$ is a fraction of a turn.
- r . Grover iterations, Trotter steps and the order of a modulo N all take r , following three different literatures. No chapter uses two of the three, which is why Chapter 6 §6.5 switches to K when it needs a Trotter-step count.

Table A.2: Scalar symbols with a book-wide meaning. Every entry was checked against the chapter that fixes it.

Symbol	Meaning	Fixed in
n	number of qubits in the working register	Ch. 1
N	2^n , the dimension of the state space. In Ch. 5 N is instead the integer being factored, and in §8.1–§8.2 the number of electrons	Ch. 1
ϵ	target accuracy	Ch. 2
t	evolution time. Ch. 5 alone writes t for the width of its counting register, where $t = 2n$	Ch. 7
Δt	one Trotter step, of duration t/r	Ch. 7
θ	the Bloch-sphere polar angle (Ch. 1); the argument of a rotation gate, in radians, $R(\theta) = e^{-i\theta\sigma/2}$ (§A.4), which Ch. 1 alone writes η ; a phase expressed as a fraction of a full turn in phase estimation, $e^{2\pi i\theta}$ with $0 \leq \theta < 1$ (Ch. 4, Ch. 5); and the Grover half-angle, $\sin \theta = \sqrt{\mu/N}$, with every full rotation written 2θ (Ch. 6, Ch. 10)	Ch. 2
c	width of the counting or clock register	Ch. 4
m	the integer measured out of that register. In Ch. 7 m is instead the number of terms in a linear combination of unitaries	Ch. 4
r	Grover iterations (Ch. 6, Ch. 10); Trotter steps (Ch. 7); the order of a modulo N (Ch. 5)	Ch. 5
w	the marked item in search (Ch. 6); the weight of a Pauli string, meaning its number of non-identity factors (Ch. 7, §B.3)	Ch. 6
μ	number of marked items	Ch. 6
g	probability that $\mathcal{A} 0\rangle$ lands in the good subspace	Ch. 6
L	number of terms in a Hamiltonian, $H = \sum_{k=1}^L H_k$	Ch. 7
K	number of Trotter steps in Chapter 6 §6.5, where both r and L are taken; the truncation order of a series (Ch. 7, Ch. 10)	Ch. 6
c_j	real coefficient of a Pauli string in $H = \sum_j c_j P_j$, and of a unitary in the LCU sum $A = \sum_i c_i U_i$	Ch. 7
Λ	$\sum_k \ H_k\ $; it is what sets the Trotter step count	Ch. 7
d	row sparsity of a matrix (Ch. 7, Ch. 9); degree of the QSP polynomial (Ch. 10). In §5.4 d is the RSA private exponent	Ch. 7
M	number of spin-orbitals, hence of qubits, in a chemistry problem. In §5.4 M is the RSA plaintext; in the amplitude-estimation schedule of Ch. 10 it is the largest exponent, $r \leq 2^M$	Ch. 8
p	number of QAOA layers. In Ch. 3 p is Simon’s hidden string, in §5.4 an RSA prime, and in the LCU of Ch. 7 the post-selection success probability	Ch. 9
κ	condition number of the matrix of a linear system	Ch. 9
N_{shots}	number of repetitions of one circuit	Ch. 9
α	normalization factor of a block encoding	Ch. 10
a	number of ancilla qubits of a block encoding; the base whose order is sought in Ch. 5	Ch. 10
$\hbar$	set to 1 from Ch. 6 §6.5 onward; see §A.8	Ch. 6

A.3 Bit ordering: the endianness trap

One convention, used everywhere in this book

Every chapter places the **most** significant bit on the top wire. An n -qubit basis state is written

$$|j\rangle = |j_0 j_1 \cdots j_{n-1}\rangle, \quad j = j_0 2^{n-1} + j_1 2^{n-2} + \cdots + j_{n-1} 2^0,$$

so the leftmost symbol in the ket is qubit 0, qubit 0 is drawn on the top wire, and qubit 0 carries the most significant bit. This is *big-endian* ordering.

In the first draft of this book, Chapters 4 and 5 used the opposite ordering. They no longer do. If you are working from an earlier version, or from the arXiv posting, check which convention it uses before comparing circuits.

Two consequences deserve to be stated plainly. Both are places where students lose time.

The SWAP layer in the QFT circuit is not a convention patch. The QFT circuit of Chapter 4 produces its output bits in reverse wire order. That is a property of the circuit, not of the labeling scheme. The final layer of SWAP gates undoes it. Under either endianness the SWAPs are needed, and under either endianness they are what lets you read the output register directly as a number. If you drop them, you must reverse the register in classical post-processing instead. Doing neither is a silent error.

Most software libraries disagree with this book. Qiskit, among others, is *little-endian*: its top wire is the *least* significant bit. Neither choice is wrong. But the same circuit drawn under the two conventions produces matrices that differ by a bit-reversal permutation, and the same measured bit string decodes to a different integer.

If your period or phase is consistently wrong, look here first

Reverse the register before the inverse QFT, or reverse the measured bit string afterwards, but do exactly one of the two.

This is the single most common bug in implementations of Chapters 4 and 5. It does not raise an error. It returns a well-formed number that happens to be the bit reversal of the right one, and continued fractions will then hand you a plausible but wrong period. Test on a case whose answer you already know. For $N = 15$ and $a = 7$ the order is $r = 4$, so a counting register of width $t = 8$ should return one of 0, 64, 128, 192 and nothing else.

A.4 Rotation angles: the factor of two

$$R_z(\theta) = e^{-i\theta Z/2}, \text{ so } e^{-i\theta Z} = R_z(2\theta)$$

The rotation gates carry a factor of $\frac{1}{2}$ in the exponent:

$$R_x(\theta) = e^{-i\theta X/2}, \quad R_y(\theta) = e^{-i\theta Y/2}, \quad R_z(\theta) = e^{-i\theta Z/2}.$$

Consequently, to implement $e^{-i\theta P}$ for a Pauli string P you need $R_z(2\theta)$. For a Hamiltonian term $c_j P_j$ evolved for time t you need $R_z(2c_j t)$.

Roughly half the errors I found while preparing this book were a missing or spurious factor of two here. The matrix $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ is $R_y(2\theta)$, not $R_y(\theta)$.

How to check. Verify on a case you know: $R_z(\pi) = \text{diag}(e^{-i\pi/2}, e^{i\pi/2}) = -iZ$. If your gate gives Z for an argument of π , you are off by two.

A.5 Fourier sign convention

I use

$$\text{QFT} |j\rangle = \frac{1}{\sqrt{N}} \sum_k e^{+2\pi i j k / N} |k\rangle,$$

with a *positive* exponent, matching the classical inverse DFT. This is the same sign used in Chapter 4. Many libraries use the negative sign. The two differ by complex conjugation. The QFT matrix is symmetric, so conjugation is the same as taking the adjoint: one convention's QFT is the other's *inverse* QFT. That is why a mismatch makes phase estimation return $1 - \theta$ instead of θ .

A.6 Occupation and ladder operators

In Chapter 8, $|0\rangle$ denotes an **unoccupied** spin-orbital and $|1\rangle$ an occupied one. Hence

$$A^+ = \frac{X - iY}{2} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} \quad (\text{raises}), \quad A^- = \frac{X + iY}{2} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \quad (\text{lowers}).$$

A^+ is lower-triangular. Under the opposite occupation convention these swap, and every sign in the Jordan–Wigner transformation flips [17].

$A^\pm$ act on a single qubit. The fermionic operators $a_i^\dagger, a_i$ that appear in Chapter 8 and in the identity table of Appendix B are those same $A_i^\pm$ dressed with a Z string,

$$a_i^\dagger \mapsto Z_0 Z_1 \cdots Z_{i-1} A_i^+, \quad a_i \mapsto Z_0 Z_1 \cdots Z_{i-1} A_i^-,$$

the string being what supplies the antisymmetry sign. The two notations are therefore not alternatives; one is built from the other.

A.7 Bravyi–Kitaev index sets

The index sets of the Bravyi–Kitaev transformation follow the notation of [25]; Chapter 8 derives them.

$P(j)$ is read from row $j - 1$

- $U(j)$: column j of β , entries *below* the diagonal.
- $P(j)$: **row** $j - 1$ of $\pi\beta^{-1}$ (empty for $j = 0$). Here π is the lower-triangular parity matrix *including the diagonal*, as in Chapter 8. With the strictly lower-triangular convention the rule gives a wrong set for every j .
- $F(j)$: row j of β^{-1} , entries *left* of the diagonal.
- $R(j) = P(j) \setminus F(j)$.

Reading $P(j)$ from row j instead of $j - 1$ yields $P(j + 1)$, the parity set of the *next* orbital, which is also a meaningful set. The mistake therefore produces a plausible table and passes small-case anticommutator tests. Check against $M = 8$, where $P(7) = \{3, 5, 6\}$ while the wrong reading gives row $7 = \{7\}$. Also: invert β over $\mathbb{F}_2$ by Gaussian elimination, never in floating point, and assert $\beta\beta^{-1} \equiv \mathbb{I} \pmod{2}$.

A.8 Units

Chapter 6 §6.5 and Chapters 7–10 set $\hbar = 1$. Chapter 8 additionally uses Hartree atomic units, $\hbar = m_e = e = 4\pi\epsilon_0 = 1$. In those units energies are measured in Hartree ($1 E_h \approx 27.211 \text{ eV}$), and *chemical accuracy* means $1.6 \times 10^{-3} E_h$.

Appendix B

Gate and Identity Reference

B.1 Single-qubit gates

The matrices below are the ones I use throughout the book. They cover the Pauli matrices, the Hadamard gate, the phase gates S , T and $P(\theta)$, and the three rotation gates.

$$\mathbb{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \quad P(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

$$R_x(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} \quad R_y(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}$$

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

B.2 Identities used repeatedly

Table B.1: Identities that recur across chapters. Every one was verified numerically during preparation.

Identity	Where used
$X = HZH$	Pauli-string circuits (Ch. 7)
$Y = (HS^\dagger)^\dagger Z (HS^\dagger)$	Pauli-string circuits (Ch. 7)
$Z = HXH$	
$XY = iZ, YZ = iX, ZX = iY$	throughout
$\{X, Y\} = \{Y, Z\} = \{Z, X\} = 0$	Trotter error, Ch. 7
$X^2 = Y^2 = Z^2 = \mathbb{I}$	closed-form exponentials
$e^{-i\theta P} = \cos \theta \mathbb{I} - i \sin \theta P$	any Pauli string P
$e^{-i\theta \phi\rangle\langle\phi } = \mathbb{I} + (e^{-i\theta} - 1) \phi\rangle\langle\phi $	Ch. 6 simulation search
$\text{SWAP} = \text{CNOT}_{12}\text{CNOT}_{21}\text{CNOT}_{12}$	Ch. 1, Ch. 5
$n_i = a_i^\dagger a_i \mapsto (\mathbb{I} - Z_i)/2$	Jordan–Wigner, Ch. 8
$x_i \leftrightarrow (\mathbb{I} - Z_i)/2$	QUBO to Ising, Ch. 9
$[W(x)^d]_{00} = T_d(x)$	QSP, Ch. 10

B.3 The Pauli exponential circuit

The single most reused construction in the book (Chapter 7, §7.4). To implement $e^{-i\theta P}$ for a Pauli string P of weight w :

1. Basis change: H on each X position, $HS^\dagger$ on each Y position, nothing on Z or $\mathbb{I}$.
2. CNOT ladder from each active qubit into the last active qubit.
3. $R_z(2\theta)$ on that qubit.
4. Reverse the ladder.
5. Reverse the basis change.

Cost, in gates, for a string of weight w (the number of non-identity factors): the ladder of step 2 links w active qubits with $w - 1$ CNOTs, and step 4 runs it again, so $2(w - 1)$ CNOTs in all; one R_z ; and $2 \times (\text{number of } X) + 4 \times (\text{number of } Y)$ single-qubit gates, so at most $4w$ of them. Each X position costs a Hadamard either side; each Y position costs $HS^\dagger$ either side, which is two gates each way. None of this depends on n , the total number of qubits: the identity positions cost nothing.

Figure B.1 carries out the five steps for $P = X \otimes \mathbb{I} \otimes Y \otimes Z$ on four qubits. Qubit q_1 carries $\mathbb{I}$, so step 1 leaves it alone and step 2 skips it: the ladder runs from the active qubits q_0 and q_2 into q_3 , the last active one. The weight is $w = 3$, giving four CNOTs and six single-qubit gates.

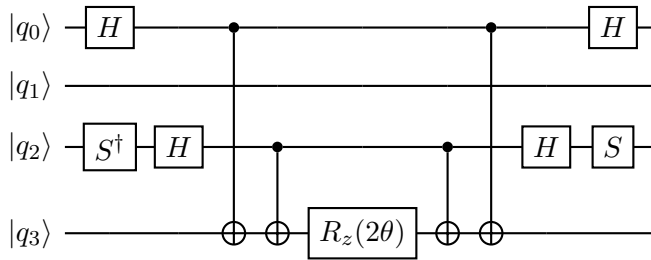


Figure B.1: The five-step Pauli exponential recipe applied to $e^{-i\theta X \otimes \mathbb{I} \otimes Y \otimes Z}$. Step 1 conjugates q_0 by H (for X) and q_2 by $HS^\dagger$ (for Y), and leaves q_1 (identity) and q_3 (Z) untouched. Step 2 is the CNOT ladder from the active qubits q_0 and q_2 into q_3 ; the inactive q_1 is skipped. Step 3 is the single $R_z(2\theta)$. Steps 4 and 5 are the mirror image, which restores the original basis and uncomputes the parity.

B.4 These gates in Qiskit

Every gate in this book exists in Qiskit, and the table below gives the name of each one. I checked the names against **Qiskit 2.5**, the current release as this book went to press. Classes come from `qiskit.circuit.library`. The method column is what you call on a `QuantumCircuit`.

Three things that bite when you move to Qiskit

The bit ordering is the other way round. Qiskit is little-endian: its top wire is the *least* significant bit, while this book is big-endian; see §A.3. Every gate below behaves the same way, but the *register* you apply it to is indexed in the opposite direction, and the unitary Qiskit reports is a bit-reversal permutation of the one printed here.

R_k is a phase gate, not a rotation gate. The R_k of the QFT is $P(2\pi/2^k)$, so its Qiskit form is `PhaseGate`, and controlled it is `CPhaseGate`. Using `RZGate` instead is the classic silent error. Chapter 2 §2.2 draws the same distinction for $P(\theta)$ and $R_z(\theta)$: the two differ by a global phase, invisible on its own, but controlling either turns that global phase into a *relative* phase, so the controlled versions are genuinely different operators and the QFT you build is wrong.

You can no longer ask how many ancillas an `MCXGate` will use. The static method `get_num_ancilla_qubits()` was deprecated in Qiskit 2.1 and is scheduled for removal in 3.0. The transpiler now chooses a synthesis method during high-level synthesis, from how many ancillas are free, so count them from the transpiled circuit rather than from the gate. Chapter 2 makes the same point where `MCXGate` is introduced.

Library APIs move faster than books. Treat the table as a starting point and check

Table B.2: The gates of this book and their Qiskit 2.5 names. `grover_operator` is a function rather than a class, and `Gate.control()` is a method called on a gate you already have. “None” in the method column means there is no dedicated `QuantumCircuit` method; append the gate instead.

This book	Meaning	Qiskit class	Method
$\mathbb{I}$	identity	<code>IGate</code>	<code>.id()</code>
X, Y, Z	Pauli matrices	<code>XGate, YGate, ZGate</code>	<code>.x(), .y(), .z()</code>
H	Hadamard	<code>HGate</code>	<code>.h()</code>
$S, S^\dagger$	phase by $\pm\pi/2$	<code>SGate, SdgGate</code>	<code>.s(), .sdg()</code>
$T, T^\dagger$	phase by $\pm\pi/4$	<code>TGate, TdgGate</code>	<code>.t(), .tdg()</code>
$P(\theta)$	$\text{diag}(1, e^{i\theta})$	<code>PhaseGate</code>	<code>.p()</code>
R_x, R_y, R_z	$e^{-i\theta\sigma/2}$, $\sigma = X, Y, Z$	<code>RXGate, RYGate, RZGate</code>	<code>.rx(), .ry(), .rz()</code>
CNOT	controlled X	<code>CXGate</code>	<code>.cx()</code>
CZ	controlled Z	<code>CZGate</code>	<code>.cz()</code>
R_k controlled	controlled $P(2\pi/2^k)$	<code>CPhaseGate</code>	<code>.cp()</code>
SWAP	exchange two qubits	<code>SwapGate</code>	<code>.swap()</code>
Toffoli, CCX	doubly controlled X	<code>CCXGate</code>	<code>.ccx()</code>
Fredkin, CSWAP	controlled SWAP	<code>CSwapGate</code>	<code>.cswap()</code>
$C^n(X)$	n controls	<code>MCXGate</code>	<code>.mcx()</code>
$C(U)$, general U	controlled unitary	<code>Gate.control()</code>	none
arbitrary U	built from a matrix	<code>UnitaryGate</code>	<code>.unitary()</code>
QFT	Fourier transform, Ch. 4	<code>QFTGate</code>	none
$e^{-i\theta P}$	Pauli exponential, Ch. 7	<code>PauliEvolutionGate</code>	none
G	Grover operator, Ch. 6	<code>grover_operator()</code>	none

the current documentation before you rely on a signature.

Appendix C

Algorithm Resource Summary

The table below puts every algorithm in the book side by side. Each row gives the number of qubits, the leading cost, and, in a separate column, what that cost actually counts. I have separated the last two on purpose. A number such as $O(n^2)$ means nothing until you know whether it counts two-qubit gates, oracle queries, uses of a block encoding, or Trotter steps, and the algorithms here count different things. The chapter cited in the last column is where each entry is derived, and the legend below defines every symbol; the table itself sits on the next page.

All counts are for the fault-tolerant setting, ignoring error-correction overhead, and every symbol carries the meaning fixed in §A.2.

Legend. Every symbol in Table C.1, in order of appearance.

- n : number of qubits in the working register; $N = 2^n$ is the dimension of its state space. In the Shor row alone, N is the integer being factored and $n = \lceil \log_2 N \rceil$ is its length in bits.
- U_f : the oracle unitary that computes the function f being queried.
- ϵ : the target accuracy, meaning the largest error tolerated in the final state or the estimated quantity.
- $k_{\max}$: the largest k for which the phase gate R_k is kept in the approximate QFT.
- c : width of the counting or clock register; U is the unitary whose eigenphase is being estimated.
- μ : number of marked items in a search over N items.
- $\mathcal{A}$: the state-preparation unitary of amplitude amplification and of MLAE; g is the probability that $\mathcal{A}|0\rangle$ lands in the good subspace; $\mathcal{Q}$ is the generalized Grover operator built from $\mathcal{A}$ and the oracle. The MLAE schedule is the sequence of Grover-iteration counts $0, 1, 2, 4, \dots$ at which $\mathcal{Q}$ is applied.
- r : number of Trotter steps.

- t : the evolution time, with $\hbar = 1$.
- L : number of terms in the Hamiltonian $H = \sum_{k=1}^L H_k$, and $\Lambda = \sum_k \|H_k\|$ the sum of their operator norms.
- value register: the fixed-width register that holds the fetched matrix element $H_{x,f(x)}$ of a sparse Hamiltonian; the extra qubit is the comparison bit. The oracle queries counted are to O_f and O_H , returning the column index $f(x)$ and the element $H_{x,f(x)}$.
- m : number of unitaries in the linear combination $A = \sum_{i=1}^m c_i U_i$.
- M : number of spin-orbitals, hence of qubits, in a chemistry problem.
- p : number of QAOA layers.
- d : row sparsity of the matrix A (HHL row) or degree of the QSVT polynomial (QSVT row).
- κ : condition number of A , the ratio of its largest to its smallest singular value.
- $|b\rangle$: the right-hand side of the linear system $A|x\rangle = |b\rangle$.
- a : number of ancilla qubits of a block encoding, and α its normalization, so that the encoded block is A/α .

What these numbers do not include

Every entry above counts logical qubits and logical operations. On error-corrected hardware, one logical qubit costs 10^3 – 10^4 physical qubits. Non-Clifford gates (T , Toffoli) dominate the runtime, while Clifford gates are nearly free. A resource estimate that counts total gates rather than T -count will be misleading by orders of magnitude. The counts also exclude state preparation, which for HHL and the QSVT linear solver can dominate everything else (Chapter 9, §9.5) [1].

Table C.1: Resource summary. Chemistry has $O(M^4)$ terms under both encodings; Bravyi–Kitaev reduces the *weight* of each term, hence its circuit depth, not the term count.

Algorithm	Qubits	Cost	What the cost counts	Ch.
Deutsch–Jozsa	$n + 1$	1	queries to the oracle U_f	3
Bernstein–Vazirani	$n + 1$	1	queries to U_f	3
Simon	$2n$	$O(n)$	queries to U_f , expected	3
QFT (exact)	n	$\frac{n(n+1)}{2} + \lfloor n/2 \rfloor$	gates: n Hadamards, $\frac{n(n-1)}{2}$ controlled phases, $\lfloor n/2 \rfloor$ SWAPs	4
Approximate QFT	n	$O(n \log n)$	gates; $k_{\max} \approx \log_2(n/\epsilon)$ controlled phases per wire	4
Phase estimation	$n + c$	$2^c - 1$	controlled applications of U , then one inverse QFT	4
Iterative phase estimation	$n + 1$	$2^c - 1$	controlled applications of U , over c rounds	4
Shor (order finding)	$3n$	$O(n^3)$	gates; the $2n$ controlled modular multiplications dominate	5
Grover	$n + \text{ancillas}$	$\lfloor \frac{\pi}{4} \sqrt{N/\mu} \rfloor$	oracle queries, one per iteration	6
Quantum counting	$n + c + \text{ancillas}$	$2^c - 1$	controlled Grover iterations, one oracle query each	6
Amplitude amplification	as $\mathcal{A}$ needs	$O(1/\sqrt{g})$	applications of $\mathcal{Q}$, each one use of $\mathcal{A}$, $\mathcal{A}^\dagger$ and the oracle	6
Trotter, 1st order	n	$r \approx \Lambda^2 t^2 / \epsilon$	Trotter steps of L Pauli exponentials each	7
Trotter, 2nd order	n	$r \approx (\Lambda^3 t^3 / \epsilon)^{1/2}$	Trotter steps of $2L - 1$ Pauli exponentials each	7
Sparse Hamiltonian simulation	$2n + \text{value register} + 1$	4	oracle queries per 1-sparse piece per Trotter step	7
LCU	$n + \lceil \log_2 m \rceil$	m	controlled unitaries in select, plus prepare and its inverse	7
Chemistry (JW encoding)	M	$O(M^4)$	Pauli terms, each of weight $O(M)$, so $O(M)$ CNOTs	8
Chemistry (BK encoding)	M	$O(M^4)$	Pauli terms, each of weight $O(\log M)$, so $O(\log M)$ CNOTs	8
QAOA (p layers)	n	$p \cdot O(n^2)$	two-qubit gates: two CNOTs per $Z_i Z_j$ term per layer	9
HHL	$n + c + 1 + \text{arithmetic}$	$O(\log(N)d^2\kappa^2/\epsilon)$	gates (runtime); preparing $ b\rangle$ excluded	9
QSVT (degree d)	$n + a$	d	uses of the block encoding, plus $d + 1$ ancilla phase rotations	10
MLAE	as $\mathcal{A}$ needs	$O(1/\epsilon)$	queries to $\mathcal{A}$ over the whole schedule	10
Qubitized simulation	$n + a$	$O(\alpha t + \log(1/\epsilon))$	uses of the block encoding of H	10
QSVT linear solver	$n + a$	$O(\kappa^2 \log(1/\epsilon))$, or $O(\kappa \log(1/\epsilon))$ variable-time	uses of the block encoding of A : degree $O(\kappa \log(1/\epsilon))$ times $O(\kappa)$ amplification rounds	10

Appendix D

Verifying an Implementation

Every algorithm chapter ends with an implementation guide. I have collected here the methodology those guides share, because in my experience the same five tests catch most bugs regardless of the algorithm.

D.1 The five tests

1. **Build a classical reference first.** Before writing any quantum code, write the brute-force classical version: exhaustive order finding, full enumeration of the cost function, dense matrix exponentiation. It is exponentially slow and it is the only thing that tells you whether the quantum version is right.
2. **Test each operator as a dense matrix.** For $n \leq 12$, build every oracle, diffuser, encoding and evolution operator explicitly and check the property that defines it: unitarity, the correct diagonal, the anticommutation relations, the right eigenvalues. Most bugs are here, not in the circuit assembly.
3. **Test on an instance with an exact answer.** Choose parameters where the algorithm is exact: eigenvalues that are exact powers of two for phase estimation, $N = 4$ for Grover, a period dividing 2^t for Shor. Any deviation is then unambiguously a bug.
4. **Check the scaling, not just the value.** Plot error against the parameter that controls it on log axes and confirm the slope: 2 for first-order Trotter, 3 for symmetric, $1/r$ for accumulated Trotter error, $1/N_{\text{queries}}$ for amplitude estimation. A correct value at one setting can hide a wrong exponent.
5. **Check that ancillas return to $|0\rangle$.** Uncomputation failures leave ancillas entangled with the system, which destroys interference and produces wrong answers with no error message. Assert it explicitly.

D.2 Symptom-to-cause table

Table D.1: Symptoms and their usual causes. Every row corresponds to an error encountered during the preparation of this book.

Symptom	Likely cause
Consistently wrong period or phase	bit ordering reversed before the inverse QFT
Rotation half or double what it should be	$R_z(\theta)$ used where $R_z(2\theta)$ needed
Success probability rises monotonically with Grover iterations	diffuser wrong (should fall after the peak)
Energies right up to a constant	identity term of the Pauli expansion dropped
Complex coefficients in a Hamiltonian	missing conjugate or a sign error in the encoding
Anticommutators fail only for large M	β^{-1} computed in floating point rather than over $\mathbb{F}_2$
Bravyi–Kitaev sets look plausible but energies wrong	$P(j)$ read from row j instead of $j - 1$
Interference vanishes; results look classical	ancilla not uncomputed
Trotter error scales as Δt not Δt^2	operator ordering wrong in the splitting
Phase estimation returns $1 - \theta$	Fourier sign convention mismatch with your library
Amplitude estimation error falls as $1/\sqrt{N_{\text{queries}}}$	schedule not resolving the phase; use an exponential schedule of r

D.3 A note on published demonstrations

Several widely reproduced demonstrations hard-code their answers:

- **Shor on $N = 15$:** many implementations choose gates knowing $r = 4$. If your modular multiplier does not take a and N as parameters, you have not implemented Shor’s algorithm [26].
- **HHL on 2×2 :** the standard HHL example [16] uses eigenvalues 1 and 2 so that a 2-qubit clock register is exact and rotation angles can be hard-coded.

Both are useful teaching circuits. Neither demonstrates the algorithm’s scaling, and I say so to my students explicitly.

Appendix E

Hints for Selected Problems

Hints, not solutions. Problems whose answers appear inline in the text are omitted.

Chapter 1

1.4 For the matrix A to be expanded, use $c_\mu = \frac{1}{2}\text{Tr}[\sigma_\mu A]$, which follows from $\text{Tr}[\sigma_\mu \sigma_\nu] = 2\delta_{\mu\nu}$ with $\sigma_0 = \mathbb{I}$.

Chapter 2

2.8 Every Gray-code path between two states at Hamming distance h takes h steps, whatever order the bits are flipped in. So any path from 111 to 000 costs $h - 1 = 2$ multi-controlled NOTs on the way out, one multi-controlled $\tilde{U}$, and the same two NOTs on the way back. The count does not change. **2.13** Uncomputation costs the same as computation; the ancilla must return to $|0\rangle$ for the outer interference to survive.

Chapter 3

3.13 Simon's algorithm needs $n - 1$ *independent* equations, not n . Each run returns a z drawn uniformly from the 2^{n-1} strings with $p \cdot z = 0$. While fewer than $n - 1$ independent vectors are in hand, their span covers at most half of those, so each run adds a new independent vector with probability at least $\frac{1}{2}$. Failing after $n - 1 + k$ runs therefore needs more than k wasted runs.

Chapter 4

4.7 Bound the error of each dropped gate by its phase angle $2\pi/2^k$, which is the right measure because R_k is a phase gate and not one of the rotation gates of §A.4. Then sum the geometric series. **4.10** Use $|\delta| \leq 2^{-c-1}$ together with $\sin x \leq x$ and $\sin x \geq 2x/\pi$ on $[0, \pi/2]$. **4.15** $2^c - 1$ applications with $c = 20$; compare with the coherence time of any plausible device.

Chapter 5

5.5 If $\gcd(l, r) = k$, the fraction l/r reduces to $(l/k)/(r/k)$ and continued fractions returns the reduced denominator. **5.9** $a^{r/2} \equiv +1$ would contradict the minimality of r . **5.13** Repeated application of V costs 2^k operations; compare with one controlled multiplier for the precomputed constant $a^{2^k} \bmod N$.

Chapter 6

6.2 At $N = 4$ the half-angle is $\theta = \arcsin \sqrt{1/4} = \pi/6$ exactly, so $(2r+1)\theta = \pi/2$ has the integer solution $r = 1$. **6.6** The oracle changes a sign, not a magnitude; measurement probabilities depend only on magnitudes. **6.15** Use $|\phi\rangle\langle\phi|^k = |\phi\rangle\langle\phi|$ for all $k \geq 1$ and sum the remaining series. **6.17** Grover halves the effective key length, so AES-256 retains 128-bit security.

Chapter 7

7.1 Expand both sides to second order; the difference is $-\frac{1}{2}[A, B]t^2$. **7.16** Compute $R_z(-\pi/2)XR_z(\pi/2)$ explicitly; the sign of the rotation matters.

Chapter 8

8.3 $|0\rangle$ is the *empty* orbital, so the raising operator is lower-triangular. **8.6** The Z -strings on qubits $< i$ appear in both terms and cancel in the product. **8.12** Row j gives the parity of orbitals with index $\leq j$; you want $< j$. **8.15** The two conserved quantities are the numbers of spin-up and spin-down electrons.

Chapter 9

9.7 Diagonalize $H(s) = (1-s)X + sZ$; the gap is $2\sqrt{(1-s)^2 + s^2}$, minimized at $s = \frac{1}{2}$. **9.11** Expand $p(0) = \frac{1}{4}\langle\Phi|(\mathbb{I} + U^\dagger)(\mathbb{I} + U)|\Phi\rangle$ and simplify; the cross terms give $2\operatorname{Re}\langle\Phi|U|\Phi\rangle$. **9.14** Write λ in binary with λ_0 most significant and λ_{c-1} least significant, as in Fig. 9.9, and evaluate the sum for each case.

Chapter 10

10.5 Put $x = \cos \theta$. Then $W(x) = e^{i\theta X}$, which is the X -rotation $R_x(-2\theta)$ named in Chapter 10, and $W(x)^d = e^{id\theta X}$. Read off the 00 entry, $\cos d\theta$. **10.10** Restrict W to the span of $|0\rangle|u_j\rangle$ and $|1\rangle|u_j\rangle$ and write the 2×2 matrix. **10.14** Use the large-order asymptotic $J_k(t) \sim \frac{1}{\sqrt{2\pi k}} \left(\frac{et}{2k}\right)^k$. **10.18** The filtered state succeeds with probability $O(1/\kappa^2)$, so amplification needs $O(\kappa)$ rounds where plain repetition needs $O(\kappa^2)$: that is the one factor recovered. The other factor of κ is the degree of the polynomial, that is the depth of a single circuit, and amplification does not touch it.

Appendix F

Answers to Selected Problems

Chapter 1

1.1 $|\psi\rangle = \frac{3}{5}|0\rangle - \frac{4i}{5}|1\rangle$; $\theta = 2 \arccos \frac{3}{5} \approx 1.855$ (106.3°), $\varphi = 3\pi/2$, so $\vec{r} = (0, -\frac{24}{25}, -\frac{7}{25})$.

1.2 $|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$, $|1\rangle = \frac{1}{\sqrt{2}}(|+\rangle - |-\rangle)$.

1.4 $c_0 = c_y = 0$, $c_x = c_z = \frac{1}{\sqrt{2}}$, that is $H = \frac{1}{\sqrt{2}}(X + Z)$.

1.5 $p(0) = \cos^2(\theta/2)$, $p(1) = \sin^2(\theta/2)$; the phase $e^{i\varphi}$ has modulus one and drops out of $|\langle 1|\psi\rangle|^2$.

1.6 $H \otimes H = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$.

1.7 (a) Not entangled: it equals $|+\rangle|0\rangle$. (b) Entangled. (c) Entangled. In (b) and (c) the 2×2 coefficient matrix has rank 2.

1.8 $\alpha|00\rangle + \beta|11\rangle$, entangled if and only if $\alpha\beta \neq 0$.

1.10 $R_z(2\pi) = -I$: a full turn returns the Bloch vector but multiplies the state by -1 , invisible as a global phase but not as a controlled operation.

Chapter 2

2.1 $C(H) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}$, block diagonal with blocks I and H , each unitary.

2.3 $H = e^{i\pi/2} R_y(\pi/2) R_z(\pi)$, that is $\alpha = \pi/2$, $\beta = 0$, $\gamma = \pi/2$, $\delta = \pi$; hence $A = R_y(\pi/4)$, $B = R_y(-\pi/4) R_z(-\pi/2)$ and $C = R_z(\pi/2)$.

2.6 $C^2(Z) = (I \otimes I \otimes H)$ Toffoli $(I \otimes I \otimes H)$; the one Toffoli costs five two-qubit gates by Fig. 2.6 with $V = \sqrt{X}$ (three controlled- V and two CNOTs).

2.7 AND: target $|0\rangle$, giving $|a, b, ab\rangle$. NOT: both controls $|1\rangle$, giving $|1, 1, \bar{t}\rangle$. FANOUT: controls $|1\rangle|a\rangle$ and target $|0\rangle$, giving $|1, a, a\rangle$.

2.8 No: every path uses two multi-controlled NOTs out, one $C^2(\tilde{U})$, and the same two NOTs back, five gates in all.

2.9 $\cos(\theta/2) = 0.8536$, $\sin(\theta/2) = 0.5210$, $\theta = 2 \arccos(\cos^2 \frac{\pi}{8}) = 1.0961$ rad = 62.80° ; the squares sum to 1.0000.

2.10 $D = 2^{11} - 2 = 2046$, so $m \gtrsim 2^{11} \log(1/\epsilon)/(f \log n) \approx 3 \times 10^3$ gates with $f = 2$: a modest number. The astronomical quantity is the number of ϵ -patches the reachable set must cover, $\epsilon^{-D} = 10^{6138}$, against roughly 10^{80} atoms.

2.12 CZ between input qubit i and the ancilla for exactly those i with $s_i = 1$; on the input register this is $O_f^\pm = \bigotimes_i Z^{s_i}$.

2.13 Without garbage, $p(0) = 1$ for constant f and 0 for balanced f . With the garbage qubit the final state is $\frac{1}{2} \sum_{x,z} (-1)^{f(x)+xz} |z\rangle |x\rangle$, so $p(0) = \frac{1}{2}$ for every f .

2.14 One gate: mark $|00\rangle$ with a doubly open-controlled gate; the result is -1 times the oracle for S , and the sign is a global phase.

Chapter 3

3.6 Writing f as the string $f(00)f(01)f(10)f(11)$: constant 0000 and 1111; balanced 0011, 0101, 0110, 1001, 1010, 1100. Deutsch–Jozsa returns 00 with probability 1 for the two constant functions and 0 for all six balanced ones.

3.7 $\Pr(00 \cdots 0) = |C_0|^2 = (1 - 2N_1/N)^2 = ((N_0 - N_1)/N)^2$, where N_b is the number of inputs with $f(x) = b$; it lies strictly between 0 and 1 when f is neither constant nor balanced.

3.8 Query k uniformly random inputs and answer “constant” iff all values agree. A constant f is never misclassified; a balanced f fools the test with probability $2^{-(k-1)}$, so $k = 1 + \lceil \log_2(1/\delta) \rceil$ queries give error at most δ ($k = 21$ for $\delta = 10^{-6}$).

3.9 Three CNOTs, one from each of qubits 0, 1, 3 into the $|-\rangle$ ancilla; the count is the Hamming weight of s .

3.11 The pairs $x \leftrightarrow x \oplus 101$ are $\{000, 101\} \mapsto f_0$, $\{001, 100\} \mapsto f_1$, $\{010, 111\} \mapsto f_2$, $\{011, 110\} \mapsto f_3$, with $f_0, \dots, f_3$ any four distinct 3-bit strings; each pair XORs to 101. Simon’s circuit on this f returns 000, 010, 101 or 111, each with probability $\frac{1}{4}$.

3.14 $M_{ij} = \Pr(\text{read } i \mid \text{prepared } j)$; column j is the recorded outcome distribution for basis state j , so it sums to 1.

3.15 Any P_{noisy} cleaner than the calibration allows works. For $P_{\text{noisy}} = (1, 0, 0, 0)^T$,

$M^{-1}P_{\text{noisy}} = (1.113, -0.055, -0.048, -0.010)^T$;

for the Bell-like counts $(0.49, 0.01, 0.01, 0.49)^T$,

$M^{-1}P_{\text{noisy}} = (0.540, -0.032, -0.036, 0.528)^T$.

Report the constrained least-squares solution with $P_i \geq 0$ and $\sum_i P_i = 1$, which is $(1, 0, 0, 0)^T$ in the first case.

Chapter 4

4.1 $\text{QFT}_2 = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix}$, with entries $\omega^{jk}/2$, $\omega = e^{2\pi i/4} = i$.

4.2 $|101\rangle = |5\rangle$, so the three factors carry phases $2\pi(0.1)_2 = \pi$, $2\pi(0.01)_2 = \pi/2$ and $2\pi(0.101)_2 = 5\pi/4$: $\text{QFT}|101\rangle = |-\rangle \otimes |+\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + e^{5\pi i/4}|1\rangle) = \frac{1}{\sqrt{8}} \sum_{k=0}^7 e^{2\pi i 5k/8} |k\rangle$.

4.3 Any basis state other than $|0\rangle^{\otimes n}$ (with $n \geq 2$) separates them; for instance $\text{QFT}|01\rangle = |-\rangle \otimes |+\rangle$ while $H \otimes H|01\rangle = |+\rangle \otimes |-\rangle$.

4.4 No. For example $\text{QFT}_2|0\rangle|+\rangle = \frac{1}{\sqrt{2}}(|+\rangle|+\rangle + |-\rangle|+\rangle)$ is entangled (Schmidt rank 2) although the input is a product state.

4.5 For $n = 4$: 4 Hadamards, $3 + 2 + 1 = 6$ controlled phases and 2 SWAPs, 12 gates in all, matching $\frac{4 \cdot 5}{2} + 2$.

4.7 Each wire loses angles $2\pi/2^k$ for $k = k_{\max} + 1, \dots$, which sum to less than $2\pi/2^{k_{\max}}$, so $\|\text{QFT} - \text{QFT}_{\text{approx}}\| \leq 2\pi n/2^{k_{\max}}$ in operator norm; requiring this to be at most ϵ gives $k_{\max} = \log_2(n/\epsilon) + \log_2(2\pi) \approx \log_2(n/\epsilon) + 2.65$. (Numerically the true error for $n = 5, 6, 7$, $k_{\max} = 2, 3, 4$ is smaller than the bound by a factor of three or more.)

4.9 With $N = 16$ and $\delta = \frac{1}{3} - \frac{m}{16}$, the probabilities for $m = 0, \dots, 15$ are (to three decimals)

0.004, 0.005, 0.008, 0.015, 0.044, **0.685**, 0.172, 0.028,

0.012, 0.007, 0.005, 0.004, 0.003, 0.003, 0.003, 0.003.

The most likely outcome is $m = 5 = 0101_2$, giving $\theta \approx 5/16 = 0.3125$, an error of $1/48$; $m = 5$ and $m = 6$ together carry probability 0.857.

4.11 $Z|0\rangle = |0\rangle$ ($\theta = 0$) and $Z|1\rangle = -|1\rangle$ ($\theta = \frac{1}{2}$). The counting register reads $m = 0$ ($00 \dots 0$) with probability $\frac{1}{2}$, leaving the second register in $|0\rangle$, or $m = 2^{c-1}$ ($10 \dots 0$) with probability $\frac{1}{2}$, leaving it in $|1\rangle$; no other outcome occurs.

4.13 $\theta = 5/8 = 0.101_2$, so $\theta_1\theta_2\theta_3 = 101$. Round 1: controlled- U^4 , kickback $2\pi \cdot \frac{20}{8} \equiv \pi$, no correction, H , measure $\theta_3 = 1$. Round 2: controlled- U^2 , kickback $\pi/2$, apply $R_2^\dagger = P(-\pi/2)$ (since $\theta_3 = 1$), phase 0, measure $\theta_2 = 0$. Round 3: controlled- U , kickback $5\pi/4$, apply $R_3^\dagger = P(-\pi/4)$ (since $\theta_3 = 1$; $\theta_2 = 0$ needs no

$R_2^\dagger$), phase π , measure $\theta_1 = 1$. Every measurement is deterministic.

4.15 $2^{20} - 1 = 1\,048\,575$ controlled applications of U in either scheme ($\sum_{t=0}^{19} 2^t$ for QPE, $\sum_{k=1}^{20} 2^{20-k}$ for IPE); at $1\,\mu\text{s}$ per application that is about one second of coherent evolution, far beyond current coherence times, so 20 bits is out of reach without error correction.

Chapter 5

5.1 Measuring output 0 leaves $\frac{1}{2}(|0\rangle + |3\rangle + |4\rangle + |7\rangle)$, the union of the cosets $\{0, 4\}$ and $\{3, 7\}$ (output 1 gives $\{1, 5\} \cup \{2, 6\}$). The inverse QFT then yields $m = 0, 2, 6$ with probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{4}$ and never $m = 4$.

5.2 $|\psi_3\rangle = \frac{1}{\sqrt{8}} \sum_{x=0}^7 |x\rangle |x \bmod 4\rangle$; measuring the output as $y_0 \in \{0, 1, 2, 3\}$ leaves $\frac{1}{\sqrt{2}}(|y_0\rangle + |y_0 + 4\rangle)$; the inverse QFT gives $\frac{1}{2} \sum_{l=0}^3 e^{-2\pi i y_0 l/4} |2l\rangle$, so $m \in \{0, 2, 4, 6\}$, each with probability $\frac{1}{4}$.

5.4 $3891/4096 = [0; 1, 18, 1, 50, 4]$ with convergents $\frac{0}{1}, \frac{1}{1}, \frac{18}{19}, \frac{19}{20}, \frac{968}{1019}, \frac{3891}{4096}$. The last with denominator below 55 is $19/20$, and $2^{20} \equiv 1 \pmod{55}$, so $r = 20$.

5.5 For $r = 12$, $\phi(12) = 4$, so 8 of the 12 values of l (all but 1, 5, 7, 11) share a factor with r : two thirds of the outcomes return a proper divisor of 12 (or $m = 0$).

5.6 $1071 = 2 \cdot 462 + 147$, $462 = 3 \cdot 147 + 21$, $147 = 7 \cdot 21$, so $\gcd = 21$; back-substituting, $21 = 462 - 3 \cdot 147 = 7 \cdot 462 - 3 \cdot 1071$, so $(x, y) = (-3, 7)$.

5.8 $a = 2$: $r = 6$, $x = 2^3 = 8$, $\gcd(7, 21) = 7$ and $\gcd(9, 21) = 3$, so $21 = 3 \times 7$.
 $a = 5$: $r = 6$ again but $5^3 = 125 \equiv 20 \equiv -1$, so the attempt fails and a new a is needed.

5.9 $N = 21$, $a = 5$: $r = 6$ and $5^3 \equiv -1 \pmod{21}$; or $N = 15$, $a = 14$: $r = 2$ and $14 \equiv -1$.

5.10 $r = 4, 2, 4, 4, 2, 4, 2$ for $a = 2, 4, 7, 8, 11, 13, 14$ (all even). $a^{r/2} = 4$ for $a = 2, 4, 7, 8, 13$, giving $\gcd(3, 15) = 3$ and $\gcd(5, 15) = 5$; $a = 11$ gives $a^{r/2} = 11$ and $\gcd(10, 15) = 5$, $\gcd(12, 15) = 3$; $a = 14$ has $a^{r/2} = 14 \equiv -1$ and fails. Six of seven choices succeed.

5.11 $r = 6$ with orbit $1, 2, 4, 8, 16, 11$. With $\omega = e^{-2\pi i/6}$, $|u_s\rangle = \frac{1}{\sqrt{6}}(|1\rangle + \omega^s|2\rangle + \omega^{2s}|4\rangle + \omega^{3s}|8\rangle + \omega^{4s}|16\rangle + \omega^{5s}|11\rangle)$ for $s = 0, \dots, 5$; in $\frac{1}{\sqrt{6}} \sum_s |u_s\rangle$ the coefficient of $|2^k\rangle$ is $\frac{1}{6} \sum_s \omega^{sk} = \delta_{k0}$, leaving $|1\rangle$.

5.12 $n = 2048$: $t = 2n = 4096$ counting plus n work qubits, $3n = 6144$ logical qubits, about 8200 with the multiplier's scratch register (or $2n + 3 = 4099$ optimized). Toffolis: $2n$ controlled multiplications, each n controlled modular additions of a few n Toffolis, doubled for uncomputation, is of order $10n^3 \approx 10^{11}$ ($n^3 \approx 8.6 \times 10^9$).

5.13 $\sum_{k=0}^{t-1} 2^k = 2^{2n} - 1 \approx N^2$ applications of V , each $O(n^2)$ gates, so $O(N^2 n^2) =$

$O(4^n n^2)$: exponential. For $N = 15$ ($t = 8$) that is 255 applications; for a 2048-bit modulus, $2^{4096} - 1$.

5.14 Take $a = 2$: $\gcd(2, 91) = 1$, $r = 12$, $x = 2^6 = 64 \not\equiv -1$, $\gcd(63, 91) = 7$, $\gcd(65, 91) = 13$; $\phi = 6 \cdot 12 = 72$; $d = 5^{-1} \bmod 72 = 29$; $M = 41^{29} \bmod 91 = 6$ (check: $6^5 \bmod 91 = 41$).

5.15 $2^6 = 64 \equiv 9$. Mod 10: $sk = 6, 8, 2, 4$ and $k^{-1} = 1, 7, 3, 9$ for $k = 1, 3, 7, 9$, so $(sk)k^{-1} = 6, 56, 6, 36 \equiv 6$ in every case; the other six k share a factor with 10 and have no inverse.

5.16 RSA-2048: $3n = 6144$ qubits. A 256-bit curve: $2t + n = 5n = 1280$, about a fifth as many, so the elliptic-curve key is the easier target although it is the one considered stronger classically.

Chapter 6

6.1 $U_f = \text{diag}(1, 1, 1, -1)$ and $V = \frac{1}{2} \begin{pmatrix} -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 \end{pmatrix}$, so $G = VU_f = \frac{1}{2} \begin{pmatrix} -1 & 1 & 1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$.

In the $\{|w^\perp\rangle, |w\rangle\}$ plane this is $\begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}$ with $\theta = \arcsin \frac{1}{2} = \pi/6$, a rotation by $2\theta = \pi/3$.

6.4 With $\theta = \arcsin \frac{1}{4} = 0.2527$, $\Pr(w) = \sin^2((2r+1)\theta)$ for $r = 0, \dots, 10$ is 0.063, 0.473, 0.908, **0.961**, 0.582, 0.126, 0.020, 0.365, 0.836, **0.992**, 0.687. Equation (6.11) gives $r = 2.608$, so $r = 3$; the highest value in the range is at $r = 9$ (the second revolution) and the worst is $r = 6$.

6.7 In round k set $R_k = \lceil (6/5)^k \rceil$, capped at $\sqrt{N}$; pick r uniformly from $\{0, \dots, R_k - 1\}$, run r iterations, measure and check f . The expected number of Grover iterations is at most $\frac{9}{2}m_0$ with $m_0 = N/(2\sqrt{\mu(N-\mu)})$, that is about $\frac{9}{4}\sqrt{N/\mu} = 72/\sqrt{\mu}$ for $N = 1024$ (valid for $\mu \leq 3N/4$), so $O(\sqrt{N/\mu})$ with no knowledge of μ . The bound is loose: the exact expectation at $\mu = 1$ is 37 iterations over about 15 rounds, against $r = 25$ for a fixed- r run that knows μ .

6.8 Eigenvalues $e^{\pm 2i\theta}$, eigenvectors $\frac{1}{\sqrt{2}} \begin{pmatrix} i \\ 1 \end{pmatrix}$ and $\frac{1}{\sqrt{2}} \begin{pmatrix} -i \\ 1 \end{pmatrix}$, from $\det(G - \lambda \mathbb{I}) = \lambda^2 - 2\lambda \cos 2\theta + 1 = 0$.

6.9 $2\theta = 2\pi \cdot 8/64 = \pi/4$, so $\theta = \pi/8$ and $\mu = 64 \sin^2(\pi/8) = 32 - 16\sqrt{2} = 9.37$, hence $\mu \approx 9$ (exact $\mu = 9$ maps to $m = 7.83$). Resolution $\delta\mu \approx \sqrt{\mu(N-\mu)} 2\pi/2^6 \approx 2.2$, so the run cannot separate $\mu = 9$ from $\mu = 10$ or $\mu = 8$.

6.13 $g_0 = 1/32$, $\theta = \arcsin \sqrt{1/32} = 0.1777$, exact $r = \pi/(4\theta) - \frac{1}{2} = 3.92$, rounded up to $r = 4$. Then $g'_0 = \sin^2(\pi/18) = 0.03015 \leq g_0 = 0.03125$, $g'_0/g_0 = 0.965$, and

$R = R_y(2 \arcsin \sqrt{g'_0/g_0}) = R_y(2.765)$, an angle of 158.4° . Four rounds then give $\Pr(w, 1) = 1$ exactly, since $9 \cdot \pi/18 = \pi/2$.

6.16 With $|w\rangle$ fixed and $|\psi\rangle$ uniform, the Frobenius-norm error is 0.0425, 0.00171 and 1.71×10^{-5} at $\Delta t = 0.5, 0.1, 0.01$; the ratio to Δt^2 is 0.170, 0.171, 0.171, converging to $\frac{1}{2} \| [H_1, H_2] \| = \frac{1}{\sqrt{2}} a \sqrt{1-a^2} = 0.171$ with $a = 1/\sqrt{16}$. (In the spectral norm every entry is smaller by $\sqrt{2}$.)

Chapter 7

7.1 The leading error is $-\frac{1}{2}[A, B]t^2$: for $t = 10^{-2}$ it accounts for all but 0.7% of $e^{iAt}e^{iBt} - e^{i(A+B)t}$ (Frobenius norm, random Hermitian A, B).

7.3 In the Frobenius norm, with $A = X$ and $B = Z$, the total errors at $r = 1, 2, 5, 10$ are 1.130, 0.513, 0.199, 0.099 for first order ($r \cdot \text{error} \approx 1.0$, so $\mathcal{O}(1/r)$) and 0.444, 0.101, 0.0156, 0.0039 for the symmetric formula ($r^2 \cdot \text{error} \approx 0.4$, so $\mathcal{O}(1/r^2)$).

7.4 For the open transverse-field chain $H = \sum_i Z_i Z_{i+1} + \sum_i X_i$ with unit couplings, $\Lambda = 9 + 10 = 19$, so the guide's rules give $r \approx \Lambda^2 t^2 / \epsilon = 3.6 \times 10^7$ (first order) and $r \approx (\Lambda^3 t^3 / \epsilon)^{1/2} = 8.3 \times 10^4$ (symmetric). Both are pessimistic: the smallest r that actually reaches operator-norm error 10^{-3} , splitting H into its ZZ and X groups, is $r = 35\,300$ and $r = 1601$ respectively.

7.5 $d + 1$ is Vizing's bound on the edge chromatic number; d alone does not suffice, since three states coupled in a triangle are 2-sparse and need three pieces. Each piece must be Hermitian so that f is an involution and $e^{-iH_j t}$ is unitary, which is what the 1-sparse subroutine and the Trotter product require.

7.6 In the order $(0, 3, 1, 2)$, at $t = 1$, $e^{-iH} = \begin{pmatrix} \cos a & -i \sin a \\ -i \sin a & \cos a \end{pmatrix} \oplus \text{diag}(e^{-ib}, e^{-ic}) = R_x(2a) \oplus e^{-i(b+c)/2} R_z(b-c)$, with the off-diagonal blocks identically zero.

7.9 $\phi = 2 \arctan \sqrt{c_1/c_0} = 2 \arccos \sqrt{c_0/(c_0 + c_1)}$; for $c_0 = 0.6$, $c_1 = 1.4$ this is $\phi = 1.982$, and $O^\dagger O = \mathbb{I}$ since $(\sqrt{c_0})^2 + (\sqrt{c_1})^2 = c_0 + c_1$.

7.10 $p = \|A|0\rangle\|^2 / \|c\|_1^2 = 2/4 = \frac{1}{2}$, since $(X + Z)|0\rangle = |1\rangle + |0\rangle$; as $(X + Z)^2 = 2\mathbb{I}$, $\|A|\psi\rangle\|^2 = 2$ for every $|\psi\rangle$ and $p = \frac{1}{2}$ always.

7.12 $c_{\mathbb{I}} = 0$, $c_X = 2$, $c_Y = 0$, $c_Z = 1$, so $H = 2X + Z$.

7.13 Yes: $X \otimes X$, $Y \otimes Y$ and $Z \otimes Z$ commute pairwise, so $e^{-iHt} = e^{-i(X \otimes X)t} e^{-i(Y \otimes Y)t} e^{-i(Z \otimes Z)t}$ exactly, with $r = 1$ and no Trotter error; the naive circuit uses 2 CNOTs per term, 6 in all.

7.14 Four CNOTs, $2(w-1)$ with weight $w = 3$: conjugate q_0 with $HS^\dagger$ and q_3 with H , ladder $q_0 \rightarrow q_3$, $q_1 \rightarrow q_3$, apply $R_z(2\theta)$ on q_3 , and undo; q_2 is untouched.

7.16 $R_z(-\pi/2) X R_z(\pi/2) = -Y$; the correct sign is $R_z(\pi/2) X R_z(-\pi/2) = Y$.

7.17 One angle per Pauli term: L parameters per Trotter layer, pL for p layers (the Hamiltonian variational ansatz of Chapter 9).

Chapter 8

8.4 $a_2^\dagger a_0 |1100\rangle = -|0110\rangle$: a_0 removes orbital 0 with sign +1 (nothing lies to its left), then $a_2^\dagger$ on $|0100\rangle$ picks up $(-1)^{k_0+k_1} = -1$. The Jordan–Wigner product $(Z_0 Z_1 A_2^+)(A_0^-)$ gives the same vector.

8.7 $a_0^\dagger a_3 + a_3^\dagger a_0 \mapsto \frac{1}{2}(X_0 Z_1 Z_2 X_3 + Y_0 Z_1 Z_2 Y_3)$, two strings of weight $w = 4$, so $2(w-1) = 6$ CNOTs each and 12 for the exponential; the two strings commute, so the product of the two factors is exact.

8.8 $|10110010\rangle \mapsto |11011100\rangle$ (running sums mod 2); back via $k_0 = p_0$ and $k_j = p_j \oplus p_{j-1}$, which returns $|10110010\rangle$.

8.10 $\beta_4 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{pmatrix}$, $\beta_4^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{pmatrix}$ over $\mathbb{F}_2$; only the last row changes, since $k_3 = b_3 \oplus b_1 \oplus b_2$.

8.11 $j = 0$: $U = \{1, 3\}$, $P = F = R = \emptyset$. $j = 1$: $U = \{3\}$, $P = F = \{0\}$, $R = \emptyset$. $j = 2$: $U = \{3\}$, $P = \{1\}$, $F = \emptyset$, $R = \{1\}$. $j = 3$: $U = \emptyset$, $P = F = \{1, 2\}$, $R = \emptyset$.

8.12 Row 4 of $\pi\beta_8^{-1}$ gives $\{3, 4\}$ and row 5 gives $\{3, 5\}$. Since $b_3 \oplus b_4 = k_0 \oplus \dots \oplus k_4$, the first is the parity of orbitals < 5 , so $P(5) = \{3, 4\}$; the second is $P(6)$.

8.14 Jordan–Wigner: 16, from $a_{15} \mapsto Z_0 \dots Z_{14} A_{15}^-$. Bravyi–Kitaev: 5, from $a_0 \mapsto X_1 X_3 X_7 X_{15} A_0^-$, that is $\log_2 M + 1$ against $\log_2 M = 4$ (the largest set, $U(0)$, has exactly $\log_2 M$ elements).

8.16 Jordan–Wigner: X on qubits 0, 1, 2, 3. Bravyi–Kitaev: β_8 sends $|11110000\rangle$ to $|10100000\rangle$, so X on qubits 0 and 2 only.

8.17 With $L \approx M^4 = 1.6 \times 10^5$ terms and $1/\epsilon^2 \approx 4 \times 10^5$ shots per term at $\epsilon = 1.6 \times 10^{-3}$, about 6×10^{10} circuit repetitions (about 6×10^9 with the 10^4 or so distinct strings that survive combining). Qubit-wise commuting grouping divides the number of settings, and so the total, by a factor of a few to ten.

8.18 Electronic ground state -1.8573 Hartree; adding the nuclear repulsion $1/R = 0.7200$ Hartree ($R = 0.735 \text{ \AA} = 1.389$ bohr) gives -1.1373 Hartree, equal to full CI to machine precision (Hartree–Fock: -1.1170). The Jordan–Wigner Hamiltonian has 15 Pauli terms, and Bravyi–Kitaev reproduces all 16 eigenvalues.

Chapter 9

9.1 Maximum cut weight 10, attained by exactly two assignments, $\mathbf{x} = (0, 1, 0, 1, 0)$ and its complement $(1, 0, 1, 0, 1)$; these are the same partition, so there is one distinct

optimal partition.

9.2 With $Q = -W$ and $c_i = \sum_j W_{ij}$, for example $\mathbf{x} = (0, 1, 0, 1, 0)$ gives 10, $(1, 0, 0, 0, 0)$ gives 4 and $(1, 1, 0, 0, 0)$ gives 4, each equal to the cut weight of (9.3).

9.3 For MaxCut the linear terms vanish and $H_C = \frac{1}{2} \sum_{i,j} W_{ij} \mathbb{I} - \frac{1}{2} \sum_{(i,j) \in E} W_{ij} Z_i Z_j$; for the instance of Fig. 9.1 the constant is $\sum_{i,j} Q_{ij}/4 + \sum_i c_i/2 = -5.5 + 11 = 5.5$, half the total edge weight.

9.5 For integers a_i with sum S , minimize $C(\mathbf{x}) = (S - 2 \sum_i a_i x_i)^2$, that is $Q = 4 \mathbf{a} \mathbf{a}^T$, $\mathbf{c} = -4S \mathbf{a}$, plus the constant S^2 . The Ising form is $H_C = (\sum_i a_i Z_i)^2 = \sum_{i \neq j} a_i a_j Z_i Z_j + (\sum_i a_i^2) \mathbb{I}$, whose ground energy is 0 exactly when an equal split exists.

9.6 Energy $-n$: each X_i has eigenvalue $+1$ on $|+\rangle$, and $-n$ is the smallest eigenvalue of $-\sum_i X_i$.

9.7 $\Delta(s) = 2\sqrt{(1-s)^2 + s^2}$, $\Delta_{\min} = \sqrt{2}$ at $s = \frac{1}{2}$, so (9.8) predicts $T \sim 1/\Delta_{\min}^2 = \frac{1}{2}$, a constant independent of anything else.

9.8 Both errors are $\mathcal{O}(\Delta t^2)$ per slice, so over p slices the total is $\mathcal{O}(p \Delta t^2) = \mathcal{O}(T^2/p)$: first order in $1/p$ at fixed T . Numerically (two qubits, $T = 4$) the error halves each time p doubles.

9.9 Six CNOTs: two per $R_{Z_i Z_j}$ and one such gate per edge; the R_z angles on single qubits vanish.

9.10 With $H_C = -\frac{3}{2} + \frac{1}{2}(Z_0 Z_1 + Z_1 Z_2 + Z_0 Z_2)$ (ground state the maximum cut, -2), $\langle H_C \rangle = -\frac{3}{2} - \frac{3}{4} \sin(4\beta) \sin(2\gamma) + \frac{3}{2} \sin^2(2\beta) \sin^2 \gamma$. The minimum is $\langle H_C \rangle = -2$, the exact optimum, at $\gamma^* = \arctan(1/\sqrt{2}) = 0.6155$, $\beta^* = \gamma^*/2 = 0.3077$; there the six maximum-cut strings each carry probability $\frac{1}{6}$.

9.13 Eigenvalues 1 and 2 with eigenvectors $|-\rangle$ and $|+\rangle$; $|x\rangle = (3|0\rangle - |1\rangle)/\sqrt{10}$; with $C = 1$ the success probability is $\frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{4} = \frac{5}{8}$.

9.14 $\theta\lambda = \frac{9}{2}\pi = 14.14$, $\frac{25}{4}\pi = 19.63$, $\frac{9}{2}\pi = 14.14$, $\frac{49}{4}\pi = 38.48$ for $\lambda = 3, 5, 6, 7$, from $\theta = \frac{3}{2}\pi, \frac{5}{4}\pi, \frac{3}{4}\pi, \frac{7}{4}\pi$; none equals π .

9.15 The small-angle form is low by 4.5%, 1.06% and 0.26%: $2 \arcsin(C/\lambda) = 1.0472, 0.5054, 0.2507$ against 1, 0.5, 0.25.

9.18 $f(\lambda) = e^{-i\gamma\lambda}$ for QAOA and $f(\lambda) = 1/\lambda$ for HHL. Neither is a polynomial; Chapter 10 approximates the first by a truncated Jacobi–Anger series and the second by a polynomial on $[1/\kappa, 1]$.

Chapter 10

10.2 $\sqrt{\mathbb{I} - A^2} = \begin{pmatrix} 0.8944 & -0.0109 \\ -0.0109 & 0.9327 \end{pmatrix}$, so

$$U = \begin{pmatrix} 0.4 & 0.2 & 0.8944 & -0.0109 \\ 0.2 & -0.3 & -0.0109 & 0.9327 \\ 0.8944 & -0.0109 & -0.4 & -0.2 \\ -0.0109 & 0.9327 & -0.2 & 0.3 \end{pmatrix}, \text{ which is real symmetric with } U^2 = \mathbb{I}.$$

10.3 $\alpha = \|c\|_1 = \sum_i |c_i|$: the $|0\rangle$ block of $O^\dagger V O$ is $\sum_i c_i U_i / \|c\|_1$.

10.4 Axis X , angle $-2 \arccos x$: with $\theta = \arccos x$, $W(x) = \cos \theta \mathbb{I} + i \sin \theta X = e^{i\theta X} = R_x(-2\theta)$, and $WW^\dagger = (x^2 + 1 - x^2)\mathbb{I}$.

10.6 $U_\phi(x) = W(x) e^{i\pi Z/4} W(x) = \begin{pmatrix} (2x^2-1+i)/\sqrt{2} & i\sqrt{2}x\sqrt{1-x^2} \\ i\sqrt{2}x\sqrt{1-x^2} & (2x^2-1-i)/\sqrt{2} \end{pmatrix}$, so $P(x) = (T_2(x) + i)/\sqrt{2}$ (even, degree $2 = d$) and $Q(x) = \sqrt{2}x$ (odd).

10.9 $W = \begin{pmatrix} \lambda & \sqrt{1-\lambda^2} \\ -\sqrt{1-\lambda^2} & \lambda \end{pmatrix}$, a rotation by $\arccos \lambda$, with eigenvalues $\lambda \pm i\sqrt{1-\lambda^2} = e^{\pm i \arccos \lambda}$.

10.12 r iterations implement T_{2r+1} : the good-state amplitude is $\sin((2r+1)\theta) = (-1)^r T_{2r+1}(\sin \theta)$ with $\sin \theta = \sqrt{M/N}$, so the success probability is $T_{2r+1}(\sqrt{M/N})^2$; overshooting is evaluating T_{2r+1} past its first extremum at $(2r+1)\theta = \pi/2$.

10.13 $J_0(5) = -0.1776$, $J_1 = -0.3276$, $J_2 = 0.0466$, $J_3 = 0.3648$, $J_4 = 0.3912$, $J_5 = 0.2611$, $J_6 = 0.1310$, $J_8 = 0.0184$, $J_{10} = 1.47 \times 10^{-3}$, $J_{15} = 4.80 \times 10^{-7}$, $J_{20} = 2.77 \times 10^{-11}$; the truncated series first has maximum error below 10^{-8} on $[-1, 1]$ at $K = 17$ (3.3×10^{-9} ; $K = 16$ gives 2.4×10^{-8}).

10.15 $\alpha = 2$ (against $\|H\| = \sqrt{2}$). Rescaling H by 10 gives $\alpha = 20$, and the αt term of $K = \mathcal{O}(\alpha t + \log(1/\epsilon))$ grows tenfold: for $t = 1$ and $\epsilon = 10^{-10}$ the truncation order goes from 13 to 42.

10.16 Odd degree 61 to 63 depending on the fit (Chebyshev interpolation of $1/u$ on $u = x^2 \in [1/64, 1]$ gives 61, a relative least-squares fit 63; $\kappa \ln(1/\epsilon) = 55$). Dividing by $\kappa = 8$ leaves $|P| \leq 0.999$ on the two intervals, but the fit peaks at $|P|/\kappa \approx 1.31$ near $x = \pm 0.07$ inside the gap, so the rescaling that QSVT needs is by about $1/(1.3\kappa)$, not $1/\kappa$.

10.17 HHL $\kappa^2/\epsilon = 10^{12}$ against $\kappa^2 \ln(1/\epsilon) = 1.4 \times 10^7$ for QSVT with amplitude amplification, a factor of about 7×10^4 ; with variable-time amplification, $\kappa \ln(1/\epsilon) = 1.4 \times 10^4$, a factor of 7×10^7 .

10.19 $\theta = \arcsin \sqrt{0.3} = 0.5796$, and the exact $\Pr[\text{good}]$ at $r = 0, 1, 2, 4, 8, 16$ are 0.300, 0.972, 0.058, 0.766, 0.173, 0.076. With $N_{\text{shots}} = 100$ the schedule costs $100(M + 2^{M+2}) = 6800$ queries ($M = 4$); raising M from 4 to 8 (6800 to 103,200 queries) takes the root-mean-square error in g from 1.3×10^{-3} to 7.8×10^{-5} , with $N_{\text{queries}} \times \text{error} \approx 8$ throughout: the $1/N_{\text{queries}}$ rate.

Bibliography

- [1] Scott Aaronson. Read the fine print. *Nature Physics*, 11(4):291–293, 2015.
- [2] Dorit Aharonov and Amnon Ta-Shma. Adiabatic quantum state generation and statistical zero knowledge. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing (STOC '03)*, pages 20–29, New York, NY, USA, 2003. Association for Computing Machinery.
- [3] Ethan Bernstein and Umesh Vazirani. Quantum complexity theory. *SIAM Journal on Computing*, 26(5):1411–1473, 1997.
- [4] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. Tight bounds on quantum searching. *Fortschritte der Physik*, 46(4–5):493–505, 1998.
- [5] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. In *Quantum Computation and Information*, volume 305 of *Contemporary Mathematics*, pages 53–74. American Mathematical Society, Providence, RI, 2002.
- [6] Sergey B. Bravyi and Alexei Yu. Kitaev. Fermionic quantum computation. *Annals of Physics*, 298(1):210–226, 2002.
- [7] Andrew M. Childs, Robin Kothari, and Rolando D. Somma. Quantum algorithm for systems of linear equations with exponentially improved dependence on precision. *SIAM Journal on Computing*, 46(6):1920–1950, 2017.
- [8] Andrew M. Childs and Nathan Wiebe. Hamiltonian simulation using linear combinations of unitary operations. *Quantum Information and Computation*, 12(11–12):901–924, 2012.
- [9] David Deutsch and Richard Jozsa. Rapid solution of problems by quantum computation. *Proceedings of the Royal Society of London A*, 439(1907):553–558, 1992.
- [10] Yulong Dong, Xiang Meng, K. Birgitta Whaley, and Lin Lin. Efficient phase-factor evaluation in quantum signal processing. *Physical Review A*, 103(4):042419, 2021.

- [11] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.
- [12] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. Quantum computation by adiabatic evolution. *arXiv preprint quant-ph/0001106*, 2000.
- [13] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: Exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*, pages 193–204, New York, NY, USA, 2019. Association for Computing Machinery.
- [14] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.
- [15] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96)*, pages 212–219, New York, NY, USA, 1996. Association for Computing Machinery.
- [16] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15):150502, 2009.
- [17] Pascual Jordan and Eugene Wigner. Über das Paulische äquivalenzverbot. *Zeitschrift für Physik*, 47(9):631–651, 1928.
- [18] A. Yu. Kitaev, A. H. Shen, and M. N. Vyalyi. *Classical and Quantum Computation*, volume 47 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 2002.
- [19] Seth Lloyd. Universal quantum simulators. *Science*, 273(5278):1073–1078, 1996.
- [20] Guang Hao Low and Isaac L. Chuang. Optimal Hamiltonian simulation by quantum signal processing. *Physical Review Letters*, 118(1):010501, 2017.
- [21] Guang Hao Low and Isaac L. Chuang. Hamiltonian simulation by qubitization. *Quantum*, 3:163, 2019.
- [22] John M. Martyn, Zane M. Rossi, Andrew K. Tan, and Isaac L. Chuang. Grand unification of quantum algorithms. *PRX Quantum*, 2(4):040203, 2021.
- [23] Sam McArdle, Suguru Endo, Alán Aspuru-Guzik, Simon C. Benjamin, and Xiao Yuan. Quantum computational chemistry. *Reviews of Modern Physics*, 92(1):015003, 2020.

- [24] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010.
- [25] Jacob T. Seeley, Martin J. Richard, and Peter J. Love. The Bravyi–Kitaev transformation for quantum computation of electronic structure. *The Journal of Chemical Physics*, 137(22):224109, 2012.
- [26] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Review*, 41(2):303–332, 1999.
- [27] Daniel R. Simon. On the power of quantum computation. *SIAM Journal on Computing*, 26(5):1474–1483, 1997.
- [28] Yohichi Suzuki, Shumpei Uno, Rudy Raymond, Tomoki Tanaka, Tamiya Onodera, and Naoki Yamamoto. Amplitude estimation without phase estimation. *Quantum Information Processing*, 19(2):75, 2020.

Index

- 3-SAT, 187
- Aaronson, Scott, 262
- ABC decomposition, 27, 45
- adiabatic quantum computation, 247
- adiabatic theorem, 247
- adjoint, 108
- AES, 160
- Aharonov, Dorit, 195
- amplitude, 8
- amplitude amplification, 176, 202, 279
 - variable-time, 289
- amplitude estimation, 100, 177, 256,
 - 281, 309
 - maximum-likelihood, 281
- ancilla, 48, 78, 117, 170, 196, 255,
 - 271, 309
 - dirty, 49, 170
- ancilla qubit, *see* ancilla
- annihilation operator, 220
- anticommutation relations, 221, 300
- antisymmetry, 221
- approximate quantum Fourier
 - transform, 106
- approximation ratio, 254
- atomic units, 219, 300
- balanced function, 77
- barren plateau, 244
- basis, 3
- basis change, 208, 234, 302
- basis set, 222
- Bayesian estimation, 121
- Bell measurement, 73
- Bell state, 31, 72
 - Bell basis, 31, 73
- Bell states, *see* Bell state
- Bennett, Charles, 65
- Bernstein, Ethan, 82
- Bernstein–Vazirani algorithm, 67, 82
- Bessel function, 285
- Bézout’s identity, 138
- big-endian convention, *see* big-endian ordering
- big-endian ordering, 20, 298
- binary expansion, 109
- binary fraction, 104
- binary odometer, 107
- birthday argument, 87
- bit oracle, 79
- bit ordering, 20, 43, 101, 135, 295
- bit-flip gate, 12
- bitwise inner product, 80
- black box, 58
- Bloch sphere, 8, 58, 104
 - Bloch vector, 11, 184
- Bloch, Felix, 8
- block encoding, 262, 270
 - normalization factor, 270
- block-diagonal form, 196, 276
- Born rule, 17
- Born, Max, 219
- Born–Oppenheimer approximation,

- 219
- boson, 219
- bra, 4
- Bravyi, Sergey, 226
- Bravyi–Kitaev transformation, 226, 300
 - β matrix, 227
 - flip set, 228
 - parity set, 228
 - update set, 228
- calibration matrix, 93
- Chebyshev polynomial, 185, 274
 - as approximation basis, 274
- chemical accuracy, 219, 300
- Childs, Andrew M., 199, 261
- circuit compilation, 36
- circuit depth, 49
- circuit diagram, 42
- classical channel, 76
- classical feed-forward, 117
- classical reference implementation, 309
- Clifford gate, 306
- Clifford+T gate set, 187
- clock register, 258, 280
- CNOT gate, 32, 45, 73, 302
- CNOT ladder, 208, 302
- collision, 87
- collision finding, 159
- column-stochastic matrix, 96
- commutator, 182, 190
- commuting terms, 194
- comparison circuit, 196
- completeness relation, 16, 162
- complex projective space, 57
- composite number, 136
- composite system, 18
- computational basis, 8, 64, 73
- condition number, 256, 288
- conical intersection, 219
- conjugate gradient, 261
- constant function, 77
- constrained least squares, 95
- continued fractions, 133
 - convergent, 133
- continuous-time quantum search, 180
- control qubit, 32
- controlled operation, 41
- controlled rotation, 258
 - binary cascade, 259
- controlled unitary, 45, 174
- controlled-NOT gate, *see* CNOT gate
- controlled-phase gate, 44, 103
- controlled-SWAP gate, 196
- controlled-U gate, *see* controlled unitary
- controlled-Z gate, 34
- coset, 128
- cost Hamiltonian, 246
- counting argument, 57
- counting register, 113, 143
- coupon collector argument, 91
- creation operator, 220
- cumulative parity, 225
- cut weight, 245
- cyclic shift, 145
- database search, 159
- decoherence, 93, 116
- deferred measurement, principle of, 135
- density matrix, 272
- derandomization, 178
- Deutsch’s algorithm, 67
- Deutsch’s problem, 81
- Deutsch, David, 77
- Deutsch–Jozsa algorithm, 78
- Diffie–Hellman key exchange, 151
- dimension, 3
- Dirac notation, 4, 295
- Dirac, Paul, 4
- direct sum decomposition, 276
- discrete Fourier transform, 100, 299

- discrete logarithm, 127
 - two-dimensional period, 152
- dyadic rational, 113
- eigenvalue, 109, 175
- eigenvalue filter, 280
- eigenvector, 109, 142, 172
- eigenvector register, 111
- electronic Hamiltonian, 219
- elliptic-curve cryptography, 151
- endianness, *see* bit ordering
- energy measurement, 234
- energy offset, 224
- entanglement, 29, 49, 71, 103
- EPR pair, 75
- error propagation, 121
- Euclid's lemma, 136
- Euclidean algorithm, 137
 - extended, 140
- Euler totient function, 133, 149
- Euler's formula, 24
- Euler's theorem, 150
- exchange operator, 219
- exponential schedule, 283
- exponential search, 169
- factor of two, 207
- factoring, 127
- Farhi, Edward, 247
- fast Fourier transform, 101
- fault tolerance, 171, 281, 305
- fault-tolerant quantum computing,
 - see* fault tolerance
- fermion, 219
- fermion-to-qubit encoding, 220
- Feynman, Richard, 189
- Fock space, 220
- Fock, Vladimir, 232
- Frobenius norm, 192
- full configuration interaction, 237
- fundamental theorem of arithmetic,
 - 136
- garbage, 65
- gate count, 212
- gate error, 92
- gate fidelity, 106
- Gaussian elimination, 50, 91, 227, 300
- geometric series, 113
- Gershgorin circle theorem, 263, 290
- global phase, 9, 44, 82, 181, 196, 246
- Goemans–Williamson algorithm, 254
- good state, 175
- graph
 - 3-regular, 254
 - weighted, 244
- Gray code, 52
- greatest common divisor, 137
- ground-state energy, 100
- Grover search, 150, 159, 269
 - diffusion operator, 54, 162, 309
 - Grover operator, 100, 161, 204, 279
 - optimal number of iterations, 166
- Grover, Lov, 159
- Hadamard basis, 10, 74
- Hadamard gate, 12, 43, 72, 102, 130, 174, 208, 301
- Hadamard test, 254
 - imaginary part, 255
- Hadamard transform, 99
- Hamiltonian simulation, 180, 189, 257, 269
- Hamming weight, 85
- Harrow, Aram, 256
- Hartree, Douglas, 232
- Hartree–Fock state, 116, 232
- Hassidim, Avinatan, 256
- Heisenberg Hamiltonian, 214
- Hermitian operator, 13
- HHL algorithm, 100, 256, 269, 310
- hidden string, 86
- Hilbert space, 5
- hopping term, 223

- hydrogen molecule, 238
- implementation guide, 309
- initial Hamiltonian, 248
- inner product, 4, 295
- instantaneous ground state, 247
- interference, 63, 71
- invariant subspace, 163, 276
- involution, 195
- irrational rotation, 55
- Ising Hamiltonian, 244
- Ising model, 214
- iterative phase estimation, 117
- Jacobi–Anger expansion, 285
- Jordan, Pascual, 223
- Jordan–Wigner transformation, 223, 299
- Jozsa, Richard, 78
- ket, 4
- Kitaev, Alexei, 117, 226
- LCU, *see* linear combination of unitaries
- Lie product formula, 191
- likelihood function
 - multimodality, 283
- linear combination of unitaries, 199, 272, 285
- linear independence, 3, 91
- linear operator, 5
- linear reversible circuit, 225
- linear system of equations, *see* linear systems problem
- linear systems problem, 256, 280
- little-endian convention, *see* little-endian ordering
- little-endian ordering, 20, 43, 101, 135, 298
- Lloyd, Seth, 190, 256
- local Hamiltonian, 190
- LOCC, 77
- logical qubit, 146, 306
- majority vote, 121
- marking a basis state, 63
- matrix inversion
 - polynomial approximation, 288
- MaxCut, 244
- maximally mixed state, 77
- measurement, 16
 - change of basis, 74
 - in the computational basis, 17
- measurement error mitigation, 92
- mid-circuit measurement, 121
- mixer Hamiltonian, 249
- modular exponentiation, 143, 310
- modular inverse, 138
- modular multiplication, 100, 127, 145
 - multiply-accumulate, 147
- modular multiplier, *see* modular multiplication
- molecular Hamiltonian, 218, 287
- Monte Carlo estimation, 177, 281
- Monte Carlo integration, *see* Monte Carlo estimation
- multi-controlled gate, 41, 170
- multi-controlled rotation, 260
- multiple solutions, 168
- Newton–Raphson division, 260
- no-cloning theorem, 74
- no-signaling, 32
- norm, 5
- normalization condition, 8
- number field sieve, 137
- number operator, 223
- oblivious amplitude amplification, 203
- occupation convention, 299
- occupation number, 220
- one-electron integral, 222
- open control, 46
- operator exponential, 24
- operator norm, 54, 194

- Oppenheimer, J. Robert, 219
- oracle, 41, 71, 128, 309
 - bit oracle, 61
 - phase oracle, 61, 78, 162
 - sparse-matrix, 194
- order finding, 127
- order of an element, 127
- orthonormal basis, 3
- outer product, 6, 162
- overshooting, 167, 279

- parity, 207
- parity transformation, 225
- particle-number conservation, 232
- Pauli basis, 206
- Pauli exclusion principle, 219
- Pauli gates, *see* Pauli matrices
- Pauli matrices, 12, 43, 181, 301
- Pauli operator, 76
- Pauli string, 190, 217, 246, 272, 299
 - exponential of, 190, 246, 302
 - weight, 210, 224, 302
- Pauli, Wolfgang, 219
- period finding, 128
 - injectivity condition, 128
- phase estimation, 60, 109, 127, 171, 257, 269, 299
- phase factors, 274
 - numerical computation of, 274
- phase gate, 23, 43, 104, 212, 301
- phase kickback, 34, 60, 110, 170
- phase rotation, 277
- phase-flip gate, 12
- polynomial approximation, 274
- post-selection, 200, 258, 288
- postulates of quantum mechanics, 8
- potential energy surface, 219
- preconditioning, 262
- prepare unitary, 200
- prime number, 136
- prime power, 141
- primitive root, 155
- private key, 149
- probabilistic error cancellation, 93
- problem Hamiltonian, 248
- product form of the QFT, 102
- product state, 29, 103
- projector, 17, 181
 - even and odd parity, 229
- promise problem, 77
- public key, 149

- QAOA, 249
 - fixed schedule, 250
 - variational form, 250
- QFT, *see* quantum Fourier transform
- Qiskit, 43, 101, 135, 303
- QPE, *see* phase estimation
- QSP, *see* quantum signal processing
- QSVT, *see* quantum singular value transformation
- quadratic speed-up, 159, 281
- quadratic unconstrained binary optimization, *see* QUBO
- quantum approximate optimization algorithm, *see* QAOA
- quantum arithmetic, 260
- quantum chemistry, 116, 217
- quantum counting, 171
- quantum error correction, 93, 305
- quantum Fourier transform, 54, 92, 99
 - circuit, 104
 - gate count, 106
 - inverse, 108, 131, 281
 - sign convention, 299
- quantum gate, 12
- quantum parallelism, 59
- quantum phase estimation, *see* phase estimation
- quantum signal processing, 200, 272
 - parity constraint, 273
 - QSP theorem, 273
- quantum singular value transformation, 243, 269

- real polynomials, 278
- theorem, 277
- unifies earlier algorithms, 278
- quantum teleportation, 75
- qubit, 8
- qubit ordering, *see* bit ordering
- qubit tapering, 232
- qubit-wise commuting group, 235
- qubitization, 200, 275
- QUBO, 244
- query complexity, 58, 72, 160, 286
- readout error, 93
- readout problem, 262
- reference state, 232
- reflection operator, 162, 204, 275
 - composition of two, 163
- relative phase, 10
- resource estimate, 306
- reversible computation, 67
- reversible embedding, 59
- ripple-borrow comparator, 199
- root of unity, 141
- rotation angle, 295
- rotation gate, 25, 43, 178, 299, 301
- rotation matrix, 172
- row-computable Hamiltonian, 194, 262
- RSA cryptosystem, 149
- sampling bound, 256
- satisfiability, 159
- Schrödinger equation, 189
- scratch space, 64
- secret string, 82
- select unitary, 200
- self-inverse unitary, 271
- separable state, 29
- Shor's algorithm, 85, 99, 140, 172, 310
- Shor, Peter, 85, 99, 127
- shot allocation, 235
- shots, 17
- sign function, 279
- signal operator, 272
- Simon's algorithm, 85, 99
- Simon, Daniel, 85
- sinc function, 114
- singular matrix, 256
- Slater determinant, 220
- Slater, John C., 220
- small-angle approximation, 260
- Solovay–Kitaev theorem, 57
- span, 3
- sparse matrix, 194, 261
- sparse-access oracle, 272
- spectral gap, 247
- spin-orbital, 220, 299
- square root of a gate, 47
- square-root trick, 139
- state preparation, 175, 206, 262, 289, 306
 - binary-tree, 206
- state preparation error, 92
- state space, 8
- STO-3G basis, 238
- SU(2), 55
- success probability, 115, 163, 200
- superposition, 71
- surface code, 187
- SWAP gate, 35, 101, 145
- symmetric cryptography, 150
- symmetric Trotter formula, 193
- symplectic representation, 236
- T gate, 24
- T-count, 306
- Ta-Shma, Amnon, 195
- target qubit, 32
- Taylor series truncation, 204
- tensor product, 7
- time-ordered product, 248
- Toffoli gate, 45, 306
- trace inner product, 206
- trial division, 136
- Trotter error, 182, 192, 249, 309

- Trotter splitting, 181
- Trotter step, 191, 248
- Trotter–Suzuki decomposition, 191, 309
 - first order, 191
 - second order, 192
- Trotterization, 54, 233, 284
- truth table, 45
- two-electron integral, 222
- two-level unitary, 50
- two-qubit gate, 32
- two-to-one function, 86

- uncomputation, 65, 147, 170, 199, 258, 309
- uniform superposition, 163
- unitarity, 309
- unitary operator, 11, 74, 109, 141
- universal gate set, 24, 55

- universality, 50
- unstructured search, 159

- variational ansatz, 212
- Vazirani, Umesh, 82
- vector space, 1

- walk operator, 275
 - eigenvalues of, 275
- wavefunction, 218
- Wiebe, Nathan, 199
- Wigner, Eugene, 223
- work register, 146

- X -string, 226

- z -rotation, 196
- Z -string, 223
- Z - Y decomposition, 26
- zero-noise extrapolation, 93